A music notation platform

Epiphany

Core Specification

A foundational specification for an open, Rust-native music notation platform: pitch and time primitives, the score graph, the layout intermediate representation, the file format, and the constraint-solver interface.

Working Draft · October 11, 2026

This specification defines the foundational layer of the Epiphany platform: the data model, engraving engine, and file format on which all higher-level features are built. The user interface, audio engine, plugin runtime, and platform integrations are layered above the core and are addressed in separate specifications.

Status of This Document

This draft is **structurally stabilized**. Substantive review passes through the core type model, graph invariants, concurrent reduction semantics, file-format atomicity, solver conformance, and the determinism contract have been completed. The architecture is no longer under active redesign; further revisions are expected to be additive (companion specifications, normative algorithm choices for the open canonical algorithms, extension registry contents) rather than structural.

The remaining blockers to full conformance are companion specifications and the canonical algorithms identified in Appendix D Section D.11:

- Several companion specifications referenced throughout the document have not yet been delivered: Appendix A catalogs them and distinguishes them from accidentally missing content.
- Two algorithms affect canonical state and have not yet received one of the dispositions in Appendix D Section D.11: the tempo curve integration algorithm and the `wallclock_to_musical` root-finding algorithm. Cross-implementation byte equality of canonical score state **MUST NOT** be claimed for outputs derived from these algorithms until they are normatively specified or profile-declared (Appendix D Section D.2). The spelling pre-pass and the notational-decomposition algorithm received the profile-declared-by-versioned-identifier disposition in Pass 12 (Requirements 2.10 and 3.16).
- Extension registry catalogs are referenced by typed identifier but not enumerated here; they are delivered as separate, versioned registry documents.

What conformance can be established now:

- The core data model can be structurally implemented and tested against the invariants in Chapter 5 Section 5.11.
- The operation framework (Chapter 6) can be implemented end-to-end against the canonical reduction algorithm.
- The file format (Chapter 8) can be implemented atomically and crash-recoverably against the fixed prelude and superblock protocol.
- Solver implementations can be developed against the interface contract (Chapter 9); reference-suite conformance becomes checkable when the Reference Suite companion is delivered.

The normative keywords **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are to be interpreted as described in RFC 2119 when they appear in all capital letters.

Conventions

Code samples Rust syntax is used for type signatures, struct definitions, and algorithmic illustrations. Code samples are illustrative unless explicitly labeled normative.

Normative requirements Statements containing **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, or **MAY** are normative. Conforming implementations must respect them.

Non-normative content Discussion, rationale, examples, and diagrams are non-normative unless explicitly stated.

Forward references The specification builds bottom-up. Later chapters depend on earlier ones; forward references are minimized but occasionally unavoidable and are marked when they occur.

Contents

Status of This Document i

Conventions ii

Reading Guide xii

1 Introduction and Scope 1
- 1.1 Purpose . . . 1
- 1.2 Scope . . . 1
- 1.3 Design Goals . . . 2
- 1.4 Document Structure . . . 2

2 Pitch 4
- 2.1 Design Principles . . . 4
- 2.2 The `Pitch` Type . . . 4
- 2.3 Scale Position . . . 5
  - 2.3.1 Octave Convention . . . 7
  - 2.3.2 The `CmnNominal` Type . . . 8
  - 2.3.3 Transposition and the `Interval` Type . . . 8
- 2.4 Acoustic Realization . . . 10
  - 2.4.1 Cents as the Offset Unit . . . 10
  - 2.4.2 Tuning Reference and Inheritance . . . 10
  - 2.4.3 Reference Pitch . . . 11
- 2.5 Spelling . . . 11
  - 2.5.1 The Spelling Attachment . . . 11
  - 2.5.2 Spelling Sources . . . 13
  - 2.5.3 Configurable Precedence . . . 14
  - 2.5.4 The Spelling Pre-Pass . . . 14
  - 2.5.5 Absent Accidentals . . . 16
  - 2.5.6 Analytical Layers . . . 16
- 2.6 Pitch Identifiers . . . 17
- 2.7 What Pitch Is Not . . . 17
- 2.8 Equality and Comparison . . . 18
- 2.9 Forward References . . . 18

3 Time and Duration 19
- 3.1 Design Principles . . . 19
- 3.2 The Rational Time Type . . . 19

3.2.1 Recommended Implementation: Inline-or-Promoted . . . 20
3.2.2 Promotion and Demotion . . . 20
3.2.3 Equality and Ordering . . . 21
3.2.4 Units . . . 21
3.3 Position and Duration as Distinct Types . . . 21
3.4 Wall-Clock Time . . . 22
3.4.1 Wall-Clock and Musical Time Coexist . . . 23
3.5 Time Anchors . . . 23
3.5.1 Anchor Resolution . . . 25
3.5.2 Anchor Stability Under Edits . . . 25
3.6 Time Signatures and Meter . . . 25
3.6.1 Irrational Meters . . . 26
3.6.2 Polymetric and Polytemporal Scores . . . 27
3.7 Tempo and the Tempo Map . . . 27
3.7.1 Conversion . . . 28
3.8 Sounding Duration and Notational Decomposition . . . 30
3.8.1 The Notational Decomposition . . . 30
3.8.2 The Decomposition Pre-Pass . . . 31
3.9 Tuplets as Grouping Objects . . . 32
3.9.1 Tuplet Nesting . . . 34
3.9.2 Tuplet Consistency . . . 34
3.10 Region Time Models . . . 35
3.10.1 Metric Time . . . 35
3.10.2 Proportional Time . . . 35
3.10.3 Aleatoric Time . . . 36
3.10.4 Concurrent Time Models . . . 38
3.11 The Notion of “Now” . . . 38
3.12 Forward References . . . 38

4 Tuning Systems and Pitch Spaces 39
4.1 Design Principles . . . 39
4.2 Pitch Spaces . . . 40
4.2.1 Position Structure . . . 40
4.2.2 Interval Algebra . . . 41
4.2.3 Nominal Registry . . . 42
4.2.4 Transposition Behavior . . . 42
4.2.5 Spelling Rule Sets . . . 43
4.3 Accidental Registries . . . 43
4.3.1 Pitch Space Modifications . . . 44
4.3.2 Accidental Engraving Metadata . . . 44
4.3.3 Combination Behavior . . . 46
4.3.4 Score-Local Extensions . . . 46
4.4 Glyph References and SMuFL . . . 46
4.4.1 SMuFL Versioning . . . 47
4.5 Tuning Systems . . . 47
4.5.1 Tuning Resolution . . . 48
4.5.2 The Resolution Contract . . . 49

4.5.3 Adaptive Tuning . . . 49
4.6 Reference Pitch . . . 51
4.6.1 Multiple Reference Pitches . . . 51
4.7 Score Tuning Context and Hierarchical Resolution . . . 51
4.7.1 Resolution Order . . . 52
4.7.2 Compatibility Constraints . . . 53
4.8 Built-in Catalog . . . 53
4.8.1 Built-in Pitch Spaces . . . 53
4.8.2 Built-in Tuning Systems . . . 54
4.8.3 Temperament Constructions . . . 55
4.8.4 The Static 5-Limit Construction . . . 60
4.8.5 Default Score Configuration . . . 60
4.9 Forward References . . . 61

5 The Score Graph 62
5.1 Design Principles . . . 62
5.2 Top-Level Score Structure . . . 62
5.2.1 Score Metadata . . . 64
5.3 Identifiers . . . 65
5.3.1 Identifier Generation . . . 67
5.3.2 System-Derived Identifier Namespace . . . 68
5.3.3 System-Derived Counter Collisions . . . 69
5.3.4 System-Derived Pitch Identity . . . 72
5.3.5 Identifier Stability . . . 72
5.4 The Event Arena . . . 72
5.4.1 The Event Type . . . 73
5.5 The Canvas . . . 80
5.5.1 Regions . . . 81
5.5.2 Region Overlap and Concurrency . . . 82
5.5.3 Staff-Based Content . . . 82
5.6 Staves, Voices, and Instruments . . . 86
5.6.1 Instruments . . . 86
5.6.2 Staves: Identity Versus Instance . . . 87
5.6.3 Voices . . . 89
5.7 Cross-Cutting Structures . . . 92
5.7.1 Slurs and Phrase Marks . . . 92
5.7.2 Ties . . . 93
5.7.3 Beams . . . 95
5.7.4 Spanners . . . 96
5.7.5 Markers . . . 97
5.7.6 Lyrics . . . 98
5.7.7 Repeat Structures . . . 99
5.7.8 Analytical Annotations . . . 99
5.7.9 Comments . . . 100
5.7.10 Graphic Gestures . . . 100
5.8 Graphic Content . . . 101
5.8.1 Graphic Objects . . . 101

5.8.2 Coordinates . . . 102
5.8.3 Time Bindings . . . 103
5.9 Parts . . . 104
5.9.1 Parts Are Projections, Not Storage . . . 105
5.10 Analysis Layers and Views . . . 105
5.10.1 Analysis Layers . . . 105
5.10.2 Views . . . 105
5.11 Graph Invariants . . . 106
5.12 Indexes and Auxiliary Structures . . . 109
5.13 Forward References . . . 110

6 Semantic Operations and Concurrent Reduction 111
6.1 Design Principles . . . 111
6.2 The Operation Framework . . . 112
6.2.1 Operation Identity and Stamps . . . 112
6.2.2 Causal Context via Dotted Version Vectors . . . 113
6.2.3 Operation Envelopes . . . 114
6.2.4 The Operation Set as CRDT . . . 117
6.2.5 OperationId Collisions and Replica Equivocation . . . 117
6.2.6 Envelope Acceptance and Malformed-Envelope Behavior . . . 119
6.2.7 Per-Replica HLC Monotonicity . . . 120
6.3 The Canonical Reduction . . . 121
6.3.1 Operation Lifecycle . . . 121
6.3.2 The Reduction Algorithm . . . 122
6.3.3 Canonical Reduction Order . . . 122
6.3.4 Operation Effects . . . 123
6.3.5 Cache and Snapshot Discipline . . . 126
6.4 Object Existence and Tombstones . . . 126
6.4.1 Add-Tombstone Semantics . . . 126
6.4.2 Tombstone Retention . . . 127
6.4.3 Reference Resolution Across Tombstones . . . 127
6.5 Conflict Records . . . 127
6.5.1 Conflict Record Type . . . 128
6.5.2 The Conflict Registry . . . 130
6.5.3 ConflictId Derivation . . . 131
6.5.4 Conflict Resolution Operations . . . 132
6.6 Transactions . . . 132
6.6.1 Transaction Descriptor Causal Ordering . . . 133
6.7 Re-Anchoring . . . 135
6.7.1 The Re-Anchor Function . . . 135
6.7.2 Total Ordering for "Nearest" . . . 136
6.7.3 The Re-Anchoring Rule Table . . . 136
6.8 Undo . . . 140
6.8.1 UndoTransaction Payload . . . 140
6.8.2 Undo Semantics . . . 141
6.9 LWW Discipline . . . 141
6.9.1 The LwwAdvisory Marker . . . 141

6.9.2 Eligible Fields . . . 142
6.10 Validation Modes . . . 143
6.11 Per-Kind Reduction Summary . . . 143
6.12 Representative Operations . . . 144
6.12.1 InsertEvent . . . 144
6.12.2 DeleteEvent . . . 146
6.12.3 CreateCrossCutting (Slur) . . . 147
6.12.4 RespellPitch . . . 148
6.12.5 ChangeRegionTimeModel . . . 149
6.12.6 Transpose . . . 150
6.12.7 SetUserSystemBreak . . . 151
6.13 Operation Catalog Conformance . . . 152
6.14 Forward References . . . 152

7 Layout Intermediate Representation 153
7.1 Design Principles . . . 153
7.2 Spatial Primitives . . . 154
7.2.1 Units . . . 154
7.2.2 Geometric Types . . . 154
7.3 Provenance . . . 155
7.4 The Stage Pipeline . . . 157
7.4.1 Pipeline Overview . . . 157
7.4.2 Stage Contracts . . . 158
7.5 LogicalLayoutIR . . . 158
7.5.1 Top-Level Structure . . . 158
7.5.2 Layout Regions . . . 159
7.5.3 The Time Axis Trait . . . 159
7.5.4 Layout Objects . . . 161
7.5.5 Note Layout: A Worked Example . . . 162
7.5.6 Engraving Decisions . . . 163
7.6 Engraving Overrides . . . 164
7.6.1 Override Definition . . . 164
7.6.2 Override Resolution . . . 165
7.7 ConstrainedLayoutIR . . . 166
7.7.1 Glyph-Level Objects . . . 166
7.7.2 Spring Slots . . . 167
7.7.3 Vertical Bands . . . 167
7.7.4 Constraints . . . 169
7.8 ResolvedLayoutIR . . . 171
7.9 RenderIR (Interface Only) . . . 174
7.10 Incremental Layout and Caching . . . 175
7.10.1 The Layout Cache . . . 175
7.10.2 Invalidation Rules . . . 176
7.10.3 Frame-Budget Requirements . . . 176
7.11 Region Uniformity . . . 176
7.12 Glyph Catalog Interface . . . 177
7.12.1 Glyph Catalog Identity for Layout Conformance . . . 178

7.13 Forward References . . . 179

8 File Format 180
8.1 Design Principles . . . 180
8.2 Durable Writes . . . 181
8.3 The Bundle Layout . . . 182
8.3.1 The Fixed Header . . . 182
8.3.2 The Superblock Slots . . . 184
8.3.3 Superblock Selection . . . 185
8.4 The Manifest . . . 186
8.4.1 Canonical and Non-Canonical Manifest Roots . . . 188
8.4.2 File, Document, and Lineage Identity . . . 189
8.4.3 Manifest Encoding . . . 190
8.5 Content Hashing . . . 191
8.5.1 Domain-Separated Preimages . . . 191
8.6 Chunks . . . 192
8.7 Operation Envelope Blocks . . . 194
8.7.1 The Operation Index . . . 195
8.8 Snapshots and Canonical Bases . . . 196
8.8.1 The Canonical Document Identity . . . 197
8.8.2 Pruning . . . 198
8.9 Blobs . . . 199
8.10 The Atomic Write Protocol . . . 199
8.10.1 Crash Recovery . . . 200
8.11 Garbage Collection and Retention . . . 200
8.11.1 Retention Policy . . . 201
8.11.2 Garbage Collection Rules . . . 201
8.12 Forward Compatibility and Edit Barriers . . . 202
8.12.1 Extension Declarations . . . 202
8.12.2 Behavior Under Unknown Extensions . . . 205
8.13 Text Projection . . . 206
8.13.1 Use Cases . . . 207
8.14 Schema Versioning . . . 207
8.15 Format Profiles . . . 209
8.16 Compression . . . 211
8.17 Streaming Reads . . . 211
8.17.1 Required Locatability . . . 211
8.17.2 Cold Open Procedure . . . 212
8.17.3 Memory Mapping . . . 212
8.18 Binary Format Companion . . . 213
8.18.1 Ratified Convention Baseline . . . 213
8.19 Forward References . . . 214

9 Constraint-Solver Interface 215
9.1 Design Principles . . . 215
9.2 The Solver Interface . . . 216
9.3 The Solver Report . . . 217

9.4 Constraint Families . . . 219
9.4.1 Strength Levels . . . 220
9.4.2 Normative Constraint Families . . . 220
9.5 Quality Metrics . . . 220
9.5.1 Pareto Frontiers as Design Target . . . 221
9.6 Conformance Tiers . . . 222
9.6.1 Minimal Layout Solver . . . 223
9.6.2 Standard Engraving Solver . . . 223
9.6.3 Advanced / Extension-Aware Solver . . . 224
9.6.4 Tier Declaration and Document Compatibility . . . 224
9.7 Determinism . . . 224
9.7.1 Within-Implementation Determinism . . . 224
9.7.2 Cross-Implementation Conformance . . . 225
9.8 Incremental Solving . . . 225
9.8.1 Invalidation Scopes . . . 225
9.8.2 Observational Equivalence . . . 226
9.8.3 Propagation Documentation . . . 226
9.9 Conformance: The Reference Suite . . . 226
9.10 Reference Algorithm . . . 227
9.11 Forward References . . . 228

10 Performance Requirements 229
10.1 Core/Product Boundary . . . 229
10.2 Design Principles . . . 230
10.3 Reference Hardware Profile . . . 230
10.4 Frame Budgets . . . 231
10.4.1 Interactive Edit Latency . . . 231
10.4.2 Multi-System Edits . . . 231
10.4.3 Full Re-Layout . . . 232
10.5 Cold Open . . . 232
10.5.1 Cold Open Measurement . . . 232
10.5.2 Streaming Read Discipline . . . 232
10.6 Memory Budgets . . . 233
10.6.1 Memory Categories . . . 233
10.6.2 Per-Object Memory Discipline . . . 233
10.7 Core Data Structures for Audio Thread Use . . . 233
10.7.1 Snapshot API . . . 234
10.8 Constraint-Solver Performance Targets . . . 234
10.9 File Format Performance . . . 235
10.10 Measurement Methodology . . . 235
10.11 Regression Testing . . . 235
10.12 Forward References . . . 236

11 Extension Points 237
11.1 Design Principles . . . 237
11.2 Notation Grammar Extensions . . . 237
11.3 Tuning System Extensions . . . 238

11.4 Glyph and Font Extensions . . . 238
11.5 Score Graph Extensions . . . 238
11.6 Layout IR Extensions . . . 239
11.7 Constraint-Solver Extensions . . . 239
11.8 File Format Extensions . . . 239
11.9 Extension Registry Contract . . . 239
11.9.1 Identifier Stability . . . 240
11.9.2 Discoverability . . . 240
11.10 Plugin Runtime Considerations . . . 240
11.11 Conformance and Extensions . . . 240
11.12 Forward References . . . 241

A Intentionally Deferred Types and Specifications 242
A.1 Companion Specifications . . . 242
A.2 Externally Provided Components . . . 243
A.3 Open Canonical Algorithms . . . 243
A.4 Extension Registry Catalogs . . . 244
A.5 Support-Type Identity Semantics . . . 244
A.6 Distinction from Open Questions . . . 246

B Glossary 247

C Bibliography and References 252

D Determinism Contract 255
D.1 Thesis . . . 255
D.2 Layers of Determinism . . . 255
D.3 Floating-Point Values in Canonical State . . . 256
D.3.1 Permitted Forms . . . 257
D.3.2 Serialization . . . 257
D.3.3 Equality . . . 257
D.4 Rounding and CPU Behavior . . . 257
D.5 Exact and Quantized Representations . . . 258
D.5.1 Quantized Layout Coordinates . . . 258
D.6 Tolerance Classes . . . 259
D.7 Ordered Iteration over Sets and Maps . . . 260
D.8 Text and Unicode . . . 261
D.9 Randomness and Parallelism . . . 262
D.9.1 Randomness . . . 262
D.9.2 Parallelism . . . 262
D.10 Compression and File Bytes . . . 262
D.11 Open Algorithm Hooks . . . 263
D.12 Conformance Statement . . . 263

E Canonical Byte-Layout Reference 265
E.1 Status of this reference . . . 265
E.2 Shared encoding conventions . . . 265
E.3 System-derived identifier function . . . 265

E.4 Discriminant tables, derivations, and encodings . . . 266
E.4.1 Object-graph identity (canonical document state) . . . 266
E.4.2 Bundle layer (chunk store, manifest, blobs) . . . 266
E.4.3 Operation layer (transaction metadata) . . . 267
E.4.4 Primitive scalar encodings . . . 267
E.5 Domain-tag registry . . . 268
E.6 Reference-implementation locks . . . 268
E.7 Layouts deferred to the companions . . . 269

F Revision History 271

Reading Guide

This document is long. The following tables summarize the most load-bearing concepts and where they are defined. They are non-normative; the chapters they reference are authoritative.

Score Graph Identity Hierarchy

The score graph is organized as a containment hierarchy of identified objects. Each level introduces a distinct concept of identity.

Level	Identifier	Role
Score	—	Document root. Carries metadata, instruments, global staves, staff groups, tuning context, tempo map, canvas.
Canvas	—	Spatial root. Partitioned into regions.
Region	`RegionId`	Spatial-temporal container. Declares time model (metric/proportional/aleatoric) and content model (staff-based/free-graphic/hybrid).
Staff	`StaffId`	Global, abstract staff identity persisting across regions. Declared once at score level.
Staff instance	`StaffInstanceId`	Region-local manifestation of a staff. One `Staff` may have multiple instances across regions.
Voice	`VoiceId`	Polyphonic line within a staff instance. Carries `VoiceOrigin`: user-declared, imported, or system-promoted.
Event	`EventId`	Rhythmic atom: pitched, unpitched, rest, indeterminate, trajectory, graphic, cue. Lives in the score's event arena.
Identified pitch	`PitchId`	Stable identity for a pitch within an event. Enables spelling attachments, respelling, stable tie pairing through chord reordering.

Canonical versus Non-Canonical

Many file-format and runtime objects exist in two forms. Canonical objects define the document; non-canonical objects accelerate access but never affect canonical state.

Object	Status	Notes
Operation envelope set	Canonical	The replicated grow-only CRDT defining the document (Chapter 6).
Operation envelope block	Canonical (storage)	Soft 1 MiB blocks on disk (Chapter 8).
Operation index	Non-canonical	Acceleration of operation-id lookup. Rebuildable.
Canonical-base snapshot	Canonical	The manifest's `canonical_base`: at most one active, with declared causal frontier and reduction algorithm version. Required for pruned documents.
Acceleration snapshot	Non-canonical	Listed in `acceleration_snapshots`. Cache only. Discardable.
Manifest	Canonical (index)	Table of roots; itself a chunk; referenced from the active superblock.
Materialized score graph	Derivative	Deterministic reduction of the operation set under the active reduction algorithm.
Layout cache, glyph cache, render artifacts	Non-canonical	Always discardable (Section 6.3.5).
Conflict registry	Canonical	Stable, addressable, user-visible records of operations that could not apply cleanly.

Operation Lifecycle

Every operation traverses four phases between user intent and its visible effect.

Phase	Description
Prepare	Local UI command validates enough to construct a well-formed envelope. Advisory only; failure here means the UI rejects the user's action. Does not bind canonical semantics.
Commit	The envelope is added to the replicated operation set. After commit, the operation is part of canonical state and **MUST NOT** be discarded outside the pruning protocol.
Reduce	During materialization, the operation is processed by the canonical reduction algorithm against working state. Deterministic: every replica with the same operation set produces the same output.

Phase	Description
Report	The reduction produces a deterministic `OperationEffect`: `Applied`, `AppliedWithRepair`, `Conflicted`, `TombstonedTarget`, or `NoOp`. Effects are visible graph facts.

Determinism Layers

The specification distinguishes five layers of determinism. Conflating them is the most common source of impossible requirements.

Layer	Contract
Canonical score determinism	Identical operation sets produce identical materialized score states across all conforming implementations, subject to all canonical algorithms having received the dispositions in Appendix D.
Canonical serialization	Same canonical state → same chunk hashes, same text projection, same canonical iteration order.
Layout determinism	Within one solver implementation at a fixed version: byte-equal output for identical input. Across implementations: reference-suite thresholds, not byte equal.
Conformance determinism	Reference-suite-based. Cross-implementation byte equality is not required for layout.
Non-canonical caches	May vary freely across runs, platforms, implementations. **MUST NOT** affect canonical state.

Solver Tiers

Solver implementations declare a conformance tier. Tiers are orthogonal to file-format profiles (Chapter 8).

Tier	Obligations
Minimal	Hard-constraint validity, within-implementation determinism, well-formed `SolveReport`, CMN + metric regions, Minimal reference-suite subset. Aesthetic quality may be mediocre.
Standard	Minimal + Standard-tier thresholds on the full reference suite, Measure-local invalidation under incremental solving, all Standard constraint families. Professional engraving quality for common-practice notation.
Advanced	Standard + proportional and aleatoric regions, microtonal accidentals, extension-contributed constraints, System-local invalidation under incremental solving.

Chapter Map

Ch.	Content
1	Scope, design principles, glossary of key terms used throughout.
2	Pitch as two intrinsic layers (scale position and acoustic), spelling subsystem with stacked accidentals, analytical layers.
3	Rational time, time anchors with `AnchorOffset`, three region time models, tempo map with explicit segment endpoints, sounding duration vs. notational decomposition.
4	Pitch spaces (including CMN and JI), tuning systems, score tuning context, accidental registries, glyph references.
5	Score graph, event arena, cross-cutting structures, regions, staff identity (Staff vs. StaffInstance), polymeter via per-staff metric grids, voice origin, graph invariants.
6	Operation framework, canonical reduction, DVV causal contexts, operation effects, tombstones, conflict records, transactions, re-anchoring, undo, LWW discipline.
7	The four-stage layout intermediate representation; provenance and incremental invalidation.
8	Fixed prelude, two superblock slots, atomic commit, content-addressed BLAKE3 chunks, operation-envelope blocks, snapshots as canonical bases, edit barriers, text projection.
9	Solver interface, SolveReport, deterministic budgets, NormalizedMetric, three conformance tiers, reference-suite conformance, six invalidation scopes.
10	Performance requirements: frame budgets, cold open, memory, audio-thread discipline, measurement.
11	Consolidated catalog of extension points by chapter of origin.
A	Intentionally deferred types: companion specifications, externally provided components, open canonical algorithms.
B	Glossary of defined terms with chapter cross-references.
C	Bibliography and references.
D	Determinism contract: the legal code for reproducibility.
E	Canonical byte-layout reference: every ratified discriminant, derivation preimage, and primitive encoding in one place, for the Binary Format companion to import.
F	Revision history.

Chapter 1

Introduction and Scope

1.1 Purpose

This document specifies the core layer of *Epiphany*, a music notation platform. The core comprises the abstract musical data model, the engraving engine that transforms that data into a typeset layout, and the on-disk file format that serializes both. Everything else built atop the platform—editing UI, audio engine, plugin runtime, collaboration, export pipelines—depends on this core and must conform to its interfaces.

The name *Epiphany* evokes both the original sense of the word—a manifestation made visible—and the experience of music itself, which strikes the listener and the creator alike. Music notation is, in a literal sense, the act of making the inaudible visible: a body of work this central to that task deserves a name of corresponding weight.

1.2 Scope

In scope:

- Pitch, time, and duration primitives.
- The score graph: the in-memory representation of musical content.
- The layout intermediate representation produced by the engraving engine.
- The file format: the canonical on-disk projection of the score graph.
- The constraint-solver interface consumed by the engraving engine.
- Extension points for non-Common-Music-Notation systems, including microtonal pitch spaces and graphic-notation regions.

Out of scope for this document:

Non-Goal

The following are explicitly out of scope for this core specification. Each is addressed by a separate specification layered atop the core.

- User interface, input handling, and editing operations.
- Audio synthesis, playback, and plugin (VST3/AU/CLAP/LV2) hosting.

- The plugin runtime (WebAssembly capability-based extensions).
- Real-time collaboration transport and federation protocols.
- Import and export pipelines for foreign formats (MusicXML, MEI, LilyPond, MIDI, etc.), although the core defines the data sufficient to make lossless interchange feasible.
- Machine-learning components (transcription, OMR, engraving assistance).
- Platform-specific integrations (file pickers, sharing, accessibility APIs).

1.3 Design Goals

Correctness over convenience. The data model must represent music accurately, even when this is harder than a convenient approximation. MIDI-style pitch numbers, for example, are inadequate and rejected; see Chapter 2.

Extensibility without bolt-ons. Microtonality, graphic notation, historical notations, and future notational systems must be expressible within the core data model, not as escape hatches.

Determinism. Given identical input, the engraving engine must produce identical output. The on-disk format must serialize deterministically.

Incrementality. The data model and layout engine must support fine-grained incremental edits without full recomputation.

Reducibility. All editing operations on the score graph must have well-defined reduction rules under canonical deterministic reduction (Chapter 6), enabling collaboration and version control as natural consequences of the data model rather than bolt-on features.

Performance. Layout and rendering of a 100-page orchestral score must remain interactive on commodity hardware. See Chapter 10 for budgets.

Privacy. The core must operate entirely offline. No core functionality may depend on network access.

1.4 Document Structure

The remainder of this document is organized bottom-up:

1. Chapter 2 defines pitch: its layered structure, the spelling subsystem, scale positions, and acoustic realization.
2. Chapter 3 defines time and duration primitives.
3. Chapter 4 defines tuning systems, pitch spaces, and accidental registries.
4. Chapter 5 defines the score graph: how primitives compose into staves, voices, regions, and the overall score structure, including the canvas/region model that accommodates non-CMN notations.
5. Chapter 6 defines the semantic operations on the score graph: the canonical set of mutations and their reduction semantics under concurrent edits.

6. Chapter 7 defines the layout intermediate representation: the contract between the score graph (input) and the engraving engine (consumer).
7. Chapter 8 defines the on-disk file format, including serialization, chunking, the operation-envelope set and its deterministic reduction, and schema evolution.
8. Chapter 9 defines the constraint-solver interface consumed by the engraving engine.
9. Chapter 10 states performance requirements and measurement methodology.
10. Chapter 11 describes the extension points by which the core supports notation grammars beyond Common Music Notation.

Chapter 2

Pitch

This chapter specifies the pitch primitive: the data type representing a single sounding (or to-be-sounded) note's identity in the score graph. Pitch is the most consequential primitive in the core because nearly every higher-level operation—transposition, key analysis, accidental rendering, playback frequency, interchange, voice-leading analysis—is defined in terms of it. The definition is therefore deliberately verbose.

2.1 Design Principles

Separation of identity layers. Pitch comprises three independent semantic layers: scale position (analytical identity), acoustic realization (sounding frequency), and spelling (notational appearance). Each is a first-class concern and **MUST** be representable independently. Conflating them is a design error.

Spelling is attachment, not field. A pitch carries its scale position and acoustic identity intrinsically. Spelling is attached externally through a separate indexed subsystem (see Section 2.5). A pitch may have zero, one, or multiple spelling attachments.

Grammar neutrality. The pitch type **MUST** support Common Music Notation as a fast path while remaining capable of representing pitches in arbitrary user-defined notation grammars. CMN is a privileged default, not a hardcoded ceiling.

Microtonality is not an extension. Microtonal pitches are representable using the same primitive as 12-tone equal-tempered pitches. No separate type, no escape hatch.

Unpitched events are not pitches. Unpitched percussion and similar events **MUST NOT** be represented using the pitch type. See Section 2.7.

2.2 The `Pitch` Type

A pitch is a record of two intrinsic layers:

```
/// A pitch's intrinsic identity. Spellings are attached externally;
/// see Section~\ref{sec:pitch:spelling}.
pub struct Pitch {
    /// The analytical identity within a pitch space. Determines
    /// transposition, key analysis, and Roman-numeral behavior.
    pub scale_position: ScalePosition,
```

```
    /// The acoustic identity: what frequency this pitch produces when
    /// sounded, expressed relative to a tuning system.
    pub acoustic: AcousticPitch,
}
```

The two layers are independent. A given scale position can be acoustically realized in many ways depending on the active tuning system; a given acoustic realization can correspond to many scale positions in different analytical contexts. The pitch carries both; neither is derivable from the other in general.

RATIONALE

Storing both layers, rather than deriving one from the other, costs a small number of bytes per pitch and saves a large amount of complexity downstream. Derivation requires the renderer, the analyzer, the transcriber, and the importer to each agree on a derivation procedure; storage makes the pitch self-describing.

```
/// A closed range of pitches, used for advisory declarations: an
/// instrument's playable compass and an indeterminate event's pitch hint.
/// Both bounds are full pitches.
pub struct PitchRange {
    pub lowest: Pitch,
    pub highest: Pitch,
}
```

A `PitchRange` is *advisory*. Whether a given pitch lies within it is decidable only when the pitch and both bounds share a pitch space that defines an order (the common CMN case); cross-space or partially-ordered comparisons are treated as "not out of range" — a sound-but-incomplete check, as with region time overlap. A range is well-formed when `lowest` does not sort above `highest` in a common ordered space. A solver **MAY** flag an out-of-range pitch but **MUST NOT** treat the range as a hard constraint. The type is defined with schema major 1, closing the gap in which `IndeterminacyHints` and `Instrument` named a `PitchRange` that no section defined.

2.3 Scale Position

A scale position locates the pitch within a *pitch space*. A pitch space is the analytical universe in which scale-degree and interval relationships are defined.

```
pub struct ScalePosition {
    /// The pitch space this position is defined within. References an
    /// identifier in the built-in pitch-space catalog
    /// ('sec:tuning:builtin'); a score selects from this catalog and
    /// does not define its own entries.
    pub space: PitchSpaceId,

    /// The position within that space. Interpretation is space-defined.
    pub position: PitchSpacePosition,
}
```

The `PitchSpacePosition` type is a tagged union supporting the common cases with a fast representation, plus a registry-id escape hatch for arbitrary grammars:

```
pub enum PitchSpacePosition {
    /// Common Music Notation: a diatonic nominal plus chromatic
    /// alteration plus octave. The fast path for tonal Western music.
    Cmn {
        nominal: CmnNominal,        // C, D, E, F, G, A, B
        alteration: i8,             // chromatic steps of the enclosing space
        octave: i8,                 // scientific pitch notation
    },

    /// N-tone integer position. Used for 12-tone serial music, EDO
    /// systems, and any space where pitches are indexed as integers.
    Integer {
        space_size: u16,            // e.g., 12, 19, 31, 53
        index: i32,                 // absolute index; octave is implicit
    },

    /// Just-intonation lattice position. Used for JI pitch spaces.
    /// Each component is the integer power of the corresponding prime
    /// in the space's prime basis. The pitch space declares the basis
    /// ordering; by convention prime 2 is first, so the leading
    /// component carries octave information.
    JiVector {
        components: Vec<i32>,
    },

    /// A registered position whose interpretation is defined by the
    /// pitch space's grammar plugin. Used for maqamat, gamelan,
    /// historical modes, and other non-lattice, non-CMN grammars.
    Registered(PositionRegistryId),
}
```

REQUIREMENT 2.1

For a `PitchSpacePosition::Cmn` enclosed by a `ScalePosition`, `alteration` **MUST** be measured in steps of that pitch space's chromatic layer. Its zero point for each nominal **MUST** be the space's `nominal_to_chromatic` mapping. A `PitchSpaceModification::CmnChromatic` value **MUST** use that same unit in every pitch space whose accidental registry contains it. Thus one step is a semitone in `cmn-12` and a quarter-tone in `cmn-24`; a flat is respectively -1 and -2.
When a bare `PitchSpacePosition::Cmn` is embedded as `ScoreTuningContext.reference.position`, the score's `default_pitch_space` **MUST** supply the chromatic layer and mapping for this rule. Any other context embedding a bare CMN position **MUST** identify the pitch space that denominates it.

REQUIREMENT 2.2

For `PitchSpacePosition::JiVector`:

- `components.len()` **MUST** equal the number of primes in the enclosing pitch space's declared prime basis.
- The basis ordering is established by the pitch space; the built-in JI pitch spaces (`ji-5limit`, `ji-7limit`, `ji-11limit`) order primes ascending starting with 2.
- The vector $[a_0, a_1, a_2, \ldots]$ denotes the ratio $p_0^{a_0} \cdot p_1^{a_1} \cdot p_2^{a_2} \cdots$ where p_i is the i-th prime in the basis. With the conventional basis $\{2, 3, 5, \ldots\}$, the first component holds octave information.
- Pitch spaces **MAY** declare octave-reduced equivalence, in which case the first component is normalized to a canonical range. Without such a declaration, full register is preserved.

REQUIREMENT 2.3

Every score **MUST** select at least one pitch space from the built-in catalog (Section 4.8). The default pitch space for CMN scores is `cmn-12`, which is diatonic-over-chromatic in structure (seven diatonic nominals A–G over twelve chromatic positions), anchored to the score's reference pitch as resolved through the score tuning context (Chapter 4). Scores **MAY** select additional pitch spaces from that catalog; pitches in different spaces cannot be directly compared and operations between them **MUST** go through an explicit space-conversion mechanism.

RATIONALE

Score-local pitch-space definition — a score introducing a pitch space of its own, beyond selecting from the built-in catalog — is deferred to a later schema major. The Chapter 4 type surface (`PitchSpace`, `PositionStructure`, `IntervalAlgebra`, and their kin) has never had a consumer; committing a registry for it to the wire now would freeze that surface permanently under `req:binfmt:frozen-layout`, before any implementation has exercised it. Selecting from the built-in catalog is the only reading the data model has ever supported: neither `Score` nor `ScoreTuningContext` carries a pitch-space or tuning-system registry, and none is added here.

2.3.1 Octave Convention

For the `Cmn` variant, the octave field uses Scientific Pitch Notation: middle C is C4. This convention is normative throughout the specification.

REQUIREMENT 2.4

The `octave` field of `PitchSpacePosition::Cmn` **MUST** follow Scientific Pitch Notation. Middle C is C4. The lowest A on a standard piano is A0. C-1 is one octave below the lowest C on a standard piano.

RATIONALE

Scientific Pitch Notation is the dominant modern convention in English-language music theory and notation software. MIDI octave numbering, where middle C is sometimes C3 and sometimes C5 depending on vendor, is rejected as a source of foot-guns. Helmholtz notation is expressible at the display layer; the data model commits to Scientific.

2.3.2 The `CmnNominal` Type

```
#[repr(u8)]
pub enum CmnNominal { C = 0, D = 1, E = 2, F = 3, G = 4, A = 5, B = 6 }
```

The discriminants are normative: they define the diatonic step ordering used by transposition algorithms.

2.3.3 Transposition and the `Interval` Type

Transposition acts on *scale position*, not on acoustic realization. It moves a pitch's analytical identity through the pitch space and leaves the tuning context that sounds it untouched. A transposition therefore requires no tuning catalog (Chapter 4) — tuning decides what frequency a scale position sounds at, not how to add an interval to one. It requires an interval:

```
/// A signed interval: the transposition primitive. Defined here and
/// referenced by Instrument.transposition (Chapter 5), which is the
/// written-versus-sounding interval of a transposing instrument
/// (a B-flat clarinet is -1 diatonic, -2 chromatic).
pub struct TranspositionInterval {
    /// Signed diatonic steps: how far the nominal moves along C..B.
    /// A perfect fifth up is +4 (C -> G); an octave up is +7.
    pub diatonic_steps: i32,

    /// Signed steps in the enclosing pitch space's chromatic layer.
    /// A perfect fifth in cmn-12 is +7 and an octave is +12.
    /// The corresponding values are space-relative.
    pub chromatic_steps: i32,
}
```

Both components are load-bearing. The diatonic component fixes the *spelling* — which nominal, hence which staff line the notehead occupies — and the chromatic component fixes the *sound*. Neither determines the other: an augmented second $(+1, +3)$ and a minor third $(+2, +3)$ sound alike and are written differently, while a diminished sixth $(+5, +7)$ and a perfect fifth $(+4, +7)$ are written a step apart and sound the same. A scalar chromatic-step count cannot express the difference, and an implementation carrying only one component is not transposing but altering.

REQUIREMENT 2.5

Let n be the `nominal`'s normative discriminant (Section 2.3.2). Let the enclosing pitch space have a `DiatonicOverChromatic` structure with C chromatic positions per octave and nominal mapping m. A `PitchSpacePosition::Cmn` position's *absolute chromatic coordinate* is

$$s = m(\texttt{nominal}) + \texttt{alteration} + C \cdot \texttt{octave}.$$

Transposing that position by `TranspositionInterval { d, c }`, where c is measured in steps of the same chromatic layer, **MUST** produce

$$\begin{aligned} \texttt{nominal}' &= \textsc{CmnNominal}\big((n+d) \bmod 7\big), \\ \texttt{octave}' &= \texttt{octave} + \lfloor (n+d)/7 \rfloor, \\ \texttt{alteration}' &= (s+c) - \big(m(\texttt{nominal}') + C \cdot \texttt{octave}'\big), \end{aligned}$$

where mod and $\lfloor \cdot \rfloor$ are Euclidean (the remainder is non-negative). The diatonic component alone selects the nominal and the octave; the alteration absorbs exactly the residue. A transposition **MUST** be refused — atomically, changing nothing — when any target satisfies any of:

- its `scale_position.position` is not `Cmn`, so the `Interval` above has no defined action on it;
- its enclosing pitch space cannot be resolved to a compatible `DiatonicOverChromatic` structure and nominal mapping;
- its `acoustic.realization` is `AcousticRealization::AbsoluteHz`, which overrides the tuning system, so moving the scale position would move the notehead without moving the sound;
- the computed `alteration`′ or `octave`′ does not fit its field.

An implementation **MUST NOT** saturate, clamp, guess a pitch-space structure, or partially apply a transposition in any of these cases.

REQUIREMENT 2.6

An implementation **MUST** fail closed when it cannot establish the pitch-space structure required to interpret a position. In particular, transposition **MUST** report a refusal when the enclosing CMN chromatic layer or nominal mapping is unavailable, and a computation claiming a 12-chromatic pitch class or absolute semitone **MUST** report unavailability unless it can establish a twelve-position chromatic layer. The `Cmn` position discriminant alone **MUST NOT** be treated as proof of either capability.

RATIONALE

The refusal cases are the four ways a transposition can silently lie. A non-CMN position has no nominal to move, so a chromatic shift would have to invent one. An unresolved pitch space supplies neither the octave size nor the nominal mapping, so arithmetic would have to guess both. An `AbsoluteHz` pitch declares its own frequency: it can

be respelled or it can be resounded, but a transposition that claims to do both does one and pretends to the other. And saturation is the worst of the four, because it is invisible — a transposition that clamps has produced a pitch nobody asked for, reported success, and destroyed the information needed to notice.
Refusing atomically, rather than skipping the offending target, follows from the same reasoning: a chord transposed except for one note is not a transposed chord, it is a different chord. Contrast the treatment of *tombstoned* targets, which are skipped rather than refused — a deleted pitch is not an untransposable pitch, it is a pitch the operation has nothing to say about.

2.4 Acoustic Realization

The acoustic layer specifies the sounding frequency. It does not carry a frequency directly; instead it references a tuning system that, given the pitch's scale position, resolves to a frequency.

```
pub struct AcousticPitch {
    /// The tuning system governing this pitch's frequency.
    /// May be inherited; see Section~\ref{sec:pitch:tuning-inherit}.
    pub tuning: TuningReference,

    /// How the tuning system resolves to a frequency.
    pub realization: AcousticRealization,
}

pub enum AcousticRealization {
    /// Resolve through the tuning system using the scale position alone.
    /// This is the default case for ordinary CMN.
    Implicit,

    /// An explicit offset in cents from what the tuning system would
    /// otherwise produce. For inflections, expressive retuning, or
    /// just-intonation adjustments within a tempered piece.
    CentsOffset(f64),

    /// An explicit absolute frequency in Hertz, overriding the tuning
    /// system. For spectral music or fixed-frequency electronic parts.
    AbsoluteHz(f64),
}
```

2.4.1 Cents as the Offset Unit

Cents (one cent equals one hundredth of an equal-tempered semitone) are the normative offset unit. `f64` provides far more precision than the ear can resolve in any context.

2.4.2 Tuning Reference and Inheritance

A `TuningReference` is either an explicit tuning-system identifier or an inheritance marker:

```rust
pub enum TuningReference {
    /// Inherit the tuning system from the enclosing scope.
    Inherit,

    /// Explicitly named tuning system.
    Explicit(TuningSystemId),
}
```

Resolution proceeds by walking outward: pitch, then voice, then staff, then region, then score. The score's tuning system **MUST** be explicit; `Inherit` at the score level is a malformed document. Tuning systems and their resolution rules are defined in detail in Chapter 4.

2.4.3 Reference Pitch

A pitch's `TuningReference` resolves, through the hierarchical walk, to a tuning system. The *reference pitch* (the position and frequency that anchor that tuning system's structure to absolute Hertz) is resolved separately through the same hierarchical walk, and is a property of the score's tuning context, not of the tuning system itself.

The pitch type does not own or imply a reference frequency. Two scores using the same tuning system (12-TET) but different references (A4 = 440 Hz versus A4 = 415 Hz) share their tuning structure entirely; only their reference pitch differs.

The full reference-pitch model, including the `ReferencePitch` type and the score tuning context that owns it, is specified in Chapter 4 (Section 4.6).

2.5 Spelling

Spelling determines what the performer sees on the page for a given pitch: the staff position, the accidental glyph (if any), and any ancillary notational marks (parenthesized accidentals, courtesy accidentals, microtonal accidentals).

Spelling **MUST NOT** be a field on the pitch type. Spellings are stored externally, indexed by pitch identifier, and managed by the spelling subsystem defined in this section.

RATIONALE

Spelling as a field on pitch causes destructive overwrites on respelling, prevents partial spelling (where only some pitches have user-chosen spellings and the rest are inferred), prevents multiple spellings per pitch (analytical layers), and prevents provenance tracking. Externalizing spelling resolves all four.

2.5.1 The Spelling Attachment

```rust
pub struct SpellingAttachment {
    /// What this attachment applies to: a single pitch or a scope.
    pub scope: SpellingScope,

    /// The directive: explicit spelling or a rule for inference.
    pub directive: SpellingDirective,
```

```
    /// Provenance of this attachment.
    pub source: SpellingSource,

    /// Tie-break priority among attachments. Higher wins. See
    /// Section~\ref{sec:pitch:precedence} for the configurable
    /// precedence model.
    pub priority: i32,

    /// Optional analytical layer. None means the engraved layer;
    /// Some(id) means an alternative analytical layer that does not
    /// affect the engraved score.
    pub layer: Option<AnalysisLayerId>,
}

pub enum SpellingScope {
    /// Applies to one specific pitch.
    Pitch(PitchId),

    /// Applies to all pitches matching the selector within a time range.
    Range {
        start: TimeAnchor,
        end: TimeAnchor,
        voices: VoiceSelector,
    },
}

pub enum SpellingDirective {
    /// An explicit spelling for a single pitch. Only valid with
    /// SpellingScope::Pitch.
    Explicit(PitchSpelling),

    /// A rule for inferring spellings within a scope.
    Rule(SpellingRule),
}

pub struct PitchSpelling {
    /// The nominal: the staff position. For CMN, this is one of A-G.
    /// For other grammars, an identifier into the grammar's nominal
    /// registry.
    pub nominal: SpellingNominal,

    /// Stack of accidentals applied to the nominal, in stacking
    /// order: innermost (closest to the notehead) first.
    /// Empty vector means no accidental glyph is drawn, which does
    /// not necessarily mean "natural"; see
    /// Section~\ref{sec:pitch:absent-accidental}.
    pub accidentals: Vec<AccidentalId>,

    /// The octave designation.
    pub octave: i8,

    /// Optional rendering hints: parenthesized, cautionary, editorial,
```

```
    /// small-print, etc. Non-normative for sounding pitch.
    pub render_hints: SpellingRenderHints,
}

pub enum SpellingNominal {
    /// CMN nominals: the fast path.
    Cmn(CmnNominal),

    /// Integer nominals for N-tone systems.
    Integer(i32),

    /// Registered nominals for grammar-specific systems.
    Registered(NominalRegistryId),
}
```

Accidental Stack Semantics

REQUIREMENT 2.7

The `accidentals` vector **MUST** be interpreted as follows:

- The stored order is normative for engraving: index 0 is the innermost glyph (closest to the notehead), with subsequent indices placed further outward.
- For built-in additive accidental systems, the pitch-space modification contributed by the stack is the sum of the modifications of its members, and is independent of stored order. Reordering the stack changes engraving appearance but **MUST NOT** change sounding pitch.
- Custom accidental systems **MAY** declare non-additive combination semantics only through their registered `AccidentalCombination` (Section 4.3). Otherwise, accidentals are treated as commutative under their pitch effect.
- Two accidentals with the same `AccidentalId` in one stack constitute a malformed spelling, unless the accidental's registered combination semantics explicitly permits repetition. The conventional double-sharp and double-flat are represented as single accidental definitions, not as repeated singles.
- An empty `accidentals` vector means no glyph is drawn; it is distinct from a vector containing only a natural sign (which explicitly cancels prior alterations and is drawn).

2.5.2 Spelling Sources

Every spelling attachment carries provenance:

```
pub enum SpellingSource {
    /// The user explicitly chose this spelling.
    UserChosen,

    /// Inferred by the spelling pre-pass from key signature and
    /// context. Lowest default precedence. The pre-pass does not
    /// store attachments (its output is a derived annotation);
```

```
    /// this variant ranks that output in the precedence order.
    Inferred,

    /// Imported from a foreign format.
    Imported { format: ForeignFormatId },

    /// Propagated from a transposition or other edit operation.
    Propagated { from: PitchId },

    /// An analytical spelling on a non-engraved layer.
    Analytical,
}
```

REQUIREMENT 2.8

Editing operations that respell a pitch **MUST** produce attachments with source `UserChosen`. Editing operations that move or transpose a pitch **MUST** produce attachments with source `Propagated`. Foreign-format importers **MUST** produce attachments with source `Imported`. The spelling pre-pass (Section 2.5.4) **MUST NOT** mint attachments: its inferred spellings are derived annotations reported with `Inferred` provenance, promoted to a stored `UserChosen` attachment only by an explicit editing operation.

2.5.3 Configurable Precedence

When multiple attachments target the same pitch on the same analysis layer, the renderer resolves the conflict by precedence. The default precedence is:

1. `UserChosen` (highest)
2. `Imported`
3. `Propagated`
4. `Inferred` (lowest)

REQUIREMENT 2.9

Every score **MUST** carry a `SpellingPrecedence` configuration. The configuration **MUST** assign a total ordering over the variants of `SpellingSource`. The configuration **MAY** differ from the default; for example, a workflow that treats MusicXML imports as authoritative may rank `Imported` above `UserChosen`. Conflicts are broken first by precedence, then by the `priority` field of the attachment (higher wins), then by canonical attachment order (the attachment appearing earliest in the score's canonical serialization wins). Attachments carry no creation timestamp: a timestamp tie-break would hang resolution on non-canonical state.

2.5.4 The Spelling Pre-Pass

When a score is materialized (on load, after an edit, or ahead of rendering), the spelling pre-pass computes an inferred spelling for every spelling-eligible pitch. Its output is a *canonical*

derived annotation: a deterministic function of the materialized score graph, the active profile, and the versioned `SpellingAlgorithmId`, recomputed on materialization and never stored as graph state or serialized into canonical chunks. Where a stored attachment targets the pitch, the resolved annotation reports that attachment's spelling and provenance per the precedence configuration (Section 2.5.3); the pre-pass supplies the `Inferred`-provenance spelling that applies when no attachment outranks it. The pre-pass **MUST** be deterministic: given the same materialized graph, precedence configuration, and algorithm version, it **MUST** produce identical annotations.

The pre-pass operates as follows:

1. Collect all pitches in the score, partitioned by voice and measure.
2. For each partition, in time order, resolve each pitch's spelling using the applicable context inputs (which of these a given algorithm version consumes is part of its versioned definition; see Requirement 2.10):
   (a) The active key signature for the staff.
   (b) The accidental context of prior pitches in the same measure and voice.
   (c) The melodic context (preceding interval).
   (d) The harmonic context (concurrent pitches in other voices).
   (e) Any active scope-level `Rule` attachments.
3. Report an `Inferred`-provenance derived annotation for each pitch.

Because derived annotations are not stored, a change to the algorithm version deterministically invalidates them without any state migration. Implementations **MAY** cache derived annotations and **MAY** recompute incrementally (re-running only the affected measure and voice, and forward through measures whose context is altered, until the context stabilizes); a cache **MUST** be invalidated when the derivation key — graph, profile, algorithm version — changes, and neither caching nor incrementality may be observable in the annotations produced.

RATIONALE

Deriving inferred spellings—rather than storing them as attachments—keeps algorithm output out of canonical state: two replicas at the same (graph, profile, algorithm version) agree on annotations byte-for-byte without ever exchanging them, an algorithm upgrade cannot strand stale inferred state, and serialization never churns on re-inference. Inferred spellings remain inspectable in the UI and editable by promotion: accepting one mints a `UserChosen` attachment through an editing operation, which then outranks the pre-pass.

REQUIREMENT 2.10

Spelling algorithm disposition (ratified Pass 12). The spelling pre-pass receives the *profile-declared by versioned identifier* disposition of Appendix D Section D.11. The reserved identifier `SpellingAlgorithmId "default"` denotes, at version 1, a Temperley-style line-of-fifths preference algorithm: each partition's tonal context is estimated as a centre of gravity on the line of fifths from the accidental, melodic, and harmonic context, and each

pitch takes the enharmonic spelling nearest that centre. The ascending-runs-take- sharps / descending-runs-take-flats convention participates *only as a tiebreak* within the centre-of-gravity rule; an isolated chromatic run with no tonal context may therefore receive the enharmonic the convention alone would not pick, and a voice-leading refinement is a future algorithm version, not a deviation. Which of the context inputs enumerated above an algorithm version consumes is part of its versioned definition: `"default"` version 1 does *not* consult declared key signatures (a key-aware refinement is a future version) and is region-time-model-independent — pitches in aleatoric and proportional regions are spelled by the same rule as metric ones, and no region-specific spelling pass exists. A profile requesting any other identifier **MUST** error; implementations **MUST NOT** silently substitute a different algorithm.

REQUIREMENT 2.11

Authored annotations for inference-ineligible targets (ratified Pass 12). A stored spelling attachment whose target the pre-pass produces *no* inferred spelling for (an inference-ineligible pitch) **MUST** still surface in the resolved derived annotations: the winning attachment under the precedence configuration (Section 2.5.3) is reported with its authored provenance, exactly as it would have been had it outranked an inferred spelling. An authored attachment is precisely how a user notates what the algorithm cannot infer; the derived-annotation surface **MUST NOT** render it invisible. Annotation taxonomy counts these authored-only resolutions distinctly from authored-over-inferred overrides. The same rule applies to the decomposition pre-pass (Section 3.8).

2.5.5 Absent Accidentals

A `PitchSpelling` with an empty `accidentals` vector means no accidental glyph is drawn at this position. This is distinct from a vector containing an explicit natural sign. The distinction matters in two cases:

- A pitch on a line affected by the key signature carries no accidental and sounds the key-signature pitch; drawing a natural would be a respelling.
- A pitch that follows an in-measure accidental on the same line and octave inherits that accidental; an empty vector means inheritance applies, while a vector containing a natural cancels the inheritance.

REQUIREMENT 2.12

The renderer **MUST** distinguish between an empty `accidentals` vector (no glyph) and a vector containing only a natural (explicit natural glyph) when materializing a spelling. Promotion from empty to natural-only, or vice versa, is a respelling operation.

2.5.6 Analytical Layers

A pitch may carry multiple spelling attachments on different analytical layers. The engraved layer (`layer: None`) governs what the performer sees. Other layers (`layer: Some(AnalysisLayerId)`)

record alternative spellings used by analytical views, theory tools, or pedagogical overlays.

The `AnalysisLayer` type itself is defined in Chapter 5 (Section 5.10); a layer identifier here refers to that authoritative definition.

REQUIREMENT 2.13

Resolution of a pitch's spelling for a given view **MUST** consider only attachments whose `layer` matches the view's active layer. The engraved view's active layer is `None`. Analytical views may set an explicit layer; if no attachment on that layer matches, the view **MUST** fall back to the engraved layer.

2.6 Pitch Identifiers

Every pitch in the score graph carries a stable identifier:

```
pub struct PitchId(pub u128);
```

REQUIREMENT 2.14

Pitch identifiers **MUST** be stable across edits: a pitch's identifier is assigned at creation and never reassigned. Pitch identifiers **MUST** be unique within a score. Pitch identifiers **MUST** be collision-resistant across distributed editing scenarios; the recommended generation strategy is a 128-bit value combining a per-replica identifier and a monotonic counter, with the exact format defined in Chapter 6.

2.7 What Pitch Is Not

Several musical phenomena resemble pitches but are not represented by the pitch type:

Unpitched events. Snare drum hits, claves, cymbal strokes, and other unpitched percussion are sounding events with staff positions but no pitched identity. They are represented by a separate `UnpitchedEvent` type (see Chapter 5). They have no scale position and no acoustic pitch in the sense defined here.

Rests. A rest is a duration without a sounding event. Rests are represented by a separate `Rest` type. They carry timing and voice information but no pitch.

Indeterminate-pitch events. Events whose pitch is deliberately indeterminate (text instructions like "highest possible note," aleatoric pitch material, certain graphic-notation events) are represented by an `IndeterminatePitchEvent` type, which may carry hints (range, contour, density) but no concrete pitch.

Pitch trajectories. Glissandi, portamenti, and continuous pitch bends are not single pitches but trajectories between or around pitches. They are represented as relationships between pitched events plus a trajectory shape, defined in Chapter 5.

RATIONALE

Forcing every sounding event into a pitch-shaped hole creates invariant-breaking edge cases throughout the codebase. Sibling types under a common `SoundingEvent` (or similar) abstraction keep each type coherent and let operations branch cleanly.

2.8 Equality and Comparison

Two pitches are *structurally equal* if and only if their scale positions and acoustic realizations are equal field-by-field. Structural equality is the default `Eq` implementation.

Three additional equivalence relations are defined:

Sounding equivalence. Two pitches are sounding-equivalent if they resolve to the same frequency under their respective tuning systems. Computed, not derived from the type.

Scale-position equivalence. Two pitches are scale-position- equivalent if their `ScalePosition` fields are equal, ignoring acoustic realization.

Enharmonic equivalence. Two pitches are enharmonically equivalent if they are sounding-equivalent under 12-tone equal temperament, regardless of their actual tuning system. Used for foreign-format export and certain analytical operations.

REQUIREMENT 2.15

Implementations **MUST** provide functions for each of the three computed equivalences. Implementations **MUST NOT** conflate structural equality with sounding or enharmonic equivalence.

2.9 Forward References

- Tuning systems, pitch spaces, and accidental registries are defined in Chapter 4.
- The score graph, including how pitches are embedded in events, voices, and staves, is defined in Chapter 5.
- `TimeAnchor`, `VoiceSelector`, `PitchId` generation, and the reduction rules governing spelling attachments under concurrent edits are defined in Chapter 6.

Chapter 3

Time and Duration

This chapter specifies the time and duration primitives: how positions and durations are represented, how time signatures and tempo maps are modelled, how notational rhythm is decomposed from sounding duration, how tuplets and other groupings work, and how the core supports metric, proportional, and aleatoric time models simultaneously.

3.1 Design Principles

Exact rational arithmetic. Musical positions and durations **MUST** be exact rationals. Floating-point time is rejected as a source of accumulated error; integer-tick time is rejected as incompatible with arbitrary tuplet nesting. Exactness is non-negotiable.

Two clocks. Musical time (measured in whole-note rationals) and wall-clock time (measured in fixed-point seconds) are distinct primitives. Conversion between them is a pure function of the tempo map.

Position and duration are distinct types. A position and a duration share an underlying representation but are not interchangeable. The algebra (position + duration → position; position − position → duration; position + position is undefined) is enforced at the type level.

Sounding duration is data; notational rhythm is derived. A note's sounding duration is intrinsic; its notated decomposition (notehead values, ties, dots, tuplet membership) is produced by a deterministic pre-pass analogous to the spelling pre-pass and is overridable per-event by an authored attachment.

Multiple time models coexist. A score **MUST** support metric, proportional, and aleatoric regions, possibly concurrent on different staves. The region's declared time model determines the interpretation of positions within it.

Time anchors, not absolute positions. Stored references to points in time **MUST** anchor to identified objects (events, measures, regions) plus offsets. Absolute positions **MUST NOT** be stored where they could shift under edits.

3.2 The Rational Time Type

The fundamental time primitive is an exact rational number. The specification states the contract; implementations are free to choose representations meeting it.

REQUIREMENT 3.1

The rational time type **MUST** behave as an exact rational with at least 32-bit numerator and 32-bit denominator range in normalized form. Operations **MUST** produce exact results; on representation overflow, the implementation **MUST** either widen to an exact unbounded representation or return an error. Silent loss of precision is forbidden. Implementations **MAY** use any representation that meets this contract, including packed inline-or-heap-promoted designs.

3.2.1 Recommended Implementation: Inline-or-Promoted

The recommended implementation packs the common case (denominators under 2^{32}) into 8 bytes and promotes to a heap-allocated arbitrary-precision rational on overflow. This is normative for conformance only inasmuch as the observable behavior must match; it is illustrative as a reference design.

```rust
/// A musical time value: position or duration. Exact rational.
pub enum RationalTime {
    /// Inline case: fits in 8 bytes.
    Small(SmallRational),

    /// Promoted case: heap-allocated arbitrary-precision rational.
    /// Used only when arithmetic overflows the inline range.
    Large(Arc<BigRational>),
}

/// The inline rational: numerator i32, denominator NonZeroU32.
/// Always normalized: gcd(|numerator|, denominator) == 1.
#[derive(Copy, Clone, PartialEq, Eq, Hash)]
pub struct SmallRational {
    numerator: i32,
    denominator: NonZeroU32,
}
```

REQUIREMENT 3.2

All instances of the small rational **MUST** be normalized: the greatest common divisor of the absolute value of the numerator and the denominator **MUST** equal one, and the denominator **MUST** be strictly positive. The constructor **MUST** enforce normalization.

3.2.2 Promotion and Demotion

Arithmetic operations on two small rationals are performed in widened intermediate types ($i64$ numerator, $u64$ denominator) and the normalized result is examined. If the result fits in the small representation, it is returned as `Small`; otherwise it is returned as `Large`.

REQUIREMENT 3.3

Arithmetic that exceeds the small representation's range **MUST** silently promote to the large representation. The user of the type **MUST** observe no behavioral difference between small-only and small-or-large arithmetic; the contract is exact arithmetic on exact rationals.
Implementations **MAY** demote a `Large` value back to `Small` when its normalized representation fits, but **MUST NOT** make demotion observable (e.g., by changing equality or hash behavior across the variant boundary).

3.2.3 Equality and Ordering

Two rational time values are equal if and only if their normalized representations represent the same rational number. Equality between `Small` and `Large` variants **MUST** compare the underlying values, not the variants.

Ordering is the standard rational ordering. Comparison between rationals in the small representation is performed by cross-multiplication in widened intermediate types to avoid overflow.

3.2.4 Units

The unit of musical time is the *whole note*. A quarter note has duration $\frac{1}{4}$. A triplet eighth has duration $\frac{1}{12}$. A quintuplet sixteenth inside a duplet half (an unusual but legal construct) has duration $\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{5} = \frac{1}{20}$.

REQUIREMENT 3.4

The whole note **MUST** be the unit of musical time. The duration of a whole note is the rational $\frac{1}{1}$. All other durations are expressed as rationals relative to the whole note.

RATIONALE

The whole note is the only unit under which durations form a multiplicative algebra with 1 as identity. Quarter-note units would make "the duration of a whole note" equal to 4, which is awkward; PPQ-tick units would force a choice of resolution. The whole-note unit also matches the etymology of note-value names: a half note is half of a whole note.

3.3 Position and Duration as Distinct Types

The rational time representation is shared between two distinct types: `MusicalPosition` and `MusicalDuration`. Both wrap a `RationalTime`. Their distinction is enforced at the type level to prevent algebraic errors.

```rust
/// A point in musical time, relative to the origin of a time region.
#[derive(Clone, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct MusicalPosition(RationalTime);
```

```rust
/// A span of musical time.
#[derive(Clone, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct MusicalDuration(RationalTime);

impl Add<MusicalDuration> for MusicalPosition {
    type Output = MusicalPosition;
    // ...
}

impl Add<MusicalDuration> for MusicalDuration {
    type Output = MusicalDuration;
    // ...
}

impl Sub<MusicalPosition> for MusicalPosition {
    type Output = MusicalDuration;
    // ...
}

// MusicalPosition + MusicalPosition is intentionally not implemented.
```

REQUIREMENT 3.5

Implementations **MUST** provide distinct types for position and duration. The type system **MUST** prevent adding two positions. The difference of two positions **MUST** yield a duration; the sum of a position and a duration **MUST** yield a position.

RATIONALE

The illustrative Rust signatures derive `Clone`, not `Copy`, because `RationalTime` contains an `Arc<BigRational>` variant that cannot satisfy `Copy`. Implementations **MAY** provide `Copy` semantics through a compact inline-or-pointer-tagged representation whose `Large` case is a non-owning interior pointer rather than an `Arc`; such implementations satisfy the contract here. Conformance does not require `MusicalPosition` or `MusicalDuration` to be `Copy`.

3.4 Wall-Clock Time

Wall-clock time is the time measured by the playback engine, by SMPTE timecode, and by external audio and video sources. It is distinct from musical time and is represented as a fixed-point integer count of a sub-second unit.

```rust
/// A point in wall-clock time, in nanoseconds from a region origin.
/// 64-bit signed: range +/- ~292 years.
#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct WallClockTime(i64);  // nanoseconds

#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord)]
```

```rust
pub struct WallClockDuration(i64);  // nanoseconds
```

REQUIREMENT 3.6

Wall-clock time **MUST** be a fixed-point integer count of nanoseconds. Floating-point wall-clock time is forbidden in stored data. Implementations **MAY** convert to floating-point for transient computation provided no stored or serialized value loses precision.

RATIONALE

Nanosecond resolution provides over four orders of magnitude more precision than the human ear can resolve and aligns with the resolution conventions of professional audio interfaces and SMPTE-derived clocks. The 292-year range is sufficient for any conceivable musical work.

3.4.1 Wall-Clock and Musical Time Coexist

A score event may be anchored in either musical or wall-clock time. Musical-time anchors are the default for ordinary notation; wall-clock anchors are used for film hit points, audio reference markers, fixed electronic parts in scores with live ensemble, and similar sync-critical content. Both anchor types may coexist in the same score, possibly on the same staff.

Conversion between musical and wall-clock time is performed through the tempo map (Section 3.7).

3.5 Time Anchors

Stored references to points in time **MUST** be expressed as *time anchors*: references to identified objects plus offsets. Absolute positions **MUST NOT** be stored where they could be invalidated by edits to the score.

```rust
pub enum TimeAnchor {
    /// Anchored to a specific event. Survives edits that do not
    /// delete the event.
    Event { id: EventId, offset: AnchorOffset },

    /// Anchored to a measure boundary. Survives measure reordering
    /// (the anchor follows the measure).
    Measure {
        id: MeasureId,
        position: MeasurePosition,
        offset: AnchorOffset,
    },

    /// Anchored to the start or end of a region.
    Region { id: RegionId, edge: RegionEdge, offset: AnchorOffset },

    /// Anchored to absolute wall-clock time within the score.
    /// Used for film and audio sync.
```

```
    WallClock { time: WallClockTime },
}

/// An offset applied to an anchor target. The variant kind is
/// constrained by the target's enclosing region's time model.
pub enum AnchorOffset {
    /// Offset in musical time. Valid for targets in metric regions
    /// and for aleatoric regions whose anchoring discipline
    /// admits musical coordinates.
    Musical(MusicalDuration),

    /// Offset in wall-clock time. Valid for targets in proportional
    /// regions and for aleatoric regions whose anchoring discipline
    /// admits wall-clock coordinates.
    WallClock(WallClockDuration),

    /// No offset; the anchor refers to the target's reference point
    /// exactly. Valid in any region.
    Zero,
}

pub enum MeasurePosition { Start, End }
pub enum RegionEdge { Start, End }
```

REQUIREMENT 3.7

Stored *references to external time points* (cross-cutting endpoints, marker locations, attachment anchors, anything pointing at an object outside the referencing object's own voice and region) **MUST** use the `TimeAnchor` type. Absolute musical positions **MUST NOT** appear as such references in stored data: a reference to "measure 47, beat 3" that does not survive a measure insertion in measure 12 is a broken reference.

An event's own position within its owning voice and region **MAY** be stored as an `EventPosition` (carrying a `MusicalPosition` for events in metric regions, or a `WallClockTime` for events in proportional regions, etc.). Event-local positions are part of the event's intrinsic data, not references to other objects. Operations that shift a voice or region update the contained events' positions directly; they do not need anchor indirection because the containment relationship makes the dependency explicit.

Implementations **MAY** compute and cache absolute positions from anchors for performance; such caches **MUST** be invalidated when their underlying anchored objects change.

REQUIREMENT 3.8

The variant of an `AnchorOffset` **MUST** agree with the time model of the anchor target's enclosing region:

- For targets in metric regions: `AnchorOffset::Musical` or `AnchorOffset::Zero`.
- For targets in proportional regions: `AnchorOffset::WallClock` or `AnchorOffset::Zero`.

- For targets in aleatoric regions: any offset variant consistent with the region's `AleatoricAnchoringDiscipline` (Section 3.10.3).

Implementations **MUST** reject anchors whose offset variant contradicts the target region's time model.

3.5.1 Anchor Resolution

Resolving a time anchor to an absolute position is a pure function of the score graph: locate the anchored object, query its position within its region, and add the offset. Resolution **MUST** be deterministic.

3.5.2 Anchor Stability Under Edits

The chosen anchor type determines what edits the anchor survives:

- An `Event` anchor survives any edit that does not delete the anchored event. Edits that move the event (insertions before it, tempo changes) move the anchor with it.
- A `Measure` anchor survives insertions and deletions of other measures, and survives reordering. It is invalidated only by deletion of the anchored measure.
- A `Region` anchor survives edits within the region. It is invalidated only by deletion of the region.
- A `WallClock` anchor is unaffected by musical edits but may correspond to different musical positions after tempo changes.

REQUIREMENT 3.9

When an anchored object is deleted, anchors targeting it **MUST** be either re-anchored to a surviving object or marked as orphaned. The semantic operations chapter (Chapter 6) specifies re-anchoring rules per operation.

3.6 Time Signatures and Meter

A time signature defines the metric organization of a passage: where bar lines fall, where beats group, and what accent pattern is implied. Time signatures are objects in the score graph; they are not merely display numerator/denominator pairs.

```rust
pub struct TimeSignature {
    /// Display form. Determines what the performer sees.
    pub display: TimeSignatureDisplay,

    /// The total duration of one measure under this signature.
    pub measure_duration: MusicalDuration,

    /// Beat groupings within the measure. Drives beaming, accent
    /// inference, and the notational-decomposition pre-pass.
    pub beat_groups: Vec<BeatGroup>,
}
```

```rust
pub enum TimeSignatureDisplay {
    /// Standard: a numerator over a power-of-two denominator.
    Standard { numerator: u16, denominator: PowerOfTwo },

    /// Compound display: e.g., 3+2+3/8.
    Compound { numerators: Vec<u16>, denominator: PowerOfTwo },

    /// Irrational meter: denominator is not a power of two.
    /// e.g., 4/6, 5/12. The denominator names a non-power-of-two
    /// note value.
    Irrational { numerator: u16, denominator: NonZeroU16 },

    /// Mixed denominators: e.g., 1/4 + 1/8 + 1/16 displayed additively.
    MixedDenominators { components: Vec<(u16, NonZeroU16)> },

    /// No visible time signature (free meter or proportional).
    None,

    /// Custom symbol (cut time, common time, etc.) or
    /// grammar-specific.
    Symbolic(SymbolId),
}

pub struct BeatGroup {
    /// Duration of this beat group.
    pub duration: MusicalDuration,

    /// Subdivision hint for beaming and accent.
    pub subdivision: Option<MusicalDuration>,

    /// Accent strength relative to other beat groups in the measure.
    /// Higher values indicate stronger accents.
    pub accent: u8,
}
```

REQUIREMENT 3.10

The sum of the durations of a time signature's beat groups **MUST** equal the measure duration. Implementations **MUST** reject time signatures whose beat groups do not sum to the measure duration.

3.6.1 Irrational Meters

Time signatures whose denominator is not a power of two (e.g., 4/6, 5/12) are first-class. Their measure duration is computed as a rational: a 4/6 measure has duration $\frac{4}{6} = \frac{2}{3}$ whole notes. No special case in the time model; only the display layer treats the unusual denominator.

3.6.2 Polymetric and Polytemporal Scores

A score **MAY** have different time signatures concurrent on different staves. Each staff carries its own meter sequence; bar lines align where the rational positions coincide, and may misalign otherwise. The score graph chapter (Chapter 5) defines how concurrent meters are represented; this chapter establishes only that the time model supports them.

3.7 Tempo and the Tempo Map

The tempo map is the function mapping musical positions to wall-clock positions. It is a piecewise definition over musical time:

```
pub struct TempoMap {
    /// Piecewise segments, in non-overlapping musical-time order.
    pub segments: Vec<TempoSegment>,

    /// Tempo before the first segment, if any. None means the first
    /// segment's start_tempo applies from time zero.
    pub initial: Option<Tempo>,
}

pub struct TempoSegment {
    /// Start of this segment, as a TimeAnchor.
    pub start: TimeAnchor,

    /// End of this segment. None means "valid until the next segment
    /// in segments order, or until the end of the score if this is
    /// the final segment." Implementations resolve None lazily.
    pub end: Option<TimeAnchor>,

    /// Tempo at the start of this segment.
    pub start_tempo: Tempo,

    /// Tempo at the end of this segment.
    /// Required if shape != Constant; ignored (and SHOULD be None)
    /// if shape == Constant. A Constant segment holds start_tempo
    /// throughout.
    pub end_tempo: Option<Tempo>,

    /// Shape of the tempo change over this segment.
    pub shape: TempoShape,
}

pub enum TempoShape {
    /// Constant tempo throughout: end_tempo MUST be None or equal to
    /// start_tempo.
    Constant,

    /// Linear interpolation from start_tempo to end_tempo.
    Linear,

    /// Exponential (continuous-rate) interpolation from start_tempo
```

```
    /// to end_tempo.
    Exponential,

    /// Arbitrary curve, parameterized by control points. The control
    /// points define intermediate tempos between start_tempo and
    /// end_tempo.
    Curve(TempoCurve),
}

pub struct Tempo {
    /// Beats per minute. The beat unit is part of the tempo
    /// specification: a quarter-note BPM and a dotted-quarter BPM
    /// at the same numeric value differ in actual rate.
    pub bpm: f64,

    /// Note value taken as one beat.
    pub beat_unit: MusicalDuration,
}
```

REQUIREMENT 3.11

The tempo map's segments **MUST** be non-overlapping when resolved to absolute musical positions, and **MUST** appear in monotonically-increasing start order. Two adjacent segments where the earlier segment's resolved end equals the later segment's resolved start form a continuous tempo curve; the later segment's `start_tempo` **SHOULD** equal the earlier segment's resolved end tempo when continuity is intended.
A segment with `end == None` extends until the next segment's resolved start or, if no later segment exists, until the end of the score. Implementations **MUST** resolve open-ended segments deterministically.
At musical positions not covered by any segment (gaps between segments, or before the first segment when `initial` is `None`), the tempo **MUST** be the most recent applicable segment's terminating tempo, or the next applicable segment's `start_tempo` if no earlier segment exists.

REQUIREMENT 3.12

A tempo segment's `shape` and its `end_tempo` **MUST** be compatible. If `shape` is `Constant` and `end_tempo` is present, it **MUST** equal `start_tempo`. If `shape` is `Linear`, `Exponential` or `Curve`, `end_tempo` **MUST** be present.
This requirement states the compatibility that is enforced. It does not determine whether a constant segment records an `end_tempo` at all.

3.7.1 Conversion

Conversion between musical and wall-clock time integrates the tempo map. For constant segments, conversion is multiplication. For linear and exponential segments, conversion has a closed-form solution. For curve segments, conversion is numerical.

REQUIREMENT 3.13

For a `TempoShape::Linear` segment, the quantity that varies linearly across the segment is *speed* — whole notes per second — as a function of fractional position within the segment, *not* beats-per-minute and *not* beat period. Speed is derived from a `Tempo` as speed $=$ bpm $\times\ w(\text{beat_unit})/60$, where $w(\cdot)$ is the beat unit's value in whole notes. Writing s_0, s_1 for the segment's start and end speeds and $u \in [0, 1]$ for fractional position, $s(u) = s_0 + (s_1 - s_0)\,u$, and the wall-clock duration of the segment of musical length ℓ is the closed form $\int_0^1 \ell\,\mathrm{d}u/s(u) = (\ell/(s_1 - s_0))\ \ln(s_1/s_0)$ (and ℓ/s_0 when $s_1 = s_0$).
A `TempoShape::Exponential` segment interpolates speed *geometrically*: $s(u) = s_0\,(s_1/s_0)^u$, i.e. a constant continuous rate of change of speed.

RATIONALE

"Linear interpolation from `start_tempo` to `end_tempo`" is ambiguous about which quantity is linear: bpm, period, or speed. We pin *speed* because it is the only beat-unit-agnostic choice. A tempo map may change beat unit across segments (a `quarter = 120` segment abutting a `dotted-quarter = 80` segment); interpolating bpm would make the wall-clock schedule depend on the arbitrary beat unit chosen to notate each endpoint, whereas interpolating speed depends only on the sounding rate. When both endpoints share a beat unit, speed-linear and bpm-linear coincide, so the common case matches a conductor's "linear accelerando" intuition. Because this choice fixes the derived wall-clock schedule for every playback engine, it is stated normatively rather than left to the implementation.

REQUIREMENT 3.14

The tempo map **MUST** expose two conversion functions: `musical_to_wallclock` and `wallclock_to_musical`. Both **MUST** be deterministic: given identical tempo maps and inputs, they **MUST** produce identical outputs across runs and platforms. Numerical integration and inversion, where used, **MUST** use a deterministic algorithm with documented tolerance and iteration bounds.
Tempo curves **MUST** be monotonically increasing in musical time (i.e., tempo may vary but musical time always advances) so that `wallclock_to_musical` is single-valued. Implementations **MUST** reject tempo segments whose shape produces a non-monotonic mapping (such curves are musically meaningless).

OPEN QUESTION

The specific numerical-integration algorithm for `TempoShape::Curve` (Romberg integration, adaptive Gauss-Kronrod, or a normative method) and the specific root-finding algorithm for `wallclock_to_musical` are unresolved. Both algorithms affect canonical state and must therefore receive one of the dispositions in Appendix D Section D.11: normatively specified in this document or a named companion, profile-declared by versioned identifier, or explicitly marked non-canonical. Until that disposition is made,

cross-implementation byte equality of derived canonical state (notably tempo-driven audio scheduling) **MUST NOT** be claimed.

RATIONALE

Determinism of tempo conversion is necessary for reproducible audio output, click-track stability, and SMPTE sync. Non-determinism here would produce playback drift that compounds over long pieces.

3.8 Sounding Duration and Notational Decomposition

An event has a single *sounding duration* (an exact rational). Its *notational decomposition* is the sequence of notehead values, augmentation dots, and ties used to draw that duration on the staff.

The decomposition is computed from the sounding duration by a deterministic pre-pass as a *canonical derived annotation* — recomputed on materialization, never stored graph state — under the same output model as the spelling pre-pass (Section 2.5.4). The user may accept the inferred decomposition or override it by authoring a decomposition attachment, which takes precedence.

3.8.1 The Notational Decomposition

```rust
pub struct NotationalDecomposition {
    /// Sequence of notated components. Their durations sum to the
    /// event's sounding duration.
    pub components: Vec<NotatedComponent>,

    /// Provenance, analogous to SpellingSource.
    pub source: DecompositionSource,
}

pub struct NotatedComponent {
    /// Base note value: whole, half, quarter, eighth, etc.
    pub base_value: NoteValue,

    /// Number of augmentation dots (0..=4 typical).
    pub dots: u8,

    /// Tuplet membership, if any.
    pub tuplet: Option<TupletId>,

    /// Whether this component is tied to the next.
    pub tied_to_next: bool,
}

pub enum DecompositionSource {
    UserChosen,
    Inferred,
```

```
    Imported { format: ForeignFormatId },
    Propagated { from: EventId },
}
```

The structure deliberately mirrors the spelling attachment model (Section 2.5): same sources, same source-rank discipline, same pre-pass output model. Unlike spelling, decomposition precedence is *not configurable* (ratified Pass 12): the fixed default source order `UserChosen` > `Imported` > `Propagated` > `Inferred` applies, with the score's canonical attachment order as tie-break among equal-rank authored attachments; `DecompositionAttachment` carries no `priority` field and the graph carries no `DecompositionPrecedence` configuration. Making precedence configurable would add a canonical `Score` field (a schema-major change under the Binary Format companion's evolution rule) for which no consumer exists; the decision may be revisited at a future schema major if a use case appears.

3.8.2 The Decomposition Pre-Pass

When a score is materialized, the decomposition pre-pass computes inferred decompositions, reported as derived annotations; an authored higher-precedence attachment overrides the inferred value in the resolved annotation. An authored attachment targeting an event the pre-pass produces *no* inferred decomposition for (ungriddable, non-metric, or an inapplicable event kind) surfaces as the resolved annotation on its own, per Requirement 2.11 — authoring is how a user notates exactly what the algorithm cannot infer. The algorithm is, in outline:

1. For each event in time order within a voice and measure, examine the sounding duration and the current beat-group context.
2. Decompose the duration into the longest leading note value that fits within the current beat group without crossing a stronger accent boundary.
3. If the duration is not exhausted, produce a tie and continue decomposition from the next beat-group boundary.
4. Augmentation dots are applied where they shorten the decomposition without crossing an accent boundary that the dot would obscure.
5. Tuplet membership is recorded for events that fall within a tuplet group (Section 3.9).

REQUIREMENT 3.15

The decomposition pre-pass **MUST** be deterministic: given identical materialized graph, configuration, and algorithm version, it **MUST** produce identical decompositions. Its output is a derived annotation, never stored graph state. Implementations **MAY** cache and **MAY** recompute incrementally (re-running only the affected voice and measure, with propagation only as far as the context is altered); neither may be observable in the annotations produced.

REQUIREMENT 3.16

Decomposition algorithm disposition (ratified Pass 12). The decomposition pre-pass receives the *profile-declared by versioned identifier* disposition of Appendix D Section D.11.

The reserved identifier `DecompositionAlgorithmId"default"` denotes, at version 2, the integer-grid metric splitter, whose scope bounds are part of its normative definition:

- barlines are each staff instance's measures: a measure starts where its region-relative anchor puts it and lasts until the next, the last as long as its governing time signature's bar (the measure's own signature, else the latest before it); where a staff instance's measures do not resolve so, barlines fall at the region's first governing meter, as at version 1;
- a first measure shorter than its signature's bar is a pickup, and its events keep their place in that bar (it begins as far in as the beats it lacks);
- within a bar, a span that is one notated value (with at most two augmentation dots, `MAX_DOTS` = 2) is written as that value; any other span is split at the strongest metric boundary it crosses, recursively, and a span crossing a barline is split there with ties;
- metric strength within a bar is dyadic from its start, and compound-meter beat grouping uses the dyadic default;
- tuplet nesting and cross-beat tuplet members are out of scope.

Version 1 honoured only the first governing meter of a region, assumed the region origin on a barline, wrote at most one dot, and split a span at any interior boundary at least as strong as its start even when one value expressed it (a half note on the second beat of a $\frac{4}{4}$ bar was two tied quarters). A wider algorithm is a *version bump*, which deterministically invalidates derived output under the derived-annotation model — no state migration. Style-choice selection among multiple valid decompositions (e.g., $\frac{3}{8}$ in $\frac{4}{4}$ as a dotted quarter versus a quarter tied to an eighth) is fixed by the versioned algorithm; user-configurable alternate rule sets, if ever wanted, are new registered identifiers. A profile requesting any identifier other than a registered one **MUST** error; implementations **MUST NOT** silently substitute.

3.9 Tuplets as Grouping Objects

A tuplet is a grouping object in the score graph that records notational intent: that a span of events forms a triplet, quintuplet, or other irregular grouping with a visible bracket and number. Tuplets do not modify the sounding durations of their members; sounding durations are always concrete rationals.

```rust
pub struct Tuplet {
    pub id: TupletId,

    /// The actual:notated ratio. A triplet is 3:2; a quintuplet is
    /// 5:4; an irregular grouping like 7:4 has ratio 7 over 4.
    pub ratio: TupletRatio,

    /// Range of events in this tuplet, by event ID.
    pub members: Vec<EventId>,

    /// Nesting: if this tuplet is itself inside another, the parent.
    pub parent: Option<TupletId>,
```

```
    /// The total sounding duration the members must sum to
    /// (Tuplet Consistency, below): a 3:2 eighth-note triplet's 1/4.
    pub required_total: MusicalDuration,

    /// Display: whether its number and bracket are shown (defined
    /// below; schema major 4).
    pub display: TupletDisplay,
}

/// How a tuplet's marks are shown. Growth of either vocabulary is by
/// appended variant.
pub struct TupletDisplay {
    pub number: TupletNumber,
    pub bracket: TupletBracket,
}

pub enum TupletNumber {
    /// The ratio's 'actual' term: a triplet's 3. The default.
    Actual,
    /// No number.
    None,
}

pub enum TupletBracket {
    /// The engraver's choice: conventionally a bracket unless the
    /// members are one beamed group. The default.
    Auto,
    /// No bracket.
    Hidden,
}

pub struct TupletRatio {
    // Fields are private: a TupletRatio is constructed only through the
    // checked 'new', so a degenerate ratio is never representable (see
    // the requirement below). The fields are read via 'actual()'/'notated()'.
    actual: u32,
    notated: u32,
}

impl TupletRatio {
    /// Returns 'None' for a degenerate ratio (either term zero, or
    /// 'actual == notated'); otherwise the well-formed ratio.
    pub fn new(actual: u32, notated: u32) -> Option<TupletRatio> { /* checked */ }
    pub fn actual(&self) -> u32 { /* ... */ }
    pub fn notated(&self) -> u32 { /* ... */ }
}
```

REQUIREMENT 3.17

A `TupletRatio` is *degenerate* if either term is zero or if `actual == notated` (a ratio that expresses no augmentation or diminution). Degenerate ratios **MUST** be rejected at construction: a conforming constructor of `TupletRatio` (and therefore of any `Tuplet`) **MUST** fail rather than produce a value with `actual == 0`, `notated == 0`, or `actual == notated`. This is a construction-time rejection, not a runtime graph invariant (Section 5.11): a degenerate ratio is never a representable graph state, so no invariant restores it after the fact.

REQUIREMENT 3.18

A tuplet's `display` is the author's record of how its marks are shown, and so is canonical state, though it is display-oriented. A conforming engraver **MUST NOT** draw the number of a tuplet whose `display.number` is `None`, nor the bracket of one whose `display.bracket` is `Hidden`; with `Actual` it draws the ratio's `actual` term, and with `Auto` the bracket is its own choice. A tuplet whose number and bracket are both hidden is still a tuplet: its members, ratio and consistency (Requirement 3.19) are unaffected. The display enters no reduction verdict and no canonical reduced state. A tuplet read from a schema major before 4, which carried none, takes the default, `Actual` and `Auto`.

3.9.1 Tuplet Nesting

Tuplets **MAY** nest to arbitrary depth. The sounding duration of an event inside nested tuplets is computed directly from its rational duration, not from the tuplet ratios; the ratios are notational and analytical.

RATIONALE

Storing sounding durations directly (rather than as base-value times a stack of tuplet ratios) keeps the time model regular: every event has a single rational duration, full stop. Moving a tuplet, splitting it, or deleting it never alters the durations of its members. Tuplets thus become a pure annotation layer, which dramatically simplifies edit operations.

3.9.2 Tuplet Consistency

REQUIREMENT 3.19

A tuplet's notated ratio **MUST** be consistent with the sounding durations of its members: the sum of member durations **MUST** equal the duration that the tuplet's notated value indicates when scaled by the ratio. For example, a 3:2 eighth-note triplet of three members has total sounding duration $\frac{1}{4}$; the sum of the three member durations **MUST** equal $\frac{1}{4}$.

> This is a structural invariant. Edit operations that would violate it **MUST** either adjust member durations or be rejected. The semantic operations chapter specifies which.

3.10 Region Time Models

A region of the score declares one of three time models. The declared model determines how positions within the region are interpreted, displayed, and played back.

```
pub enum RegionTimeModel {
    /// Metric time: measures, beats, exact rational positions.
    /// The dominant model for Western notated music.
    Metric(MetricTimeModel),

    /// Proportional time: events placed in space, where horizontal
    /// position is wall-clock time. No measures, no beats.
    Proportional(ProportionalTimeModel),

    /// Aleatoric time: events with partial or unspecified ordering.
    Aleatoric(AleatoricTimeModel),
}
```

3.10.1 Metric Time

```
pub struct MetricTimeModel {
    /// Sequence of meter changes within this region.
    pub meters: Vec<MeterChange>,

    /// Tempo map for this region. May inherit from the score.
    pub tempo: TempoMapReference,
}
```

Metric regions have measures, beats, and time signatures as described in Sections 3.6 and 3.7. Positions within them are `MusicalPosition` values.

3.10.2 Proportional Time

```
pub struct ProportionalTimeModel {
    /// Total wall-clock duration of the region.
    pub duration: WallClockDuration,

    /// Horizontal space per second. Drives the engraver's spacing.
    /// Stored as a layout hint; the actual spacing is decided by the
    /// constraint solver.
    pub space_per_second: SpaceUnit,

    /// Optional reference grid markings (vertical lines every N
    /// seconds, for example) for performer reference.
    pub grid: Option<ProportionalGrid>,
}
```

Events in proportional regions carry `WallClockTime` positions directly. There are no measures, no beats, no rational durations. Notational duration is computed from wall-clock duration by the notational decomposition pre-pass using a proportional-mode rule set, which generally produces unbeamed events with explicit duration notations or graphic-duration symbols.

Requirement 3.20

Events in a proportional region **MUST** use `WallClockTime` for positions and `WallClockDuration` for durations. The tempo map **MUST NOT** be applied to convert proportional-region positions; they are already in wall-clock time.

3.10.3 Aleatoric Time

```rust
pub struct AleatoricTimeModel {
    /// Ordering constraints among events: a directed acyclic graph
    /// where an edge a -> b means "a precedes b."
    pub ordering: EventOrderingDAG,

    /// Anchoring discipline: declares what event coordinate kinds
    /// this region permits and whether duration-bound kinds may
    /// be mixed. See Section~\ref{sec:time:aleatoric-anchoring}.
    pub anchoring: AleatoricAnchoringDiscipline,

    /// Optional per-event interval bounds. An event with start
    /// bounds [t1, t2] may begin at any time in that window.
    /// Bound kinds are constrained by the anchoring discipline.
    pub bounds: HashMap<EventId, EventBounds>,

    /// Approximate or maximum total duration, for layout.
    pub duration_hint: WallClockDuration,
}

pub enum AleatoricAnchoringDiscipline {
    /// Events in this region carry musical-time coordinates only.
    Musical,

    /// Events in this region carry wall-clock coordinates only.
    WallClock,

    /// Events may carry either kind, but each event's position and
    /// duration kinds must agree with each other. Mixed-kind
    /// duration bounds are not permitted.
    EitherPerEvent,

    /// Events may carry either kind freely, and duration bounds
    /// may mix kinds (one bound musical, the other wall-clock).
    /// Used for free-time scores synchronized to external media.
    FreelyMixed,
}

pub struct EventBounds {
```

```
    pub start: Option<TimeBounds>,
    pub end: Option<TimeBounds>,
}

pub enum TimeBounds {
    MusicalRange { min: MusicalPosition, max: MusicalPosition },
    WallClockRange { min: WallClockTime, max: WallClockTime },
    Unbounded,
}
```

Aleatoric regions support both unordered event collections (any permutation of events is valid) and time-bracket notation (events with specified start and end windows, à la Cage's later notation). The DAG encoding admits both as cases.

Anchoring Discipline

REQUIREMENT 3.21

Every aleatoric region **MUST** declare an `AleatoricAnchoringDiscipline`. Events within the region **MUST** carry coordinate kinds consistent with the discipline:

- `Musical`: events use `EventPosition::Musical` and `EventDuration::Musical` (or `EventDuration::Indeterminate` whose bounds are musical).
- `WallClock`: events use `EventPosition::WallClock` and `EventDuration::WallClock` (or `EventDuration::Indeterminate` whose bounds are wall-clock).
- `EitherPerEvent`: events choose per-event, but an event's position and duration kinds **MUST** agree, and `DurationBounds` **MUST** use a single `ConcreteDuration` variant.
- `FreelyMixed`: no agreement required; `DurationBounds` **MAY** mix kinds.

Implementations **MUST** reject events whose coordinate kinds violate the region's declared discipline.

REQUIREMENT 3.22

The ordering DAG **MUST** be acyclic. Cycles **MUST** be rejected at construction. Two events with no path between them in the DAG are unordered.

REQUIREMENT 3.23

Every event referenced by an aleatoric region's `ordering` DAG, and every event used as a key in its `bounds` map, **MUST** be an event of that same region. Naming an event that does not exist is a dangling reference, governed by graph invariant 10; naming an event that exists in a *different* region is a distinct defect, and this requirement is what forbids it. Neither the ordering DAG nor the bounds map may reach outside the region whose time model declares them.

Rationale

A DAG with optional interval bounds expresses every aleatoric form the project intends to support: unordered collections (empty DAG), partial orderings (sparse DAG), strict orderings (chain DAG), and time-brackets (DAG plus bounds). Higher-level constructs (mobile-form notation, statistical scores) can be built atop this primitive in later specifications without altering the core.

3.10.4 Concurrent Time Models

Different staves in the same score **MAY** declare different time models. A metric solo line over a proportional aleatoric texture, for example, is expressible: each staff's region declares its own model. Synchronization between concurrent time models is by wall-clock time or by explicit cue points (see Chapter 5).

3.11 The Notion of "Now"

The core does not define playback state; it does not have a notion of "current position" or "transport time." Such state belongs to the audio engine, which consumes the core's time model. The core defines only the static time structure of the score.

3.12 Forward References

- Event identifiers (`EventId`), measure identifiers (`MeasureId`), and region identifiers (`RegionId`) are defined in Chapter 5.
- The full event taxonomy (pitched, unpitched, rest, indeterminate, trajectory) is defined in Chapter 5.
- Re-anchoring rules for time anchors when their targets are deleted are defined per-operation in Chapter 6.
- `SpaceUnit` is defined normatively in Chapter 5 as of schema major 2 (it entered canonical state through `StaffLineConfiguration`). `TupletDisplay` is defined above: the author's choice to hide a tuplet's marks is part of the score (schema major 4). Other purely display-oriented types are defined in Chapter 7.

Chapter 4

Tuning Systems and Pitch Spaces

This chapter specifies the three intertwined registries that complete the primitives layer: *pitch spaces* (the analytical universes in which scale-position arithmetic lives), *accidental registries* (the catalogs of position-modifying glyphs), and *tuning systems* (the frequency-resolution mechanisms). It also specifies the hierarchical resolution rules that determine which tuning applies to each pitch in a score.

4.1 Design Principles

Pitch space and tuning system are distinct. A pitch space defines what pitches exist and how they relate analytically. A tuning system defines what frequencies those pitches produce. The two are independent: a single pitch space (e.g., CMN) admits many tuning systems (12-TET, meantone, well-temperaments, just intonation, Pythagorean); a single tuning structure may be anchored at different reference frequencies.

Accidentals belong to pitch spaces. An accidental defines a *position modification* within a pitch space. The tuning system handles the frequency consequences of that modification. The chromatic semitone implied by a sharp has different cent-values in 12-TET, quarter-comma meantone, and Pythagorean tuning, but it is the same accidental denoting the same position.

Reference pitch is score-level, not tuning-level. A score declares a reference pitch (position plus frequency); the tuning system defines structure relative to that reference. "12-TET at A=440" and "12-TET at A=415" share the same tuning system; the reference differs.

Resolution is hierarchical. Tuning, pitch space, and accidental registry are resolved by walking outward from the pitch: pitch, voice, staff, region, score. Inheritance markers permit any level to defer to the enclosing scope.

Built-in defaults, extensible by users. The core ships a baseline catalog of pitch spaces, tuning systems, and accidental registries sufficient for the common 95% case. Scores and grammar plugins extend these without forking.

SMuFL primary, custom fallback. Glyph references default to SMuFL codepoints. Custom glyphs are supported as fallback. SMuFL version is recorded per score.

4.2 Pitch Spaces

A pitch space is the analytical universe in which scale-degree and interval relationships are defined. It is the algebra; the tuning system is the physics.

```rust
pub struct PitchSpace {
    pub id: PitchSpaceId,

    /// Human-readable name (e.g., "CMN 12-tone chromatic").
    pub name: String,

    /// Optional description and provenance notes.
    pub description: Option<String>,

    /// What positions exist and how they are structured.
    pub positions: PositionStructure,

    /// How intervals between positions are defined.
    pub interval_algebra: IntervalAlgebra,

    /// The accidental registry valid in this pitch space.
    pub accidental_registry: AccidentalRegistryId,

    /// The nominal registry: named position-letters
    /// (e.g., A-G for CMN).
    pub nominal_registry: NominalRegistryId,

    /// Rules governing how pitches transpose within this space.
    pub transposition: TranspositionBehavior,

    /// Default rules for the spelling pre-pass when this pitch space
    /// is active.
    pub spelling_rules: SpellingRuleSet,
}
```

4.2.1 Position Structure

The `PositionStructure` declares the shape of the pitch space. Three families cover the vast majority of useful cases; a fourth is the escape hatch for grammars defined entirely by plugin.

```rust
pub enum PositionStructure {
    /// Chromatic: a fixed number of equally-numbered positions per
    /// octave, with no hierarchical diatonic substructure.
    /// Includes EDOs and serial 12-tone.
    Chromatic { positions_per_octave: u16 },

    /// Diatonic over chromatic: a smaller set of named nominals
    /// (e.g., A-G for CMN) plus accidental modifications reaching
    /// the full chromatic range. CMN's structure.
    DiatonicOverChromatic {
        nominals_per_octave: u16,
        chromatic_positions_per_octave: u16,
```

```
        /// Mapping from each nominal to its position in the
        /// chromatic layer (e.g., C=0, D=2, E=4, F=5, G=7, A=9, B=11
        /// for CMN).
        nominal_to_chromatic: Vec<u16>,
    },

    /// Just intonation lattice: positions defined by exact rational
    /// ratios from a tonic, organized along prime-axis dimensions
    /// (3-limit, 5-limit, 7-limit, etc.).
    JiLattice {
        limit: u8,                      // prime limit
        generators: Vec<JiRatio>,       // one per prime dimension
    },

    /// Grammar-defined: structure is opaque to the core and resolved
    /// by a grammar plugin. Used for maqam, gamelan, raga frameworks,
    /// and other systems whose position structure is not flat or
    /// hierarchical in the above senses.
    Registered(PositionStructureRegistryId),
}
```

REQUIREMENT 4.1

The `DiatonicOverChromatic` variant **MUST** have a `nominal_to_chromatic` mapping of length equal to `nominals_per_octave`, with each entry strictly less than `chromatic_positions_per_octave`. The mapping **MUST** be strictly increasing.

4.2.2 Interval Algebra

An interval algebra defines how distances between positions are computed and how transposition is realized. For chromatic spaces the algebra is integer arithmetic modulo the position count; for diatonic-over-chromatic the algebra distinguishes diatonic from chromatic intervals (a diatonic third may correspond to three or four chromatic semitones depending on quality).

```
pub enum IntervalAlgebra {
    /// Single-axis integer arithmetic. Two positions are subtracted
    /// to yield a single integer interval.
    Chromatic,

    /// Two-axis arithmetic distinguishing diatonic and chromatic
    /// components. CMN's algebra: a major third is (3 diatonic steps,
    /// 4 chromatic steps); a diminished fourth is (3 diatonic steps,
    /// 4 chromatic steps) -- distinguishable from the major third only
    /// by spelling intent.
    DiatonicChromatic,

    /// Multi-axis JI lattice arithmetic.
    JiVector { dimensions: u8 },

    /// Grammar-defined algebra.
    Registered(IntervalAlgebraRegistryId),
```

```
}
```

4.2.3 Nominal Registry

A nominal registry catalogs the named position-letters of a pitch space. For CMN this is the seven-element set $\{C, D, E, F, G, A, B\}$. For 12-tone serial music the nominal registry may simply be the integers 0 through 11. For maqam the nominals are the names of maqam degrees.

```
pub struct NominalRegistry {
    pub id: NominalRegistryId,
    pub nominals: Vec<NominalDefinition>,
}

pub struct NominalDefinition {
    pub id: NominalId,

    /// Canonical name.
    pub name: String,

    /// Display glyph, typically a letter or symbol.
    pub display: GlyphReference,

    /// Default position in the pitch space's chromatic layer, if
    /// applicable.
    pub default_position: Option<u16>,
}
```

4.2.4 Transposition Behavior

Transposition behavior governs how scale positions move under interval operations and how spellings follow.

```
pub enum TranspositionBehavior {
    /// Diatonic transposition: shift by N diatonic steps within a
    /// chosen key. Accidentals adjust to fit the destination key.
    Diatonic,

    /// Chromatic transposition: shift by N chromatic steps. Spellings
    /// follow rules of the SpellingRuleSet.
    Chromatic,

    /// Compound: support both, parameterized at the operation site.
    Compound,

    /// Grammar-defined.
    Registered(TranspositionRegistryId),
}
```

4.2.5 Spelling Rule Sets

A pitch space carries a default spelling rule set that the spelling pre-pass (Section 2.5.4) consults when inferring spellings for unspelled pitches.

```
pub struct SpellingRuleSet {
    pub id: SpellingRuleSetId,
    pub name: String,

    /// Algorithmic family this rule set belongs to. The specific
    /// algorithm is referenced by id and resolved against the
    /// implementation's registered spelling algorithms.
    pub algorithm: SpellingAlgorithmId,

    /// Algorithm-specific parameters.
    pub parameters: SpellingParameters,
}
```

OPEN QUESTION

The default spelling algorithm is now ratified (Requirement 2.10: `"default"` = Temperley-style line-of-fifths preference, version 1). Still open: the catalog of *additional* registered spelling algorithms (candidates include Longuet-Higgins line-of-fifths distance, Cambouropoulos pitch-spelling, and Meredith PS13) and their parameter schemas, which are normative once registered.

4.3 Accidental Registries

An accidental registry catalogs the position-modifying glyphs valid within a pitch space. Each accidental defines a glyph, a position modification, and engraving metadata.

```
pub struct AccidentalRegistry {
    pub id: AccidentalRegistryId,
    pub name: String,
    pub accidentals: Vec<AccidentalDefinition>,

    /// Whether scores may extend this registry with score-local
    /// accidentals. Most registries permit this; some (e.g., a
    /// closed conformance profile) may not.
    pub extensible: bool,
}

pub struct AccidentalDefinition {
    pub id: AccidentalId,

    /// Canonical name. Used for serialization, accessibility, and
    /// foreign-format export.
    pub name: String,

    /// Glyph reference, typically SMuFL.
```

```
    pub glyph: GlyphReference,

    /// The position modification this accidental applies.
    pub modification: PitchSpaceModification,

    /// Engraving metadata.
    pub engraving: AccidentalEngraving,

    /// Combination behavior with other accidentals.
    pub combination: AccidentalCombination,
}
```

4.3.1 Pitch Space Modifications

```
pub enum PitchSpaceModification {
    /// CMN-style integer offset in chromatic steps of the enclosing
    /// pitch space ('req:pitch:alteration-unit').
    CmnChromatic(i8),

    /// Integer step offset in an EDO position space.
    EdoSteps(i16),

    /// Modification expressed as an exact rational ratio. Used for
    /// JI accidentals such as the HEJI syntonic-comma symbols
    /// (81/80) and septimal-comma symbols (64/63).
    JiRatio { numerator: i32, denominator: NonZeroU32 },

    /// Modification expressed in cents. Used for Sagittal-style
    /// precise microtonal accidentals.
    Cents(f64),

    /// Modification defined by a grammar plugin.
    Registered(ModificationRegistryId),
}
```

REQUIREMENT 4.2

An accidental's modification **MUST** be expressible in the interval algebra of every pitch space that references its registry. A `CmnChromatic` modification is valid only in spaces with `DiatonicChromatic` or compatible algebra; an `EdoSteps` modification is valid only in `Chromatic` or `Registered` spaces with appropriate compatibility. Implementations **MUST** reject scores referencing an accidental in a space whose algebra does not admit the modification.

4.3.2 Accidental Engraving Metadata

`AccidentalEngraving` is reachable from canonical score state — via `AccidentalDefinition` inside `ScoreAccidentalExtensions`, which hangs off `ScoreTuningContext` — so every field it carries must itself be canonical. Its bounding box therefore cannot reuse Chapter 7's `BoundingBox`,

which is built on `StaffSpace` (single-precision `f32`) and belongs to the non-canonical resolved-layout cache. This chapter defines its own box type over `SpaceUnit` instead:

```rust
/// A bounding box over canonical space units, used for engraving
/// metadata that lives in canonical score state. Distinct from
/// Chapter 7's 'BoundingBox' (built on 'StaffSpace', single precision,
/// for the non-canonical resolved-layout cache): this type's edges are
/// 'SpaceUnit' ('CanonicalF64'), per
/// 'req:determinism:canonical-floating-point'.
pub struct EngravingBoundingBox {
    pub left: SpaceUnit,
    pub right: SpaceUnit,
    pub top: SpaceUnit,
    pub bottom: SpaceUnit,
}
```

```rust
pub struct AccidentalEngraving {
    /// Bounding box in canonical space units, relative to the glyph's
    /// anchor point.
    pub bounding_box: EngravingBoundingBox,

    /// Where the glyph attaches to the note: typically the geometric
    /// center or a custom anchor for compound glyphs.
    pub anchor: AnchorPoint,

    /// Advance width for horizontal spacing computations.
    pub advance_width: SpaceUnit,

    /// Stacking order when multiple accidentals attach to one note.
    /// Lower values are placed closer to the notehead.
    pub stacking_order: i32,

    /// Whether this glyph should be drawn with parentheses by default
    /// (e.g., editorial or cautionary accidentals).
    pub default_parenthesized: bool,
}
```

RATIONALE

Engraving metadata that lives in canonical state must carry canonical precision. Chapter 7's coordinates are single-precision by requirement (Requirement 7.1) and reach the wire in the `LayoutCache`, which the Binary Format companion defines as an independent *non-canonical* chunk — so `StaffSpace` is correct there. But `accidental_extensions` lives on `ScoreTuningContext`, which *is* canonical, and Requirement D.3 requires canonical stored floats to be finite IEEE 754 binary64. `EngravingBoundingBox` makes `AccidentalEngraving` wholly core-typed, over the same `SpaceUnit` that `advance_width` already uses, and removes what would otherwise be a backwards dependency from this chapter onto Chapter 7. Chapter 7's `BoundingBox` is unaffected and remains the correct type for resolved-layout coordinates.

4.3.3 Combination Behavior

Most accidentals do not combine; a note carries one accidental at a time. Some systems (HEJI, certain microtonal notations) permit stacking multiple accidental glyphs on a single note to express compound modifications.

```
pub enum AccidentalCombination {
    /// Stands alone; replaces any prior accidental on the same note.
    Solitary,

    /// May stack with members of the listed compatibility groups.
    /// Stacking order is determined by the engraving metadata.
    Stacking { compatible_groups: Vec<AccidentalGroupId> },
}
```

4.3.4 Score-Local Extensions

A score **MAY** extend a referenced accidental registry with additional accidental definitions, provided the registry's `extensible` flag is true. Extensions are stored on the score and override or augment the base registry during resolution.

```
pub struct ScoreAccidentalExtensions {
    pub base: AccidentalRegistryId,
    pub additions: Vec<AccidentalDefinition>,
    pub overrides: Vec<AccidentalDefinition>,
}
```

4.4 Glyph References and SMuFL

The Standard Music Font Layout (SMuFL) is the normative source for glyph references in the core. Every standard accidental, notehead, articulation, dynamic, and ornament has a SMuFL codepoint. Non-standard glyphs are supported via custom glyph references.

```
pub enum GlyphReference {
    /// SMuFL codepoint, resolved against the active SMuFL font.
    Smufl(u32),

    /// Custom glyph defined by the score or a plugin.
    Custom(CustomGlyphId),

    /// Composite glyph: multiple glyph references rendered as a
    /// single accidental. Used for compound HEJI symbols and similar.
    Composite(Vec<GlyphReference>),
}
```

4.4.1 SMuFL Versioning

REQUIREMENT 4.3

Every score **MUST** declare the SMuFL version it targets. Resolving a SMuFL glyph reference whose codepoint is not present in the active font's SMuFL version **MUST** produce a deterministic fallback (the reference SMuFL substitution glyph, or a clearly-marked missing-glyph indicator); it **MUST NOT** silently fail.

```
pub struct SmuflVersion {
    pub major: u16,

    /// The fractional part, normalized to hundredths.
    pub minor_centi: u16,
}
```

A SMuFL version's fractional part is stored *normalized to hundredths* in `minor_centi`, not as the literal digits that follow the decimal point: 1.12 → 12, 1.18 → 18, 1.20 → 20, 1.3 → 30, 1.4 → 40. SMuFL versions are decimal fractions, and this repository's published minor releases did not arrive in digit order: 1.12 and 1.18 shipped before 1.3 and 1.4. A literal-digit encoding (`minor: 3` for 1.3, `minor: 12` for 1.12) is **non-conforming** because the derived ordering on `(major, minor)` then sorts 1.3 and 1.4 *before* 1.12, backwards versus the real release history. The hundredths normalization also collapses the 1.2/1.20 spelling ambiguity: both denote the same release and both normalize to 20. This is the single `SmuflVersion` type used throughout the specification — by `SmuflVersionRequirement` below and, in Chapter 7, by `GlyphCatalog::smufl_version()` (Section 7.12) and `GlyphCatalogIdentity` (Section 7.12.1).

RATIONALE

For reference, SMuFL's minor releases in their real order, each with its normalized `minor_centi` value: 1.12 (12), 1.18 (18), 1.20 (20), 1.3 (30), 1.4 (40).

```
pub struct SmuflVersionRequirement {
    /// Minimum SMuFL version required by this score.
    pub minimum: SmuflVersion,

    /// SMuFL version this score was authored against. Used for
    /// detecting whether newer glyphs are in use.
    pub authored_against: SmuflVersion,
}
```

4.5 Tuning Systems

A tuning system maps positions in a pitch space to frequencies. Given a reference pitch (position plus frequency) and the system's resolution rules, every position in the space resolves to a determinate frequency.

```rust
pub struct TuningSystem {
    pub id: TuningSystemId,

    /// Human-readable name.
    pub name: String,

    /// The pitch space whose positions this tuning resolves.
    pub pitch_space: PitchSpaceId,

    /// How the tuning resolves positions to frequencies, given a
    /// reference pitch. The reference itself is supplied separately
    /// (see Section~\ref{sec:tuning:reference}).
    pub resolution: TuningResolution,

    /// Optional historical or provenance notes.
    pub description: Option<String>,
}
```

4.5.1 Tuning Resolution

```rust
pub enum TuningResolution {
    /// N-tone equal temperament: each step is the Nth root of the
    /// octave ratio. Includes 12-TET when N=12.
    EqualTemperament { divisions_per_octave: u16 },

    /// Explicit per-position ratios. Each entry is a ratio relative
    /// to the reference position.
    PerPositionRatios(Vec<PositionRatio>),

    /// Procedural definition: a registered tuning function that
    /// computes frequencies from a reference. Historical temperaments
    /// (Werckmeister, Vallotti, meantone variants) live here.
    Function {
        function: TuningFunctionId,
        parameters: TuningParameters,
    },

    /// A base resolution with per-position overrides. Used for
    /// split-accidental keyboards and similar irregular tunings.
    Overlay {
        base: Box<TuningResolution>,
        overrides: Vec<PositionOverride>,
    },

    /// Imported from a tuning file (Scala .scl, MIDI Tuning Standard,
    /// etc.). The data is captured at import; the original file is
    /// referenced for provenance but is not required at runtime.
    Imported {
        format: TuningFileFormat,
        data: Arc<ImportedTuningData>,
    },
```

```
    /// Adaptive tuning: resolution depends on the harmonic context
    /// at the moment of sounding. Used for adaptive JI and similar
    /// systems.
    Adaptive {
        function: AdaptiveTuningFunctionId,
        parameters: AdaptiveTuningParameters,
    },
}
```

4.5.2 The Resolution Contract

REQUIREMENT 4.4

Every tuning resolution **MUST** expose a deterministic function from pitch-space position (plus, where applicable, harmonic context) to frequency in Hertz, given a reference pitch. Determinism **MUST** hold across runs and platforms; numerical computations **MUST NOT** depend on floating-point rounding modes.

4.5.3 Adaptive Tuning

Adaptive tuning systems compute frequency from position *plus harmonic context*. A registered adaptive tuning function **MAY** resolve position per sonority — for example, tuning a given E differently depending on whether it sounds as the major third of a C chord, the perfect fifth of an A chord, or a passing tone — subject only to the purity constraint below (Requirement 4.5). The built-in `ji-adaptive-5limit`, at its pinned version 1 (Requirement 4.6), does not: it consumes only the harmonic context's tonal centre.

```
pub struct HarmonicContext {
    /// Concurrently sounding pitches at the moment of resolution.
    pub concurrent: Vec<PitchId>,

    /// Recently sounding pitches, with decay weighting.
    pub recent: Vec<(PitchId, f64)>,

    /// The active key or modal center, if known.
    pub key_context: Option<KeyContext>,

    /// User-supplied context hints attached to the score.
    pub hints: Vec<ContextHint>,
}
```

The harmonic context is constructed by the audio engine (or by analysis tooling) and passed to the tuning resolution function. Static tuning systems ignore it; adaptive tuning systems consume it.

REQUIREMENT 4.5

Adaptive tuning resolution **MUST** be a pure function of position and harmonic context. Implementations **MAY** cache resolution results, but **MUST** invalidate caches when the harmonic context changes.

REQUIREMENT 4.6

The built-in adaptive function, version 1. The catalog identifier `ji-adaptive-5limit` resolves to `TuningResolution::Adaptive` with function identity `"default-v1"` — the version is part of the machine-visible identifier string, not prose beside it. Version 1 is a pure function of (position, anchor pitch class):

- the position resolves through the construction of Requirement 4.12, transposed so the anchor takes the role of $1/1$;
- the anchor is the tonal centre supplied by the harmonic context (Requirement 4.7 where that context is score-graph-derived); C (chromatic position 0) when no tonal centre is supplied;
- `concurrent`, `recent`, `hints`, `parameters`, and mode are *ignored* by version 1, exactly as `"default"` version 1 of Requirement 2.10 ignores the context inputs it does not consult: consuming any of them is a new version, not a silent behaviour change under the same identity;
- every result derives from the anchor and the reference pitch alone. No adjustment is ever carried forward from a previous resolution, so the construction is comma-drift-free by shape, not by a correction step.

An unregistered or unknown `AdaptiveTuningFunctionId` **MUST** be a hard error; there is no silent fallback. `AdaptiveTuningFunctionId` remains an extension point and other functions **MAY** be registered, but the built-in `ji-adaptive-5limit` is bound to `"default-v1"` and that binding **MUST NOT** be overridden.

REQUIREMENT 4.7

Where a `HarmonicContext`'s tonal centre is derived from a score graph, the anchor pitch class **MUST** be computed from the prevailing key signature as

$$\texttt{anchor_pc} = (7 \times \texttt{fifths}) \bmod 12,$$

reading `fifths` (`KeySignature`, Chapter 5) as the major tonic; mode is not consulted, so a signature of 0 anchors at C whether the prevailing key is C major or A minor. The *prevailing* key signature is the one on the staff containing the pitch being resolved, taken from that staff's `key_sequence` (`StaffInstance::key_sequence`, Chapter 5) at the latest `KeySignatureChange` whose anchor is at or before the pitch's onset. Both selections — staff and time — **MUST** be made this way: without them, a polytonal or modulating score would resolve differently in two conforming implementations, which Requirement 4.4 forbids.

4.6 Reference Pitch

The reference pitch is the anchor that converts the tuning system's structural ratios into absolute frequencies. It is a property of the score, not of the tuning system.

```
pub struct ReferencePitch {
    /// The pitch-space position chosen as the reference.
    /// Conventionally A4 in CMN, but configurable.
    pub position: PitchSpacePosition,

    /// The frequency in Hertz assigned to that position.
    /// Conventional values: 440 Hz (modern standard), 415 Hz (Baroque),
    /// 430 Hz (early Classical), 442 Hz (some modern orchestras).
    pub frequency_hz: f64,
}
```

Requirement 4.8

Every score **MUST** declare a reference pitch. The reference pitch **MUST** be expressible as a valid position within the score's default pitch space. When that position is `Cmn`, the default pitch space **MUST** supply its chromatic layer and nominal mapping under Requirement 2.1. The frequency **MUST** be positive and finite.

4.6.1 Multiple Reference Pitches

A score **MAY** declare additional reference pitches at the staff or region level, overriding the score default. This supports mixed-instrumentation scores in which different instruments use different concert-pitch references (e.g., a piece for period and modern instruments with different A frequencies).

4.7 Score Tuning Context and Hierarchical Resolution

The *score tuning context* is the top-level structure declaring the tuning environment for a score. Per-scope overrides extend it.

```
pub struct ScoreTuningContext {
    /// Default pitch space for the score.
    pub default_pitch_space: PitchSpaceId,

    /// Default tuning system. Its pitch_space must match (or be
    /// compatible with) the default_pitch_space.
    pub default_tuning_system: TuningSystemId,

    /// The reference pitch for the score.
    pub reference: ReferencePitch,

    /// Score-local accidental registry extensions.
    pub accidental_extensions: Vec<ScoreAccidentalExtensions>,

    /// SMuFL version targeting.
```

```
    pub smufl: SmuflVersionRequirement,

    /// Per-scope overrides, in the order they apply.
    pub overrides: Vec<TuningOverride>,
}

pub struct TuningOverride {
    pub scope: TuningScope,
    pub pitch_space: Option<PitchSpaceId>,
    pub tuning_system: Option<TuningSystemId>,
    pub reference: Option<ReferencePitch>,
}

pub enum TuningScope {
    Voice(VoiceId),
    Staff(StaffId),
    Region(RegionId),
    Range { start: TimeAnchor, end: TimeAnchor, voices: VoiceSelector },
}
```

4.7.1 Resolution Order

For a given pitch, resolution proceeds by walking outward through scopes from most specific to most general until each component (pitch space, tuning system, reference) is determined.

REQUIREMENT 4.9

Resolution **MUST** proceed in the following order, halting at the first scope that supplies a non-inherited value for each component:

1. The pitch's own `AcousticPitch` (an explicit `TuningReference::Explicit` or `AcousticRealization::AbsoluteHz` short-circuits the walk).
2. The voice containing the pitch.
3. The staff containing the voice.
4. Each region enclosing the pitch, from innermost to outermost.
5. The score's default tuning context.

Each of the three components (pitch space, tuning system, reference) is resolved independently along this chain.

RATIONALE

Independent per-component resolution allows a passage to override only the parts that differ. A period-instrument inset in a modern orchestra score may override only the reference frequency (415 Hz instead of 440 Hz) while inheriting the pitch space and tuning structure from the parent context. Conversely, a microtonal passage in an otherwise 12-TET score may override the pitch space and tuning system but inherit the reference.

4.7.2 Compatibility Constraints

Requirement 4.10

When a tuning override is applied, the resolved tuning system's declared `pitch_space` **MUST** either equal the resolved pitch space or be declared compatible with it via a registered compatibility mapping. Implementations **MUST** reject configurations in which the resolved pitch space and the resolved tuning system's pitch space disagree without a compatibility mapping.

4.8 Built-in Catalog

The core ships with a baseline catalog of pitch spaces, tuning systems, and accidental registries sufficient for the common case. The catalog is normative: a conforming implementation **MUST** provide these identifiers with the specified semantics.

4.8.1 Built-in Pitch Spaces

Identifier	Description
`cmn-12`	Common Music Notation, 7 diatonic nominals (A–G) over 12 chromatic positions. Standard sharps, flats, double-sharps, and double-flats. The default for Western tonal and post-tonal music.
`cmn-24`	CMN over 24 chromatic positions with `nominal_to_chromatic` $= [0, 4, 8, 10, 14, 18, 22]$ and quarter-tone alteration steps. A flat is -2 and a half-flat is -1. Used for Arabic-derived microtonal practice in CMN-compatible notation.
`edo-19`	19-tone equal division of the octave. Notable for its closer-to-just major thirds.
`edo-22`	22-tone equal division. Distinguishes harmonic intervals collapsed in 12-TET.
`edo-31`	31-tone equal division. Approximates quarter-comma meantone almost exactly.
`edo-53`	53-tone equal division. Excellent approximation of 5-limit JI.
`edo-72`	72-tone equal division. Common in research and contemporary microtonal practice.
`ji-5limit`	5-limit just intonation lattice with HEJI accidentals. Three-dimensional; prime basis $\{2, 3, 5\}$ in ascending order (leading component is the octave, prime 2).
`ji-7limit`	7-limit JI lattice with HEJI accidentals. Four-dimensional; prime basis $\{2, 3, 5, 7\}$ in ascending order.
`ji-11limit`	11-limit JI lattice with HEJI accidentals. Five-dimensional; prime basis $\{2, 3, 5, 7, 11\}$ in ascending order.

Identifier	Description
`maqam-base`	Skeletal maqam framework with quarter-flat and quarter-sharp `CmnChromatic` accidentals, denominated by the space's chromatic layer under Requirement 2.1. Deep coverage of specific maqamat is the province of grammar plug-ins.
`gamelan-slendro`	Five-tone slendro framework.
`gamelan-pelog`	Seven-tone pelog framework.

RATIONALE

The three JI spaces include prime 2 in their basis, per Requirement 2.2, rather than assuming octave equivalence. A `JiVector` is an absolute position, not a pitch class: without the prime-2 exponent, `ji-5limit` could not distinguish C4 from C5. None of these three declares octave-reduced equivalence, so full register is preserved, as that requirement's default already states.

Conformance note. At this revision the default spelling pre-pass is structurally 12-chromatic and therefore reports `spelling_unavailable` for `cmn-24` positions. This does not prevent storing or engraving an explicitly authored spelling, nor does it change the space-relative transposition algebra; automatic 24-chromatic spelling inference is deferred.

4.8.2 Built-in Tuning Systems

Identifier	Description
`tet-12`	12-tone equal temperament. The default.
`tet-19`	19-tone equal temperament.
`tet-22`	22-tone equal temperament.
`tet-31`	31-tone equal temperament.
`tet-53`	53-tone equal temperament.
`tet-72`	72-tone equal temperament.
`pythagorean`	Pure-fifth (3:2) Pythagorean tuning.
`meantone-1/4-comma`	Quarter-comma meantone (pure major thirds).
`meantone-1/6-comma`	Sixth-comma meantone.
`meantone-1/5-comma`	Fifth-comma meantone.
`werckmeister-iii`	Werckmeister III well temperament.
`werckmeister-iv`	Werckmeister IV well temperament.
`vallotti`	Vallotti well temperament.
`kirnberger-ii`	Kirnberger II well temperament.
`kirnberger-iii`	Kirnberger III well temperament.
`young-ii`	Thomas Young's second temperament.
`ji-static-5limit-C`	Static 5-limit JI anchored to C tonic.
`ji-static-5limit-G`	Static 5-limit JI anchored to G tonic.
`ji-static-5limit-D`	Static 5-limit JI anchored to D tonic.

Identifier	Description
`ji-adaptive-5limit`	Adaptive 5-limit JI; resolves based on harmonic context.

REQUIREMENT 4.11

All built-in catalog identifiers **MUST** resolve in a conforming implementation. The semantics specified above are normative. Implementations **MAY** provide additional built-ins; they **MUST NOT** redefine the semantics of those listed.

4.8.3 Temperament Constructions

This subsection gives the normative construction for each of the ten non-equal-tempered, non-just-intonation identifiers in the catalog above. Per the pitch-space/tuning-system independence stated in Section 4.1, all ten are constructions over `cmn-12`'s twelve chromatic positions per octave (C, C♯/D♭, D, …, B) — not over the open-ended `ji-5limit` lattice pitch space, which is a separate built-in with its own structure. Cents figures throughout are derived from the stated ratio or fraction-of-comma, never primary.

`pythagorean`

A chain of eleven pure 3/2 fifths (twelve notes); every other interval is stacked fifths reduced by octaves. No comma is tempered anywhere in the chain — the entire Pythagorean comma ($531441/524288 \approx 23.460$ cents) is concentrated in the single interval where the chain does not close.

The twelve-note selection is a choice of *which* eleven consecutive fifths to keep, not a consequence of "pure fifths" alone. The conventional cut runs E♭–B♭–F–C–G–D–A–E–B–F♯–C♯–G♯ (C at the center), leaving the wolf on the diminished sixth G♯–E♭. An equally valid alternative cuts the spiral one step the other way (chain D♭–…–F♯, wolf at F♯–D♭); either is a legitimate Pythagorean tuning. The E♭–G♯ cut is the construction this identifier denotes.

Note	Ratio	Cents (derived)
C	1/1	0.000
C♯	2187/2048	113.685
D	9/8	203.910
E♭	32/27	294.135
E	81/64	407.820
F	4/3	498.045
F♯	729/512	611.730
G	3/2	701.955
G♯	6561/4096	815.640
A	27/16	905.865
B♭	16/9	996.090
B	243/128	1109.775

The closing wolf fifth G♯ →E♭ (octave-reduced) is $262144/177147 \approx 678.495$ cents.

Closure (non-circulating, by design). Eleven pure fifths at 701.955 c each, plus the closing wolf at 678.495 c: $11 \times 701.955 + 678.495 = 8400.000$ c exactly (seven octaves, as any assignment of twelve distinct pitch classes must sum to by construction). Relative to twelve *pure* fifths ($12 \times 701.955 = 8423.460$ c), this construction is short by exactly $8423.460 - 8400.000 = 23.460$ c — one Pythagorean comma, concentrated entirely on the single G♯–E♭ interval rather than distributed. That concentration is what makes the tuning non-circulating: no pair of its twelve notes can be treated as interchangeable fifths the way a well temperament's can.

`meantone-1/4-comma`, `meantone-1/5-comma`, `meantone-1/6-comma`

Meantone is a *regular* temperament: every one of the twelve fifths in the chain is tempered by the same fraction of the *syntonic* comma ($81/80 \approx 21.506$ cents) — not the Pythagorean comma used by the well temperaments below. All twelve fifths are tempered uniformly, so there is no "which fifths" selection the way there is for a well temperament; there is still a twelve-note chain-closing choice, structurally identical to `pythagorean`'s, and the same conventional cut places the wolf between G♯ and E♭.

The tempered fifth is $(3/2) \cdot (80/81)^{1/n}$ for $1/n$-comma. For $n = 4$ this collapses to the exact form $5^{1/4}$; $n = 5$ and $n = 6$ are irrational and are given only as derived cents.

Variant	Fifth ratio	Cents (derived)	Narrowing vs. pure $3/2$
1/4-comma	$(3/2)(80/81)^{1/4} = 5^{1/4}$ exactly	696.578	5.377 c $= \frac{1}{4} \cdot 21.506$ c
1/5-comma	$(3/2)(80/81)^{1/5}$	697.654	4.301 c $= \frac{1}{5} \cdot 21.506$ c
1/6-comma	$(3/2)(80/81)^{1/6}$	698.371	3.584 c $= \frac{1}{6} \cdot 21.506$ c

Four ascending 1/4-comma fifths, lowered by two octaves, give the just major third: $4 \times 696.578 - 2 \times 1200 = 386.31$ c, the cents of $5/4$ — the defining property of quarter-comma meantone.

Closure (non-circulating, by design; all three). Because all twelve fifths are tempered equally, the residual wolf is the twelfth, closing interval forced to bring the total to 8400 c (seven octaves):

Variant	11 × tempered fifth	Wolf (12th, closing)	Wolf vs. pure $3/2$
1/4-comma	7662.363 c	737.637 c	+35.682 c (wide)
1/5-comma	7674.191 c	725.809 c	+23.854 c (wide)
1/6-comma	7682.077 c	717.923 c	+15.968 c (wide)

All three wolves land on the wide side of a pure fifth — the opposite sense from `pythagorean`'s narrow wolf.

`werckmeister-iii`

Fifths C–G, G–D, D–A, and B–F♯ are each narrowed by 1/4 of the *Pythagorean* comma; the remaining eight fifths are pure. Because only four of twelve fifths are tempered and the pure

fifths absorb none of the comma, all twelve notes remain usable as a tonic: there is no wolf.

Fifths	Tempering
C–G, G–D, D–A, B–F♯	narrow, 1/4 Pythagorean comma (4)
A–E, E–B, F♯–C♯, C♯–G♯, G♯–E♭, E♭–B♭, B♭–F, F–C	pure (8)

Tempered-fifth cents: $(3/2)/(3^{12}/2^{19})^{1/4} \approx 696.090$ (narrowing 5.865 c $= \frac{1}{4} \cdot 23.460$ c).

Closure. $4 \times 5.865 = 23.460$ c exactly — the chain closes.

`werckmeister-iv`

Fifths C–G, D–A, E–B, F♯–C♯, and B♭–F are each narrowed by 1/3 of the Pythagorean comma; fifths G♯–D♯ and E♭–B♭ are each *widened* by 1/3 of the Pythagorean comma; the remaining five fifths are pure.

Fifths	Tempering
C–G, D–A, E–B, F♯–C♯, B♭–F	narrow, 1/3 Pythagorean comma (5)
G♯–D♯, E♭–B♭	wide, 1/3 Pythagorean comma (2)
G–D, A–E, B–F♯, C♯–G♯, F–C	pure (5)

Narrow-fifth cents: $(3/2)/(3^{12}/2^{19})^{1/3} \approx 694.135$ (narrowing 7.820 c $= \frac{1}{3} \cdot 23.460$ c). Wide-fifth cents: $(3/2) \cdot (3^{12}/2^{19})^{1/3} \approx 709.775$ (widening 7.820 c). No wolf: all twelve notes remain usable as a tonic.

Closure. $5 \times 7.820 - 2 \times 7.820 = 3 \times 7.820 = 23.460$ c exactly — the chain closes.

`vallotti`

Fifths F–C, C–G, G–D, D–A, A–E, and E–B (six consecutive) are each narrowed by 1/6 of the Pythagorean comma; the other six fifths are pure.

Fifths	Tempering
F–C, C–G, G–D, D–A, A–E, E–B	narrow, 1/6 Pythagorean comma (6)
B–F♯, F♯–C♯, C♯–G♯, G♯–E♭, E♭–B♭, B♭–F	pure (6)

Tempered-fifth cents: $(3/2)/(3^{12}/2^{19})^{1/6} \approx 698.045$ (narrowing 3.910 c $= \frac{1}{6} \cdot 23.460$ c). No wolf.

Closure. $6 \times 3.910 = 23.460$ c exactly — the chain closes.

kirnberger-ii

Fifths D–A and A–E are each narrowed by 1/2 the syntonic comma; fifth F♯–D♭ (i.e. F♯–C♯, named with the flat spelling because the chain below is built outward from D♭) is narrowed by a schisma (the difference between the Pythagorean and syntonic commas, $32805/32768 \approx 1.9537$ c); the remaining nine fifths are pure.

Fifths	Tempering
D♭–A♭, A♭–E♭, E♭–B♭, B♭–F, F–C, C–G, G–D, E–B, B–F♯	pure (9)
D–A, A–E	narrow, 1/2 syntonic comma (2)
F♯–D♭ (closing)	narrow, 1 schisma (1)

Cents by note, built by stacking the chain D♭–A♭–E♭–B♭–F–C–G–D–A–E–B–F♯ from C = 1/1:

Note	Cents (derived)
C	0.000
D♭	90.225
D	203.910
E♭	294.135
E	386.314
F	498.045
F♯	590.224
G	701.955
A♭	792.180
A	895.112
B♭	996.090
B	1088.269

Closure. 2×10.753 (half-syntonic-comma fifths) $+1 \times 1.9537$ (schisma fifth) $= 21.5063 + 1.9537 = 23.4600$ c exactly — the chain closes.

kirnberger-iii

Fifths C–G, G–D, D–A, and A–E (four consecutive) are each narrowed by 1/4 the syntonic comma; fifth F♯–D♭ is narrowed by a schisma (same position and same chain skeleton as `kirnberger-ii`); the remaining seven fifths are pure.

Fifths	Tempering
D♭–A♭, A♭–E♭, E♭–B♭, B♭–F, F–C, E–B, B–F♯	pure (7)

Fifths	Tempering
C–G, G–D, D–A, A–E	narrow, 1/4 syntonic comma (4)
F♯–D♭ (closing)	narrow, 1 schisma (1)

Cents by note, same chain skeleton as `kirnberger-ii`, from C = 1/1:

Note	Cents (derived)
C	0.000
D♭	90.225
D	193.157
E♭	294.135
E	386.314
F	498.045
F♯	590.224
G	696.578
A♭	792.180
A	889.735
B♭	996.090
B	1088.269

Closure. 4×5.377 (quarter-syntonic-comma fifths) $+1 \times 1.9537$ (schisma fifth) $= 21.5063 + 1.9537 = 23.4600$ c exactly — the chain closes.

young-ii

Thomas Young's *second* temperament. Fifths C–G, G–D, D–A, A–E, E–B, and B–F♯ (six consecutive) are each narrowed by 1/6 of the Pythagorean comma; the remaining six fifths are pure.

Fifths	Tempering
C–G, G–D, D–A, A–E, E–B, B–F♯	narrow, 1/6 Pythagorean comma (6)
F♯–C♯, C♯–G♯, G♯–E♭, E♭–B♭, B♭–F, F–C	pure (6)

This is the same six-tempered/six-pure, 1/6-Pythagorean-comma construction as `vallotti`, rotated: `young-ii`'s tempered run starts at C; `vallotti`'s starts at F. Tempered-fifth cents are therefore identical to `vallotti`'s: ≈ 698.045 (narrowing 3.910 c $= \frac{1}{6} \cdot 23.460$ c). No wolf.

Closure. $6 \times 3.910 = 23.460$ c exactly — the chain closes.

4.8.4 The Static 5-Limit Construction

REQUIREMENT 4.12

The static 5-limit just-intonation systems are constructed from the contiguous lattice block $\{3^a 5^b \mid a \in [-1, 2],\ b \in [-1, 1]\}$ — twelve cells, octave-reduced, assigned in ascending order to the twelve chromatic positions of `cmn-12` starting from the anchor, which takes the role of $1/1$. The block is generated by its bounds; nothing is selected and nothing is discarded.

The twelve, anchor-relative (position 0 is the anchor):

Step	Cell	Ratio	Cents (derived)
0	$3^0 5^0$	$1/1$	0.000
1	$3^{-1} 5^{-1}$	$16/15$	111.731
2	$3^2 5^0$	$9/8$	203.910
3	$3^1 5^{-1}$	$6/5$	315.641
4	$3^0 5^1$	$5/4$	386.314
5	$3^{-1} 5^0$	$4/3$	498.045
6	$3^2 5^1$	$45/32$	590.224
7	$3^1 5^0$	$3/2$	701.955
8	$3^0 5^{-1}$	$8/5$	813.686
9	$3^{-1} 5^1$	$5/3$	884.359
10	$3^2 5^{-1}$	$9/5$	1017.596
11	$3^1 5^1$	$15/8$	1088.269

`ji-static-5limit-C`, `ji-static-5limit-G`, and `ji-static-5limit-D` are this one construction at three anchors, the named note taking the role of $1/1$. The catalog keeps exactly these three rows; additional anchors are an implementation extension and **MUST NOT** be read as a conformance obligation.

No 12-note set built this way is inversionally symmetric: $45/32$ is present and its inverse $64/45$ is not.

4.8.5 Default Score Configuration

A newly-created score with no explicit tuning context **MUST** default to:

- Pitch space: `cmn-12`.
- Tuning system: `tet-12`.
- Reference pitch: A4 = 440 Hz.
- SMuFL version: the highest SMuFL version supported by the implementation.

4.9 Forward References

- `VoiceId`, `StaffId`, `RegionId`, `VoiceSelector`, and the score graph's hierarchical organization are defined in Chapter 5.
- `KeyContext` is not defined anywhere in this specification; completing it is out of scope for this document. Whatever completes it **MUST** expose a tonal-centre chromatic pitch class consistent with Requirement 4.7. Chapter 5 defines `KeySignature`, not `KeyContext`. `ContextHint` and `AdaptiveTuningParameters` are likewise undefined; the built-in `ji-adaptive-5limit`, version `1` (Requirement 4.6), ignores both, so defining them now would freeze a type surface on a chapter with no consumer.
- The specific spelling algorithms referenced by `SpellingAlgorithmId` remain an open question; see the related open question in Section 2.5.4.

Chapter 5

The Score Graph

This chapter specifies the score graph: the in-memory representation of all musical content in a score. The graph is the canonical truth about the music; layout, serialization, and editing operations are downstream projections and consumers of it. This chapter is the largest in the specification because it threads together every primitive defined in Chapters 2–4 into a coherent whole.

5.1 Design Principles

Hybrid topology. The graph is a tree of containment overlaid with cross-cutting structures that hold references. The tree determines existence: deleting a parent deletes its descendants. Cross-cutting structures hold references that may become dangling and require re-anchoring per the rules defined in this chapter and in Chapter 6.

Canvas before staff. The spatial root of a score is the canvas, a 2D space partitioned into regions. Staves exist inside regions, not as a top-level concept. This permits regions of free graphic content to coexist with regions of staff-based CMN without forcing one paradigm to subsume the other.

Arena storage. Events are stored in a flat arena owned by the score. Voices, cross-cutting structures, and analysis layers hold event identifiers, not events. Arena storage enables fast bulk iteration, simple cross-cutting reference, and clean CRDT semantics.

Parts are projections. A part is a view onto the score defined by a part-extraction specification. Parts do not live in the canvas; they are separate objects that select and reshape canvas content for printed extraction.

Typed identifiers. Every named object kind has its own identifier type. Cross-kind confusion is a compile-time error.

Identifiers are stable and deterministic. Identifiers are assigned at creation, never reassigned, and generated by a scheme that is deterministic per replica.

5.2 Top-Level Score Structure

The score is the root object. All other objects in the specification exist within or reference into a score. Conflict records are deliberately *not* part of the score graph: the `ConflictRegistry` produced by canonical reduction is a component of the canonical *materialized state* (Chapter 6),

recomputed with it, and persisting there until resolved or dismissed by explicit operations.

```rust
pub struct Score {
    /// Bibliographic and authorship metadata.
    pub metadata: ScoreMetadata,

    /// The spatial root: regions and their staff/graphic content.
    pub canvas: Canvas,

    /// Abstract instrument definitions used by staves.
    pub instruments: Vec<Instrument>,

    /// Globally-identified staves of the score. Each region's
    /// staff instances reference these by StaffId.
    pub staves: Vec<Staff>,

    /// Optional staff groupings (grand staves, choral groups,
    /// bracketed orchestral sections). Used by engraving and parts.
    pub staff_groups: Vec<StaffGroup>,

    /// Per-part view definitions for extraction.
    pub parts: Vec<PartDefinition>,

    /// Cross-cutting structures: slurs, ties, hairpins, markers,
    /// graphic gestures, comments, and analytical annotations.
    pub cross_cutting: CrossCuttingRegistry,

    /// Tuning systems, pitch spaces, and reference pitch.
    pub tuning_context: ScoreTuningContext,

    /// The score-level tempo map. Regions may declare local tempo
    /// maps that override this.
    pub tempo_map: TempoMap,

    /// Flat arena of all events in the score.
    pub events: EventArena,

    /// Spelling attachments index (see Chapter 2).
    pub spelling_attachments: SpellingAttachmentIndex,

    /// Notational decomposition attachments index (see Chapter 3).
    pub decomposition_attachments: DecompositionAttachmentIndex,

    /// Analysis layers and view definitions.
    pub analysis_layers: Vec<AnalysisLayer>,
    pub views: Vec<ViewDefinition>,

    /// Replica identifier and identifier generation state.
    pub identity: IdentityContext,

    /// Each visiting voice's home staff, the staff it belongs to
    /// whatever staff its notes are drawn on (schema major 5). A voice
    /// absent here is at home on its instance's staff.
```

```
    pub voice_homes: BTreeMap<VoiceId, StaffId>,
}

pub struct StaffGroup {
    pub id: StaffGroupId,
    pub name: Option<String>,
    pub kind: StaffGroupKind,
    /// A denormalized projection of group membership, maintained from
    /// \texttt{Staff.group} by reduction (P13-S16, ratified disposition A;
    /// supersedes genesis tranche G3a's disposition B).
    /// \texttt{Staff.group} below is the sole authority; this field is
    /// derived from it and the two \MUST{} agree in both directions, which
    /// graph invariant~21 enforces. \texttt{CreateStaffGroup} refuses a
    /// non-empty carried \texttt{members}: membership changes only by
    /// writing \texttt{Staff.group}. Under disposition B this field was
    /// authored directly and MAY have been stale in both directions;
    /// neither state is reachable now.
    pub members: Vec<StaffId>,
}

pub enum StaffGroupKind {
    /// Grand staff: typically piano, organ, harp.
    GrandStaff,
    /// Bracketed group: orchestral sections.
    Bracket,
    /// Sub-bracket: divisi within a section.
    SubBracket,
    /// Choral group: SATB and similar.
    Choral,
    /// Custom group defined by a layout extension.
    Registered(StaffGroupKindRegistryId),
}
```

5.2.1 Score Metadata

```
pub struct ScoreMetadata {
    pub title: Option<String>,
    pub subtitle: Option<String>,
    pub composer: Option<String>,
    pub lyricist: Option<String>,
    pub arranger: Option<String>,
    pub copyright: Option<String>,
    pub creation_timestamp: Timestamp,
    pub modification_timestamp: Timestamp,
    pub additional: Vec<MetadataEntry>,
}

pub struct MetadataEntry {
    pub key: String,
    pub value: MetadataValue,
}
```

```rust
/// The value of an additional metadata entry (defined schema major 2).
/// Closed small union; growth is by appended variant.
pub enum MetadataValue {
    Text(String),
    Integer(i64),
    Flag(bool),
}

/// A calendar timestamp (defined schema major 2): nanoseconds since the
/// Unix epoch, UTC, no zone. Distinct from WallClockTime, which is
/// *performance* time within a score. The zero value is the "unset"
/// convention (genesis without a declared creation time).
pub struct Timestamp(pub i64);
```

The metadata structure is deliberately small. Extended bibliographic, catalog, and editorial-apparatus metadata is the province of foreign- format mappings and analytical layers, not the core. `additional` is an *ordered authored list*, not a map (defined schema major 2): entry order is preserved verbatim in canonical bytes, and duplicate keys are permitted — foreign formats legitimately carry repeated keys (multiple contributors, multiple identifiers of one scheme). Consumers wanting map semantics take the first entry per key.

REQUIREMENT 5.1

Timestamps are strictly authored (ratified with schema major 2). `creation_timestamp` and `modification_timestamp` are stored, *authored* values: genesis takes the creation timestamp explicitly (the empty-document constructor defaults it to the epoch-zero unset convention), and a `SetMetadata` operation carries both verbatim as part of the metadata value. Implementations **MUST NOT** write either field implicitly — an automatic modification-time update would rewrite canonical bytes on every edit, churning content addresses and breaking replica byte-agreement for identical operation histories. A client that wants a live modification time surfaces one from non-canonical state (the operation log's stamps); it does not store it here.

5.3 Identifiers

Every named object in the graph carries a typed 128-bit identifier.

```rust
pub struct EventId(u128);
pub struct VoiceId(u128);
pub struct StaffId(u128);
pub struct StaffInstanceId(u128);
pub struct StaffGroupId(u128);
pub struct RegionId(u128);
pub struct InstrumentId(u128);
pub struct PartDefinitionId(u128);
pub struct MeasureId(u128);
pub struct BarlineAlignmentGroupId(u128);
pub struct SlurId(u128);
pub struct TieId(u128);
```

```rust
pub struct BeamId(u128);
pub struct SpannerId(u128);
pub struct MarkerId(u128);
pub struct AnalyticalAnnotationId(u128);
pub struct CommentId(u128);
pub struct GraphicObjectId(u128);
pub struct GraphicGestureId(u128);
pub struct TimeSignatureId(u128);
pub struct ConflictId(u128);
pub struct TransactionId(u128);
// ... and so on for every named object kind

/// A tagged identifier over every typed identifier kind in the
/// score graph. Used wherever an object is referenced generically
/// (cross-cutting endpoints, conflict affected_objects, repair
/// records, edit barriers). The variant tag is part of canonical
/// content: distinct variants with the same underlying u128 are
/// distinct TypedObjectIds.
pub enum TypedObjectId {
    Event(EventId),
    Pitch(PitchId),
    Voice(VoiceId),
    Staff(StaffId),
    StaffInstance(StaffInstanceId),
    StaffGroup(StaffGroupId),
    Region(RegionId),
    Instrument(InstrumentId),
    PartDefinition(PartDefinitionId),
    Measure(MeasureId),
    BarlineAlignmentGroup(BarlineAlignmentGroupId),
    Slur(SlurId),
    Tie(TieId),
    Beam(BeamId),
    Spanner(SpannerId),
    Marker(MarkerId),
    AnalyticalAnnotation(AnalyticalAnnotationId),
    Comment(CommentId),
    GraphicObject(GraphicObjectId),
    GraphicGesture(GraphicGestureId),
    TimeSignature(TimeSignatureId),
    AnalysisLayer(AnalysisLayerId),
    Tuplet(TupletId),
    RepeatStructure(RepeatStructureId),
    LyricLine(LyricLineId),
    ChordSymbol(ChordSymbolId),
    View(ViewId),
    /// Extension-defined object kind, identified by registry id
    /// plus the extension's own 128-bit identifier.
    Registered(ObjectKindRegistryId, u128),
}

impl TypedObjectId {
    /// Canonical byte form for hashing, ordering, and equality.
```

```rust
    /// Variant tag is encoded as a 16-bit big-endian discriminant
    /// followed by the variant payload's canonical bytes.
    pub fn canonical_bytes(&self) -> Vec<u8> { /* ... */ }
}
```

REQUIREMENT 5.2

The `TypedObjectId` variant tag **MUST** be encoded as a 16-bit big-endian discriminant followed by the variant payload's canonical bytes. The discriminant of each variant is its declaration-order index in the following closed table; this assignment is normative and stable, and **MUST NOT** be reordered:

Disc.	Variant	Disc.	Variant
0	`Event`	14	`Spanner`
1	`Pitch`	15	`Marker`
2	`Voice`	16	`AnalyticalAnnotation`
3	`Staff`	17	`Comment`
4	`StaffInstance`	18	`GraphicObject`
5	`StaffGroup`	19	`GraphicGesture`
6	`Region`	20	`TimeSignature`
7	`Instrument`	21	`AnalysisLayer`
8	`PartDefinition`	22	`Tuplet`
9	`Measure`	23	`RepeatStructure`
10	`BarlineAlignmentGroup`	24	`LyricLine`
11	`Slur`	25	`ChordSymbol`
12	`Tie`	26	`View`
13	`Beam`	27	`Registered`

The object-kind set is closed at these 28 discriminants (`0`–`27`). Every named object kind in the score graph has a dedicated discriminant; kinds introduced by registered extensions **MUST** use discriminant `27` (`Registered`) and are distinguished by their `ObjectKindRegistryId`, never by extending this table. The `Registered(reg, raw)` payload is encoded as the 128-bit `reg.canonical_bytes()` (16 big-endian bytes) followed by the 16 big-endian bytes of the `raw` `u128`, for a total canonical form of $2 + 16 + 16 = 34$ bytes. `ObjectKindRegistryId` is a 128-bit value.

5.3.1 Identifier Generation

REQUIREMENT 5.3

Identifiers **MUST** be 128-bit values composed of a 64-bit replica identifier and a 64-bit monotonic counter local to that replica. The replica identifier **MUST** be unique among all replicas of the score; collision resistance is achieved by random initialization of the replica identifier at creation time with at least 64 bits of entropy from a cryptographically appropriate source. The counter **MUST** be monotonically increasing within a replica and **MUST NOT** be reused, even if the corresponding object is deleted.

```rust
pub struct IdentityContext {
    /// This replica's identifier. Generated at score creation.
    pub replica_id: ReplicaId,

    /// Monotonic counter for new identifiers. Per-kind counters
    /// optional; a single counter suffices.
    pub next_counter: u64,
}

pub struct ReplicaId(u64);

impl ReplicaId {
    /// Reserved replica identifier for deterministically-derived
    /// system identifiers (system-promoted voices, content-derived
    /// conflict IDs, future deterministic synthetic identifiers).
    /// User-authored replicas MUST NOT use this value.
    pub const SYSTEM_DERIVED: ReplicaId = ReplicaId(0xffff_ffff_ffff_ffff);
}
```

RATIONALE

Replica-plus-counter identifiers are deterministic given a replica and a creation order, which simplifies reasoning about CRDT merge semantics and reproducible builds of generated scores. UUIDv7 was considered and rejected: the timestamp embedding leaks ordering information that the CRDT layer would have to reconstruct anyway, and the random portion gives less collision resistance per bit than a properly initialized replica identifier.

5.3.2 System-Derived Identifier Namespace

Some identifiers are not author-authored: they arise during deterministic reduction (system-promoted voices, content-derived conflict identifiers, attachment-deduplication identifiers). To prevent collisions with user-authored identifiers, the spec reserves a dedicated replica namespace.

REQUIREMENT 5.4

The replica identifier `ReplicaId::SYSTEM_DERIVED` (`0xffff_ffff_ffff_ffff`) is reserved for deterministically-derived system identifiers. User-authored replicas **MUST NOT** use this value as their `ReplicaId`. Implementations creating a new score **MUST** reject this value when generating the local replica identifier and **MUST** regenerate the random portion until a non-reserved value is obtained.
Because a system-derived identifier is content-derived, the identified object's intrinsic content is *immutable*: a reduction **MUST** refuse an operation that would rewrite the intrinsic content of a `SYSTEM_DERIVED`-namespace object in place (ratified Pass 12; the Operation Catalog pins the precondition and its failure reason). Rewriting the content would silently invalidate the id's content derivation; the sanctioned path is minting a replacement object.

System-derived identifiers within the `ReplicaId::SYSTEM_DERIVED` namespace **MUST** have their 64-bit counter portion derived deterministically by BLAKE3 truncation:

```
fn derive_system_counter(
    domain_tag: &[u8; 8],
    canonical_inputs: &[u8],
) -> u64 {
    let mut preimage = Vec::new();
    preimage.extend_from_slice(domain_tag);
    preimage.extend_from_slice(canonical_inputs);
    let hash = blake3(&preimage);
    u64::from_be_bytes(hash[0..8].try_into().unwrap())
}
```

The built-in (reserved) domain tags for system-derived identifiers are a closed set of three:

- `"MUSCSVCE"` for system-promoted voices (Section 5.6.3).
- `"MUSCSPCH"` for system-derived pitches (rare; only when promotion logic introduces a synthetic pitch; Section 5.3.4).
- `"MUSCSANM"` for integrity-anomaly identifiers (Section 5.3.3). Anomalies are core, not an extension concern, so their tag is built in alongside the other two.
- Additional domain tags introduced by registered extensions **MUST** begin with `"MUSCS"`, have length exactly 8 bytes, and **MUST NOT** collide with the three reserved tags above.

Two replicas reducing the same operation set **MUST** derive identical system identifiers from identical canonical inputs.

5.3.3 System-Derived Counter Collisions

The 64-bit counter portion of a system-derived identifier is derived by BLAKE3 truncation. Truncation to 64 bits offers sufficient collision resistance against accidental collision (roughly 2^{32} system-derived identifiers of the same kind before a 50% chance of any pair colliding), but is weaker than the rest of the identifier story and is potentially vulnerable to deliberate construction of colliding canonical inputs. A collision means the identity layer has failed: canonical state derived from a colliding identifier cannot be trusted.

```
/// An integrity anomaly that takes the document out of ordinary
/// canonical operation. Distinct from ConflictRecord (an
/// ordinary canonical-state fact): integrity anomalies indicate
/// that the structural assumptions underlying canonical state
/// have failed.
pub struct IntegrityAnomaly {
    pub id: IntegrityAnomalyId,
    pub kind: IntegrityAnomalyKind,
}

pub enum IntegrityAnomalyKind {
    /// A system-derived identifier counter collision.
    SystemIdentifierCollision {
```

```
        kind: ObjectKind,
        colliding_counter: u64,
        input_set_a: SerializedCanonicalInputs,
        input_set_b: SerializedCanonicalInputs,
    },
    /// An OperationSlot is Equivocated. The slot's candidates
    /// are referenced; reduction has excluded the operation.
    OperationSlotEquivocated {
        operation_id: OperationId,
    },
    /// A replica's stream has been quarantined per the HLC
    /// monotonicity rule. The AnomalousReplicaSegment is
    /// referenced.
    ReplicaStreamQuarantined {
        replica: ReplicaId,
        first_bad_counter: u64,
    },
    /// A registered anomaly defined by an extension or
    /// transport.
    Registered(IntegrityAnomalyRegistryId),
}

/// The kind of object whose system-derived identifier collided.
/// Open vocabulary, but intentionally narrower than TypedObjectId:
/// only the kinds actually minted into the system-derived namespace
/// appear here.
pub enum ObjectKind {
    Voice,    // system-promoted voices (MUSCSVCE)
    Pitch,    // content-derived synthetic pitches (MUSCSPCH)
    Registered(OperationKindRegistryId),
}
```

REQUIREMENT 5.5

An `IntegrityAnomalyId` **MUST** be derived by the system-derived identifier function of Section 5.3.2 with the reserved domain tag `"MUSCSANM"` over the anomaly kind's canonical bytes: `derive_system_id::<IntegrityAnomalyId>(b"MUSCSANM", &kind.to_canonical_bytes())`. Because the identity is content-derived from the kind, two replicas observing the same structural failure derive the same anomaly identifier and therefore agree on anomaly identity across the network.

REQUIREMENT 5.6

`ObjectKind` (the kind field of `SystemIdentifierCollision`) is an *open* vocabulary whose normative core set is exactly `Voice` and `Pitch`, with a `Registered` escape for extension-introduced system-derived kinds. It is deliberately narrower than `TypedObjectId`'s 28-kind table (Requirement 5.2): `ObjectKind` enumerates only the kinds that are actually minted into the `ReplicaId::SYSTEM_DERIVED` namespace — today, promoted voices and synthetic pitches — because those are the only kinds for which a system-derived counter

collision is possible. A new core variant is added here only when a new object kind begins being minted into the system-derived namespace.
The discriminant byte of `ObjectKind` is its declaration-order index in the listing above: `Voice` $= 0$, `Pitch` $= 1$, `Registered` $= 2$. Because `ObjectKind` is the `kind` field of a `SystemIdentifierCollision` anomaly, this byte enters the canonical-input preimage of the resulting `IntegrityAnomalyId` (Requirement 5.5); two replicas observing the same structural failure agree on its identity only if they agree on this byte. The assignment is therefore normative and stable and **MUST NOT** be reordered.

REQUIREMENT 5.7

Implementations **MUST** perform a collision check during reduction whenever a new system-derived identifier is minted. If a newly derived 64-bit counter, within the same typed identifier kind and within the `ReplicaId::SYSTEM_DERIVED` namespace, collides with a previously-derived counter from *different* canonical inputs:

- An `IntegrityAnomaly` of kind `SystemIdentifierCollision` is recorded in the document's diagnostic integrity-anomaly register. This register is distinct from the canonical conflict registry: conflict records are ordinary canonical-state facts; integrity anomalies indicate the structural assumptions underlying canonical state have failed.
- Canonical reduction **MUST NOT** continue past the collision. The colliding object's identifier is held pending recovery: neither input set is allowed to occupy the collided counter in canonical state.
- Readers **MUST** open such a bundle in diagnostic recovery mode, surfacing the collision prominently and prohibiting ordinary editing of the affected portion of canonical state.
- Recovery is by external action: a profile-declared deterministic policy, a transport-level credential revocation, or an explicit user reconciliation operation. No automatic in-spec resolution is defined.

Authoring implementations **SHOULD** detect predictable collisions before authoring (e.g., by canonicalizing inputs with additional disambiguating data such as the causally-greatest operation visible at authoring time) and **SHOULD NOT** rely on collision frequency being negligible in adversarial scenarios.

RATIONALE

Moving system-derived collisions out of `ConflictKind` reflects the difference in kind between the two. A `ConflictRecord` is an ordinary canonical-state fact: the user has two valid musical edits that conflict, and the resolution is a normal editing action. A system-derived identifier collision is not a musical conflict; it is a structural failure of the identity layer. Treating it as a conflict would let canonical state grow under the collision, which is precisely the wrong behavior for an integrity failure. Treating it as a diagnostic anomaly halts canonical growth until external recovery resolves the structural issue.

5.3.4 System-Derived Pitch Identity

When promotion logic introduces a synthetic pitch (rare), its `PitchId` is content-addressed from the pitch's intrinsic identity rather than allocated from a replica counter, so that all replicas mint the same identifier for the same synthetic pitch.

REQUIREMENT 5.8

A system-derived `PitchId` **MUST** be derived by the function of Section 5.3.2 with domain tag `"MUSCSPCH"` over a fixed canonical byte form of the pitch's *intrinsic identity*, defined as the pair (*scale position*, *acoustic realization*). All variable-width string components of this preimage (nominal names, accidental and tuning catalog references) **MUST** be length-prefixed and **MUST** be NFC-normalized at the derivation boundary, so that canonically-equivalent Unicode spellings derive the same identifier.
The acoustic realization's *tuning reference* is always part of intrinsic identity, including the `TuningReference::Inherit` case: `Inherit` is encoded as a distinct presence marker, not as the absence of a field. Consequently a synthetic pitch that inherits its tuning and one that names an explicit tuning are *distinct* identities even when they would resolve to the same frequency in context. This makes synthetic-pitch identity stable under later changes to the inherited tuning environment, and aligns Invariant 11's uniqueness check with a fixed, context-free preimage.

5.3.5 Identifier Stability

REQUIREMENT 5.9

An identifier, once assigned, **MUST** never be reassigned, even after deletion of the corresponding object. Stale identifiers encountered in stored references **MUST** be detected and reported by re-anchoring rules (see Chapter 6); they **MUST NOT** silently match a different object.

5.4 The Event Arena

Events are the rhythmic atoms of the score: pitched events, unpitched events, rests, indeterminate events, trajectories, graphic events, and cues. All events of all kinds in a score are stored in a single arena.

```rust
pub struct EventArena {
    /// Storage backing for events. Implementation-defined; the
    /// observable contract is O(1) lookup by EventId and stable
    /// addresses for the lifetime of the event.
    storage: ArenaStorage<Event>,

    /// Auxiliary index for fast time-range queries.
    time_index: EventTimeIndex,
}
```

REQUIREMENT 5.10

Event lookup by `EventId` **MUST** be $O(1)$ amortized. Implementations **MAY** use any storage layout meeting this contract; vector-of-events with stable indices, slot-allocator schemes, and generational arenas are all acceptable.

5.4.1 The Event Type

```
pub enum Event {
    Pitched(PitchedEvent),
    Unpitched(UnpitchedEvent),
    Rest(Rest),
    Indeterminate(IndeterminateEvent),
    Trajectory(TrajectoryEvent),
    Graphic(GraphicEvent),
    Cue(CueEvent),
}
```

Every event variant carries:

- Its own `EventId`.
- Its *voice membership*: a `VoiceId` identifying the unique voice that owns it.
- Its *position* within that voice (either a `MusicalPosition` or `WallClockTime`, depending on the enclosing region's time model).

Voice membership and position are stored on the event so that operations addressing an event can resolve its context without traversing parent containers.

Event Position and Duration

An event's temporal coordinates use union types matching its enclosing region's time model. Position is unioned over musical and wall-clock forms; duration is unioned over musical, wall-clock, and indeterminate forms.

```
pub enum EventPosition {
    Musical(MusicalPosition),
    WallClock(WallClockTime),
}

pub enum EventDuration {
    Musical(MusicalDuration),
    WallClock(WallClockDuration),
    Indeterminate(DurationBounds),
}

/// Bounded interval expressing an indeterminate duration. Bounds are
/// concrete (not themselves indeterminate); this prevents recursive
/// indeterminacy in the type system.
pub struct DurationBounds {
    pub lower: Option<ConcreteDuration>,
```

```
    pub upper: Option<ConcreteDuration>,
}

pub enum ConcreteDuration {
    Musical(MusicalDuration),
    WallClock(WallClockDuration),
}
```

REQUIREMENT 5.11

Event position and duration variants **MUST** agree with the time model of the enclosing region:

- Metric regions accept `EventPosition::Musical` and `EventDuration::Musical` only.
- Proportional regions accept `EventPosition::WallClock` and `EventDuration::WallClock` only.
- Aleatoric regions **MAY** accept any combination consistent with the region's declared anchoring discipline (see Section 3.10.3).

Implementations **MUST** reject events whose coordinate variants contradict their enclosing region's time model.

REQUIREMENT 5.12

If both bounds of a `DurationBounds` are present, they **MUST** use the same `ConcreteDuration` variant, unless the enclosing aleatoric region's anchoring discipline explicitly permits mixed-kind bounds.

Identified Pitches

Pitches embedded in events are wrapped in `IdentifiedPitch` records that pair each pitch with a stable `PitchId` (Section 2.6). The identifier enables spelling attachments, respelling operations, pitch-level reduction rules under concurrent edits, and stable tie pairing through chord reordering.

```
pub struct IdentifiedPitch {
    pub id: PitchId,
    pub pitch: Pitch,
}
```

REQUIREMENT 5.13

Every pitch embedded in an event **MUST** be wrapped in an `IdentifiedPitch`. The contained `PitchId` **MUST** be unique within the score's pitch-identity index. Pitch identifiers **MUST** be minted atomically with their containing event: the operation creating an event mints `EventId` and the `PitchId`s of all embedded pitches in a single causal step.

Pitched Events

```rust
pub struct PitchedEvent {
    pub id: EventId,
    pub voice: VoiceId,
    pub position: EventPosition,
    pub duration: EventDuration,

    /// One or more identified pitches. A single pitch is a
    /// single-note event; multiple pitches are a chord.
    pub pitches: Vec<IdentifiedPitch>,

    /// The marks the event carries, a canonical set (schema
    /// major 5): articulations, bowings, technical, brass and jazz
    /// marks, tremolo and arpeggio.
    pub marks: Vec<EventMark>,

    /// Reserved: a dynamic is a point marker (Marker, kind
    /// Dynamic), not an event field, so no writer fills this
    /// slot, and a decoder refuses a filled one by name.
    pub dynamic: Option<DynamicMark>,

    /// Ornaments attached to the event, a canonical set (schema
    /// major 5). Ornament resolutions (the realized notes of a
    /// trill, mordent, etc.) are a playback concern.
    pub ornaments: Vec<Ornament>,

    /// Stem configuration: direction, length adjustment, hidden.
    pub stem: StemConfiguration,

    /// Whether this event is a grace note, and if so how it is
    /// notated (schema major 5).
    pub grace: Option<Grace>,
}

/// A mark an event carries (schema major 5). Growth is by appended
/// variant.
pub enum EventMark {
    Staccato, Staccatissimo, Spiccato, Tenuto, DetachedLegato,
    Accent, Marcato, Stress, Unstress,
    UpBow, DownBow, Harmonic, OpenString, Stopped, SnapPizzicato,
    Scoop, Plop, Doit, Falloff,
    /// A single-note tremolo, its strokes through the stem.
    Tremolo { strokes: u8 },
    /// A two-note tremolo with the next event of the voice.
    TremoloWithNext { strokes: u8 },
    /// A chord's arpeggio.
    Arpeggio { direction: ArpeggioDirection },
}

pub enum ArpeggioDirection { Plain, Up, Down }

/// An ornament with the accidentals shown above and below it.
```

```rust
pub struct Ornament {
    pub kind: OrnamentKind,
    pub accidental_above: Option<AccidentalId>,
    pub accidental_below: Option<AccidentalId>,
}

pub enum OrnamentKind { Trill, Mordent, InvertedMordent, Turn, InvertedTurn }

/// A grace note's notation: its kind, its notated value and dots
/// (its duration being zero), and its order among the graces at
/// its position.
pub struct Grace {
    pub kind: GraceKind,
    pub value: NoteValue,
    pub dots: u8,
    pub order: u16,
}

pub enum GraceKind {
    /// Slashed.
    Acciaccatura,
    Appoggiatura,
}
```

REQUIREMENT 5.14

A grace note (an event whose `grace` is present) **MUST** have a musical duration of zero, and an event whose `grace` is absent **MUST** have a positive one. A grace note occupies no voice time: it stands at an event's onset and never strictly inside another event's span in its voice, and a voice's events at one position stand with its graces first, in ascending `order` and then by identifier, before the event of positive duration there. Its principal is positional: a grace whose position holds no other event stands alone, drawn before that column.

REQUIREMENT 5.15

An event's `marks` and `ornaments` **MUST** be canonical sets: at most one of each kind, in ascending order of kind (the discriminant of an `EventMark`, the `kind` of an `Ornament`).

REQUIREMENT 5.16

A `PitchedEvent` **MUST** have at least one pitch. Empty pitch lists are forbidden; use `Rest` for the no-pitch case.

Unpitched Events

```rust
pub struct UnpitchedEvent {
    pub id: EventId,
```

```
    pub voice: VoiceId,
    pub position: EventPosition,
    pub duration: EventDuration,

    /// Staff line position for the unpitched event (e.g., snare
    /// drum at the middle line of a 5-line staff, or position 0
    /// for a single-line staff).
    pub staff_position: StaffPosition,

    /// The unpitched instrument identifier, for sound mapping.
    pub instrument_member: UnpitchedMemberId,

    pub marks: Vec<EventMark>,
    pub dynamic: Option<DynamicMark>,
    pub stem: StemConfiguration,
    pub grace: Option<Grace>,
}
```

Rests

```
pub struct Rest {
    pub id: EventId,
    pub voice: VoiceId,
    pub position: EventPosition,
    pub duration: EventDuration,

    /// Optional explicit vertical positioning. If None, the engraver
    /// chooses based on voice and meter context.
    pub vertical_position: Option<StaffPosition>,

    /// Whether this rest is visible. Invisible rests preserve timing
    /// but do not render.
    pub visible: bool,
}
```

Indeterminate Events

```
pub struct IndeterminateEvent {
    pub id: EventId,
    pub voice: VoiceId,
    pub position: EventPosition,
    pub duration: EventDuration,

    /// What aspect of the event is indeterminate.
    pub indeterminacy: IndeterminacyKind,

    /// Optional hints constraining the indeterminacy.
    pub hints: IndeterminacyHints,
}

pub enum IndeterminacyKind {
```

```
    /// "Any pitch in this range/contour."
    Pitch,

    /// "Any duration within these bounds." (Realized via
    /// EventDuration::Indeterminate on the event itself.)
    Duration,

    /// "Any of these alternatives."
    Choice,

    /// Combination of multiple indeterminacies.
    Compound(Vec<IndeterminacyKind>),
}

pub struct IndeterminacyHints {
    pub pitch_range: Option<PitchRange>,
    pub pitch_contour: Option<ContourHint>,
    pub density: Option<DensityHint>,
    pub textual_instruction: Option<String>,
    pub duration_bounds: Option<DurationBounds>,
    pub alternatives: Vec<EventId>,
}
```

Trajectory Events

A trajectory event represents a continuous pitch motion: a glissando, portamento, or pitch bend.

```
pub struct TrajectoryEvent {
    pub id: EventId,
    pub voice: VoiceId,
    pub position: EventPosition,
    pub duration: EventDuration,

    /// Start and end pitch endpoints. Either may reference another
    /// event's pitch by id, or may carry an explicit identified
    /// pitch local to the trajectory.
    pub start: TrajectoryEndpoint,
    pub end: TrajectoryEndpoint,

    /// Shape of the trajectory between endpoints.
    pub shape: TrajectoryShape,

    /// Visual representation.
    pub display: TrajectoryDisplay,
}

pub enum TrajectoryEndpoint {
    /// Reference to a pitch belonging to another event.
    EventPitch(PitchId),

    /// An explicit identified pitch local to this trajectory.
```

```
    /// Such pitches participate in spelling, identity, and editing
    /// the same way embedded pitches do.
    ExplicitPitch(IdentifiedPitch),
}

pub enum TrajectoryShape {
    Linear,
    Exponential,
    Curve(Vec<TrajectoryControlPoint>),
    Stepwise(Vec<IdentifiedPitch>),
}
```

Graphic Events

A graphic event is an event whose primary content is graphic (drawn or constructed) rather than notated through the standard staff system. Used within hybrid and free-graphic regions for events that nevertheless occupy a rhythmic slot in a voice.

```
pub struct GraphicEvent {
    pub id: EventId,
    pub voice: VoiceId,
    pub position: EventPosition,
    pub duration: EventDuration,

    /// Reference to the graphic object(s) realizing this event.
    pub graphics: Vec<GraphicObjectId>,

    /// Optional playback parameter bindings (see
    /// Section~\ref{sec:graph:timebinding}).
    pub playback_bindings: Vec<PlaybackBinding>,
}
```

Cue Events

A cue is a small-print rendering of music from another voice or instrument, displayed as a guide.

```
pub struct CueEvent {
    pub id: EventId,
    pub voice: VoiceId,
    pub position: EventPosition,
    pub duration: EventDuration,

    /// The source events being cued.
    pub source: Vec<EventId>,

    /// Rendering hints: how the cue should be drawn (size, label,
    /// abbreviation of the source instrument's name, etc.).
    pub rendering: CueRendering,
}
```

5.5 The Canvas

The canvas is the spatial root of the score. It contains regions.

The score root and the canvas remain *structural givens*, not operation products (ratified Pass 12; narrowed by `spec/RULING_GENESIS_PERSISTENCE.md`): neither object is ever minted, addressed, or deleted by an operation, and there is still no `TypedObjectId` kind for either. This is deliberate — a document that could mint its own root would admit a genesis race under concurrent editing, and every operation would need a defined semantics against a not-yet-existing root. The decision may be revisited only if an addressable multi-canvas model is adopted at a future schema major.

What the genesis ruling reverses is the narrower claim this paragraph used to make alongside the above: that the score's *contents* also arrive outside the operation set, as part of the same empty-score-plus-bundle genesis act. They do not. Every mutable field of `Score` becomes operation-authored under the genesis tranche (`spec/PLAN_GENESIS_OPS.md`): a document created empty (`Score::empty`) and given only operations can materialize a note-bearing score, reaching `Canvas.layout_defaults` and `Score.spelling_precedence` (genesis tranche G2a) among other surfaces. The root and the canvas themselves are the load-bearing exception that survives intact.

```rust
pub struct Canvas {
    /// Regions in this canvas. Each region declares its time and
    /// content models and occupies a time and staff extent.
    pub regions: Vec<Region>,

    /// Canvas-level configuration: paper size, margins, default
    /// system layout, page-break behavior.
    pub layout_defaults: CanvasLayoutDefaults,
}

/// Canvas-level layout defaults. Page size and margins are in staff
/// spaces; a solver reads them as its default page geometry and MAY
/// override per solve. The type is defined with schema major 1 (its field
/// was named on the canvas from the first draft; only its type was
/// undefined).
pub struct CanvasLayoutDefaults {
    pub page_size: CanvasSize,
    pub margins: CanvasMargins,
}

/// A page size in staff spaces (1 staff space = staff height / 4).
pub struct CanvasSize {
    pub width: f64,
    pub height: f64,
}

/// Page margins in staff spaces.
pub struct CanvasMargins {
    pub top: f64,
    pub right: f64,
    pub bottom: f64,
```

```
    pub left: f64,
}
```

The default is **A4 portrait at an 8 mm staff** (1 staff space = 2 mm): page 105×148.5 staff spaces, 7.5-staff-space margins, hence a 90×133.5 content area. Each staff-space scalar is realized canonically as a `CanonicalF64` leaf; the reference engraver maps a `CanvasLayoutDefaults` onto its internal page geometry. This closes the long-standing gap in which the canvas named `layout_defaults` but no chapter defined `CanvasLayoutDefaults`.

5.5.1 Regions

```rust
pub struct Region {
    pub id: RegionId,

    /// Time model: metric, proportional, or aleatoric.
    pub time_model: RegionTimeModel,

    /// Content model: staff-based, free graphic, or hybrid.
    pub content: RegionContent,

    /// Range in time that this region occupies.
    pub time_extent: TimeExtent,

    /// Range in vertical staff space that this region occupies.
    /// A region may cover all staves (full-score region) or a subset
    /// (a free-graphic gesture occupying only one staff's vertical
    /// band, for example).
    pub staff_extent: StaffExtent,

    /// Optional tempo and tuning overrides scoped to this region.
    pub local_tempo_map: Option<TempoMap>,
    pub local_tuning_overrides: Vec<TuningOverride>,

    /// Whether spanners (slurs first of all) may cross this region's
    /// boundary into an adjacent region. Default false: cross-region
    /// spanning is not permitted unless a region opts in. Added with
    /// schema major 1.
    pub permits_spanning_slurs: bool,
}
```

For a slur (or other spanner) whose endpoints lie in *different* regions, permission is governed conjunctively (ratified Pass 12): the boundary is permeable only when *both* endpoint regions set `permits_spanning_slurs` — a region that forbids spanning is never crossed against its declaration, regardless of which side the spanner starts on. The check is an authoring-time advisory only; it never alters canonical reduction.

```rust
pub enum RegionContent {
    /// Staff-based notation: one or more staves carrying voices and
    /// events.
    StaffBased(StaffBasedContent),
```

```
    /// Free graphic content: no staves, only graphic objects placed
    /// in the region's coordinate space.
    FreeGraphic(GraphicContent),

    /// Hybrid: staves with overlaid graphic content. The graphic
    /// layer is rendered above (or below, configurably) the staff
    /// layer.
    Hybrid {
        staves: StaffBasedContent,
        overlay: GraphicContent,
        overlay_below_staves: bool,
    },
}

pub struct TimeExtent {
    pub start: TimeAnchor,
    pub end: TimeAnchor,
}

pub struct StaffExtent {
    /// Identifiers of the globally-identified staves whose content
    /// this region carries. A region carries one StaffInstance per
    /// listed StaffId. The same StaffId may appear in the
    /// staff_extent of adjacent regions; this is the mechanism by
    /// which a staff's identity persists across regions.
    pub staves: Vec<StaffId>,
}
```

5.5.2 Region Overlap and Concurrency

REQUIREMENT 5.17

Regions **MAY** overlap in time if their staff extents are disjoint. Regions **MUST NOT** overlap in both time and staff extent. The graph's construction **MUST** reject configurations violating this constraint.

RATIONALE

Concurrent regions on disjoint staves are essential for the hybrid notation feature: a metric staff and a proportional staff sounding simultaneously are two regions with overlapping time extents but disjoint staff extents. Overlapping time *and* staff would mean two regions claim the same content, which is ambiguous.

5.5.3 Staff-Based Content

A staff-based region carries one or more staff *instances*, each of which manifests a globally-identified staff for the duration of the region. The score's metric organization is per-staff (admitting polymeter); barline alignment between staves is recorded as an explicit relationship

when meters coincide.

```rust
pub struct StaffBasedContent {
    /// Staff instances appearing in this region. Each references a
    /// globally-identified Staff and provides region-local content.
    pub staff_instances: Vec<StaffInstance>,

    /// Default metric grid for this region. Staves without their own
    /// local_metric_grid inherit this grid. None means the region
    /// has no default metric organization (acceptable for transitional
    /// regions or for regions where every staff declares its own).
    pub default_metric_grid: Option<MetricGrid>,

    /// Explicit barline alignment groups: places where measures
    /// across multiple staves are declared to coincide. Used for
    /// engraving barlines that span multiple staves under polymeter
    /// and for analytical purposes. Implementations MAY derive
    /// some alignment groups automatically from coincident measure
    /// starts; explicit groups are authoritative.
    pub barline_alignment_groups: Vec<BarlineAlignmentGroup>,

    /// System breaks declared by the user. The engraver may
    /// override these if region settings permit (see Chapter 6).
    pub user_system_breaks: Vec<TimeAnchor>,

    /// Page breaks declared by the user. Same override rule.
    pub user_page_breaks: Vec<TimeAnchor>,
}

/// A region-local manifestation of a globally-identified Staff.
pub struct StaffInstance {
    pub id: StaffInstanceId,

    /// The globally-identified staff this instance manifests.
    pub staff: StaffId,

    /// Voices appearing on this staff instance.
    pub voices: Vec<Voice>,

    /// Clef changes within this instance.
    pub clef_sequence: Vec<ClefChange>,

    /// Key signature changes within this instance.
    pub key_sequence: Vec<KeySignatureChange>,

    /// Optional local metric grid: if Some, overrides the region's
    /// default_metric_grid for this staff (polymetric case).
    /// If None, the staff inherits the region's default_metric_grid.
    pub local_metric_grid: Option<MetricGrid>,

    /// Measures belonging to this staff instance, in order. Measures
    /// are per-staff (admitting polymeter); alignment across staves
    /// is recorded in barline_alignment_groups.
```

```rust
    pub measures: Vec<Measure>,

    /// Optional per-instance instrument override. None means inherit
    /// from the global Staff's declared instrument.
    pub instrument_override: Option<InstrumentId>,

    /// Optional per-instance staff line configuration override.
    /// None means inherit from the global Staff's default.
    pub staff_lines_override: Option<StaffLineConfiguration>,

    /// Whether this instance is visible. Hidden instances preserve
    /// content but are not rendered (used for empty-staff suppression).
    pub visible: bool,
}

/// The clef content model (ratified schema major 2; the value types
/// landed with the visible slice and supersede the earlier ClefId
/// identifier sketch). A clef is a value, not a registry reference.
pub enum ClefShape {
    /// G clef (treble family) -- reference pitch G4.
    G,
    /// F clef (bass family) -- reference pitch F3.
    F,
    /// C clef (alto / tenor family) -- reference pitch middle C (C4).
    C,
    /// Unpitched percussion clef -- no diatonic reference.
    Percussion,
}

pub struct Clef {
    pub shape: ClefShape,
    /// The staff line (1 = bottom) the shape's reference pitch sits on.
    pub line: i8,
    /// Signed octave displacement (e.g., -1 for the vocal tenor G clef).
    pub octave_shift: i8,
}

/// A key signature on the circle of fifths, validated to the
/// conventional CMN range -7 (seven flats) ..= +7 (seven sharps).
/// Constructed only through a checked constructor; decoders reject
/// out-of-range values.
pub struct KeySignature {
    fifths: i8, // private; KeySignature::new rejects |fifths| > 7
}

/// A clef change at a point in a staff instance.
pub struct ClefChange {
    pub anchor: TimeAnchor,
    pub clef: Clef,
}

/// A key-signature change at a point in a staff instance.
pub struct KeySignatureChange {
```

```
    pub anchor: TimeAnchor,
    pub key: KeySignature,
}

/// A per-staff (or per-region-default) metric organization.
pub struct MetricGrid {
    /// Sequence of meter changes within the scope of this grid.
    pub meter_sequence: Vec<MeterChange>,
}

pub struct MeterChange {
    /// Where this meter takes effect, expressed as an anchor within
    /// the enclosing staff or region.
    pub anchor: TimeAnchor,

    /// The active time signature from this point until the next
    /// MeterChange or the end of the grid's scope.
    pub time_signature: TimeSignatureId,
}

/// A declaration that the measure boundaries of two or more staff
/// instances coincide at a particular point.
pub struct BarlineAlignmentGroup {
    pub id: BarlineAlignmentGroupId,

    /// The staff instances participating in this alignment, paired
    /// with the measures whose boundaries coincide.
    pub members: Vec<BarlineAlignmentMember>,
}

pub struct BarlineAlignmentMember {
    pub staff_instance: StaffInstanceId,
    pub measure: MeasureId,
    pub position: MeasurePosition,
}

pub struct Measure {
    pub id: MeasureId,
    pub start: TimeAnchor,
    pub time_signature: Option<TimeSignatureId>,
    pub explicit_number: Option<u32>,
    pub number_visibility: MeasureNumberVisibility,
}
```

REQUIREMENT 5.18

Measures **MUST** belong to a single `StaffInstance`, not to the enclosing region. This admits polymeter: different staves in the same region **MAY** carry different meter sequences and consequently different measure boundaries. Barline coincidences across staves **MUST** be expressed via `BarlineAlignmentGroup` entries when explicit alignment is authoritative; otherwise alignment is derived at engraving time from coincident

measure-start positions.

5.6 Staves, Voices, and Instruments

5.6.1 Instruments

An instrument is an abstract definition of a sounding entity.

```rust
pub struct Instrument {
    pub id: InstrumentId,
    pub name: String,
    pub abbreviation: Option<String>,

    /// MIDI program, sound library mapping, or unpitched instrument
    /// definitions. The structure is detailed in the audio engine
    /// specification (out of scope here); the core stores the
    /// configuration opaquely.
    pub sound_config: SoundConfiguration,

    /// Default transposition (e.g., B-flat clarinet sounds a major
    /// second below written).
    pub transposition: Option<TranspositionInterval>,

    /// Default playable range. Engraving may flag out-of-range
    /// pitches; not a hard constraint.
    pub range: Option<PitchRange>,

    /// Default clef and staff line configuration for staves of this
    /// instrument. The clef is the content-bearing Clef value defined
    /// in this chapter (shape, line, octave shift) -- the value model
    /// ratified with the visible slice supersedes the earlier ClefId
    /// identifier sketch (reconciled schema major 2).
    pub default_clef: Clef,
    pub default_staff_lines: StaffLineConfiguration,

    /// For unpitched instruments: definitions of each playable
    /// member (snare, kick, ride bell, etc.) and their staff
    /// positions.
    pub unpitched_members: Vec<UnpitchedMember>,
}

/// Opaque sound configuration (defined schema major 2). The audio
/// engine specification owns the structure; the core stores the bytes
/// verbatim and never interprets them. Empty is the default.
pub struct SoundConfiguration(pub Vec<u8>);

// TranspositionInterval (defined schema major 2) is specified in
// Chapter 2, "Transposition and the Interval Type": the diatonic and
// chromatic step counts of an interval, and its action on a scale
// position. Instrument.transposition is the written-versus-sounding
// interval of a transposing instrument (a B-flat clarinet is
```

```
// -1 diatonic, -2 chromatic).
//
// Its ACTION is pinned (Chapter 2). What remains unimplemented is its
// AUTOMATIC APPLICATION at the instrument boundary: nothing in the core
// yet respells a written part into a sounding one, or resolves either to
// a frequency. Instrument.transposition is therefore still advisory --
// but for that reason, not for want of interval algebra.

/// One playable member of an unpitched instrument (defined schema
/// major 2): the identifier UnpitchedEvent.instrument_member
/// references, its display name, and its default staff position.
pub struct UnpitchedMember {
    pub member: UnpitchedMemberId,
    pub name: String,
    pub staff_position: StaffPosition,
}
```

`UnpitchedMemberId` is an *instrument-scoped small value* (a plain 32-bit integer), not a member of the 128-bit typed-identifier family — it is minted by the author of the instrument definition, not by replicas, and participates in no content derivation. Member values *should* be unique within one instrument's `unpitched_members`; the authoring advisory layer flags a duplicate, and resolution is by first match in list order (canonical bytes preserve the authored order), so a duplicate is deterministic, merely shadowed. An `UnpitchedEvent.instrument_member` with no matching entry resolves to *no member definition*: the event still renders (its own `staff_position` field governs placement — the member's `staff_position` is the *authoring default* copied onto new events, not a render-time override), and sound mapping falls back to the instrument's `sound_config`. This no-match tolerance is load-bearing: every pre-major-2 instrument has an empty member list while its events carry member values. `StaffPosition`'s vertical convention (which integer is which line, and its orientation) remains *deliberately deferred* to the Chapter 7 vertical model, with the unpitched-rendering tranche as its landing site; until then the field is preserved and compared verbatim, and no implementation may claim conformant unpitched *rendering*.

5.6.2 Staves: Identity Versus Instance

The specification distinguishes two staff concepts:

`Staff` is the *global, abstract* staff identity persisting across the entire score. A staff is the thing a performer is reading: the "Flute 1" staff, the "Piano upper staff." A staff is declared once in the score and is referenced by staff instances in any region where it appears.

`StaffInstance` is the *region-local manifestation* of a staff: the voices, measures, clef changes, key changes, and metric grid that belong to that staff for the duration of one region. A single `Staff` may have multiple `StaffInstance`s across the score; each instance belongs to exactly one region.

```
pub struct Staff {
    pub id: StaffId,

    /// Human-readable name (e.g., "Flute 1", "Violin II").
    pub name: String,
```

```
    /// Optional abbreviated name for subsequent systems.
    pub abbreviation: Option<String>,

    /// The instrument this staff renders by default. May be
    /// overridden per-instance.
    pub instrument: InstrumentId,

    /// Default clef for new instances of this staff (the
    /// content-bearing Clef value defined in this chapter; reconciled
    /// schema major 2).
    pub default_clef: Clef,

    /// Default staff line configuration.
    pub default_staff_lines: StaffLineConfiguration,

    /// Visual grouping: which staff group (if any) this staff
    /// belongs to (e.g., piano grand staff, choral group). The sole
    /// authority for group membership (P13–S16, ratified disposition A;
    /// supersedes genesis tranche G3a's disposition B): every consumer
    /// \MUST{} read membership from this field, and writing it is the only
    /// way membership changes. \texttt{StaffGroup.members} above is a
    /// projection maintained from this field by reduction; the two \MUST{}
    /// agree in both directions, which graph invariant~21 enforces. Under
    /// disposition B that projection was authored directly and MAY have
    /// disagreed with this field in either direction.
    pub group: Option<StaffGroupId>,
}

pub struct StaffLineConfiguration {
    pub line_count: u8,
    pub line_spacing: SpaceUnit,
    pub line_style: LineStyle,
    pub bracket: Option<StaffBracketKind>,
}

/// A dimension in staff spaces (defined schema major 2). Staff line
/// spacing is relative to the global staff space; 1.0 is a
/// normal-size staff, smaller values yield cue/ossia staves.
pub struct SpaceUnit(pub CanonicalF64);

/// A line drawing style (defined schema major 2). Shared by staff
/// lines and the slur/tie/spanner style records below.
pub enum LineStyle {
    Solid,
    Dashed,
    Dotted,
    /// A trill's or a glissando's wave (schema major 5).
    Wavy,
}

/// A per-staff bracket adornment (defined schema major 2), distinct
/// from StaffGroup-level bracketing.
```

```rust
pub enum StaffBracketKind {
    Brace,
    Bracket,
}
```

The score's top-level structure carries a list of `Staff` objects alongside its list of `Instrument` objects; each region's staff instances reference these by `StaffId`.

REQUIREMENT 5.19

Every `StaffInstance.staff` **MUST** resolve to a `Staff` declared at the score level. A single `StaffId` **MAY** be referenced by multiple `StaffInstance`s in different regions: this is how a staff maintains continuous identity across the score's regions.
A `StaffInstance` **MUST** belong to exactly one region. The region's `staff_instances` list **MUST** contain the instance, and the instance's content is owned by that region.
Continuous attachments (e.g., key signatures, clef changes, metric grids) **MAY** appear on multiple staff instances of the same `Staff` when those instances are adjacent in time; the engraving pipeline reconstructs continuous staff appearance from per-instance content.

5.6.3 Voices

A voice is a polyphonic line within a staff instance. Voices hold ordered references to events; the events themselves live in the score's arena.

```rust
pub struct Voice {
    pub id: VoiceId,

    /// Ordered references to events in this voice, sorted by
    /// position. Each event has voice == this voice's id.
    pub events: Vec<EventId>,

    /// Default stem direction for this voice. Common conventions:
    /// voice 1 stems up, voice 2 stems down on the same staff.
    pub default_stem_direction: Option<StemDirection>,

    /// Whether this voice is the staff's primary voice (for purposes
    /// of rest placement, beam direction defaults, and similar
    /// engraving decisions).
    pub is_primary: bool,

    /// Provenance of this voice: user-declared, imported, or
    /// system-promoted by concurrent-edit conflict resolution.
    pub origin: VoiceOrigin,
}

pub enum VoiceOrigin {
    /// Created by an explicit user action.
    UserDeclared,

    /// Imported from a foreign format.
```

```
    Imported { format: ForeignFormatId },

    /// System-promoted to resolve a concurrent-edit collision.
    /// The original_voice is the voice into which the user had
    /// intended to insert; the promoted voice carries the event
    /// authored by losing_operation.
    SystemPromoted {
        winning_operation: OperationId,
        losing_operation: OperationId,
        original_voice: VoiceId,
    },
}
```

REQUIREMENT 5.20

The number of voices per staff instance **MUST** be unbounded in the data model. Engraving heuristics (Chapter 7 and beyond) may flag visual issues with high voice counts; the graph admits any number.

REQUIREMENT 5.21

Every event **MUST** belong to exactly one voice. Voice membership is stored on the event (the `voice` field) and **MUST** agree with the membership of the voice's `events` list: for any event e in voice v's events list, e.`voice` **MUST** equal v.`id`.

System-Promoted Voices

When concurrent operations cause an `InsertEvent` collision that cannot be resolved by ordering alone (because both events have positive duration and would overlap within the target voice), the deterministically-losing event is placed in a newly allocated voice with `origin: VoiceOrigin::SystemPromoted`. This preserves the non-overlap invariant without rejecting user intent.

REQUIREMENT 5.22

The identifier of a system-promoted voice **MUST** be derived deterministically from a fixed function of:

1. The enclosing staff instance's `StaffInstanceId`.
2. The original target voice's `VoiceId`.
3. The winning operation's `OperationId`.
4. The losing operation's `OperationId`.

The derivation is the system-derived identifier function of Section 5.3.2 with domain tag `"MUSCSVCE"` over a fixed 64-byte preimage formed by concatenating the four identifiers in the order listed above, each as its 16-byte big-endian canonical form:

```
fn derive_promoted_voice_id(
```

```
    staff_instance: StaffInstanceId,
    original_voice: VoiceId,
    winning_op: OperationId,
    losing_op: OperationId,
) -> VoiceId {
    // 4 * 16 = 64 bytes, each component big-endian.
    let mut inputs = Vec::with_capacity(64);
    inputs.extend_from_slice(&staff_instance.canonical_bytes());
    inputs.extend_from_slice(&original_voice.canonical_bytes());
    inputs.extend_from_slice(&winning_op.canonical_bytes());
    inputs.extend_from_slice(&losing_op.canonical_bytes());
    derive_system_id::<VoiceId>(b"MUSCSVCE", &inputs)
}
```

The enclosing staff instance is recovered from graph containment at reduction time; the operation does not store it redundantly. Determinism is required so that all replicas independently arrive at the same promoted voice identity. This derivation is the same one whose result Invariant 18 (Section 5.11) verifies; the reducer that mints the voice and the verifier that checks it consume one derivation.

REQUIREMENT 5.23

The promotion rule generalizes from the pairwise case to any number of concurrent overlapping inserts into the same voice by an order-independent pre-pass:

1. Bucket the concurrent `InsertEvent` operations by their target `(staff_instance, original_voice)`.
2. Within a bucket, walk the operations in ascending `OperationId` order, maintaining a *retained set* of mutually non-overlapping inserts that stay in the original voice.
3. An operation whose inserted interval does not overlap any member of the retained set joins the retained set. An operation whose interval overlaps some retained member is a *loser* and is promoted; the lowest-`OperationId` retained member it overlaps is its `winning_operation`.

Because the walk is by `OperationId` and the retained set is built deterministically, every replica promotes the same operations and derives the same promoted-voice identities regardless of delivery order. "Overlap" is interval overlap of the half-open inserted spans `[start, end)`: the rule applies to *partial* overlaps, not only to inserts that share an identical start position. This is a deliberate widening of the pairwise rule's identical-start-position language; it is the correct reading because any positive-duration overlap breaks the voice non-overlap invariant (Invariant 3), not just a shared onset.

REQUIREMENT 5.24

System-promoted voices are first-class graph objects: they appear in the staff instance's `voices` list, they participate in cross-cutting structures, and they are visible to users. They are *not* hidden or transient. The layout pipeline (Chapter 7) **SHOULD** provide a visual indication that a voice was system-promoted until the user normalizes it (via

a later `NormalizePromotedVoice` or `MergeVoices` operation, specified in the operation catalog).
Promoted voices **MUST NOT** silently become user-authored voices; doing so would convert a CRDT conflict resolution into musical authorship without the user's consent.

REQUIREMENT 5.25

Within a voice, events **MUST** be sorted by position and **MUST NOT** overlap in time. Concurrent material on the same staff **MUST** be expressed via multiple voices.

5.7 Cross-Cutting Structures

Cross-cutting structures hold references into the tree (typically event identifiers) and represent musical phenomena that span multiple events. The registry is partitioned by type for both type safety and fast indexed lookup.

```rust
pub struct CrossCuttingRegistry {
    pub slurs: Vec<Slur>,
    pub ties: Vec<Tie>,
    pub beams: Vec<Beam>,
    pub tuplets: Vec<Tuplet>,
    pub spanners: Vec<Spanner>,
    pub markers: Vec<Marker>,
    pub repeats: Vec<RepeatStructure>,
    pub analytical: Vec<AnalyticalAnnotation>,
    pub comments: Vec<Comment>,
    pub graphic_gestures: Vec<GraphicGesture>,
    pub lyrics: Vec<Lyric>,
    pub chord_symbols: Vec<ChordSymbol>,
}
```

5.7.1 Slurs and Phrase Marks

```rust
pub struct Slur {
    pub id: SlurId,

    /// First and last events under the slur.
    pub start_event: EventId,
    pub end_event: EventId,

    /// Slur kind: legato slur, phrase mark, articulation slur,
    /// editorial.
    pub kind: SlurKind,

    /// Optional curvature override. The engraver computes default
    /// curvature; this field overrides it.
    pub curvature_override: Option<CurvatureOverride>,
```

```
    /// Visual style: solid, dashed, dotted; line thickness curve.
    pub style: SpanStyle,
}

/// The class of a slur (defined schema major 2). The v1-to-v2
/// migration default is Legato.
pub enum SlurKind {
    /// An ordinary legato slur.
    Legato,
    /// A phrase mark (typically longer, over sub-phrases).
    Phrase,
    /// An articulation slur (e.g., over a two-note sigh figure).
    Articulation,
    /// An editorial slur (rendered distinctly, e.g., dashed).
    Editorial,
}

/// Which side of the notes a curve arcs toward (defined schema
/// major 2).
pub enum CurveDirection {
    Above,
    Below,
}

/// An authored curvature override (defined schema major 2). The
/// engraver computes default curvature; each present field overrides
/// that component of it. Consumed by the Standard engraving tier;
/// stored and preserved at every tier.
pub struct CurvatureOverride {
    pub direction: Option<CurveDirection>,
    /// Arc height at the apex.
    pub height: Option<SpaceUnit>,
}

/// The visual style of a spanning mark (defined schema major 2): one
/// shared record for Slur.style, Tie.style, and Spanner.style.
/// Defaults: solid, engraver-chosen thickness.
pub struct SpanStyle {
    pub line: LineStyle,
    /// Line thickness; None = the engraver's default.
    pub thickness: Option<SpaceUnit>,
}
```

5.7.2 Ties

```
pub struct Tie {
    pub id: TieId,

    /// The two events being tied. The end event's pitch(es) must
    /// match the corresponding pitches in the start event.
    pub start_event: EventId,
    pub end_event: EventId,
```

```rust
    /// For chord ties: which pitches in the start event are tied
    /// to which in the end event, expressed by stable PitchId pairs.
    /// None means all pitches are tied by enharmonic matching in
    /// pitch-id-ascending order.
    pub pitch_pairing: Option<Vec<(PitchId, PitchId)>>,

    /// Tie class: determines the validation profile applied to this
    /// tie. The default Standard class requires same-voice immediate
    /// adjacency; other classes relax the rule for editorial,
    /// cross-voice, or notation-specific use.
    pub class: TieClass,

    /// Visual style (the shared SpanStyle record, schema major 2).
    pub style: SpanStyle,
}

pub enum TieClass {
    /// Standard CMN tie: same voice, immediate adjacency in voice
    /// order, pitches enharmonically equivalent.
    Standard,

    /// Editorial tie: connects pitches the editor considers
    /// continuous, spanning intervening rests or invisible events.
    /// Drawn with editorial style (dashed or bracketed) by default.
    Editorial,

    /// Cross-voice tie: connects pitches across voice boundaries
    /// within the same staff instance. Used for keyboard divisi,
    /// re-voicing across hands, and similar constructions.
    CrossVoice,

    /// Laissez-vibrer tie: a tie with no defined end event, drawn
    /// trailing from the start event. The end_event field is
    /// permitted to equal the start_event for this class, with
    /// pitch_pairing self-referential.
    LaissezVibrer,

    /// Registered class with custom validation behavior, defined
    /// by a notation grammar plugin.
    Registered(TieClassRegistryId),
}
```

RATIONALE

Pairing by stable `PitchId` rather than by positional index preserves tie identity through chord-reordering edits and through CRDT-style concurrent modification of the chord's pitch list. Index-based pairing would silently retarget when pitches were inserted, deleted, or reordered within a chord.

The tie class permits common editorial and contemporary-notation practices (editorial dashed ties spanning rests, cross-voice ties in keyboard scores, laissez-vibrer ties without

a notated end) to be expressed without relaxing the structural invariants of the standard tie.

REQUIREMENT 5.26

Tie validation is class-specific:

- `TieClass::Standard`: the start and end events **MUST** be in the same voice; the start event **MUST** immediately precede the end event in that voice's event sequence; the paired pitches **MUST** be enharmonically equivalent under the active tuning system.
- `TieClass::Editorial`: the start and end events **MUST** be in the same voice and the start **MUST** precede the end in voice order, but intervening events are permitted. Paired pitches **MUST** be enharmonically equivalent.
- `TieClass::CrossVoice`: the start and end events **MAY** be in different voices on the same staff instance. The start's resolved position **MUST** be less than or equal to the end's. Paired pitches **MUST** be enharmonically equivalent.
- `TieClass::LaissezVibrer`: the end event **MAY** equal the start event, in which case the tie has no notated destination. No adjacency requirement applies.
- `TieClass::Registered`: validation is defined by the registered class's contract; the registry **MUST** specify adjacency and pitch-equivalence requirements for the class.

5.7.3 Beams

```rust
pub struct Beam {
    pub id: BeamId,
    pub events: Vec<EventId>,
    pub level: u8,                  // primary beam level

    /// Sub-beam structure for inner subdivisions. Each entry
    /// describes a beam segment at a deeper level.
    pub sub_beams: Vec<SubBeam>,

    /// Beam angle and stem-length overrides.
    pub geometry_override: Option<BeamGeometryOverride>,
}

/// A beam segment at a deeper subdivision level (defined schema
/// major 2): a contiguous subset of the owning beam's events beamed
/// together at 'level' (strictly deeper than the owning beam's
/// primary level).
pub struct SubBeam {
    pub level: u8,
    pub events: Vec<EventId>,
}

/// An authored beam-geometry override (defined schema major 2). Each
/// present field overrides the engraver's computed geometry. Consumed
```

```rust
/// by the Standard engraving tier; stored and preserved at every tier.
pub struct BeamGeometryOverride {
    /// Beam slope: staff spaces of rise per staff space of run
    /// (dimensionless, hence not a SpaceUnit).
    pub slope: Option<CanonicalF64>,
    /// Vertical displacement of the beam from its default placement
    /// (positive = up).
    pub offset: Option<SpaceUnit>,
}
```

5.7.4 Spanners

A spanner is a generic spanning mark: hairpins, octave lines, pedal lines, trill extensions, and similar.

```rust
pub struct Spanner {
    pub id: SpannerId,
    pub kind: SpannerKind,
    pub start: TimeAnchor,
    pub end: TimeAnchor,

    /// Which staff or staves this spanner attaches to. Most spanners
    /// target a single staff; some (system-level dynamics, pedal
    /// lines spanning grand staff) target multiple.
    pub staves: Vec<StaffId>,

    /// Visual style (the shared SpanStyle record, schema major 2).
    pub style: SpanStyle,
}

pub enum SpannerKind {
    /// An unclassified spanning mark: renders as a plain line or
    /// bracket. First variant deliberately (schema major 2): it is
    /// the v1-to-v2 migration default -- a v1 spanner carried no
    /// kind, and Generic is the honest translation of that absence.
    Generic,
    Hairpin(HairpinDirection),
    OctaveLine(OctaveOffset),
    PedalLine(PedalKind),
    TrillExtension,
    Glissando,
    Portamento,
    TextLine(TextLineDefinition),
    Bracket(BracketKind),
    /// A pedal line drawn without its sign, the line alone (schema
    /// major 5); PedalLine is drawn with it.
    PedalBracket(PedalKind),
}

/// Hairpin orientation (defined schema major 2).
pub enum HairpinDirection {
    Crescendo,
```

```
    Diminuendo,
}

/// Octave-line displacement in signed octaves (defined schema
/// major 2): +1 = 8va, -1 = 8vb, +2 = 15ma, -2 = 15mb. Zero is
/// representable but degenerate; the authoring advisory layer flags
/// it, reduction does not.
pub struct OctaveOffset(pub i8);

/// Pedal-line kind (defined schema major 2).
pub enum PedalKind {
    Sustain,
    Sostenuto,
    UnaCorda,
}

/// A text line's content (defined schema major 2); the dash pattern
/// comes from the spanner's style.
pub struct TextLineDefinition {
    pub text: Text,
}

/// A bracket spanner's shape (defined schema major 2). Growth is by
/// appended variant.
pub enum BracketKind {
    Square,
}
```

5.7.5 Markers

A marker is a point mark (schema major 5): a dynamic, a fermata, a breath mark, a caesura, staff text, a tempo mark, a rehearsal mark, a segno or a coda. Its placement is the engraver's, not stored.

```
pub struct Marker {
    pub id: MarkerId,
    pub anchor: TimeAnchor,
    pub kind: MarkerKind,
}

/// Growth is by appended variant.
pub enum MarkerKind {
    Dynamic(Dynamic),
    Fermata(Fermata),
    Breath(BreathMark),
    Caesura(CaesuraMark),
    /// Staff text.
    Text(Text),
    /// Tempo text and an optional metronome mark, beside the tempo
    /// map's SetTempoSegment with no stored link.
    Tempo(TempoMark),
    Rehearsal(Text),
```

```
    Segno,
    Coda,
}

pub enum Dynamic {
    Ppppp, Pppp, Ppp, Pp, P, Mp, Mf, F, Ff, Fff, Ffff,
    Fffff, Ffffff, Fp, Sf, Sfz, Sffz, Sfp, Sfpp, Rf, Rfz, Fz, Niente,
    Other(Text),
}

pub struct Fermata { pub shape: FermataShape, pub inverted: bool }
pub enum FermataShape { Normal, Short, Long, VeryShort, VeryLong }
pub enum BreathMark { Comma, Tick }
pub enum CaesuraMark { Normal, Thick, Short, Curved }

pub struct TempoMark {
    pub text: Option<Text>,
    pub metronome: Option<Metronome>,
}

pub struct Metronome {
    pub beat: NoteValue,
    pub dots: u8,
    pub per_minute: Text,
}
```

5.7.6 Lyrics

A lyric is one syllable on an event (schema major 5).

```
pub struct Lyric {
    pub id: LyricLineId,
    pub event: EventId,
    /// The verse, from 1.
    pub verse: u16,
    pub text: Text,
    pub syllabic: Syllabic,
    /// Whether an extender line follows the syllable.
    pub extension: bool,
}

pub enum Syllabic { Single, Begin, Middle, End }

/// Text held to Unicode Normalization Form C.
pub struct Text(String);
```

REQUIREMENT 5.27

An event **MUST** carry at most one `Lyric` per verse.

REQUIREMENT 5.28

Every `Text` value **MUST** be in Unicode Normalization Form C; a decoder **MUST** reject any other form.

5.7.7 Repeat Structures

```rust
pub struct RepeatStructure {
    pub id: RepeatStructureId,
    pub kind: RepeatKind,
    pub start: TimeAnchor,
    pub end: TimeAnchor,

    /// For voltas: which endings apply on which passes.
    pub voltas: Vec<Volta>,
}

pub enum RepeatKind {
    SimpleRepeat { count: u32 },
    DaCapo { end_target: TimeAnchor },
    DalSegno { segno: TimeAnchor, end_target: TimeAnchor },
    Volta,
}

/// One volta bracket (defined schema major 2): the passes it applies
/// on and the time span it covers. A RepeatStructure of kind Volta
/// carries one entry per ending (e.g., a first ending Volta with
/// endings = [1] and a second with endings = [2]).
pub struct Volta {
    /// The pass numbers this ending plays on (1-based), ascending.
    pub endings: Vec<u32>,
    pub start: TimeAnchor,
    pub end: TimeAnchor,
}
```

The `endings` constraints (non-empty, 1-based, strictly ascending) are *advisory*, on the `OctaveOffset` pattern: a violating value is representable, decoders and reduction accept it, and the authoring validation layer flags it. Rendering treats an unlistable ending set as a plain bracket.

The v1-to-v2 migration default for a kindless v1 repeat structure is `SimpleRepeat` with `count` = 2: a v1 `RepeatStructure` *meant* a repeat, and playing the span twice is the conventional semantics of an unadorned repeat sign. Voltas default empty.

5.7.8 Analytical Annotations

Analytical annotations record analyses of the music: Roman numerals, form labels, motivic brackets, set-theory analyses, Schenkerian graphs. They are first-class and may appear on dedicated analysis layers.

```rust
pub struct AnalyticalAnnotation {
    pub id: AnalyticalAnnotationId,
```

```
    pub kind: AnalyticalKind,
    pub anchor: AnnotationAnchor,

    /// Optional analysis layer. None means the engraved layer;
    /// Some(id) targets a specific analysis layer.
    pub layer: Option<AnalysisLayerId>,

    pub content: AnalyticalContent,
}

pub enum AnalyticalKind {
    RomanNumeral,
    FunctionalLabel,
    SetTheoryLabel,
    SchenkerianGraph,
    FormSection,
    MotivicBracket,
    Custom(CustomAnalyticalKindId),
}

pub enum AnnotationAnchor {
    Event(EventId),
    Range { start: TimeAnchor, end: TimeAnchor },
    Region(RegionId),
}
```

5.7.9 Comments

Comments are review-mode annotations: discussion threads anchored to points or ranges in the score.

```
pub struct Comment {
    pub id: CommentId,
    pub anchor: AnnotationAnchor,
    pub thread: Vec<CommentMessage>,
    pub resolved: bool,
}

pub struct CommentMessage {
    pub author: AuthorId,
    pub timestamp: Timestamp,
    pub body: String,
}
```

5.7.10 Graphic Gestures

A graphic gesture is a drawn or constructed graphic element that spans events, staves, or regions and is not contained within a single graphic region. Used for performance-direction sweeps, multi-staff curved gestures, and similar.

```
pub struct GraphicGesture {
```

```
    pub id: GraphicGestureId,

    /// The graphic objects composing this gesture. Stored in the
    /// canvas's graphic-object storage.
    pub objects: Vec<GraphicObjectId>,

    /// Anchoring: how this gesture is positioned relative to score
    /// content.
    pub anchoring: GestureAnchoring,

    /// Optional playback parameter bindings.
    pub playback_bindings: Vec<PlaybackBinding>,
}

pub enum GestureAnchoring {
    /// Anchored to events: the gesture moves with them.
    Events(Vec<EventId>),

    /// Anchored to a time and staff range.
    Range {
        start: TimeAnchor,
        end: TimeAnchor,
        staves: Vec<StaffId>,
    },

    /// Free canvas coordinates: does not follow score edits.
    Free(CanvasCoordinates),
}
```

5.8 Graphic Content

Free-graphic and hybrid regions carry graphic content: vector primitives, drawn strokes, text, and images. Graphic content is expressed in the region's local coordinate space.

```
pub struct GraphicContent {
    pub objects: Vec<GraphicObject>,

    /// The region's local coordinate system. Region transforms
    /// map this to canvas coordinates.
    pub coordinate_system: LocalCoordinateSystem,
}
```

5.8.1 Graphic Objects

```
pub struct GraphicObject {
    pub id: GraphicObjectId,

    /// The object's geometric kind.
    pub kind: GraphicObjectKind,
```

```
    /// Transform applied within the region's coordinate space.
    pub transform: Transform2D,

    /// Layer ordering: higher values draw on top of lower.
    pub layer: i32,

    /// Visual style: stroke color, fill, line weight, opacity.
    pub style: GraphicStyle,

    /// Optional time binding. See Section~\ref{sec:graph:timebinding}.
    pub time_binding: Option<TimeBinding>,
}

pub enum GraphicObjectKind {
    Path(PathData),
    Shape(ShapePrimitive),
    Text(TextData),
    Image(ImageData),
    Group(Vec<GraphicObjectId>),
    Stroke(StrokeData),
}

pub enum ShapePrimitive {
    Line { start: Point2D, end: Point2D },
    Rectangle { origin: Point2D, size: Size2D },
    Ellipse { center: Point2D, radii: Size2D },
    Polygon(Vec<Point2D>),
    Bezier(Vec<BezierSegment>),
}

pub struct StrokeData {
    /// Pressure-sensitive stylus stroke samples: position, pressure,
    /// tilt, and time per sample.
    pub samples: Vec<StrokeSample>,
}

pub struct StrokeSample {
    pub position: Point2D,
    pub pressure: f32,        // 0.0..=1.0
    pub tilt_xy: (f32, f32),  // radians
    pub timestamp: WallClockTime,
}
```

5.8.2 Coordinates

REQUIREMENT 5.29

Graphic objects **MUST** be stored in region-local coordinates. The region's transform maps local coordinates to canvas coordinates. This permits regions to be moved, resized, or otherwise transformed without rewriting the coordinates of every contained object.

5.8.3 Time Bindings

A graphic object **MAY** carry an optional time binding, which assigns a temporal meaning to the object beyond its visual appearance. Time bindings are the mechanism by which a drawn curve becomes a performance instruction (a filter sweep, an amplitude envelope, a density trajectory).

```rust
pub struct TimeBinding {
    /// What musical or wall-clock range this graphic spans.
    pub time_extent: TimeExtent,

    /// What parameter, if any, this graphic drives. None means the
    /// graphic is decorative; Some specifies a named parameter.
    pub parameter: Option<ParameterBinding>,

    /// How the graphic's geometry maps to parameter values.
    pub mapping: ParameterMapping,
}

pub struct ParameterBinding {
    pub target: ParameterTarget,
    pub parameter_name: String,
    pub value_range: ParameterRange,
}

pub enum ParameterTarget {
    /// A specific event's parameter (e.g., this event's velocity).
    Event(EventId),

    /// A voice-level parameter (e.g., voice expression CC).
    Voice(VoiceId),

    /// A staff-level parameter.
    Staff(StaffId),

    /// A region-level parameter.
    Region(RegionId),

    /// A score-global parameter (e.g., master gain).
    Score,
}

pub enum ParameterMapping {
    /// Vertical position in region coordinates maps linearly to
    /// parameter value.
    VerticalLinear,

    /// Vertical position with exponential mapping.
    VerticalExponential,

    /// Stroke pressure maps to parameter value.
    StrokePressure,
```

```
    /// Stroke velocity maps to parameter value.
    StrokeVelocity,

    /// Custom mapping defined by a plugin.
    Registered(MappingRegistryId),
}
```

RATIONALE

Time bindings make graphic-notation playback expressible in the core data model without committing to specific audio engine semantics. The core stores what parameter a graphic drives and how its geometry maps to values; the audio engine spec consumes this binding and applies it. Decorative graphics (binding is `None`) remain pure visual content.

5.9 Parts

A part is a per-instrument or per-section view onto the score, intended for printed extraction or focused performer use. Parts live separately from the canvas; they are projections defined by a part-extraction specification.

```rust
pub struct PartDefinition {
    pub id: PartDefinitionId,

    /// Human-readable name (e.g., "Flute 1", "Violins II").
    pub name: String,

    /// Which staves are included in this part, in order from top
    /// to bottom.
    pub staves: Vec<StaffId>,

    /// Part-specific layout overrides: page size, system count,
    /// cue insertions, multirest consolidation, page-turn placement.
    pub layout_overrides: PartLayoutOverrides,

    /// Per-event visibility overrides specific to this part (e.g.,
    /// hide a cue in the full score but show it in the part).
    pub visibility_overrides: Vec<VisibilityOverride>,

    /// Auto-cue insertions: source staves to draw cues from when
    /// the part has long rests.
    pub auto_cue_sources: Vec<StaffId>,
}
```

5.9.1 Parts Are Projections, Not Storage

REQUIREMENT 5.30

A part **MUST NOT** store musical content directly; it **MUST** consist only of references to score content and overrides. Edits to score content **MUST** propagate to all parts that reference that content.

5.10 Analysis Layers and Views

5.10.1 Analysis Layers

Analysis layers, introduced for spelling in Section 2.5.6 and used by analytical annotations, are first-class objects on the score.

```
pub struct AnalysisLayer {
    pub id: AnalysisLayerId,
    pub name: String,
    pub description: Option<String>,

    /// Which views display this layer.
    pub visible_in_views: Vec<ViewId>,
}
```

5.10.2 Views

A view is a recipe for displaying a configuration of the score: which analysis layers are active, which parts are rendered, what overrides apply. Views are how the system supports condensed scores, analytical views, study scores, and lead-sheet projections.

```
pub struct ViewDefinition {
    pub id: ViewId,
    pub name: String,
    pub kind: ViewKind,

    /// Which analysis layers are active in this view.
    pub active_layers: Vec<AnalysisLayerId>,

    /// View-specific overrides.
    pub overrides: ViewOverrides,
}

pub enum ViewKind {
    FullScore,
    Part(PartDefinitionId),
    CondensedScore(CondensationSpec),
    LeadSheet(LeadSheetSpec),
    Analysis(AnalysisViewSpec),
    Custom(CustomViewKindId),
}
```

5.11 Graph Invariants

The score graph maintains a set of structural invariants. Implementations **MUST** preserve these invariants across all edits.

REQUIREMENT 5.31

The following invariants hold over every well-formed score graph:

1. Every event in the arena has a valid `voice` field pointing to a voice that lists the event in its `events` vector.
2. Every event in a voice's events list has its `voice` field pointing back to that voice.
3. Events within a voice are sorted by position and do not overlap in time; a grace note has zero duration and only a grace note does, and a position's graces stand before its event of positive duration, in their order and then by identifier (Requirement 5.14).
4. Every event's `EventPosition` and `EventDuration` variants agree with the time model of its enclosing region (Section 5.4.1). For aleatoric regions, agreement is governed by the region's declared `AleatoricAnchoringDiscipline`.
5. Every voice belongs to exactly one `StaffInstance`; every `StaffInstance` belongs to exactly one region; every region belongs to the canvas.
6. Every `StaffInstance.staff` resolves to a `Staff` declared at the score level. A single `StaffId` **MAY** be referenced by multiple `StaffInstance`s in different regions (this is how continuous staff identity is expressed across regions); a `StaffId` **MUST NOT** be referenced by two `StaffInstance`s in the same region.
7. Region time and staff extents do not simultaneously overlap (Section 5.5). A region's `staff_extent` lists exactly the `StaffId`s manifested by the region's `staff_instances`, with no duplicates.
8. Measures belong to exactly one `StaffInstance`. A region's metric organization is per-staff (admitting polymeter); barline alignment across staves is recorded via `BarlineAlignmentGroup` entries when explicit alignment is authoritative.
9. Every `TimeAnchor`'s `AnchorOffset` variant agrees with the time model of the target object's enclosing region (Section 3.5).
10. Except where the re-anchoring rules of Chapter 6 explicitly permit transient dangling states during edits, every graph reference resolves to an extant object. Every cross-cutting structure's references resolve to extant objects in the graph, except where explicit re-anchoring rules permit transient dangling states during edits (see Chapter 6). Tempo-map conditions that are not reference resolution — segment ordering and non-overlap — are governed by Requirement 3.11, not by this invariant. The complete reference surface is:
    - `Slur.start_event` — live event.
    - `Slur.end_event` — live event.
    - `Tie.start_event` — live event.
    - `Tie.end_event` — live event.

- `Beam.events` — live event.
- `SubBeam.events` — live event.
- `Tuplet.members` — live event.
- `Tuplet.parent` — extant tuplet.
- `Spanner.staves` — declared staff.
- `Spanner.start` — anchor target.
- `Spanner.end` — anchor target.
- `Marker.anchor` — anchor target.
- `RepeatStructure.start` — anchor target.
- `RepeatStructure.end` — anchor target.
- `RepeatStructure.kind` — anchor target.
- `RepeatStructure.voltas` — anchor target.
- `ChordSymbol.anchor` — anchor target.
- `AnalyticalAnnotation.anchor` — anchor target, extant region, live event.
- `AnalyticalAnnotation.layer` — declared analysis layer.
- `Comment.anchor` — anchor target, extant region, live event.
- `GraphicGesture.objects` — stored graphic object.
- `GraphicGesture.anchoring` — anchor target, declared staff, live event.
- `Lyric.event` — live event.
- `Staff.instrument` — declared instrument.
- `StaffInstance.instrument_override` — declared instrument.
- `Staff.group` — declared staff group.
- `StaffGroup.members` — declared staff.
- `PartDefinition.staves` — declared staff.
- `ViewDefinition.active_layers` — declared analysis layer.
- `MetricTimeModel.meters` — declared time signature.
- `StaffBasedContent.default_metric_grid` — declared time signature.
- `Measure.time_signature` — declared time signature.
- `StaffInstance.local_metric_grid` — declared time signature.
- `NotatedComponent.tuplet` — extant tuplet.
- `IndeterminacyHints.alternatives` — live event.
- `TrajectoryEvent.start` — live pitch.
- `TrajectoryEvent.end` — live pitch.
- `GraphicEvent.graphics` — stored graphic object.
- `CueEvent.source` — live event.
- `TempoSegment.start` — anchor target.
- `TempoSegment.end` — anchor target.

- `Score.voice_homes` — declared staff, extant voice.

11. Every identifier in the graph is unique within its kind. Identifiers reserved for system-derived objects (notably `VoiceId`s of system-promoted voices) **MUST** be derived from the deterministic function specified for their kind and **MUST NOT** collide with user-authored identifiers.
12. Every `IdentifiedPitch` in the arena has a `PitchId` that is unique within the score's pitch-identity index.
13. Every `SpellingScope::Pitch(PitchId)` resolves to a live or tombstoned `PitchId` in the pitch-identity index. A live target resolves to exactly one `IdentifiedPitch`; a tombstoned target is preserved for operation-log replay, undo, conflict reporting, and historical annotation, and **MUST NOT** be silently re-matched to a different live pitch.
14. Every notational decomposition attachment's target `EventId` resolves to a live or tombstoned event; attachments to tombstoned events follow the same preservation discipline as for tombstoned pitches.
15. For live decomposition attachments, the targeted event's duration matches the decomposition's component sum.
16. Every tuplet's member event durations sum to the tuplet's structurally-required total (Section 3.9). Operations that would break this invariant **MUST** include compensating changes (Section 6.12.2).
17. Every `Tie`'s `pitch_pairing` entries reference `PitchId`s that resolve to identified pitches belonging to the start and end events respectively. The class-specific adjacency, voice-membership, and pitch-equivalence rules of Section 5.7.2 hold for the tie's declared `TieClass`.
18. Every `Voice` declares an `origin` consistent with how it was created: `UserDeclared` for explicit user actions, `Imported` for foreign-format imports, `SystemPromoted` for voices allocated by concurrent-edit conflict resolution. The `VoiceId` of a `SystemPromoted` voice **MUST** be the deterministic derivation specified in Section 5.6.3.
19. Every `BarlineAlignmentGroup`'s members reference `StaffInstance`s within the same region and `Measure`s belonging to those instances. Group members **MUST NOT** reference staff instances or measures across region boundaries.
20. (Genesis tranche G3b.) A measure's declared time signature `Measure.time_signature`, when it resolves, AGREES with the effective metric grid's active signature at the measure's start; and consecutive measure starts within a staff instance are separated by the governing signature's `measure_duration()` (BOUNDARY consistency). `time_signature: None` exempts only the agreement clause — the inherited meter still governs boundary consistency. This invariant **MUST NOT** duplicate the signature-*resolution* check above (invariant 10): it compares only already-resolving signatures. It ABSTAINS — raises no violation — wherever the comparison or the distance cannot be decided (base-ingested data may predate this rule); this is deliberate abstention, not a soundness gap. A first measure has no predecessor, so only the BOUNDARY clause is vacuous for

it; the AGREEMENT clause is not predecessor-dependent and still applies (unless `time_signature` is `None` or agrees with the governing grid). A first measure may be a pickup, shorter than its bar: the second measure's distance from it may be any positive duration up to the governing signature's `measure_duration()` (phase X3, P13-S19). Every later distance is a full bar; a short measure after the first is not modelled.

21. (P13-S16.) `Staff.group` and `StaffGroup.members` AGREE in both directions: whenever a `Staff` names a `StaffGroup`, that group's `members` contains the staff; and whenever a group's `members` lists a staff, that staff's own `group` names that group. `Staff.group` is the sole authority and `members` is the projection maintained from it, so a violation means the projection has drifted — a maintenance gap leaving a staff absent from `members`, or a stale entry listing a staff that no longer names the group. The two directions fail for different reasons and are checked independently. This invariant ABSTAINS on *dangling* membership: a `members` entry naming a staff that is not declared at all is invariant 10's reference-resolution concern, not a disagreement, since an absent staff has no `group` to compare against.

Implementations **MUST** reject graph configurations that violate any of the above invariants. Edits proposed in Chapter 6 must either preserve invariants directly or include compensating changes that restore them within the same operation.

This enumeration contains exactly **21** invariants. (Earlier summary material, including the QUICKSTART, referred to "18 graph invariants"; a subsequent revision of this document corrected that to 19; genesis tranche G3b (`spec/CONTRACT_GENESIS_G3B_MEASURE.md`) then appended a 20th, measure-meter consistency, and P13-S16 (`spec/CONTRACT_P13S16_PROJECTION.md`) a 21st, staff-group membership agreement — the authoritative count is 21, matching this enumeration and the reference implementation.) These 21 are *runtime* invariants: they hold over every well-formed graph and are restored by compensating changes when an edit would break them.

Distinct from them are three *construction-time* rejections defined in Chapter 3, which a conforming constructor **MUST** enforce before a value ever enters the graph, and which no runtime invariant restores because the offending value is never a representable graph state:

1. A `TimeSignature`'s beat groups **MUST** sum to the measure duration (Section 3.6).
2. An `EventOrderingDAG` **MUST** be acyclic (Requirement 3.22).
3. A `TupletRatio` **MUST NOT** be degenerate (Section 3.9, Requirement 3.17).

5.12 Indexes and Auxiliary Structures

Performance-critical operations require indexed access. The specification states the required indexes; implementations **MAY** construct additional indexes provided they are kept consistent.

Requirement 5.32

Implementations **MUST** maintain at least the following indexes:

Event time index. Maps (region, voice) to a sorted view of event positions, supporting $O(\log n)$ time-range queries.

Cross-cutting reference index. Maps each object identifier to the cross-cutting structures referencing it, supporting $O(1)$ amortized lookup for re-anchoring.

Measure index. Maps measure number (and measure id) to `MeasureId` for fast navigation.

Spelling attachment index. Maps each `PitchId` to its attachments, partitioned by analysis layer.

Indexes **MUST** be invalidated and rebuilt (or incrementally updated) on the corresponding edits. The semantic operations chapter specifies which operations invalidate which indexes.

5.13 Forward References

- The complete catalog of semantic operations, their preconditions, postconditions, and re-anchoring rules is in Chapter 6.
- The mapping from score graph to layout intermediate representation is defined in Chapter 7.
- The on-disk serialization of every type introduced in this chapter is defined in Chapter 8.
- Some engraving-specific types (`StemConfiguration`, `DynamicMark`, and similar) are introduced informally here and fully defined in Chapter 7 where their interaction with the engraver is specified. The clef/key content model (`Clef`, `ClefShape`, `KeySignature`, `ClefChange`, `KeySignatureChange`) and `LineStyle` are defined normatively *in this chapter* as of schema major 2; the former `ClefId` identifier sketch is retired in favour of embedded `Clef` values.

Chapter 6

Semantic Operations and Concurrent Reduction

This chapter specifies the semantic operations on the score graph: the mutations through which the data model becomes a live model. It defines the operation framework, the canonical reduction procedure by which a set of operations becomes a materialized score state, the object existence model with tombstones, the conflict-record machinery, the re-anchoring rules, transactions, and undo. The chapter ends with representative operation specifications.

The collaboration mechanics specified here are the foundation on which Chapter 8's synchronization protocols build. The materialized score is the deterministic reduction of an operation set; the file format stores that operation set together with optional acceleration snapshots.

6.1 Design Principles

The operation set is canonical. A score's state is defined by the set of operations that have been committed to it. Any materialized graph is a deterministic reduction of that set; caches, snapshots, and partial reductions are acceleration structures, never the source of truth.

The replicated operation set is a CRDT; the materialized graph is not. The grow-only set of operation envelopes with compact causal metadata is a true CRDT: replicas independently accumulate envelopes and converge on the same set. The materialized score graph is not itself claimed to be a CRDT; it is the deterministic reduction of that set, defined by the reduction algorithm specified here.

Pairwise operation merge is rejected. Musical operations do not, in general, commute in intention: transpose-then-respell differs musically from respell-then-transpose; delete-then-tie differs from tie-then-delete. The framework therefore does not attempt to make every pair of operations algebraically commute. Instead, operations are reduced in a canonical order, and that order determines the outcome.

Conflicts, no-ops, and rejections are replicated facts. No operation silently disappears because it could not apply. Every operation has a deterministic `OperationEffect` recorded as part of the materialized state; every replica reducing the same operation set records the same effects with the same reasons.

Identifier-based targeting. Operations target objects by identifier, never by structural path.

Identifiers are stable under concurrent edits; paths are not.

Tombstones preserve identity. Deleted objects retain their identifiers as tombstones. References to tombstoned identifiers are not silently invalidated; they enter a tombstoned-target state unless the operation kind specifies deterministic repair.

Transactions materialize atomically. Primitive members of a transaction **MUST NOT** be independently visible as committed score states. Either the whole transaction reduces successfully, or the transaction transitions to a conflict effect with a recorded conflict.

Undo is forward, not backward. Undo is a new operation that computes a compensating edit against the current materialized state; it is not literal time travel. History remains append-only.

Two validation modes for advisory checking. Operations are subject to strict precondition checking in *authoring mode* (interactive edits) and lenient checking in *replay mode* (loading historical operations or applying remote operations). Validation modes apply only to advisory preconditions; invariant preconditions are enforced unconditionally.

6.2 The Operation Framework

6.2.1 Operation Identity and Stamps

Operations carry two related but distinct concepts: *identity* (who authored the operation, with what counter) and *stamp* (when the operation was committed, for ordering purposes).

```rust
/// Stable identity of an operation. Determined at authoring time.
pub struct OperationId {
    pub replica: ReplicaId,
    pub counter: u64,
}

/// Ordering metadata for an operation. Used to derive the canonical
/// reduction order; not part of the operation's identity.
pub struct OperationStamp {
    pub hlc: HybridLogicalClock,
    pub id: OperationId,
}

/// A hybrid logical clock combines wall-clock and logical components.
pub struct HybridLogicalClock {
    /// Physical time component, in canonical units (see Chapter 8).
    pub physical_time: WallClockTime,
    /// Logical counter, advanced when physical time does not.
    pub logical_counter: u32,
}
```

REQUIREMENT 6.1

`OperationId` **MUST** be stable: an operation's identity is fixed at the moment of authoring and **MUST NOT** change with reordering, retransmission, or merging into operation

sets at other replicas.
`OperationStamp` **MUST** be set by the authoring replica using a hybrid logical clock that respects causal order: for any operation A that is in operation B's causal predecessor set, A's stamp **MUST** be strictly less than B's under the canonical comparison (Section 6.3.3).

6.2.2 Causal Context via Dotted Version Vectors

Operations carry a compact causal context, not an exhaustive predecessor list. This is essential: exhaustive predecessor lists scale linearly with history and become untenable.

```rust
/// A compact causal context: for each replica known to the authoring
/// replica, the highest contiguous counter observed, plus any
/// individual operation identifiers known but not contiguous (the
/// "dots").
pub struct CausalContext {
    /// Contiguous counter highest known per replica.
    pub vector: BTreeMap<ReplicaId, u64>,

    /// Individual operations known but not yet contiguous in the
    /// vector. Used when an authoring replica has observed an
    /// operation whose causal predecessors include operations not
    /// yet present.
    pub dots: BTreeSet<OperationId>,
}
```

REQUIREMENT 6.2

Causal contexts **MUST** be expressed as dotted version vectors as defined above. Exhaustive predecessor lists **MUST NOT** be used as the canonical causal representation; they may appear in debugging output or in the operation log's expanded form, but the wire and storage representation of causal context is the DVV.
The per-replica counter floor is **zero-based** and normative: `vector[r] == n` asserts that *every* operation `(r, 0..=n)` is a causal predecessor — the contiguous range begins at counter `0`, not `1`. A conforming implementation **MUST NOT** adopt a one-based floor: doing so would shift the contiguous/dot boundary and make two implementations disagree about which operations are pending versus ready. The membership check ("is operation `(r, c)` covered?") is `c <= vector[r]` or `(r, c) in dots`; an implementation **MAY** evaluate it by walking the known operation ids rather than materializing the full `0..=n` range, so a sparse high-counter context does not force work proportional to the counter value.
Interval tree clocks and other compact causal representations **MAY** appear in future revisions as performance optimizations but are non-normative in this specification.

RATIONALE

Dotted version vectors are well-understood, well-tested, and the right complexity trade-off for the human-scale collaboration model Epiphany targets. Their normative use here closes the high-priority review item about exhaustive predecessor lists scaling badly with history.

6.2.3 Operation Envelopes

An operation is transmitted, stored, and reduced as an *envelope*: the operation's payload together with its identity, ordering stamp, causal context, and optional transaction grouping.

```
pub struct OperationEnvelope {
    /// Stable identifier of this operation.
    pub id: OperationId,

    /// Author of this operation (may differ from replica in shared
    /// authoring sessions where multiple authors share a replica).
    pub author: AuthorId,

    /// Ordering stamp. Used for canonical reduction order; never
    /// for identity.
    pub stamp: OperationStamp,

    /// Compact causal context: what operations were visible to the
    /// authoring replica when this operation was created.
    pub causal_context: CausalContext,

    /// Optional transaction grouping. Multiple operations sharing a
    /// TransactionId are reduced atomically.
    pub transaction: Option<TransactionId>,

    /// The mutation itself.
    pub payload: OperationPayload,
}

pub enum OperationPayload {
    Primitive(OperationKind),

    /// A meta-operation that explicitly resolves a previously-recorded
    /// conflict (see Section~\ref{sec:semops:conflicts}).
    ResolveConflict(ResolveConflictPayload),

    /// A meta-operation that compensates for a previously-committed
    /// transaction; the realization of "undo" (see
    /// Section~\ref{sec:semops:undo}).
    UndoTransaction(UndoTransactionPayload),

    /// A meta-operation that resolves an equivocated operation slot
    /// by naming the chosen candidate envelope (see
    /// Section~\ref{sec:semops:equivocation}; payload schema in the
    /// Operation Catalog companion).
```

```
    ResolveEquivocation(ResolveEquivocationPayload),
}

/// The catalog of primitive operation kinds. The variants listed
/// here are normative for the core; additional kinds are introduced
/// by registered extensions via the Registered variant. The full
/// catalog including detailed payload schemas is the Operation
/// Catalog companion specification
/// (Appendix~\ref{app:deferred}).
pub enum OperationKind {
    // Event operations
    InsertEvent(InsertEventOp),
    DeleteEvent(DeleteEventOp),
    ModifyEvent(ModifyEventOp),

    // Pitch and tuning operations
    RespellPitch(RespellPitchOp),
    Transpose(TransposeOp),
    // The faithful transposition (Push 4a); Transpose above is frozen.
    TransposeInterval(TransposeIntervalOp),
    InsertIdentifiedPitch(InsertIdentifiedPitchOp),
    DeleteIdentifiedPitch(DeleteIdentifiedPitchOp),
    ModifyIdentifiedPitch(ModifyIdentifiedPitchOp),

    // Cross-cutting operations
    CreateCrossCutting(CreateCrossCuttingPayload),
    DeleteCrossCutting(DeleteCrossCuttingPayload),
    ModifyCrossCutting(ModifyCrossCuttingPayload),

    // Repeat-structure operations
    CreateRepeatStructure(CreateRepeatStructureOp),
    DeleteRepeatStructure(DeleteRepeatStructureOp),

    // Region and structure operations
    ChangeRegionTimeModel(ChangeRegionTimeModelOp),
    InsertRegion(InsertRegionOp),
    DeleteRegion(DeleteRegionOp),
    InsertStaffInstance(InsertStaffInstanceOp),
    DeleteStaffInstance(DeleteStaffInstanceOp),
    InsertStaff(InsertStaffOp),
    CreateVoice(CreateVoiceOp),
    DeleteVoice(DeleteVoiceOp),

    // Genesis tranche G1: mints an abstract instrument on the score root ---
    // the single missing link between Score::empty and a note.
    CreateInstrument(CreateInstrumentOp),

    // Metric-model operations
    SetTimeSignature(SetTimeSignatureOp),
    SetTempoSegment(SetTempoSegmentOp),
    SetMetricGrid(SetMetricGridOp),

    // Score settings
```

```
    SetMetadata(SetMetadataOp),
    // Genesis tranche G2a: the two major-0 settings setters, on the
    // SetMetadata advisory-LWW pattern.
    SetCanvasLayoutDefaults(SetCanvasLayoutDefaultsOp),
    SetSpellingPrecedence(SetSpellingPrecedenceOp),
    // Genesis tranche G2b: the sole genesis payload born at schema major 3,
    // on the same advisory-LWW pattern. Carries TuningContextSettings, the
    // authored subset of ScoreTuningContext (accidental_extensions is not
    // on the wire and is left untouched by reduction).
    SetTuningContext(SetTuningContextOp),

    // Genesis tranche G3a: the four remaining root-level Score entity
    // mints, on the InsertStaff set-union mint pattern. Staff.group is the
    // sole authority for group membership (see the Staff/StaffGroup
    // declarations below); CreateStaffGroup refuses a non-empty carried
    // members, and the projection is maintained from Staff.group by
    // reduction (P13-S16, disposition A).
    CreateStaffGroup(CreateStaffGroupOp),
    CreatePartDefinition(CreatePartDefinitionOp),
    CreateAnalysisLayer(CreateAnalysisLayerOp),
    CreateView(CreateViewOp),

    // Genesis tranche G3b: the final rung of the genesis ladder. Appends a
    // Measure to a live StaffInstance (StaffInstance.measures), on the
    // InsertStaffInstance parent-carrying mint pattern rather than the
    // bare-value shape above -- Measure has no back-pointer to its parent.
    CreateMeasure(CreateMeasureOp),

    // X3.1: mints a Tuplet into the cross-cutting registry. Like repeats,
    // tuplets are not CrossCuttingValue kinds on the wire. Preconditioned on
    // live members and parent and on the members' durations filling the
    // tuplet's required total.
    CreateTuplet(CreateTupletOp),

    // X3.6: set, replace or remove the clef or key-signature change at a
    // musical offset in a live staff instance's region (structural LWW, keyed
    // by instance and position, like a meter change).
    SetClef(SetClefOp),
    SetKeySignature(SetKeySignatureOp),

    // X4b: set or clear a voice's home staff (structural LWW, keyed by
    // the voice).
    SetVoiceHome(SetVoiceHomeOp),

    // Layout-semantic operations
    SetUserSystemBreak(SetUserSystemBreakOp),
    SetUserPageBreak(SetUserPageBreakOp),
    SetStaffLayout(SetStaffLayoutOp),

    // Transaction declaration. Carries metadata; member primitives
    // reference the transaction by id in their envelopes.
    DeclareTransaction(TransactionDescriptor),
```

```
    // Extension-defined primitive operation.
    Registered(OperationKindRegistryId, SerializedRegisteredOp),
}
```

6.2.4 The Operation Set as CRDT

REQUIREMENT 6.3

The replicated operation set is a grow-only CRDT: replicas accumulate envelopes by gossip, broadcast, or any other delivery mechanism, and converge on the same set when they have observed the same envelopes. The set is set-valued, not multiset-valued: an envelope is a member of the set or it is not, with no count.
A replica **MUST NOT** discard envelopes from the operation set except by an explicit pruning operation (Chapter 8), which preserves canonical state through retained base snapshots and the post-pruning operation set.

6.2.5 OperationId Collisions and Replica Equivocation

Two received envelopes may carry the same `OperationId`. Acceptance must not depend on arrival order: if it did, two replicas observing the same envelopes in different orders could diverge on which one is canonical. The operation set therefore maps each `OperationId` to an `OperationSlot` rather than directly to an envelope.

```
pub enum OperationSlot {
    /// Exactly one canonical envelope is known for this
    /// OperationId. The envelope reduces normally.
    Single(OperationEnvelope),

    /// Two or more distinct canonical envelopes have been
    /// observed for this OperationId. The slot is equivocated.
    /// The operation produces no canonical effect until external
    /// recovery resolves the equivocation.
    Equivocated {
        operation_id: OperationId,
        /// Every distinct canonical envelope observed for this
        /// OperationId. Identified by content hash to avoid
        /// duplicating large payloads. Sorted lexicographically by
        /// hash for deterministic enumeration.
        candidates: BTreeSet<EnvelopeHash>,
    },
}

/// BLAKE3-256 hash of a canonical envelope serialization with
/// domain tag "MUSCENVH".
pub struct EnvelopeHash(pub [u8; 32]);
```

REQUIREMENT 6.4

Slot transitions are determined as follows, independently of arrival order:

- If no slot exists for the incoming envelope's `OperationId`: the incoming envelope is well-formedness checked (Section 6.2.6) and, if well-formed, a `Single` slot is created.
- If a `Single` slot exists and the incoming envelope is byte-identical (canonical serialization) to the slot's envelope: the duplicate is dropped silently; the slot is unchanged.
- If a `Single` slot exists and the incoming envelope's canonical bytes differ from the slot's envelope: the slot transitions to `Equivocated` with both envelopes' hashes in its `candidates` set. The previously-canonical envelope is retained in a diagnostic candidate store keyed by its hash; the incoming envelope is likewise retained. Neither envelope contributes to canonical reduction.
- If an `Equivocated` slot exists: the incoming envelope's hash is added to its `candidates` set if not already present; otherwise the duplicate is dropped. The envelope is retained in the candidate store. The slot remains `Equivocated`.

An equivocated slot **MUST NOT** contribute to canonical reduction. Reduction proceeds as if the operation does not exist: any operation that causally depends on the equivocated `OperationId` is reduced under the same rule as a missing causal predecessor (the dependent operation is retained but produces no effect until the equivocation is resolved). Resolution is by explicit recovery action, not by arrival order or content. Three resolution mechanisms are permitted:

- *Transport-level credential revocation*: the collaboration transport revokes the equivocating replica's credentials and the user accepts one envelope timeline as canonical via an explicit reconciliation operation.
- *Local user choice*: the user (or an authorized administrator) issues an explicit `ResolveEquivocation` operation naming the `OperationId` and the chosen `EnvelopeHash`, promoting the slot back to `Single`.
- *Profile-declared deterministic policy*: a conformance profile **MAY** declare a deterministic selection function (e.g., "lowest canonical envelope hash wins"). This is a policy choice, not a default. A bundle reduced under a profile with a declared resolution policy is canonical under that profile; the same bundle reduced under a profile without such a policy remains anomalous.

If a bundle on disk contains an equivocated slot without a resolution policy or pending resolution operation, the bundle is anomalous. Readers **MUST** open it, if at all, in a diagnostic recovery mode that surfaces the equivocation prominently and that prohibits ordinary editing of the affected portion of canonical state.

RATIONALE

Replica equivocation (the same replica producing two distinct envelopes with the same `OperationId`) is a real failure mode: replica-key compromise, software bug, replay attack, or filesystem corruption can all produce it. The `OperationSlot` model makes equiv-

ocation a representable, order-independent state: two replicas that have observed the same set of envelopes converge on the same slot states regardless of the order of observation. Treating the first arrival as canonical would let a malicious early-arriver pin canonical state at one replica while a late-arriver pins different state at another replica; the slot model avoids that asymmetry by refusing to elevate either candidate without an explicit resolution action.
Recovery is a policy choice rather than a built-in default because the right answer depends on context: a collaborating team with credential revocation will resolve differently from a single-user backup-restore scenario, which differs again from an adversarial-document forensic scenario.

6.2.6 Envelope Acceptance and Malformed-Envelope Behavior

Operation envelopes arrive from local authoring and from remote replicas. Acceptance rules determine when an envelope enters the operation set.

Requirement 6.5

An incoming envelope is well-formed if and only if all of the following hold:

- Its `OperationId` consists of a 64-bit `ReplicaId` that is not `ReplicaId::SYSTEM_DERIVED` (operations are not authored by the system namespace) and a 64-bit counter.
- Its `stamp.id` is byte-identical to its top-level `id` field: an envelope's stamp **MUST** address the same operation as the envelope itself. This eliminates an ambiguity where an authoring replica could otherwise stamp an envelope as belonging to a different operation than its identity declares.
- Its `stamp.hlc.physical_time` is a finite, non-negative integer value, expressed in the canonical time unit (Chapter 8).
- Its causal context's DVV references only well-formed replica identifiers and non-negative integer counter values; dots are well-formed `OperationId`s.
- Its payload deserializes successfully against the profile's declared operation catalog.

An envelope that is not well-formed **MUST** be rejected at reception. Rejection is recorded in the local diagnostic log but does *not* enter the canonical operation set. Remote replicas **MAY** retransmit; persistent rejection indicates a non-conforming peer.
Acceptance **MUST NOT** require trust in the envelope's `stamp.hlc` content: a malicious or misconfigured peer may emit envelopes with implausible future physical times or zero/extreme logical counters. The canonical reduction order (Section 6.3.3) consumes the stamp as ordering metadata; it does not validate plausibility. Implementations **MAY** surface clock-skew warnings in diagnostics but **MUST NOT** reorder envelopes for plausibility reasons that would deviate from the canonical reduction order.

6.2.7 Per-Replica HLC Monotonicity

Per-replica monotonicity is a property of the *set* of accepted envelopes, not of arrival order. Out-of-order delivery is normal in gossip systems and is not a violation.

```rust
/// A range of envelopes from a single replica that have been
/// identified as monotonicity-violating and excluded from
/// ordinary canonical reduction. Retained for diagnostic and
/// recovery purposes.
pub struct AnomalousReplicaSegment {
    pub replica: ReplicaId,
    /// The smallest counter at or after which the replica's
    /// stream is anomalous. All envelopes from this replica
    /// with counter >= first_bad_counter are excluded from
    /// canonical reduction.
    pub first_bad_counter: u64,
    /// The detected anomaly reason.
    pub reason: ReplicaAnomalyReason,
    /// Operation ids in the excluded segment, retained for
    /// diagnostic recovery. Sorted by counter.
    pub excluded: Vec<OperationId>,
}

pub enum ReplicaAnomalyReason {
    /// The replica produced an envelope at counter c2 with a
    /// stamp tuple strictly less than a previously-known
    /// envelope at counter c1 < c2.
    HlcMonotonicityViolation {
        violating_pair: (OperationId, OperationId),
    },
    /// A registered anomaly reason defined by an extension or
    /// transport.
    Registered(ReplicaAnomalyRegistryId),
}
```

REQUIREMENT 6.6

For any two envelopes accepted into the operation set that share the same authoring `ReplicaId`, with operation- counter values $c_1 < c_2$:

> The stamp tuple `(stamp.hlc.physical_time, stamp.hlc.logical_counter, id.counter)` of the envelope with counter c_1 **MUST** be lexicographically less than or equal to the corresponding tuple of the envelope with counter c_2.

Arrival order **MUST NOT** be used to determine whether this property holds. If a replica receives counter 10 before counter 9, that is a property of delivery, not of monotonicity. If two envelopes from the same replica violate this property (their stamp tuples are inconsistent with their counter order), the document carries a **replica equivocation anomaly**. An `AnomalousReplicaSegment` is created (or extended) for the offending replica. The segment's `first_bad_counter` is the smaller of the two counters in the violating pair. Effects on canonical reduction:

- Every envelope from the offending replica with counter $\geq$ `first_bad_counter` is excluded from canonical reduction. Such envelopes are retained on disk and in memory (the CRDT property does not allow silent removal) but are held in the `AnomalousReplicaSegment`'s `excluded` list, not in the canonical operation set.
- Envelopes from the offending replica with counter $<$ `first_bad_counter` remain in the canonical operation set and reduce normally.
- Any operation in the canonical set that causally depends on an excluded envelope reduces under the missing-causal-predecessor rule: the dependent operation is retained, produces no canonical effect, and is held pending resolution.
- The document is marked as containing a replica- equivocation anomaly. Ordinary editing **MAY** continue on the unaffected portion of canonical state; collaborative synchronization with the equivocating replica **MUST** be suspended until external resolution.

Resolution paths mirror the `OperationSlot` equivocation rules (Section 6.2.5): transport-level credential revocation followed by an explicit reconciliation operation, an explicit local recovery action, or a profile-declared deterministic policy.
Authentication of replica identities and revocation of compromised replicas are the responsibility of the collaboration transport (Appendix A). This chapter specifies only local detection, segment formation, exclusion from canonical reduction, and the structural shape of resolution.

RATIONALE

Excluding the violating segment from canonical reduction rather than letting it reduce-but-mark-tainted is the safer choice: a tainted reduction still produces canonical-looking state derived from operations the spec has flagged as untrustworthy. Excluding the segment keeps canonical state derived only from operations that satisfy the monotonicity invariant, at the cost of holding pending operations that causally depended on the excluded segment.

6.3 The Canonical Reduction

The materialized score graph is the deterministic reduction of the operation set. This section specifies how that reduction is performed.

6.3.1 Operation Lifecycle

Every operation traverses four phases:

Prepare. The local UI command validates enough to construct a well-formed operation envelope. *Prepare is advisory validation only*: failure here means the UI rejects the user's action and never authors the operation. Prepare does not bind semantics; an operation that successfully prepares at one replica **MAY** be reduced to a conflict effect at another.

Commit. The operation envelope is added to the replicated operation set. After commit, the operation is part of canonical state and **MUST NOT** be discarded outside the pruning

protocol.

Reduce. During materialization, the operation is processed by the canonical reduction algorithm against the current materialized state. Reduction is deterministic: every replica with the same operation set produces the same materialized state.

Report. The reduction produces a deterministic `OperationEffect` for each operation. Effects are visible graph facts: users can inspect what each operation did, and subsequent operations can address conflicts and no-ops directly.

6.3.2 The Reduction Algorithm

Requirement 6.7

The materialized score state is computed from the operation set as follows:

1. Sort the operation set into the canonical reduction order (Section 6.3.3).
2. Group consecutive operations that share a transaction id into transaction blocks.
3. For each non-transaction operation, apply the operation to the working state, computing its `OperationEffect`.
4. For each transaction block, apply all member operations atomically: either all succeed and the transaction reduces to `Applied` (with each member's individual effect recorded), or some member fails its invariant preconditions, in which case the whole transaction reduces to a `Conflicted` effect with a recorded conflict.
5. Record every effect in the materialized state's effect log (which is part of canonical state, not implementation detail).

Given the same operation set, conformant implementations **MUST** produce score states with byte-identical structural content and byte-identical effect logs. Differences in implementation language, storage layout, threading model, or hardware **MUST NOT** produce divergent reductions.

6.3.3 Canonical Reduction Order

Requirement 6.8

The canonical reduction order is the lexicographic ordering by:

1. Causal order: for operations A and B, if A is in B's causal context (transitively, via DVV closure), then A precedes B.
2. For operations not ordered by causal order (i.e., concurrent operations), lexicographic order by:
   (a) `stamp.hlc.physical_time` (ascending)
   (b) `stamp.hlc.logical_counter` (ascending)
   (c) `stamp.id.replica` (ascending, lexicographic on the replica identifier's canonical byte form)
   (d) `stamp.id.counter` (ascending)

> Causal order strictly dominates HLC order: wall-clock skew may affect the ordering of concurrent operations but **MUST NOT** cause a causal predecessor to sort after a causal successor. The HLC authoring rule (Section 6.2) guarantees this by construction.

6.3.4 Operation Effects

```rust
/// The deterministic effect of an operation under canonical
/// reduction. Recorded as part of materialized state.
pub enum OperationEffect {
    /// The operation applied cleanly with no compensating changes.
    Applied,

    /// The operation applied with deterministic compensating changes
    /// (e.g., re-anchoring records, attachment migration).
    AppliedWithRepair { repairs: Vec<RepairRecord> },

    /// The operation could not apply cleanly; a conflict record was
    /// created in the conflict registry.
    Conflicted { conflict: ConflictId },

    /// The operation's target was tombstoned. The operation is
    /// preserved in the operation set but has no effect on the
    /// materialized graph beyond the recorded effect.
    TombstonedTarget { target: TypedObjectId },

    /// The operation reduces to no effect. The reason is recorded
    /// and is part of canonical state.
    NoOp { reason: NoOpReason },
}

pub enum NoOpReason {
    /// The operation's target was tombstoned by a causally-prior
    /// operation, and the target operation has no compensating
    /// repair rule.
    TargetTombstoned,

    /// The operation duplicates a causally-prior operation's effect.
    AlreadyApplied,

    /// A later operation in the canonical order subsumed this
    /// operation's effect (e.g., a later DeleteEvent on the same
    /// target supersedes an earlier RespellPitch).
    SupersededByLaterOperation { superseder: OperationId },

    /// An invariant precondition that was satisfied at authoring
    /// time fails when the operation is reduced under concurrent
    /// edits. Distinguished from Conflicted because the operation's
    /// intent is not preserved: it simply has no applicable effect.
    /// The failure reason is a typed enum, not a free-form string:
    /// canonical effects MUST NOT contain free-form text, since two
    /// implementations could otherwise produce divergent canonical
```

```
    /// state while agreeing semantically.
    PreconditionFailedUnderReduction {
        reason: PreconditionFailureReason,
    },

    /// The operation belonged to a transaction whose other members
    /// failed; the transaction was conflicted, and this operation
    /// produced no individual effect.
    TransactionConflict,
}

pub enum PreconditionFailureReason {
    /// The target object identifier did not exist in the working
    /// state at the moment of reduction.
    TargetMissing,

    /// The target object was tombstoned by a causally-prior
    /// operation. Distinguished from NoOpReason::TargetTombstoned
    /// in that the kind's rules permitted an attempt at
    /// repair-on-tombstone, but the repair itself failed precondition.
    TargetTombstoned,

    /// The operation requires a region time model (e.g., metric)
    /// different from the one currently in effect at the target
    /// position.
    WrongRegionTimeModel,

    /// A tuplet compensation declared by the operation is invalid
    /// against the current tuplet structure at the target.
    TupletCompensationInvalid,

    /// An event duration the operation specifies is invalid in
    /// the target voice or region (e.g., overlapping a tied
    /// chord-onset boundary in an incompatible way).
    EventDurationInvalid,

    /// The operation's target position falls outside the region
    /// declared by its envelope, or addresses a region that does
    /// not exist.
    PositionOutsideRegion,

    /// A pitch-space or tuning-context precondition failed: e.g.,
    /// the operation declared a pitch in a pitch space that is not
    /// active at the target region.
    PitchSpaceMismatch,

    /// A voice precondition failed: the operation targeted a
    /// voice that does not exist or has been tombstoned.
    VoiceMissing,

    /// An extension-declared precondition failed. The extension
    /// is identified; the extension's own catalog enumerates the
    /// specific precondition codes.
```

```
    ExtensionPrecondition(ExtensionPreconditionId),

    /// Registered precondition code defined by a versioned
    /// registry.
    Registered(PreconditionFailureRegistryId),
}

/// A deterministic compensating change made during reduction. Each
/// RepairRecord describes one repair the reduction performed to
/// preserve graph invariants in the presence of a tombstoning or
/// migrating operation. Recorded as part of canonical state.
pub struct RepairRecord {
    /// The kind of repair performed.
    pub kind: RepairKind,

    /// The object affected by this repair.
    pub target: TypedObjectId,
}

pub enum RepairKind {
    /// A reference was re-anchored to a surviving target per the
    /// re-anchoring rule table (Section~\ref{sec:semops:reanchor}).
    Reanchored {
        from: TypedObjectId,
        to: TypedObjectId,
        reason: ReanchorReason,
    },

    /// A spanner or beam had members removed but enough remained
    /// for the structure to survive.
    SpannerTruncated {
        removed_members: Vec<TypedObjectId>,
    },

    /// A user-content reference was orphaned (target lost; reference
    /// preserved for diagnostic purposes).
    Orphaned,

    /// A reference whose existence required its target was
    /// cascade-deleted.
    CascadeDeleted,

    /// An attachment transitioned to tombstoned-target state.
    AttachmentTombstoned,

    /// A voice was promoted to a system-derived voice identity due
    /// to concurrent insertion collision.
    VoicePromoted {
        from: VoiceId,
        to: VoiceId,
    },

    /// A tuplet was compensated according to the operation's
```

```
    /// declared TupletCompensation.
    TupletCompensated {
        compensation_kind: TupletCompensationKind,
    },

    /// A registered extension-defined repair kind.
    Registered(RepairKindRegistryId),
}
```

REQUIREMENT 6.9

Every operation in the operation set **MUST** produce exactly one `OperationEffect` under reduction. Effects **MUST** be deterministic: every replica reducing the same operation set in the canonical order produces the same effect for each operation, including identical `NoOpReason` values where applicable.

6.3.5 Cache and Snapshot Discipline

REQUIREMENT 6.10

Caches, snapshots, and partial reductions are acceleration structures only. They **MUST NOT** affect the canonical materialized state, which is defined solely by the operation set, the active conformance profile, and the normative reduction algorithm. Implementations **MUST** invalidate or recompute caches whenever any of the following change:

- The visible operation set (additions, or pruning operations under the file-format protocol).
- The causal order derived from any envelope's causal context.
- The reduction algorithm version declared by the active conformance profile.
- The active conformance profile itself.

Cache hits **MUST** return data that is bit-identical to the data a fresh reduction would produce. Caches that cannot guarantee this property **MUST** be invalidated.

6.4 Object Existence and Tombstones

6.4.1 Add-Tombstone Semantics

Object lifetimes in the materialized graph follow add-tombstone semantics: an operation creates an object by minting a stable identifier, and an operation deletes an object by tombstoning that identifier. Tombstoned identifiers are retained; their resolution yields a deletion record rather than the original object.

REQUIREMENT 6.11

Identifiers **MUST** have one of three states in the materialized graph:

- **Live.** The identifier resolves to a current object.
- **Tombstoned.** The identifier resolves to a deletion record carrying the operation that performed the deletion and the operation that originally minted the identifier.
- **Unknown.** The identifier has not been seen by this replica. This state is observable in collaborative scenarios where envelopes arrive out of order; it resolves to either Live or Tombstoned once causal predecessors arrive.

Implementations **MUST NOT** resurrect tombstoned identifiers (recreating an object with a previously-tombstoned id) except through an explicit inverse-of-delete operation in the operation set. Independent re-creation is forbidden.

6.4.2 Tombstone Retention

REQUIREMENT 6.12

Tombstones **MUST** be retained while any of the following hold:

- Pending replication: operations referencing the tombstoned identifier may not yet have been observed by all replicas.
- Pending undo: a transaction containing the deletion is within undo range.
- Active conflict resolution: a conflict record references the tombstoned identifier and has not been resolved.
- Attachment resolution: a tombstoned-target attachment remains in the score for historical or diagnostic purposes.

Tombstone garbage collection is governed by the pruning protocol in Chapter 8. Tombstones **MUST NOT** be discarded outside that protocol.

6.4.3 Reference Resolution Across Tombstones

REQUIREMENT 6.13

A reference to a tombstoned identifier resolves to a `TombstonedTarget` state rather than being silently invalidated. Operations targeting tombstoned identifiers reduce to `TombstonedTarget` or `NoOp` effects according to their kind's declared rules (Section 6.12). Re-anchoring rules (Section 6.7) define how cross-cutting structures and attachments respond to tombstoned references at reduction time.

6.5 Conflict Records

When an operation cannot apply cleanly, the reduction produces a `Conflicted` effect referencing a `ConflictRecord` in the canonical conflict registry (a component of materialized state; see below). Conflict records are first-class: stable, addressable, visible to users, and addressable

by subsequent operations.

6.5.1 Conflict Record Type

```rust
pub struct ConflictRecord {
    /// Content-derived identifier (Section below).
    pub id: ConflictId,

    /// The operations that participated in producing this conflict.
    /// At least two for a true conflict; one for an operation that
    /// failed precondition checking under reduction.
    pub caused_by: Vec<OperationId>,

    /// The kind of conflict, governing what resolution options
    /// exist.
    pub kind: ConflictKind,

    /// Objects affected by this conflict, for diagnostic and UI
    /// navigation purposes.
    pub affected_objects: Vec<TypedObjectId>,

    /// Current resolution state. Conflicts begin Unresolved; a
    /// later ResolveConflict operation transitions them to Resolved
    /// or Dismissed.
    pub resolution_state: ConflictResolutionState,
}

pub enum ConflictKind {
    /// Two concurrent operations attempted to write the same
    /// non-LWW field. The winning operation's effect is in the
    /// materialized state; the losing operation's effect is in the
    /// conflict record for user inspection.
    StructuralFieldCollision {
        winner: OperationId,
        loser: OperationId,
        field: FieldPath,
    },

    /// A transaction's primitive members were partially applicable
    /// under reduction; the whole transaction was rejected.
    TransactionConflict {
        transaction: TransactionId,
        failed_members: Vec<OperationId>,
    },

    /// An operation targeted a tombstoned object and could not be
    /// repaired by the kind's declared rules.
    TombstonedTarget {
        target: TypedObjectId,
        operation: OperationId,
    },

    /// Re-anchoring failed: no deterministic target could be found.
```

```
    ReanchorFailure {
        original_referent: TypedObjectId,
        referencing_object: TypedObjectId,
    },

    /// A change-region-time-model operation produced contained
    /// events whose coordinate kinds are incompatible with the new
    /// time model; the migration could not be performed
    /// deterministically.
    TimeModelMigrationFailure {
        region: RegionId,
        incompatible_events: Vec<EventId>,
    },

    /// A registered extension operation's reduction failed in a
    /// way described by the extension. Opaque to the core.
    ExtensionConflict {
        kind_id: ConflictKindRegistryId,
        details: SerializedExtensionConflict,
    },
}

pub enum ConflictResolutionState {
    Unresolved,
    Resolved { by: OperationId, action: ResolutionAction },
    Dismissed { by: OperationId },
}

// See Requirement (field-collision effect tag): for a
// StructuralFieldCollision the winner (later op) reads Conflicted;
// the earlier op retains Applied.

pub enum ResolutionAction {
    /// Accept the losing operation's effect (replacing the winner).
    AcceptLoser,
    /// Reapply the winner explicitly (no semantic change; clears the
    /// conflict).
    KeepWinner,
    /// User-authored replacement that supersedes both.
    Override { override_operation: OperationId },
    /// Re-anchor to a user-chosen target.
    Reanchor { new_target: TypedObjectId },
    /// Dismiss the conflict without changing the materialized graph:
    /// the user acknowledges it and accepts the current (winner)
    /// state. Selects the Dismissed resolution state.
    Dismiss,
    /// Custom resolution for registered conflict kinds.
    Registered(ResolutionRegistryId),
}
```

REQUIREMENT 6.14

The discriminant byte of `ResolutionAction` is its declaration-order index in the listing above: `AcceptLoser` = 0, `KeepWinner` = 1, `Override` = 2, `Reanchor` = 3, `Dismiss` = 4, `Registered` = 5. A `ResolutionAction` is encoded into the `ResolveConflict` operation payload and therefore into that operation's content hash, so this assignment is normative and stable and **MUST NOT** be reordered. `Dismiss` was assigned `4` ahead of `Registered` (`5`) when the `Dismissed` resolution state was made reachable by an authored operation (Section 6.5.4 below), fixing the wire form against a silent shift.

REQUIREMENT 6.15

When two concurrent operations write the same non-LWW field and reduction records a `StructuralFieldCollision`, the per-operation `OperationEffect` is assigned as follows: the *winner* — the operation later in canonical order, whose choice materializes and whose processing created the conflict record — reads `Conflicted{conflict}`; the *earlier* (losing) operation retains the `Applied` effect it received when it was first reduced. The winner carries the flag because it is the single operation that both materializes the field and observes the collision, which makes the assignment independent of the order in which envelopes arrive at any replica.

RATIONALE

The alternative — tagging the loser `Conflicted` (or `NoOp{SupersededByLaterOperation}`) because its intent did not survive — is defensible and arguably more intuitive for a UI that surfaces "your edit was overridden." We pin *winner-carries* instead because it is computed at the moment the collision is detected (when the winning operation is reduced against already-present state), needs no second pass to retroactively re-tag an earlier operation, and is manifestly order-independent. A UI that wishes to tell the losing author their edit was overridden reads the `StructuralFieldCollision` record's `loser` field rather than the loser operation's effect.

6.5.2 The Conflict Registry

Canonical materialized state carries a top-level conflict registry (the score graph does not — conflict records are a reduction product, not authored content):

```
pub struct ConflictRegistry {
    pub records: Vec<ConflictRecord>,
}
```

REQUIREMENT 6.16

The conflict registry is part of canonical materialized state. Conflict records persist until they are resolved or dismissed by explicit `ResolveConflict` operations. Conflict records **MUST NOT** be silently removed; their lifetime is governed by the operation set, not by UI state.
The layout pipeline (Chapter 7) **SHOULD** provide visual indication of unresolved conflicts at their affected objects; this is the user-facing manifestation of the conflict registry's contents.

6.5.3 ConflictId Derivation

Conflict records are produced during deterministic reduction, not authored by replicas. Their identifiers **MUST** therefore be content-derived, not minted from a local replica-plus-counter sequence. Otherwise, two replicas reducing the same operation set would produce conflict registries with disagreeing identifiers, breaking canonical state.

REQUIREMENT 6.17

Each `ConflictId` **MUST** be derived deterministically from the conflict's content via BLAKE3 truncation:

```
fn derive_conflict_id(
    kind: &ConflictKind,
    causing_operations: &[OperationId],
    affected_objects: &[TypedObjectId],
) -> ConflictId {
    let mut preimage = Vec::new();
    preimage.extend_from_slice(b"MUSCCONF");
    preimage.extend_from_slice(&kind.canonical_bytes());

    // Sort causing operations lexicographically by canonical bytes
    let mut sorted_ops = causing_operations.to_vec();
    sorted_ops.sort_by(|a, b| a.canonical_bytes().cmp(&b.canonical_bytes()))
        ;
    for op in &sorted_ops {
        preimage.extend_from_slice(&op.canonical_bytes());
    }

    // Sort affected objects lexicographically by canonical bytes
    let mut sorted_objs = affected_objects.to_vec();
    sorted_objs.sort_by(|a, b| a.canonical_bytes().cmp(&b.canonical_bytes())
        );
    for obj in &sorted_objs {
        preimage.extend_from_slice(&obj.canonical_bytes());
    }

    let hash = blake3(&preimage);
    ConflictId(u128::from_be_bytes(
        hash[0..16].try_into().unwrap()
```

```
        ))
    }
```

The canonical `ConflictKind` byte representation **MUST** include the kind discriminant and all kind payload (winning operation, losing operation, field path, transaction id, tombstoned target, original referent, region, extension kind identifier, etc.), so that distinct conflicts have distinct preimages by construction. Implementations **MUST NOT** introduce an ordinal or local counter into the preimage; conflicts that share kind, causing operations, and affected objects are by definition the same conflict.

RATIONALE

Content-derived identifiers are the only mechanism that makes the conflict registry a deterministic materialized fact. Every replica reducing the same operation set produces the same conflict records with the same identifiers, addressable by subsequent `ResolveConflict` operations whose payloads carry those identifiers by reference.
Truncation from 256 to 128 bits is safe: the birthday-collision probability for any practical conflict count is vanishingly small, and the same truncation discipline already governs other 128-bit identifiers in the format.

6.5.4 Conflict Resolution Operations

```rust
pub struct ResolveConflictPayload {
    pub target: ConflictId,
    pub action: ResolutionAction,
}
```

REQUIREMENT 6.18

A `ResolveConflict` operation transitions a conflict's state from `Unresolved` to one of two states selected by its `action`. The `ResolutionAction::Dismiss` action transitions the conflict to `Dismissed` (no semantic change to the materialized graph beyond the conflict record's state: the user acknowledges the conflict and accepts the current winner state). Every other `action` transitions the conflict to `Resolved` with that action applied. Both resolution states are thus reachable by an authored operation; `Dismiss` is the authored selector for `Dismissed`, closing the prior gap in which `Dismissed` was a representable state with no operation to reach it.
Concurrent `ResolveConflict` operations on the same conflict are themselves subject to canonical reduction: the earlier operation under the canonical order is applied; the later operation reduces to `NoOp` with reason `AlreadyApplied` if its action is equivalent, or to `Conflicted` with a meta-conflict record if its action differs.

6.6 Transactions

Transactions are first-class replicated operation groups. They reduce atomically: either every primitive member applies, or the whole transaction conflicts.

```rust
pub struct TransactionDescriptor {
    pub id: TransactionId,

    /// Human-readable label, for the undo history and diagnostics.
    pub label: String,

    /// Optional categorization (used by UIs and analytics).
    pub category: Option<TransactionCategory>,
}

pub struct TransactionId(pub u128);

/// Open vocabulary: a minimal core set plus a Registered escape for
/// extension-defined categories.
pub enum TransactionCategory {
    NoteEntry,
    Structural,
    Layout,
    Import,
    Registered(OperationKindRegistryId),
}
```

REQUIREMENT 6.19

`TransactionCategory` is an *open* vocabulary. Its normative core set is exactly `NoteEntry`, `Structural`, `Layout`, and `Import`; any other category **MUST** be expressed as `Registered(OperationKindRegistryId)` rather than by extending the core set. The field is advisory (consumed by UIs and analytics, never by canonical reduction), so the minimal core set is the right tradeoff: under-specifying is cheap to extend through `Registered`, whereas baking speculative categories into the core vocabulary is not. The discriminants follow declaration order (`NoteEntry` $= 0$ through `Registered` $= 4$).

Each primitive operation belonging to a transaction carries the transaction's identifier in its envelope's `transaction` field. The transaction descriptor itself is a separate operation envelope (`DeclareTransaction`) whose payload contains the `TransactionDescriptor`.

6.6.1 Transaction Descriptor Causal Ordering

The canonical reduction order is causal-first, then HLC, then operation id (Section 6.3.3). For the descriptor to be available when its members are reduced, the ordering between descriptor and members must be guaranteed by causal dependency, not by HLC stamps.

REQUIREMENT 6.20

Every primitive operation member of a transaction **MUST** causally depend on the transaction's `DeclareTransaction` envelope: the descriptor's `OperationId` **MUST** appear in the member's `causal_context` (either in the per-replica DVV range or among its dots). Authoring implementations **MUST** observe and include the descriptor in the causal context of each member they author.

Delivery order need not respect this dependency: an out-of-order delivery in which a member arrives before its descriptor is normal and is handled by the canonical reduction algorithm, which sorts by causal order before reducing.
At reduction time:

- If the descriptor is present in the operation set and causally precedes every member: the transaction is well-formed; it is reduced atomically per the rule below.
- If a primitive operation declares membership in a transaction whose `DeclareTransaction` envelope is absent from the operation set, or whose envelope is present but does not causally precede the member: the member is malformed against the transaction model. It reduces to `NoOp{reason: TransactionConflict}` and contributes to a synthesized conflict record of kind `ConflictKind::TransactionConflict` naming the malformed member and the missing-or-misordered descriptor. The member is preserved in the operation set (the CRDT property prohibits removal) but produces no effect on canonical state beyond the recorded conflict.

Rationale

Requiring causal dependency rather than HLC ordering matches how transactions actually arise at the authoring replica: a user begins an edit (descriptor), then commits its primitives. The primitives can causally observe the descriptor because they are authored after it on the same replica. The canonical reduction order then handles them correctly without any special transaction-extraction phase.
This rule also makes equivocating descriptors impossible: a member cannot claim membership in a transaction whose descriptor it has not seen.

Requirement 6.21

Transactions reduce atomically. During reduction:

- All primitive members of a transaction are gathered into a transaction block at the point of the earliest member's canonical position.
- The block is reduced as a single unit. Each member is evaluated against the working state produced by prior members within the same block.
- If every member's invariant preconditions are satisfied, the transaction reduces to `Applied`; each member's individual effect is recorded as part of the transaction's composite effect.
- If any member's invariant preconditions fail, the entire transaction reduces to `Conflicted`: a conflict record is created with kind `ConflictKind::TransactionConflict`, listing the failed members; no member's effect is applied; each member's individual effect is recorded as `NoOp{reason: TransactionConflict}`.

Primitive members of a transaction **MUST NOT** be observable as independently committed score states. Observers see either the pre-transaction state or the fully applied post-transaction state; they never see a partial intermediate.

Rationale

Atomic transaction reduction is essential for the tuplet compensation model (Section 6.12.2): the delete operation and its compensating insert (or rewrite, or cascade) are committed together as a single transaction, so the score is never observably broken between them.
Local materialization may stream a transaction's primitives to the UI for responsive feedback, but the materialized canonical state visible to remote observers transitions only at transaction boundaries.

6.7 Re-Anchoring

When an operation tombstones an object, every other object referencing it must respond deterministically. The response is specified as a total deterministic function.

6.7.1 The Re-Anchor Function

```
pub enum ReanchorResult {
    /// The reference is replaced with a reference to a new target.
    Reanchored {
        new_target: TypedObjectId,
        reason: ReanchorReason,
    },

    /// The reference is preserved but its target is tombstoned.
    /// The referencing object remains in the graph in a
    /// tombstoned-target state.
    TombstonedTarget,

    /// The referencing object is marked orphaned but retained.
    Orphaned,

    /// Re-anchoring cannot be performed deterministically; a
    /// conflict is recorded.
    Conflicted { conflict: ConflictId },

    /// The referencing object is cascade-deleted (tombstoned).
    CascadeDeleted,
}

pub enum ReanchorReason {
    SameVoiceNearer,
    SameStaffInstanceNearer,
    SameStaffNearer,
    SameRegionNearer,
    ExplicitFallback,
    DeclaredByExtension(ReanchorReasonRegistryId),
    /// A rank-4 (same-canvas) proximity survivor. Appended in
    /// Pass 12 (P12-C4); its wire discriminant is 6 because
    /// DeclaredByExtension already owned 5 when it was appended.
```

```
    SameCanvasNearer,
}
```

A re-anchor that selects a rank-4 (same-canvas) survivor records `SameCanvasNearer` (ratified Pass 12); before the variant was appended, such repairs were recorded `ExplicitFallback`, a recording this revision supersedes.

6.7.2 Total Ordering for "Nearest"

The notion of "nearest surviving anchor" is defined as a total ordering. There is no discretionary search and no implementation freedom to choose among "equally near" candidates.

REQUIREMENT 6.22

For an object kind K requiring re-anchoring to the nearest surviving K, "nearest" is computed as the strict lexicographic minimum over the surviving candidates of:

1. Containment proximity (smaller is closer): same voice (0), same staff instance (1), same staff (2), same region (3), same canvas (4). Candidates further away than the kind's declared maximum proximity are excluded.
2. Absolute time distance from the tombstoned referent's resolved position, measured under a canonical conversion (musical time within metric regions, wall-clock time within proportional regions, declared per-region for aleatoric regions).
3. Direction preference: forward (0) before backward (1), unless the kind declares the opposite preference.
4. Object identifier (lexicographic on the typed identifier's canonical byte form, ascending) as the final tie-break.

If no candidate satisfies the kind's declared proximity bound, the result is `Orphaned` (for user-content kinds such as comments and analytical annotations) or `CascadeDeleted` (for kinds whose existence requires their target, such as ties and beams).
Numeric proximity bounds, direction preferences, and the orphaned-versus-cascade choice are declared per kind in the re-anchoring rule table below; they are part of the kind's conformance contract and **MUST NOT** be reinterpreted by implementations.

6.7.3 The Re-Anchoring Rule Table

The following table specifies, for each (referencing kind, referent kind) pair, the re-anchoring action under canonical reduction. The table is normative.

Referencing	Referent	Action	Parameters
Slur	Endpoint event	Re-anchor to the nearest surviving endpoint; cascade-delete only when no endpoint survives	a two-endpoint slur collapses onto its sole survivor (a reference-clean degenerate form); proximity-aware re-targeting is a deferred refinement
Slur	Interior event	No action	—
Tie	Either endpoint	Cascade-delete	Tie's existence requires both endpoints.
Beam	Member event	Truncate. Cascade-delete if fewer than two members remain	proximity max: same voice
Tuplet	Member event	Operation **MUST** provide tuplet compensation (Section 6.12.2); otherwise the operation conflicts at precondition check. A member tombstoned with no compensation to declare (an undo of the transaction that inserted it, or a cascade) cascade-deletes the tuplet	—

Referencing	Referent	Action	Parameters
Spanner	Anchor	Re-anchor to the nearest surviving anchor; cascade-delete only when no anchor survives	as for slurs (surviving-endpoint collapse); per-spanner-kind proximity bounds are a deferred refinement
Repeat structure	Anchor	Re-anchor to the nearest surviving anchor; cascade-delete only when no anchor survives	as for spanners; the rule spans *every* event-referencing anchor site of the structure — `start/end`, the kind's jump targets (`DaCapo.end_target`, `DalSegno.segno/end_target`), and each volta's span. Among multiple surviving candidates "nearest" is currently the deterministic identifier-order minimum; proximity-aware (four-key) selection is a deferred refinement, as for spanners (ratified with the repeat-authoring pair, schema-major-2 revision)
Marker	Anchor	Re-anchor to nearest event in same staff instance	proximity max: same staff instance; orphan on failure
Lyric	Event	Cascade-delete	a syllable belongs to its event (schema major 5)
Comment	Anchor	Orphan	User content never silently deleted.
Cue event	Source event	Cascade-delete on any source deletion	a multi-source cue cascades when *any* source dies (ratified Pass 12): losing any source breaks the cue's quotation integrity; truncate-while-any-source-survives was considered and rejected

Referencing	Referent	Action	Parameters
Graphic gesture	Anchor event	Re-anchor to nearest surviving event of same staff instance; for Free anchoring, no action; for Range anchoring, truncate	proximity max: same staff instance. Truncate (ratified Pass 12): a dead event-anchored range endpoint moves to its containing region's edge — a start endpoint to the region start, an end endpoint to the region end, zero offset
Trajectory event	Endpoint pitch	Re-anchor: replace `EventPitch(PitchId)` with `ExplicitPitch` capturing the tombstoned pitch's last known value	—
Analytical annotation	Anchor	Re-anchor to time range preserving original extent; orphan if range cannot be reconstructed	orphaning is the sanctioned outcome (ratified Pass 12) for a wall-clock (region-relative) or indeterminate event span that no stored `Range` anchor form can express without region-origin resolution
Spelling attachment	Target pitch	Transition to tombstoned-target state	Attachment is preserved for diagnostic and undo purposes.
Decomposition attachment	Target event	Transition to tombstoned-target state	Attachment preserved for diagnostic and undo purposes.
Voice	Containing staff instance	Cascade-delete	—
Event	Containing voice	Cascade-delete	—
Region	Canvas	Refuse (canvas is not deletable)	—

Referencing	Referent	Action	Parameters
Part definition	Referenced staff	Remove staff from part's staff list; orphan if part becomes empty	—

Requirement 6.23

Re-anchoring **MUST** be performed as part of the same reduction step that tombstones the referent. The resulting graph state **MUST** satisfy every invariant in Section 5.11. Re-anchoring actions **MUST** be recorded as `RepairRecord` entries in the triggering operation's effect.

6.8 Undo

Undo is not literal time travel: it is the authoring of a new operation that compensates for a previously-committed transaction against the current materialized state. This preserves the append- only operation set and is correct under concurrent edits.

6.8.1 UndoTransaction Payload

```rust
pub struct UndoTransactionPayload {
    /// The transaction being undone.
    pub target: TransactionId,

    /// Policy governing how to undo when the target's effects
    /// have been partially superseded by later operations.
    pub policy: UndoPolicy,
}

pub enum UndoPolicy {
    /// Compute the inverse semantic operations against the current
    /// materialized state. If any cannot apply cleanly (because the
    /// target objects have been tombstoned, fields have been
    /// further modified, etc.), the entire undo conflicts.
    StrictInverse,

    /// Undo what remains valid; record conflicts or no-ops for the
    /// rest. The undo operation succeeds, but partial.
    BestEffort,

    /// Also undo operations that are causally or semantically
    /// dependent on the target transaction. Dependency is computed
    /// from causal contexts and explicit dependency links;
    /// implementations MUST NOT infer broad musical dependence
    /// heuristically.
```

```
    Cascade,
}
```

6.8.2 Undo Semantics

REQUIREMENT 6.24

An `UndoTransaction` operation is committed to the operation set like any other operation; it does not modify history. Its reduction computes a compensating edit against the materialized state at the point of its canonical position:

- For `StrictInverse`: the inverse of each primitive in the target transaction is computed against the current working state. If every inverse applies cleanly, the undo transaction reduces to `Applied` with all compensating effects recorded. If any inverse cannot apply (target tombstoned, field modified, etc.), the undo transaction reduces to `Conflicted` with a conflict record.
- For `BestEffort`: inverses that apply cleanly are applied; inverses that cannot are recorded as `NoOp` effects with their reasons. The undo transaction reduces to `AppliedWithRepair`.
- For `Cascade`: the dependency set is computed first (causal closure plus explicit dependency links). The transaction's inverses, plus inverses of dependents in reverse order, are applied per the chosen sub-policy (typically `StrictInverse`).

Equality of the resulting graph state to the pre-target state is defined as *content equivalence*: every live object's fields, every attachment, every cross-cutting structure matches. Operation identifiers, stamps, undo stacks, caches, and storage layout are *not* required to match.

RATIONALE

Defining undo as a forward operation eliminates the contradiction in earlier passes where undo had to "restore previous state" while remote edits had been integrated. The compensating-edit approach respects the operation set's append-only nature and generalizes cleanly to concurrent collaboration: undoing a transaction whose effects have been entwined with remote edits is expressible as a deterministic compensating operation, possibly partial.

6.9 LWW Discipline

Last-writer-wins reduction is permitted only for explicitly-marked advisory scalar fields. Structural musical objects are governed by their kind's operation-specific reduction rules.

6.9.1 The LwwAdvisory Marker

Fields eligible for LWW reduction are explicitly marked as `LwwAdvisory`. No unmarked field may use LWW semantics.

```rust
/// Marker attribute, applied to fields whose concurrent modifications
/// are resolved by last-writer-wins under the canonical reduction
/// order.
#[attribute_only]
pub struct LwwAdvisory;

// Example field declarations:
//
//   #[lww_advisory]
//   pub title: Option<String>
//
//   #[lww_advisory]
//   pub composer: Option<String>
```

6.9.2 Eligible Fields

REQUIREMENT 6.25

The following classes of field **MAY** be marked `LwwAdvisory` and reduced by last-writer-wins:

- Score metadata: title, subtitle, composer, lyricist, copyright, work number, opus, dedication, free-form annotations.
- View preferences: which layers are active, which parts are shown, viewport, zoom level (per-replica state in addition; only the explicitly shared portions are LWW).
- Layout hint preferences: default page size, default margins, default staff spacing recommendations.
- Page-break and system-break preferences on layout-semantic structures (the user's stated preference, distinct from the engraver's decision).
- Optional advisory display hints on cross-cutting structures: line color preference, line-style preference, label-position preference. Endpoints, target identities, semantic class, and attachment ownership **MUST NOT** be LWW.

All other fields, including every field carrying musical structure, identity, ownership, or semantics, **MUST** use operation-specific reduction rules defined per operation kind. Implementations **MUST NOT** default to LWW for unmarked fields.

RATIONALE

LWW is acceptable for fields where the worst case of arbitrary deterministic resolution is benign: two authors editing the composer's name concurrently is a metadata collision, not a musical loss. LWW is catastrophic for fields where wholesale replacement destroys musical meaning: two authors editing a chord's pitch list concurrently must not produce "one author's chord wins." The marker discipline makes the boundary explicit and auditable.

6.10 Validation Modes

Operations are subject to precondition checking. Two modes distinguish authoring contexts:

Authoring mode. Interactive edits. All preconditions are enforced: invariant preconditions (which preserve graph invariants) and advisory preconditions (which encode user-intent constraints, range checks, and style policy).

Replay mode. Replaying historical operations or applying remote operations under reduction. Only invariant preconditions are enforced; advisory preconditions are skipped, since they represent the authoring replica's local policy at the moment of authoring, not invariants of the canonical state.

REQUIREMENT 6.26

Every operation specification **MUST** classify each precondition as invariant or advisory. Invariant preconditions **MUST** hold in all modes; advisory preconditions **MUST** hold in authoring mode and **MAY** fail silently in replay mode.
Implementations **MUST NOT** reclassify a precondition without a corresponding revision to this specification.

6.11 Per-Kind Reduction Summary

The following table summarizes the reduction discipline for each representative operation kind. Detailed specifications appear in the next section.

Operation	Reduction Rule	Concurrency Outcomes
InsertEvent	Position-keyed insertion. Concurrent same-position inserts whose durations overlap promote the lexicographically-greater operation to a system-promoted voice (Section 5.6.3)	Voice promotion; non-overlap invariant preserved by allocation.
DeleteEvent	Tombstone the event and contained pitch identifiers. Apply tuplet compensation from the operation. Re-anchor referencing structures	Delete-wins against concurrent field modifications on the same event; concurrent same-target deletes are idempotent.

Operation	Reduction Rule	Concurrency Outcomes
RespellPitch	Field overwrite by canonical reduction order; the later operation wins. Earlier respellings of the same pitch are preserved in the conflict record if they differ	Conflict record records the losing spelling for user inspection.
Transpose	Apply in canonical reduction order. Pitch identifiers preserved. Operations do not commute	Reduction order decides; conflict record entries for non-trivial intervals only when explicit dependency rules require.
CreateCrossCutting	Set union. Concurrent additions of distinct cross-cutting structures all succeed	Tombstoned endpoints cause re-anchoring or cascade-delete per kind.
ChangeRegionTimeModel	Structural migration. Contained events whose coordinate kinds become incompatible cause the operation to conflict	Concurrent contained edits authored before the migration may conflict; explicit rebase rules per migration kind.
SetUserSystemBreak	LwwAdvisory on per-location break preference	Last-writer-wins on the break state at the same anchor.

6.12 Representative Operations

The remainder of the chapter specifies a representative selection of operations, one from each major category. The full catalog is delivered as a separate conformance specification.

These examples illustrate the contract that every operation in the catalog **MUST** satisfy. Each operation specification includes its reduction rule under the canonical reduction model (Section 6.3).

6.12.1 InsertEvent

Category: Event operations.

```
pub struct InsertEventOp {
    /// The voice the event is inserted into.
    pub voice: VoiceId,

    /// Position within the voice.
    pub position: EventPosition,
```

```
    /// The event to insert. Its EventId and any contained PitchIds
    /// are minted at operation-authoring time as a single causal
    /// step. The event's voice field must match the target voice.
    pub event: Event,
}
```

Invariant preconditions:

- The target voice exists.
- The event's `voice` field equals the target voice's id.
- The event's `id` is not present elsewhere in the arena.
- Every `PitchId` contained in the event (in an `IdentifiedPitch`, a trajectory endpoint, or a stepwise trajectory shape) is unique within the score's pitch-identity index.
- The position is within the enclosing region's time extent.
- The event's position and duration variants agree with the enclosing region's time model.
- Inserting at the position does not produce an event overlap within the voice. Concurrent inserts that would otherwise produce overlap are resolved per the reduction rule below.

Advisory preconditions (authoring mode only):

- For pitched events, every pitch is within the instrument's declared range, if any.
- The event's duration does not extend past a region boundary in a way that would require splitting.

Effect: The event is added to the arena; the voice's event list is updated to include the event in sorted position. Every contained `PitchId` is registered as live in the pitch-identity index. Any attachments declared on the event (spellings, decompositions) are added to their respective indexes.

Postconditions:

- The voice's events list remains sorted and non-overlapping.
- The event arena's lookup index resolves the new event.
- The event time index includes the new event for its region and voice.
- Every contained pitch is resolvable through the pitch-identity index.

Inverse: `DeleteEvent` of the inserted event, with captured state equal to the event itself plus any attachments added.

Reduction rule: Position-keyed insertion with voice promotion on collision. Concurrent inserts at the same position whose durations would overlap resolve as follows: the operation with the lexicographically-greater `OperationId` is placed in a newly allocated voice on the same staff instance, with `VoiceOrigin::SystemPromoted` (Section 5.6.3). The new voice's `VoiceId` is derived deterministically from the staff instance, the original target voice, and the two operations' identifiers, so all replicas independently allocate the same promoted voice. The non-overlap invariant is preserved by allocation rather than by rejection. Subsequent user normalization operations can merge the promoted voice back into the target, rename it, or accept it as an intentional second voice.

6.12.2 DeleteEvent

Category: Event operations.

```
pub struct DeleteEventOp {
    pub event: EventId,

    /// Compensation for tuplet membership, if any. Required when the
    /// target event is a member of one or more tuplets.
    pub tuplet_compensation: TupletCompensation,
}

pub enum TupletCompensation {
    /// Target is not in any tuplet; no compensation required.
    NotInTuplet,

    /// Replace the deleted event with a rest of the same duration.
    /// The rest receives a freshly-minted EventId; the deleted
    /// event's EventId and PitchIds are tombstoned.
    ReplaceWithRest { new_rest: Rest },

    /// Rewrite the enclosing tuplet(s) to remain structurally
    /// consistent. The v1 payload names the affected tuplets by
    /// id only; it does not carry rewritten values (see the
    /// precondition below).
    RewriteTuplets { tuplets: Vec<TupletId> },

    /// Cascade-delete the entire tuplet group(s) containing the
    /// target event. The listed tuplet ids are cascaded.
    CascadeDeleteTuplets { tuplets: Vec<TupletId> },
}
```

Invariant preconditions:

- The target event exists in the arena (i.e., is live, not tombstoned).
- If the target event belongs to one or more tuplets, the `tuplet_compensation` field **MUST** be a variant other than `NotInTuplet`, and the chosen compensation **MUST** preserve the tuplet structural consistency invariant (Section 3.9) after the delete is applied.
- If the target event does not belong to any tuplet, the `tuplet_compensation` field **MUST** be `NotInTuplet`.
- For `ReplaceWithRest`: the replacement rest's duration **MUST** equal the deleted event's duration; the rest's voice and position **MUST** match the deleted event.
- For `RewriteTuplets`: the v1 payload carries only tuplet ids, not the rewritten ratio and member values a reducer would need to demonstrate that the resulting structure preserves the tuplet consistency invariant. Graph-aware reduction **MUST** therefore refuse the variant as an ill-formed compensation (a precondition no-op) rather than fabricate rewritten values. A future payload revision carrying full rewrites (per-tuplet ratio and members, each of whose member-duration sums **MUST** match the tuplet's structurally-required total) reopens the variant; until then `ReplaceWithRest` and `CascadeDeleteTuplets` are the applicable compensations.

- For `CascadeDeleteTuplets`: every tuplet listed **MUST** contain the target event as a member.

Effect: The event is removed from the arena and from its voice's event list. The event's `EventId` and every contained `PitchId` are tombstoned in their respective identity indexes: they remain resolvable to "deleted" markers and **MUST NOT** be reused for new objects. The chosen tuplet compensation is applied in the same delta: a replacement rest is inserted or the tuplets are cascade-deleted as specified (a `RewriteTuplets` compensation is refused at precondition check; see above). Cross-cutting structures referencing the event or its pitches are re-anchored per the rule table (Section 6.7). Live attachments targeting the event or its pitches transition to tombstoned-target state and are preserved for replay, undo, and historical reference; they are not silently deleted.

Postconditions:

- The event is no longer live in the arena.
- Every cross-cutting structure that referenced the event has been re-anchored or cascade-deleted.
- The event's identifier and every contained pitch identifier are tombstoned and resolvable only to a deletion record.
- The tuplet structural consistency invariant holds for every tuplet that was previously consistent.

Inverse: `InsertEvent` reconstructing the event from captured state, re-promoting the tombstoned identifiers to live, reversing the tuplet compensation (delete the replacement rest; restore the original tuplet structure; un-cascade tuplet deletions), and re-creating any cross-cutting structures that were cascade-deleted as a consequence. Re-promotion preserves all stable identifiers; new identifiers **MUST NOT** be minted by the inverse.

Reduction rule: Set intersection with delete-wins. Concurrent deletion and modification of the same event resolves to deletion; the modification is dropped. Concurrent re-anchoring on the same referent resolves by the deterministic total ordering specified in Section 6.7.

RATIONALE

Requiring explicit tuplet compensation prevents primitive deletion from leaving the score in a state that violates the tuplet structural consistency invariant. The compensation is part of the same delta as the deletion, so concurrent observers see either the pre-delete state or a fully consistent post-delete state, never an intermediate broken state.

6.12.3 CreateCrossCutting (Slur)

Category: Cross-cutting structure operations.

```
pub struct CreateCrossCuttingOp {
    pub structure: CrossCuttingStructure,
}

pub enum CrossCuttingStructure {
    Slur(Slur),
```

```
    Tie(Tie),
    Beam(Beam),
    Spanner(Spanner),
    Marker(Marker),
    Repeat(RepeatStructure),
    Analytical(AnalyticalAnnotation),
    Comment(Comment),
    GraphicGesture(GraphicGesture),
}
```

The enum above is the data model's full cross-cutting family; the *wire* vocabulary the cross-cutting operations admit at this revision (`CrossCuttingValue`, Binary Format companion) is the four-kind subset `Tie`/`Slur`/`Beam`/`Spanner`. A `RepeatStructure` is authored by the dedicated `CreateRepeatStructure`/`DeleteRepeatStructure` pair (Operation Catalog REPEAT STRUCTURES; ratified schema-major-2 revision), not through `CreateCrossCutting`; the remaining variants await their own authoring primitives.

Invariant preconditions (Slur case):

- The slur's id is unique within the cross-cutting registry.
- The slur's start and end events both exist.
- The start and end events are in the same voice.
- The start event precedes (or equals) the end event in voice order.

Advisory preconditions (Slur case):

- The slur does not span a region boundary, unless explicitly permitted by region configuration.

Effect: The slur is added to the cross-cutting registry's slur list. The cross-cutting reference index is updated.

Postconditions:

- The slur is queryable by id and by referenced events.

Inverse: `DeleteCrossCutting` of the created slur.

Reduction rule: Set union. Concurrent creation of multiple slurs over overlapping ranges is valid: both slurs exist.

6.12.4 RespellPitch

Category: Pitch and tuning operations.

```
pub struct RespellPitchOp {
    pub pitch: PitchId,
    pub new_spelling: PitchSpelling,
}
```

Invariant preconditions:

- The target pitch exists in some event in the arena (i.e., is live, not tombstoned).
- The new spelling is valid in the active pitch space: every accidental in the spelling's `accidentals` stack belongs to the active accidental registry, the nominal exists in the space's nominal registry, and the octave is in range.
- Duplicate accidentals in the spelling's stack are absent unless the accidental's registered combination semantics explicitly permit repetition.
- The new spelling represents the same scale position as the pitch's current spelling, or is enharmonically equivalent under the active tuning system. (Respelling does not change sounding pitch.)

Effect: A spelling attachment with source `UserChosen` is created (or replaces an existing `UserChosen` attachment) targeting the pitch. The spelling attachment index is updated. The pitch's `PitchId` is unaffected.

Postconditions:

- Spelling resolution for the pitch yields the new spelling, subject to the precedence configuration.
- The pitch's scale position is unchanged.
- The pitch's acoustic realization is unchanged.
- The pitch's `PitchId` is unchanged.

Inverse: A `RespellPitch` restoring the prior spelling (captured), or a `RemoveAttachment` if no prior `UserChosen` attachment existed.

Reduction rule: Field overwrite by canonical reduction order. Concurrent respellings of the same live pitch resolve to the operation later in canonical order; the materialized spelling is that operation's choice. The losing respelling is recorded in a `ConflictRecord` of kind `StructuralFieldCollision` if the two respellings represent different spellings (i.e., differ in nominal, accidental stack, or octave). The per-operation effect assignment follows the general field-collision rule (Requirement 6.15): the later operation — the winner, which materializes — reads `Conflicted`, and the earlier operation retains `Applied`. Identical concurrent respellings reduce idempotently: the later operation produces `NoOp{reason: AlreadyApplied}` with no conflict recorded.

6.12.5 ChangeRegionTimeModel

Category: Region operations.

```
pub struct ChangeRegionTimeModelOp {
    pub region: RegionId,
    pub new_time_model: RegionTimeModel,
    pub position_remapping: PositionRemapping,
}

pub enum PositionRemapping {
    /// Preserve absolute time positions where possible. Events that
    /// cannot be remapped (e.g., metric events under a proportional
    /// model with no implied tempo) are converted by the supplied
    /// conversion function.
    PreserveTime { fallback: PositionConverter },
```

```
    /// Discard event positions and reinsert by user-provided
    /// position mapping.
    Reassign(HashMap<EventId, EventPosition>),
}
```

Invariant preconditions:

- The target region exists.
- The new time model is internally consistent.
- The position remapping covers every event in the region, or the remapping is `PreserveTime` with a defined fallback.

Effect: The region's `time_model` is replaced. Every event in the region has its `position` field updated according to the remapping. The tempo map's applicability is recomputed.

Postconditions:

- Every event in the region has a valid position in the new time model.
- The region's invariants (events sorted, non-overlapping, within time extent) hold.

Inverse: `ChangeRegionTimeModel` restoring the prior time model, with a remapping captured from the pre-state.

Reduction rule: Structural migration. The operation is applied at its canonical position. Contained events whose coordinate kinds become incompatible with the new time model after migration cause the operation to reduce to `Conflicted` with a `ConflictKind::TimeModelMigrationFailure` record listing the incompatible events; the migration does not apply. Concurrent operations on the same region that author events in the prior time model and reduce after the migration may themselves reduce to `Conflicted` if their coordinate kinds disagree with the new model. Concurrent same-target migrations resolve by canonical order; the earlier operation applies and the later operation reduces to `Conflicted` with a structural-field-collision conflict (the two migrations are not generally commutable).

RATIONALE

Time-model migration is a structural rebase, not a field set. Treating it as LWW would silently destroy contained musical content whose coordinate kinds depend on the prior time model. Conflicted reductions force the user to address the incompatibility explicitly.

6.12.6 Transpose

Category: Pitch and tuning operations. This is a compound operation, defined as a sequence of primitives, listed here as an illustration of compounds.

User-facing parameters:

- Target scope (events, voice, staff, region, or selection).
- Interval (diatonic, chromatic, or compound).

- Spelling policy (preserve user spellings, respell to fit a target key, or apply a custom rule).
- Tuning policy (preserve, recompute under new tuning, or apply offsets).

Primitive decomposition: A transpose operation expands into:

1. For each target pitch, a `ReplaceEvent` updating the inner `Pitch` values while preserving every `IdentifiedPitch`'s `PitchId`. Transposition does not mint new pitch identifiers; it rewrites the contents of existing identified pitches.
2. For each pitch whose spelling is recomputed, a `RespellPitch` targeting the unchanged `PitchId` with source `Propagated`.
3. Where the transposition crosses a clef boundary, optional `ChangeStaffConfiguration` operations to insert clef changes.

The compound operation is recorded as a single transaction. Each primitive within the transaction has its own re-anchoring and reduction semantics. Identifier preservation ensures that attachments, analytical layers, and cross-cutting references targeting the transposed pitches continue to resolve correctly after the operation.

6.12.7 SetUserSystemBreak

Category: Layout-semantic operations.

```
pub struct SetUserSystemBreakOp {
    pub anchor: TimeAnchor,
    pub region: RegionId,
    pub present: bool,
}
```

Invariant preconditions:

- The region exists and is staff-based.
- The anchor resolves within the region.

Effect: The region's `user_system_breaks` list is updated to include or exclude the anchor. Layout caches that depend on system breaks are invalidated.

Postconditions:

- The region's user system breaks list reflects the change.
- Layout output, if produced subsequently, accounts for the change subject to the engraver's override rules (Chapter 7).

Inverse: `SetUserSystemBreak` with the opposite `present` value.

Reduction rule: LwwAdvisory on per-anchor break preference. Two concurrent operations setting the same break to `true` reduce idempotently. Concurrent set-true and set-false on the same anchor resolve by canonical reduction order; the later operation's preference is materialized. No conflict record is produced; the break preference is an advisory field eligible for LWW reduction (Section 6.9).

6.13 Operation Catalog Conformance

This chapter specifies the framework. The full catalog of operations is delivered as a separate conformance specification that extends this chapter.

Requirement 6.27

A conforming implementation **MUST** provide every operation enumerated in the conformance catalog. Each operation **MUST** satisfy the framework requirements stated in this chapter:

- Declared preconditions classified as invariant or advisory.
- Specified effect on the score graph.
- Specified postconditions, including which invariants are preserved.
- A defined inverse operation.
- A declared reduction rule under the canonical reduction model (Section 6.3).
- Specified re-anchoring behavior for any deletions or moves it triggers.

Open Question

The conformance catalog is presently under development as a separate document. It is expected to comprise approximately 60–80 primitive operations and an open-ended set of compound operations. The catalog is normative once published; this specification is non-final until the catalog is delivered.

6.14 Forward References

- Operation envelope storage on disk (envelope block layout, causal-summary metadata, the operation index, snapshot canonical-base semantics) is specified in Chapter 8.
- Conflict-resolution algorithms (how a `ResolveConflict` operation transitions a conflict's state, and the full conflict-record contract) are specified in this chapter (Section 6.5) and in the Operation Catalog companion (Appendix A).
- The interaction between layout-semantic operations (system breaks, page breaks, engraving overrides) and the layout pipeline is specified in Chapter 7.
- The conformance catalog of all operations is a separate deliverable that extends this chapter.

Chapter 7

Layout Intermediate Representation

This chapter specifies the layout intermediate representation (Layout IR): the data structures and pipeline that transform the score graph into a typeset visual layout. The IR sits between the score graph and two downstream consumers: the constraint solver (Chapter 9), which resolves spacing and positioning, and the renderer (specified outside this document), which produces final visual output (pixels, PDF, SVG).

The IR is a pipeline of distinct stages, each with its own type and each with a well-defined contract for the next. The pipeline is the spine of the engraving system; the engraving algorithms themselves (beam grouping, slur curvature, accidental ordering, casting-off) are the consumers and producers of stages and are specified outside this document as engraving-algorithm specifications layered atop this IR.

7.1 Design Principles

Pipeline of stages. The transformation from score graph to visual output is a pipeline of four IR stages: `LogicalLayoutIR` (structural projection without spatial information), `ConstrainedLayoutIR` (with constraint inputs but unresolved positions), `ResolvedLayoutIR` (with all positions resolved), and `RenderIR` (renderer-bound primitives, defined only at the interface level here). Each stage has its own type. Compilation between stages is unidirectional.

Shared provenance. Every IR object at every stage carries a reference back to the score graph object that originated it. This enables selection, editing back-references, error reporting, and incremental layout invalidation.

Engraving decisions are explicit. When the engraver makes a decision (stem direction, accidental ordering, beam consolidation), the decision is recorded in the IR. The decision can be inspected, overridden, and traced.

Overrides exist in both worlds. User-set engraving overrides are authoritative in the score graph and projected into the IR during the logical stage. The IR may also generate transient overrides for downstream stages.

Spring-based spacing. Time-to-space projection produces elastic spring parameters per time slot. The constraint solver resolves springs system-wide. This applies to both horizontal and vertical layout.

Staff-space units, f32 precision. Spatial coordinates within the IR are expressed in staff spaces using single-precision floating point. Conversion to absolute units (points, millimeters) occurs only at the render boundary.

Glyph metrics live elsewhere. The IR references glyphs by identifier and queries metrics from a font catalog. Metrics are not embedded in IR objects.

Incremental layout, fine-grained. The IR supports incremental re-engraving via per-object dependency tracking on score graph objects. Edits invalidate only the layout objects whose dependencies changed.

Uniform region containers. Staff-based, free-graphic, and hybrid regions produce structurally identical `LayoutRegion` containers with differing time projections and content kinds.

7.2 Spatial Primitives

7.2.1 Units

The IR uses two unit types:

```
/// Staff space: the fundamental unit of music engraving.
/// One staff space equals the distance between adjacent staff lines.
#[derive(Copy, Clone, PartialEq, PartialOrd, Debug)]
pub struct StaffSpace(pub f32);

/// Point: 1/72 inch. Used at the render boundary for absolute
/// dimensions (page sizes, font sizes, output coordinates).
#[derive(Copy, Clone, PartialEq, PartialOrd, Debug)]
pub struct Point(pub f32);

/// Scaling context for converting between staff spaces and points.
pub struct ScaleContext {
    /// Points per staff space. Depends on the staff size chosen for
    /// the score; typical values are 4--10 points per staff space.
    pub points_per_staff_space: f32,
}
```

REQUIREMENT 7.1

Spatial coordinates within the IR (in stages `LogicalLayoutIR`, `ConstrainedLayoutIR`, and `ResolvedLayoutIR`) **MUST** be expressed in staff spaces. Conversion to points occurs at the `RenderIR` boundary.
Single-precision floating point (`f32`) **MUST** be used for IR coordinates. Engraving precision requirements are bounded by printer resolution (typically 600–2400 DPI); single precision provides over 7 significant figures, comfortably exceeding any engraving precision requirement.

7.2.2 Geometric Types

```
#[derive(Copy, Clone, Debug)]
```

```rust
pub struct Point2D {
    pub x: StaffSpace,
    pub y: StaffSpace,
}

#[derive(Copy, Clone, Debug)]
pub struct Size2D {
    pub width: StaffSpace,
    pub height: StaffSpace,
}

#[derive(Copy, Clone, Debug)]
pub struct Rect {
    pub origin: Point2D,
    pub size: Size2D,
}

#[derive(Copy, Clone, Debug)]
pub struct BoundingBox {
    pub left: StaffSpace,
    pub right: StaffSpace,
    pub top: StaffSpace,
    pub bottom: StaffSpace,
}

#[derive(Copy, Clone, Debug)]
pub struct Transform2D {
    pub matrix: [[f32; 3]; 3],
}
```

7.3 Provenance

Every IR object carries a provenance record tracing it back to the score graph object that originated it. Provenance is uniform across all four stages; provenance records are shared structurally between stages so that an object passing through the pipeline retains a consistent identity.

```rust
pub struct Provenance {
    /// The primary source: the score graph object this IR object
    /// represents or derives from.
    pub source: TypedObjectId,

    /// For IR objects that derive from synthesizing engraving
    /// decisions rather than directly from score graph objects, the
    /// synthesis kind. Examples: an automatically inserted natural
    /// sign, a generated rest, a system break.
    pub synthesis: Option<SynthesisKind>,

    /// Additional source dependencies. An IR object may depend on
    /// multiple score graph objects (e.g., a beam depends on its
    /// member events). All dependencies are tracked for
```

```
    /// incremental layout.
    pub dependencies: Vec<TypedObjectId>,

    /// IR object's stable identifier across re-layouts where the
    /// underlying source is unchanged.
    pub stable_id: LayoutObjectId,
}

pub enum SynthesisKind {
    /// An accidental inserted to cancel a prior alteration.
    CancellationAccidental,

    /// A natural sign generated by key signature rules.
    KeySignatureNatural,

    /// A rest inserted to fill a voice gap.
    GeneratedRest,

    /// A system or page break selected by the engraver.
    EngravedBreak,

    /// A multimeasure rest combining consecutive empty measures.
    MultimeasureRest,

    /// A cautionary key or time signature at a system break.
    Cautionary,

    /// A custom synthesis declared by a layout extension.
    Registered(SynthesisRegistryId),
}

pub struct LayoutObjectId(pub u128);
```

REQUIREMENT 7.2

Every layout object at every IR stage **MUST** carry a non-empty `Provenance` record. Objects whose direct `source` does not correspond to a score graph object (engraver-synthesized objects) **MUST** declare a `synthesis` kind. The `dependencies` list **MUST** include every score graph object whose change should invalidate this layout object.

REQUIREMENT 7.3

System-spanning continuation. A line primitive (stroke) that spans a system boundary is split into per-system segments; the segments after the first are engraver-synthesized and **MUST** declare a `synthesis` kind. The reserved registered kind `SynthesisKind::Registered(SYSTEM_CONTINUATION_SYNTHESIS)` names this continuation. Its `stable_semantic_instance_key` (per Requirement 7.4) is the pair `(original, ordinal)` — the pre-split stroke's object and the segment's order within the split — which is stable for a fixed set of system breaks. Because `LayoutObjectId`s are

non-canonical and re-derived per layout, this key need only be stable within a layout, not across relayouts that move the breaks.

REQUIREMENT 7.4

A `LayoutObjectId` **MUST** be stable across re-layouts whose underlying source is unchanged. It **MUST** be derived by domain-separated BLAKE3 truncation with the reserved layout domain tag `"MUSCLOID"` over a key that depends on how the object is manifested:

- A layout object that manifests a single score-graph object exactly once is keyed on `source.canonical_bytes()`.
- A multiply-manifested object — one score-graph object that appears in more than one layout context, e.g. a staff manifested in two regions — is keyed on the pair `(source, region)`, so each manifestation receives a distinct, stable id.
- A synthesized object (no direct score-graph `source`; `synthesis` is `Some`) is keyed on the triple `(source, synthesis_kind, stable_semantic_instance_key)`, where the `stable_semantic_instance_key` distinguishes multiple synthesized objects sharing a source and synthesis kind by a semantically stable discriminator (not a layout-position ordinal, which would not survive relayout).

`LayoutObjectId`s are *non-canonical*: they are not part of document state and do not enter any content hash, so the `"MUSCLOID"` tag lives in the layout namespace and is not one of the reserved *canonical* system tags of Section 5.3.2. The fixed derivation is pinned for incremental-relayout correctness and provenance back-reference stability; its consumers are the solver and renderer (Track A), not the interchange track. Because these ids never enter document state, no stored or interchanged artifact depends on this derivation. The v0 reference implementation realizes this derivation as of Pass-12 item P12-I2: `epiphany-determinism` reserves the built-in `"MUSCLOID"` layout tag and the `layout-ir` provenance derivations (single, multiply-manifested, and synthesized) route through it, the synthesized case no longer borrowing `"MUSCCONF"`. Because the ids are non-canonical, doing so changed layout-id values but no stored or interchanged artifact.

7.4 The Stage Pipeline

7.4.1 Pipeline Overview

```
ScoreGraph
   |
   |  Engraving pass:
   |  - Project structure
   |  - Make engraving decisions (stems, beams, accidentals)
   |  - Apply user overrides
   v
LogicalLayoutIR
   |
   |  Spacing pass:
```

```
    |  - Compute spring parameters per time slot
    |  - Compute glyph bounding boxes
    |  - Build collision constraints
    v
ConstrainedLayoutIR
    |
    |  Constraint solving (Chapter 9):
    |  - Resolve horizontal positions
    |  - Resolve vertical positions
    |  - Resolve page and system breaks
    v
ResolvedLayoutIR
    |
    |  Render projection (out of scope):
    |  - Convert to renderer primitives
    |  - Apply page templates
    |  - Generate draw calls
    v
RenderIR
```

7.4.2 Stage Contracts

REQUIREMENT 7.5

Each pipeline stage **MUST** accept its input stage as a pure function: identical inputs (plus identical configuration) **MUST** produce identical outputs. The stages are deterministic by contract.
Stage transitions **MUST** preserve provenance: every output object **MUST** carry a `Provenance` record either inherited from its input or newly synthesized with appropriate `synthesis` kind.

7.5 LogicalLayoutIR

`LogicalLayoutIR` is the structural projection of the score graph into layout objects, with all engraving decisions made but spatial positions unresolved. It is the output of the engraving pass and the input to the spacing pass.

7.5.1 Top-Level Structure

```rust
pub struct LogicalLayoutIR {
    /// Source score graph reference (typically a snapshot pointer
    /// or version identifier).
    pub source: ScoreVersion,

    /// Layout regions, one per score graph region.
    pub regions: Vec<LayoutRegion>,

    /// Engraving decisions made during this pass.
    pub engraving_decisions: Vec<EngravingDecision>,
```

```
    /// User overrides projected from the score graph.
    pub overrides: Vec<EngravingOverride>,

    /// Cross-region layout objects (slurs spanning regions, etc.).
    pub cross_region: Vec<CrossRegionObject>,
}
```

7.5.2 Layout Regions

```
pub struct LayoutRegion {
    pub provenance: Provenance,

    /// Coordinate system local to this region. The region's
    /// transform maps local coordinates to canvas coordinates.
    pub coordinate_system: LocalCoordinateSystem,

    /// Time axis: how time positions map to horizontal positions.
    /// A tagged union over the three built-in axis kinds plus a
    /// registered variant for extension-defined axes. The enum
    /// form is canonical for serialization, hashing, and
    /// conformance comparison; implementations MAY use a trait
    /// object internally for dynamic dispatch.
    pub time_axis: TimeAxisModel,

    /// Vertical extent: the set of staff bands this region occupies.
    pub vertical_extent: VerticalExtent,

    /// Layout objects within this region.
    pub objects: Vec<LayoutObject>,
}

/// Canonical representation of a region's time axis. Variants
/// correspond to the three built-in region time models; the
/// Registered variant carries extension-defined axes by typed
/// registry id and serialized payload.
pub enum TimeAxisModel {
    Metric(MetricTimeAxis),
    Proportional(ProportionalTimeAxis),
    Aleatoric(AleatoricTimeAxis),
    Registered(TimeAxisRegistryId, SerializedRegisteredAxis),
}
```

7.5.3 The Time Axis Trait

For dynamic dispatch and ergonomic implementation, the `TimeAxisModel` enum's payload types share a trait:

```
pub trait TimeAxis: Send + Sync {
    /// The time model kind this axis implements.
    fn kind(&self) -> TimeAxisKind;
```

```
    /// Project a time position to a horizontal spring slot.
    /// Returns a slot identifier; spring parameters are queried
    /// separately.
    fn project(&self, time: TimePoint) -> SpringSlotId;

    /// Enumerate the spring slots in time order.
    fn slots(&self) -> &[SpringSlotId];

    /// For incremental layout: which slots are affected by a change
    /// to the given time range?
    fn affected_slots(&self, range: TimeRange) -> Vec<SpringSlotId>;
}

pub enum TimeAxisKind {
    Metric,
    Proportional,
    Aleatoric,
    Registered(TimeAxisRegistryId),
}

pub enum TimePoint {
    Musical(MusicalPosition),
    WallClock(WallClockTime),
}

pub enum TimeRange {
    Musical { start: MusicalPosition, end: MusicalPosition },
    WallClock { start: WallClockTime, end: WallClockTime },
}
```

The three built-in implementations:

```
pub struct MetricTimeAxis {
    /// Measure boundaries and their projected spring slots.
    pub measures: Vec<MeasureProjection>,

    /// Per-time-slot spring parameters, indexed by slot id.
    pub slots: Vec<SpringSlot>,
}

pub struct ProportionalTimeAxis {
    /// Wall-clock duration of the region.
    pub duration: WallClockDuration,

    /// Linear horizontal projection: space per second.
    pub space_per_second: StaffSpace,

    pub slots: Vec<SpringSlot>,
}

pub struct AleatoricTimeAxis {
    /// DAG of event ordering.
```

```
    pub ordering: EventOrderingDAG,

    /// Spring slots are organized by topological-order layers.
    pub slots: Vec<SpringSlot>,
}
```

7.5.4 Layout Objects

A `LayoutObject` is the unit of engraving. At the `LogicalLayoutIR` stage, layout objects are composite: a single `Note` object contains its notehead, stem, flag, accidental, articulations, and so on. At the `ConstrainedLayoutIR` stage these are flattened into individual glyph objects.

```
pub enum LayoutObject {
    /// Composite note object: notehead(s), stem, flag, accidental(s),
    /// articulations, ornaments, dot(s).
    Note(NoteLayout),

    /// Composite chord object: multiple noteheads sharing stem and
    /// articulations.
    Chord(ChordLayout),

    /// Rest with its glyph kind (whole, half, quarter, ...).
    Rest(RestLayout),

    /// Beam group: multiple notes/chords beamed together.
    BeamGroup(BeamGroupLayout),

    /// Tuplet visual: bracket and number.
    TupletDisplay(TupletDisplayLayout),

    /// Slur or phrase mark: control points and style.
    Slur(SlurLayout),

    /// Tie: same as slur but connecting same-pitch events.
    Tie(TieLayout),

    /// Spanner: hairpins, octave lines, pedal lines, etc.
    Spanner(SpannerLayout),

    /// Marker: rehearsal marks, section labels, tempo marks.
    Marker(MarkerLayout),

    /// Bar line: simple, double, final, repeat, etc.
    BarLine(BarLineLayout),

    /// Clef, key signature, time signature glyphs.
    Clef(ClefLayout),
    KeySignature(KeySignatureLayout),
    TimeSignatureDisplay(TimeSignatureDisplayLayout),

    /// Staff: the staff lines themselves.
    Staff(StaffLayout),
```

```
    /// Text: lyrics, dynamics text, expressive text.
    Text(TextLayout),

    /// Free graphic: path, shape, image, stroke.
    Graphic(GraphicLayout),

    /// Multimeasure rest: consolidated empty measures.
    MultimeasureRest(MultimeasureRestLayout),

    /// Cue: small-print rendering of source material.
    Cue(CueLayout),

    /// Trajectory: glissando, portamento line.
    Trajectory(TrajectoryLayout),

    /// Generic group: implementation-defined.
    Group(GroupLayout),
}
```

7.5.5 Note Layout: A Worked Example

To illustrate the composite-object pattern at the logical stage:

```rust
pub struct NoteLayout {
    pub provenance: Provenance,

    /// Reference to the score graph event.
    pub event: EventId,

    /// The notehead glyph and its staff position.
    pub notehead: NoteheadLayout,

    /// The stem, if any.
    pub stem: Option<StemLayout>,

    /// Flag, if any (single-note events with eighth-or-shorter
    /// duration that are not part of a beam group).
    pub flag: Option<FlagLayout>,

    /// Accidental glyphs attached to the note, in stacking order.
    pub accidentals: Vec<AccidentalLayout>,

    /// Augmentation dots.
    pub dots: Vec<DotLayout>,

    /// Articulations attached to the note.
    pub articulations: Vec<ArticulationLayout>,

    /// Ornament glyphs.
    pub ornaments: Vec<OrnamentLayout>,
```

```
    /// Ledger lines required to display the note's staff position.
    pub ledger_lines: Vec<LedgerLineLayout>,

    /// Engraving decisions made for this note.
    pub decisions: NoteDecisions,
}

pub struct NoteDecisions {
    pub stem_direction: StemDirection,
    pub accidental_visible: bool,
    pub accidental_parenthesized: bool,
    pub notehead_shape: NoteheadShape,
    /// Voice membership influences default stem direction;
    /// overrides may change it.
    pub stem_direction_source: DecisionSource,
}

pub enum DecisionSource {
    /// Decision derived from automatic engraving rules.
    Automatic,

    /// Decision derived from a user override in the score graph.
    UserOverride(EngravingOverrideId),

    /// Decision derived from an IR-stage override.
    IrOverride,
}
```

7.5.6 Engraving Decisions

The pipeline records engraving decisions explicitly so they can be inspected, overridden, and replayed.

```
pub struct EngravingDecision {
    pub id: EngravingDecisionId,
    pub target: LayoutObjectId,
    pub kind: EngravingDecisionKind,
    pub source: DecisionSource,
}

pub enum EngravingDecisionKind {
    StemDirection(StemDirection),
    BeamGrouping(BeamGroupingChoice),
    AccidentalOrdering(Vec<AccidentalId>),
    SlurDirection(SlurDirection),
    LedgerLineCount(u8),
    EnharmonicSpelling(PitchSpelling),
    SystemBreak,
    PageBreak,
    NoteheadShape(NoteheadShape),
    RestPosition(StaffPosition),
    // ... extensible
```

```
}
```

7.6 Engraving Overrides

Engraving overrides are user assertions that the engraver's default decision should be replaced. Overrides are authoritative in the score graph and projected into the IR during the logical stage.

7.6.1 Override Definition

```
pub struct EngravingOverride {
    pub id: EngravingOverrideId,

    /// What this override targets. May reference a score graph object
    /// (typical) or an IR-synthesized object (transient overrides).
    pub target: OverrideTarget,

    /// The override itself.
    pub kind: OverrideKind,

    /// Binding strength.
    pub priority: OverridePriority,

    /// Provenance: who set the override and when.
    pub origin: OverrideOrigin,
}

pub enum OverrideTarget {
    /// Targets a score graph object. Authoritative; survives reload.
    ScoreGraph(TypedObjectId),

    /// Targets an IR-synthesized object. Transient; cleared on
    /// re-engraving unless explicitly re-applied.
    IrSynthesized(LayoutObjectId),
}

pub enum OverrideKind {
    StemDirection(StemDirection),
    SlurDirection(SlurDirection),
    SlurCurvature(CurvatureOverride),
    AccidentalParenthesized(bool),
    AccidentalVisible(bool),
    NoteheadShape(NoteheadShape),
    BeamGeometry(BeamGeometryOverride),
    SystemBreak { anchor: TimeAnchor },
    PageBreak { anchor: TimeAnchor },
    HiddenObject,
    CustomPosition(Point2D),
    EnharmonicSpelling(PitchSpelling),
    LedgerLineSuppression,
    DotPlacement(DotPlacement),
```

```
    // ... extensible
}

// System- and page-break overrides address a *position*, not an
// object: the break kind carries the break's TimeAnchor, while the
// override's ScoreGraph target names the owning region. They are
// projected during the logical stage from the score graph's
// authoritative user_system_breaks / user_page_breaks lists
// (Chapter 5). Break authorship (author, timestamp) lives in the
// operation log, not the materialized break lists, so projected
// break overrides carry OverrideOrigin::Internal until the
// snapshot-undo refinement (P11-C8) surfaces authorship.

pub enum OverridePriority {
    /// The engraver MUST honor this override. Failure produces an
    /// error rather than a silent override-of-the-override.
    Hard,

    /// The engraver SHOULD honor this override. The engraver may
    /// override it only under documented exceptional conditions
    /// (system overflow, unresolvable collision); when it does, the
    /// override is recorded in the resulting decisions with source
    /// IrOverride.
    Soft,
}

pub enum OverrideOrigin {
    User { author: AuthorId, timestamp: Timestamp },
    Import { format: ForeignFormatId },
    Plugin { plugin: PluginId },
    Internal,
}
```

7.6.2 Override Resolution

REQUIREMENT 7.6

Overrides **MUST** be applied during the engraving pass producing `LogicalLayoutIR`. Hard overrides **MUST** be honored or the pass **MUST** report an error. Soft overrides **MUST** be honored except where doing so would produce an invalid layout; when a soft override is not honored, the resulting decision **MUST** record the override and the reason for its non-application.

REQUIREMENT 7.7

Overrides whose targets reference objects in the score graph (the `ScoreGraph` variant) **MUST** be preserved across re-engraving. Overrides targeting IR-synthesized objects (the `IrSynthesized` variant) are transient; they are cleared on re-engraving unless explicitly re-applied. Implementations **MUST** not silently promote IR-synthesized over-

rides to score-graph overrides without explicit user action.

REQUIREMENT 7.8

Break-override attribution. When the engraver honours a user break override, the resulting decision **MUST** carry `DecisionSource::UserOverride(id)` naming the override. Because a normalized break *constraint* (`SystemBreakAt` / `PageBreakAt`, Section 7.7) carries no override identity, this attribution is threaded through the projection: the `ConstrainedLayoutIR` carries a `break_origins` sidecar, populated when the logical stage projects the break, mapping each user-anchored break slot to its override id. The normalized constraint record is deliberately *not* widened to carry override identity — attribution is a projection concern, not a constraint-solver input.

7.7 ConstrainedLayoutIR

`ConstrainedLayoutIR` is the output of the spacing pass: the logical IR with composite objects flattened to individual glyphs, each glyph carrying its bounding box, anchor, and constraint inputs to the solver.

REQUIREMENT 7.9

Minimal-tier constraint-emission floor (ratified Pass 12). The spacing pass **MUST** emit at least the following constraint set — the *Minimal-tier floor*, the testable acceptance surface for the `Minimal` conformance tier (Chapter 9): no-collision chains between successive notehead-bearing columns; per-glyph containment within the owning region's frame; and one constraint per user break override (system and page). Richer emission sets (inter-band collision, kerned optical adjacency, duration- proportional spring preferences) belong to higher tiers, whose floors are defined when those tiers are ratified.

7.7.1 Glyph-Level Objects

```rust
pub struct GlyphObject {
    pub provenance: Provenance,

    /// The glyph to render. Reference into a font catalog; metrics
    /// are queried from the catalog rather than embedded here.
    pub glyph: GlyphReference,

    /// The spring slot this glyph belongs to (horizontal time slot).
    pub horizontal_slot: SpringSlotId,

    /// The vertical band this glyph belongs to.
    pub vertical_band: VerticalBandId,

    /// Bounding box of the glyph relative to its anchor, in staff
    /// spaces. Looked up from the font catalog at IR construction
    /// time and cached here.
```

```
    pub bounding_box: BoundingBox,

    /// Anchor point: where the glyph's reference position sits
    /// within its bounding box.
    pub anchor: Point2D,

    /// Layer ordering: higher draws on top.
    pub layer: i32,

    /// Visual style applied to the glyph.
    pub style: GlyphStyle,
}
```

7.7.2 Spring Slots

A spring slot is a time slot's spacing parameters: the constraint solver's input for resolving horizontal positions.

```
pub struct SpringSlot {
    pub id: SpringSlotId,
    pub time: TimePoint,

    /// Minimum width: the smallest the slot may compress to. The
    /// constraint solver MUST NOT compress below this.
    pub min_width: StaffSpace,

    /// Preferred width: the natural width under unconstrained
    /// spacing. Derived from Gourlay-style proportional rules.
    pub preferred_width: StaffSpace,

    /// Maximum width, if bounded. None means unbounded above.
    pub max_width: Option<StaffSpace>,

    /// Stretch factor: how readily this slot stretches when more
    /// width is available than preferred. Higher = more elastic.
    pub stretch_factor: f32,

    /// Compress factor: how readily this slot compresses when less
    /// width is available than preferred. Higher = more compressible.
    pub compress_factor: f32,

    /// Glyphs belonging to this slot, ordered by stacking
    /// considerations (accidentals to left of notehead, etc.).
    pub members: Vec<GlyphObjectId>,
}
```

7.7.3 Vertical Bands

Vertical layout uses the same spring model. A *vertical band* is a horizontal slice of the canvas (typically a staff or an inter-staff gap) with its own spring parameters.

```rust
pub struct VerticalBand {
    pub id: VerticalBandId,
    pub kind: VerticalBandKind,

    pub min_height: StaffSpace,
    pub preferred_height: StaffSpace,
    pub max_height: Option<StaffSpace>,
    pub stretch_factor: f32,
    pub compress_factor: f32,

    /// Glyphs belonging to this band.
    pub members: Vec<GlyphObjectId>,
}

pub enum VerticalBandKind {
    Staff(StaffId),
    InterStaffGap,
    InterSystemGap,
    MarginBand,
}
```

REQUIREMENT 7.10

Engraving-coverage gaps are surfaced, not guessed. A projection that cannot engrave an object faithfully — a pitch with no resolved spelling, a glyph the bound catalog (7.12) does not carry — **MUST** record a `LayoutDiagnostic` naming the score-graph object and the kind of gap, and **MUST** still place the object: a fallback notehead on the clef reference line, or a zero-extent traced anchor that keeps its provenance. It **MUST NOT** silently substitute a plausible shape, and it **MUST NOT** drop the object.
Both halves matter. Dropping the object breaks the round-trip surjection (7.3) — a hit-test can no longer find it, and an editor can no longer select what the author wrote. Guessing a shape produces a score that looks engraved and is wrong, with nothing in the IR to say so. A diagnostic is not an error: the layout is still *renderable*, and a conformance claim is unaffected. It is the record that the engraving is incomplete at a named place.

REQUIREMENT 7.11

Primitive band ownership is declared, not inferred. Every primitive the projection presents to the solver in `ConstrainedLayoutIR` — `GlyphObject`, `Stroke`, and `Curve` alike — **MUST** declare the `VerticalBandId` of the band that owns it, and that band **MUST** exist in `vertical_bands`. A vertical solver **MUST** take a primitive's owning staff from that declaration and **MUST NOT** infer it from the primitive's geometry. Content owned by no staff (a page-margin annotation, a structure spanning several staves) names a band whose kind is not `Staff`.
This is a correctness requirement, not a convenience. A primitive's drawn position does not determine its owner: a stem shares its horizontal column with the staff above, and a slur's endpoints are *lifted clear* of its own staff by construction, landing in the inter-staff zone where the nearest notehead routinely belongs to the neighbouring staff. A solver

that guesses from proximity will attribute such a primitive to the wrong staff and, on renegotiating the gaps between a system's staves, tear it off the notes it belongs to.
A band **MUST** therefore exist for every staff of a region, whether or not any glyph landed in it, and for the region's margin — otherwise a primitive that names one has nowhere to point. A band MAY carry no members.
Only glyphs are *members* of a band (`VerticalBand::members`), which is what realizes the spring solve; a stroke's or curve's band reference is one-way. What becomes of the declaration downstream is 7.14.

7.7.4 Constraints

The IR carries explicit constraint inputs for the solver. Beyond glyphs it presents the non-glyph primitives — `Stroke` and `Curve`, whose shapes are listed with 7.13 — since the solver positions those too:

```rust
pub struct ConstrainedLayoutIR {
    pub source: ScoreVersion,

    pub regions: Vec<ConstrainedLayoutRegion>,

    /// Horizontal spring slots across all regions.
    pub horizontal_slots: Vec<SpringSlot>,

    /// Vertical bands.
    pub vertical_bands: Vec<VerticalBand>,

    /// Glyph-level layout objects.
    pub glyphs: Vec<GlyphObject>,

    /// Non-glyph line primitives (staff lines, stems, barlines,
    /// ledger lines, brackets) presented to the solver alongside
    /// the glyphs.
    pub strokes: Vec<Stroke>,

    /// Cubic-Bezier curve primitives (slurs), likewise.
    pub curves: Vec<Curve>,

    /// Additional constraints not captured by spring parameters.
    pub constraints: Vec<LayoutConstraint>,

    /// Which user break override each projected break constraint came
    /// from, so a solver can cite it in the decision it records.
    pub break_origins: Vec<BreakOrigin>,

    /// Engraving decisions, carried forward from the logical stage.
    pub engraving_decisions: Vec<EngravingDecision>,

    /// Engraving-coverage gaps, surfaced rather than hidden.
    pub diagnostics: Vec<LayoutDiagnostic>,

    /// The glyph catalog this IR's metrics were drawn from.
```

```
    pub catalog: GlyphCatalogIdentity,
}

/// A projected break constraint's user-override attribution.
pub struct BreakOrigin {
    pub slot: SpringSlotId,
    pub class: BreakClass,
    pub override_id: EngravingOverrideId,
}

/// An object the projection could not engrave faithfully.
pub struct LayoutDiagnostic {
    pub source: TypedObjectId,
    pub kind: LayoutDiagnosticKind,
}

pub enum LayoutDiagnosticKind {
    /// A pitch reached the constrained pass with no resolved (or
    /// non-CMN) spelling; its notehead is placed on the clef
    /// reference line, but its true staff position is unknown.
    MissingSpelling,

    /// A glyph the bound catalog does not carry; the object is
    /// carried as a traced anchor rather than drawn at a guessed
    /// shape.
    UnbundledGlyph(GlyphReference),
}

pub enum LayoutConstraint {
    /// Two glyphs must not overlap in 2D space.
    NoCollision { a: GlyphObjectId, b: GlyphObjectId },

    /// A glyph must align with another along an axis.
    Align {
        a: GlyphObjectId,
        b: GlyphObjectId,
        axis: Axis,
    },

    /// A glyph must be positioned within a region of space.
    PositionWithin {
        glyph: GlyphObjectId,
        region: Rect,
    },

    /// A range of slots must end at a system boundary.
    SystemBreakAt { slot: SpringSlotId, kind: BreakKind },

    /// A range of slots must end at a page boundary.
    PageBreakAt { slot: SpringSlotId, kind: BreakKind },

    /// Custom constraint declared by a layout extension.
    Registered(ConstraintRegistryId, ConstraintParameters),
```

```
}

pub enum BreakKind {
    Hard,
    Soft,
}
```

REQUIREMENT 7.12

Break-constraint satisfaction. A `SystemBreakAt` at `slot` is *satisfied* iff the final resolved layout starts a system at that slot; a `PageBreakAt` iff the final layout starts a page there. A slot that already begins a system (respectively page) — in particular a region's first slot — is trivially satisfied. Satisfaction is a predicate on the output `ResolvedLayoutIR`, evaluable by any consumer, not on the solver's internal spring state. A solver's tier claim (Section 7.8) reports which `Hard` break constraints its layout satisfies; a `Soft` break is advisory and MAY be left unsatisfied.

7.8 ResolvedLayoutIR

`ResolvedLayoutIR` is the output of the constraint solver: every glyph, stroke, and curve has a definitive position. This is the IR consumed by the renderer.

```
pub struct ResolvedLayoutIR {
    pub source: ScoreVersion,

    /// Pages of the resolved score.
    pub pages: Vec<ResolvedPage>,

    /// Glyph-level objects with resolved positions.
    pub glyphs: Vec<ResolvedGlyph>,

    /// Non-glyph line primitives (staff lines, stems, barlines,
    /// volta brackets, ...), positioned alongside the glyphs.
    pub strokes: Vec<Stroke>,

    /// Cubic-Bezier curve primitives (slurs, ...), positioned
    /// alongside the glyphs.
    pub curves: Vec<Curve>,

    /// Engraving decisions, including any made by the constraint
    /// solver (e.g., soft system breaks placed by casting-off).
    pub engraving_decisions: Vec<EngravingDecision>,
}

pub struct ResolvedPage {
    pub provenance: Provenance,
    pub number: u32,
    pub size: Size2D,
    pub margins: Margins,
    pub systems: Vec<ResolvedSystem>,
```

```
    pub free_objects: Vec<GlyphObjectId>,
}

pub struct ResolvedSystem {
    pub provenance: Provenance,
    pub bounding_box: Rect,
    pub staves: Vec<ResolvedStaff>,
    pub measures: Vec<ResolvedMeasure>,
}

pub struct ResolvedGlyph {
    pub provenance: Provenance,
    pub glyph: GlyphReference,
    pub position: Point2D,
    pub transform: Option<Transform2D>,
    pub bounding_box: BoundingBox,
    pub style: GlyphStyle,
    pub layer: i32,
}

/// A line primitive the solver positions (a staff line, stem,
/// barline, or bracket).
pub struct Stroke {
    pub provenance: Provenance,
    pub from: Point2D,
    pub to: Point2D,
    pub thickness: StaffSpace,
    pub layer: i32,
    pub style: GlyphStyle,

    /// The vertical band this stroke belongs to: its owning staff,
    /// declared by the projection that emitted it. A vertical solver
    /// reads ownership from here rather than inferring it from
    /// geometry. Unlike a glyph, a stroke is not listed in
    /// 'VerticalBand::members'; this is a one-way reference.
    pub vertical_band: VerticalBandId,
}

/// A cubic-Bezier curve primitive (a slur), its four control
/// points in 'p0'..'p3' drawing order.
pub struct Curve {
    pub provenance: Provenance,
    pub p0: Point2D,
    pub p1: Point2D,
    pub p2: Point2D,
    pub p3: Point2D,
    pub thickness: StaffSpace,
    pub layer: i32,
    pub style: GlyphStyle,

    /// The line pattern the renderer strokes the path with
    /// (a slur's authored 'SpanStyle.line'): solid, dashed, or dotted.
    pub line: LineStyle,
```

```
    /// The vertical band this curve belongs to (see 'Stroke'). A
    /// slur's endpoints are lifted clear of its own staff, so its
    /// owner is not recoverable from its geometry.
    pub vertical_band: VerticalBandId,
}
```

REQUIREMENT 7.13

Non-glyph resolved primitives (ratified schema-major-2, Phase F). Beyond glyphs, `ResolvedLayoutIR` carries non-glyph *line-stroke* primitives (staff lines, stems, barlines, brackets) and *cubic-Bézier curve* primitives (slurs). Each **MUST** carry provenance — the basis of hit-testing (7.3) — and is positioned by the solver like a glyph: a spanning stroke or curve is re-spaced by the same horizontal coordinate map its endpoint columns move under. These primitives are *non-canonical*: like layout identifiers and glyph positions, they are not part of document state and enter no content hash (7.4). A conformant renderer draws them alongside the glyphs; the renderer encoding itself is out of scope (7.9).

REQUIREMENT 7.14

Band ownership through the resolved stage. A resolved `Stroke` and `Curve` **MUST** retain the `vertical_band` they were given in `ConstrainedLayoutIR` (7.11), so a solver that re-spaces or relocates them — casting off into systems, or renegotiating the gaps between a system's staves — can attribute each one to its owning staff. A `ResolvedGlyph` carries no band: a glyph's ownership is consumed during the solve and its outcome is already baked into the glyph's resolved position, so the field would be dead weight in a rendering fingerprint.

Band ownership is *non-canonical layout-attribution metadata*. It enters no content hash and no canonical encoding (7.4): it draws nothing, so two layouts differing only in it are the same rendered layout. A conformant canonical encoding of `ResolvedLayoutIR` therefore omits `vertical_band` even from the strokes and curves that carry it.

REQUIREMENT 7.15

Minimal-tier repeat and volta rendering (ratified schema-major-2, Phase F). A `RepeatStructure` (5.7.7) whose kind draws barlines (`SimpleRepeat`, `Volta`) **MUST** render a repeat barline at each of its resolved boundaries — the precomposed repeat sign, either replacing a coinciding measure barline or standing at the boundary column when none coincides, and the repeat-dot pair beside a region-closing final barline (which is never replaced). Each `Volta` **MUST** render a bracket spanning its resolved extent, with its ending numbers. A boundary that does not resolve to a spacing column draws *no* ink: a traced anchor preserves the structure's provenance rather than placing a mark at a false position. The jump kinds (`DaCapo`, `DalSegno`) draw no Minimal-tier marks — their segno / coda / instruction text awaits a text primitive — and a repeat spanning more than one region renders in no single region at this tier. The precise glyph vocabulary and

spacing are engraving-algorithm concerns (7.13).

REQUIREMENT 7.16

Slur rendering (ratified schema-major-2, Phase F; extended Push 3). A `Slur` (5.7.1) **MUST** render as a cubic-Bézier `Curve` arcing between its two endpoint event columns, honoring an authored `CurvatureOverride` (direction and apex height) when present and choosing them otherwise. A slur whose endpoint does not resolve to a column on a single staff of one region draws *no* curve: a traced anchor preserves its provenance rather than a floating arc. An authored non-`Solid` `SpanStyle` line (dashed, dotted) is rendered faithfully as that pattern (its `LineStyle` rides the `Curve`); an implementation that instead defers the pattern **MUST** surface the deferral (a layout diagnostic), never silently rendering solid. A slur whose span crosses a system break **MUST** split into per-system sub-curves — the first carrying the slur's provenance, the rest engraver-synthesized continuations (7.3) — rather than drawing whole in one system. The curvature-computing algorithm remains an engraving-specification concern (7.13).

7.9 RenderIR (Interface Only)

The `RenderIR` stage is the renderer's input. Its full specification is delivered separately; this section defines only the interface contract.

```
pub trait RenderIRProducer {
    /// Convert resolved IR to renderer-bound primitives.
    fn produce(
        &self,
        resolved: &ResolvedLayoutIR,
        scale: ScaleContext,
        config: RenderConfiguration,
    ) -> RenderIR;
}

pub struct RenderConfiguration {
    /// Target output: PDF, SVG, on-screen, print.
    pub target: RenderTarget,

    /// Color space and color management options.
    pub color: ColorConfiguration,

    /// Anti-aliasing and rasterization options.
    pub rasterization: RasterizationConfiguration,
}
```

REQUIREMENT 7.17

Implementations producing `RenderIR` **MUST** preserve provenance from `ResolvedLayoutIR`: every renderer primitive **MUST** be traceable to its originating `ResolvedLayoutIR` primitive — a `ResolvedGlyph`, `Stroke`, or `Curve` — and therefore to its score graph source. This is the basis of hit-testing, selection, and back-reference navigation in the UI.
The full RenderIR type and its production rules are specified in the renderer specification, outside the scope of this document.

7.10 Incremental Layout and Caching

The pipeline supports incremental layout via per-object dependency tracking. An edit to a score graph object invalidates only the IR objects whose provenance lists it as a dependency.

7.10.1 The Layout Cache

```
pub struct LayoutCache {
    /// Reverse index: maps score graph object ids to the layout
    /// objects depending on them.
    pub dependencies: DependencyIndex,

    /// Cached IR stages, partitioned for granular invalidation.
    pub logical: HashMap<RegionId, LogicalRegionCache>,
    pub constrained: HashMap<RegionId, ConstrainedRegionCache>,
    pub resolved: HashMap<SystemId, ResolvedSystemCache>,

    /// Cached layout-stage outputs at finer granularity for
    /// performance-critical paths.
    pub fine_cache: FineLayoutCache,
}

pub struct DependencyIndex {
    /// For each score graph object, the layout objects depending
    /// on it. Reverse-indexed to make invalidation O(k) where k is
    /// the number of dependents.
    pub forward: HashMap<TypedObjectId, Vec<LayoutObjectId>>,

    /// For each layout object, its dependencies. Used to verify and
    /// rebuild the reverse index on cache writes.
    pub reverse: HashMap<LayoutObjectId, Vec<TypedObjectId>>,
}
```

7.10.2 Invalidation Rules

Requirement 7.18

When a semantic operation (Chapter 6) is applied to the score graph, the following invalidation **MUST** occur:

1. Every layout object whose `Provenance` lists a mutated score graph object in its `source` or `dependencies` is invalidated.
2. Invalidation propagates: layout objects that depend on invalidated layout objects (via cross-region references, spanning structures, etc.) are also invalidated.
3. Invalidated objects are cleared from the cache at their respective stages.
4. Invalidated systems trigger casting-off re-evaluation for themselves and any downstream systems whose horizontal extent may shift.

Re-engraving runs from the first invalidated stage forward, using cached values for non-invalidated objects.

7.10.3 Frame-Budget Requirements

Requirement 7.19

The incremental layout system **MUST** support the frame-budget requirements established in Chapter 10: an edit affecting a single system on a 100-page orchestral score **MUST** re-layout in under 16.7 ms on the reference hardware profile.
Achieving this requires (a) fine-grained dependency tracking as specified above; (b) cached glyph metrics in the font catalog; (c) parallel re-engraving where the affected systems are independent; (d) the constraint solver supporting partial re-resolution of affected slots only (see Chapter 9).

7.11 Region Uniformity

Staff-based, free-graphic, and hybrid regions produce structurally identical `LayoutRegion` containers. The differences are expressed in:

- The `time_axis`: metric, proportional, or aleatoric.
- The `objects` mix: staff-based regions contain primarily notation objects; free-graphic regions contain primarily `Graphic` objects; hybrid regions contain both.

Requirement 7.20

All three region kinds **MUST** use the same `LayoutRegion` container type. The constraint solver **MUST** treat regions uniformly: it consumes spring slots and constraints without branching on region kind.

Rationale

Uniform containers permit the constraint solver to be agnostic to region kind, simplifying its implementation. The differences between region kinds are encoded in the `TimeAxis` implementation and the object mix, not in container structure. This is the same insight that makes the file format and semantic operations layers tractable: structural uniformity at the containers, polymorphism in the contents.

7.12 Glyph Catalog Interface

The IR references glyphs by identifier; glyph metrics are queried from a font catalog at IR construction. This section defines the interface; the catalog implementation is outside the core specification. `SmuflVersion`, below, is the same type declared by a score's `SmuflVersionRequirement` (Section 4.4): the glyph catalog's version and the tuning context's declared version are one type, not two.

```
pub trait GlyphCatalog: Send + Sync {
    /// Resolve a glyph reference to its metrics.
    fn metrics(&self, glyph: &GlyphReference) -> Option<GlyphMetrics>;

    /// Resolve a glyph reference to its rendering data.
    fn render_data(&self, glyph: &GlyphReference) -> Option<GlyphRenderData>;

    /// SMuFL version supported.
    fn smufl_version(&self) -> SmuflVersion;
}

pub struct GlyphMetrics {
    pub bounding_box: BoundingBox,
    pub advance_width: StaffSpace,
    pub anchors: HashMap<AnchorName, Point2D>,
}

pub struct GlyphRenderData {
    /// Outline data: path commands suitable for vector rendering.
    pub outline: Vec<PathCommand>,

    /// Optional pre-rendered bitmap for raster targets.
    pub bitmap: Option<GlyphBitmap>,
}
```

Requirement 7.21

Implementations **MUST** use a glyph catalog for metric and render-data lookups during IR construction. Metrics **MUST NOT** be duplicated in the IR; the IR carries glyph identifiers and queries the catalog. Implementations **MAY** cache metrics within IR objects as a performance optimization provided cached values are invalidated when the underlying catalog changes.

7.12.1 Glyph Catalog Identity for Layout Conformance

Layout output depends on glyph metrics. For reproducible layout, the exact glyph catalog used **MUST** be identifiable. `smufl_version` below is a `SmuflVersion` (Section 4.4), ordered by its fraction-normalized `minor_centi` so a catalog identity's version compares correctly against SMuFL's real release history.

```
/// A reproducibility-quality identifier for the glyph catalog
/// used to produce a layout. Required for any layout conformance
/// claim that depends on byte-equal output across runs.
pub struct GlyphCatalogIdentity {
    /// SMuFL version targeted.
    pub smufl_version: SmuflVersion,

    /// Identifier of the specific font in use (e.g., Bravura,
    /// Petaluma, Leland).
    pub font_id: FontId,

    /// Optional semantic version of the font, if the font
    /// publisher uses versioned releases.
    pub font_version: Option<SemVer>,

    /// Content hash of the metric data made available to the
    /// layout solve. The hash is over a canonical serialization
    /// of every glyph's metrics (bounding box, advance width,
    /// named anchors) for every glyph referenced by the
    /// ConstrainedLayoutIR delivered to the solver, including
    /// glyphs that are candidates but do not appear in the final
    /// ResolvedLayoutIR. Computed by BLAKE3 with the domain
    /// tag "MUSCFNTM".
    pub metrics_hash: ContentHash,
}
```

REQUIREMENT 7.22

Any layout conformance claim subject to byte-equal output (within-implementation determinism per Chapter 9) **MUST** declare the `GlyphCatalogIdentity` under which it was produced. Two layouts produced with different `metrics_hash` values are layouts under different inputs; byte equality is not expected.

Implementations that cannot identify their glyph catalog at this granularity **MUST** mark their layout output as *implementation-specific* or *non-canonical*; conformance suites **MUST NOT** accept such output as byte-equivalent to a reference solver's output.

The `metrics_hash` is computed over the glyphs that are *inputs* to the layout solve (those referenced by the `ConstrainedLayoutIR`), not only over glyphs in the final layout. Two solver invocations with different available candidate-glyph metrics are operating on different inputs even if they happen to converge on the same final glyph set; this rule makes that difference visible in the `metrics_hash`.

The hash is computed only over glyphs the layout solve actually consulted, not over the entire font catalog, so that font additions (new glyphs unused by the solve) do not

> invalidate prior layouts.

7.13 Forward References

- The constraint solver consuming `ConstrainedLayoutIR` and producing `ResolvedLayoutIR` is specified in Chapter 9.
- The serialization of layout cache data for cross-session persistence is specified in Chapter 8.
- The complete RenderIR specification, the engraving algorithm specifications (beam grouping, slur curvature, casting-off, accidental ordering), the font catalog implementation, and the renderer are delivered as separate specifications layered on top of this core document.

Chapter 8

File Format

This chapter specifies the on-disk file format. Chapter 6 established that the canonical document state is the deterministic reduction of an operation-envelope set. This chapter specifies how that operation set, along with acceleration structures, profile declarations, and extension declarations, is laid out on disk in a way that is crash-recoverable, atomically updatable, and forward- compatible.

The byte-level encoding details (record layouts, varint conventions, exact field bit widths) are deferred to a companion document, the Binary Format specification. This chapter specifies *what* is stored, how the parts relate, and how the format behaves under concurrent edits, crashes, and forward-compatibility scenarios; the Binary Format specifies *how the bits are laid out*.

The file format's name is `musc` and its file extension is `.musc`.

8.1 Design Principles

Canonical state is the operation set. The canonical document is the operation-envelope set (Chapter 6). After pruning, the canonical document is a retained base snapshot plus the operation envelopes after that snapshot's causal frontier. Everything else stored in the bundle (snapshots, layout caches, indexes, rendered artifacts) is an acceleration structure and **MUST** be discardable.

Fixed prelude, atomic superblock commit. The file begins with a fixed-size header followed by two fixed-size superblock slots. The superblocks are the only mutable on-disk objects in the bundle. Commits append new chunks, write a new manifest chunk, then flip the active superblock by writing to the currently-inactive slot. This makes commits atomic with respect to crashes.

Content-addressed immutable chunks. Every other on-disk object is content-addressed by BLAKE3 of its uncompressed payload (with domain-separated preimage). Chunks are immutable once written. Duplicate content shares storage automatically.

Manifest as a table of roots. The manifest does not carry user-facing data. It carries the operation-set roots, the retained snapshot references, blob references, profile declarations, extension declarations, and the text-projection root. User-facing metadata (title, composer, etc.) lives in operation envelopes or in materialized state, not in the physical header.

Crash recovery is by superblock selection. Recovery is simply: read both superblocks, vali-

date, and select the highest valid generation. Anything not reachable from the selected superblock is unreachable garbage and can be reclaimed by a later conservative GC pass.

Forward compatibility through opaque preservation and edit barriers. Unknown extension chunks are preserved across reads and writes. Unknown required extensions either prevent open or force read-only mode. Unknown optional extensions are preserved unless an edit crosses a declared edit barrier.

Streaming reads. The bundle is designed so that opening a large score touches the prelude, the active superblock, the manifest, and a small set of bootstrap chunks (the most recent canonical-base snapshot plus the envelope index). Per-region content loads on demand.

Deterministic text projection of semantic content. The format admits a deterministic projection to a canonical s-expression form for diff tooling and version control. The text projection preserves canonical document semantics (operation envelopes, profile declarations, extension declarations, reduced state). It does *not* preserve chunk offsets, compression choices, cache chunks, garbage regions, or superblock generation numbers.

No encryption, no DRM. The format **MUST NOT** include encryption, access control, or rights-management features. Privacy at rest is the responsibility of the filesystem and operating system. The format is open and offers no mechanism by which user content can be locked away from its author or from other implementations.

NON-GOAL

Digital rights management (DRM), content encryption, hardware-binding, license enforcement, and any mechanism that could lock user content to a particular vendor or platform are explicitly out of scope. The format is designed for openness, durability, and user ownership of data. Implementations **MUST NOT** add proprietary extensions intended to restrict access to or use of user content.

8.2 Durable Writes

This chapter refers throughout to "durable flushes" of file contents. The definition is platform-abstracted.

Durable flush. The platform operation that makes previously-written bytes durable according to the host filesystem contract: `fsync` on POSIX-like systems, `FlushFileBuffers` or `NtFlushBuffersFileEx` on Windows, equivalent calls on other platforms. A successful durable flush guarantees that bytes written before the call are on the persistent storage medium when the call returns.

REQUIREMENT 8.1

Implementations **MUST** use the platform's durable flush primitive at the points required by the atomic write protocol (Section 8.10). Implementations **MAY**NOT rely on filesystem ordering guarantees or write-back caching alone; durability **MUST** be requested explicitly at the commit point.

8.3 The Bundle Layout

A `.musc` bundle is a single file whose first 576 bytes are a fixed prelude: a 64-byte header followed by two 256-byte superblock slots. The remainder of the file holds variable-length content: chunk payloads, the manifest chunk, blob payloads, the text projection (if present), and unreachable garbage from prior commits that has not yet been collected.

```rust
pub struct BundleLayout {
    /// Fixed 64-byte header at offset 0.
    pub header: FixedHeader,

    /// Two fixed-size superblock slots at offsets 64 and 320.
    pub superblock_a: Superblock,  // at offset 64
    pub superblock_b: Superblock,  // at offset 320

    /// Variable-length content begins at offset 576.
    /// All chunks, the manifest, and blobs live here.
    pub body: ContentRegion,
}
```

8.3.1 The Fixed Header

The header is exactly 64 bytes at file offset zero. It identifies the format and locates the superblock slots. The header never changes after the file is created.

```rust
pub struct FixedHeader {
    /// Magic bytes: ASCII "MUSCBND\0" (8 bytes).
    pub magic: [u8; 8],

    /// Major format version. Major version changes are
    /// non-backward-compatible.
    pub format_major: u16,

    /// Minor format version. Minor version changes are
    /// backward-compatible.
    pub format_minor: u16,

    /// Length of this header in bytes (currently always 64).
    /// Reserved for future expansion via header_length growth.
    pub header_length: u32,

    /// Offset of superblock slot A. Currently always 64.
    pub superblock_a_offset: u64,

    /// Offset of superblock slot B. Currently always 320.
    pub superblock_b_offset: u64,

    /// 128-bit file UUID, set at creation of the physical bundle.
    /// Changes on Save As (the physical bundle is a new file).
    /// Distinct from document_id, which identifies the logical work.
    pub file_uuid: FileUuid,
```

```
    /// Reserved bytes at offsets 48..60. MUST be zero on write;
    /// MUST be ignored on read. Future minor versions MAY use
    /// these bytes for additional fields whose absence is
    /// interpreted as zero.
    pub reserved: [u8; 12],

    /// CRC-32C of bytes 0..60 of this header (i.e., all preceding
    /// fields plus the reserved bytes). At fixed offset 60..64.
    pub header_crc: u32,
}
```

REQUIREMENT 8.2

The header **MUST** be exactly 64 bytes at file offset zero. Its contents **MUST NOT** change after file creation, except for the case where a major-version upgrade rewrites the entire file. Readers **MUST** verify the magic bytes and the header CRC before consulting any other part of the file.
Reserved bytes **MUST** be zero when written by current implementations. Readers **MUST** ignore the values of reserved bytes (future versions may use them).
The `file_uuid` is fixed for the lifetime of a physical bundle and persists across commits to that bundle. A filesystem-level byte copy preserves the `file_uuid` (the copy is the same physical bundle duplicated). Save As, export-as-new-bundle, and derivative-work creation **MUST** assign a fresh `file_uuid`; the resulting bundle is a new physical bundle, distinct from its source.
The `file_uuid` identifies the physical bundle, not the logical work. Two bundles with the same `file_uuid` are byte-level copies of the same physical bundle; two bundles with the same `document_id` (defined in the manifest; see Section 8.4.2) are versions of the same logical work, which may or may not share their `file_uuid`.

The Container Epoch

The `format_major` field is not only a wire-layout discriminator. It is the container's *epoch*: the statement of which guarantees held at the moment the file was created. Because the header never changes after creation, the epoch is a property of the file's creation and cannot drift as the document is committed to — which is precisely what makes it able to carry a provenance guarantee that a later generation must not be able to inherit.

Major 1 is the first epoch to use that capacity. A major-1 container is one whose every base-bearing commit was validated against a supplied reduction authority; a major-0 container carries no such guarantee, and no sequence of commits can promote one into the other.

REQUIREMENT 8.3

A reader **MUST** classify `format_major` three ways rather than testing it for equality with the version it writes: 0 is *legacy* and **MUST** be decoded deliberately as such; 1 is *current*; any other value **MUST** be rejected as an unsupported format version. The classification **MUST** be carried as a value the reader can act on, not re-derived by comparison at each use site.

Behaviour is then determined by the container's epoch together with the presence of a canonical base:

- *Legacy, no canonical base* — the bundle **MAY** be opened. Nothing whose validity cannot be established is exposed.
- *Legacy, canonical base present* — the bundle **MUST** be refused. A base predating any reduction authority is not safe materialized state, and **MUST NOT** be served in a read-only or otherwise restricted mode: a restricted view of unverifiable canonical state is still unverifiable canonical state, presented confidently.
- *Legacy, a commit attempts to add or replace a canonical base* — the commit **MUST** be refused, and the implementation **MUST** direct the writer to repack into a fresh major-1 container. This is the rule that makes the epoch non-inheritable, and it is the reason a header field can carry this guarantee where a superblock field could not.
- *Current, no canonical base* — the bundle **MAY** be opened, and **MAY** be committed to.
- *Current, canonical base present or introduced* — the base **MUST** be validated against the reduction authority the opening implementation supplies.

Where a container's canonical base disagrees with its active superblock, that disagreement is *corruption* and **MUST** be reported as a malformed bundle *before* any epoch rule above is applied, in either epoch. Ordering the epoch rules first would collapse tampering into staleness and erase the distinction the reduction authority depends on.

8.3.2 The Superblock Slots

Each superblock slot is exactly 256 bytes. The bundle has two slots (A at offset 64, B at offset 320). One is active; the other is the write target for the next commit. Atomic commit is the act of writing the inactive slot and durably flushing it.

```
pub struct Superblock {
    /// Magic bytes: ASCII "MUSCSUPR" (8 bytes).
    pub magic: [u8; 8],

    /// Generation counter. The active superblock is the one with
    /// the highest valid generation.
    pub generation: u64,

    /// Offset of the manifest chunk for this generation.
    pub manifest_offset: u64,

    /// Length of the manifest chunk in bytes.
    pub manifest_length: u64,

    /// BLAKE3 content hash of the manifest chunk's uncompressed
    /// payload, with domain-separated preimage
    /// (Section~\ref{sec:format:hashing}).
    pub manifest_hash: ContentHash,

    /// Schema version of the manifest at this generation.
```

```
    pub manifest_schema_version: SchemaVersion,

    /// Reduction algorithm version that produced any
    /// canonical-base snapshots in this manifest.
    pub reduction_algorithm_version: ReductionAlgorithmVersion,

    /// Profile under which this superblock is valid.
    pub profile_id: ProfileId,

    /// Commit state. Distinguishes a fully-committed superblock
    /// from one that was written but whose commit had not yet
    /// completed when the writer crashed (rare; the protocol does
    /// not normally leave such a state visible).
    pub commit_state: CommitState,

    /// Wall-clock timestamp at commit, in canonical units.
    /// Advisory only; superblock selection is by generation, not
    /// by timestamp.
    pub commit_timestamp: WallClockTime,

    /// CRC-32C of the bytes of this superblock up to (but not
    /// including) the CRC field. Lets readers reject torn writes.
    pub superblock_crc: u32,

    /// Reserved padding to 256 bytes. MUST be zero on write;
    /// MUST be ignored on read.
    pub reserved: [u8; /* computed */ 256 - HEADER_FIELDS_SIZE],
}

pub enum CommitState {
    /// Fully committed. The manifest at manifest_offset is the
    /// canonical state for this generation.
    Committed,

    /// Reserved for future use; writers MUST NOT produce
    /// non-Committed values in this version of the format.
    Reserved(u32),
}
```

8.3.3 Superblock Selection

REQUIREMENT 8.4

On open, readers **MUST** perform the following selection:

1. Read both superblock slots.
2. For each slot, validate: magic bytes, CRC, that the referenced manifest chunk's actual BLAKE3 hash matches the declared `manifest_hash`, and that `commit_state` is `Committed`. A slot failing any of these checks is *not valid* for ordinary selection.
3. A slot whose `commit_state` is not `Committed` is invalid for ordinary selection but

MAY be inspected in diagnostic recovery mode. Writers in this format version **MUST NOT** produce such states under normal commit; observing one indicates either a crashed writer (recovery falls back to the other slot per the atomic-write protocol) or a non-conforming writer.

4. If both slots are valid and their generations differ by more than one, the reader **MUST** report an integrity anomaly (the file may have been tampered with, accidentally merged, or written by a non-conforming implementation). The reader **MAY** continue in read-only recovery mode using the highest-generation valid slot; it **MUST NOT** silently treat the file as normal.
5. If both slots are valid and their generations differ by exactly one, the slot with the higher generation is active.
6. If both slots are valid and their generations are *equal*, the reader **MUST** compare the slots' *load-bearing* selection fields: `manifest_hash`, `manifest_schema_version`, `reduction_algorithm_version`, and `profile_id`. The advisory fields `commit_timestamp` and the physical chunk offset/length are *excluded* from this comparison.
   - If every load-bearing field is equal, the two slots select equivalent canonical state; the reader **MUST** deterministically choose slot A and open normally.
   - If any load-bearing field differs, the reader **MUST** report a format-layer `IntegrityAnomaly` of kind `DivergentSameGeneration` and open the bundle read-only. Two genuinely different committed states cannot share a generation under a conforming writer (the atomic-write protocol increments the generation on every commit), so the divergence is a structural failure, not a state the reader may silently resolve by picking one.
7. If exactly one slot is valid, that slot is active.
8. If neither slot is valid, the file is corrupt; readers **MUST** surface this as a hard error and **MUST NOT** synthesize state.

RATIONALE

The advisory fields are excluded from the equal-generation comparison because they do not determine which canonical state a slot selects: `commit_timestamp` is wall-clock metadata, and the physical offset/length merely locate the manifest chunk in the file. Two slots that agree on the four load-bearing fields name the same canonical document even if they were written at different times or to different offsets, so treating an advisory-only difference as divergence would raise false anomalies on benign repacks. Conversely, a difference in any load-bearing field means the two slots disagree about canonical state itself, which is the precise condition that must not be silently resolved.

8.4 The Manifest

The manifest is a table of roots and declarations. It is itself a chunk (content-addressed, immutable). Each commit writes a new manifest chunk and points the inactive superblock slot at it.

```rust
pub struct Manifest {
    /// Identifier of the logical work. Stable across Save As
    /// operations and across copies that are intended to remain
    /// versions of the same work. May persist across forks if
    /// the user explicitly preserves it.
    pub document_id: DocumentId,

    /// Optional identifier of a shared ancestor document. Used by
    /// version-control workflows and fork-tracking to express
    /// genealogy. Absent for documents with no declared ancestry.
    pub lineage_id: Option<LineageId>,

    /// Identifier of this manifest. Each commit produces a new
    /// ManifestId; old manifests are retained in the chunk store
    /// until garbage collection removes them subject to the
    /// retention policy.
    pub manifest_id: ManifestId,

    /// Generation counter matching the superblock that references
    /// this manifest.
    pub generation: u64,

    /// Operation-envelope blocks defining the canonical document.
    /// Block ordering is irrelevant for canonical state (envelopes
    /// are a set), and the canonical manifest encoding sorts chunk
    /// references ascending by their encoded form (Appendix D), so
    /// a writer cannot order this list by stamp; stamp-ordered
    /// scanning goes through operation_block_summaries below.
    pub operation_roots: Vec<ChunkRef>,

    /// Per-block summary metadata (Section on operation-envelope
    /// blocks below), keyed by the block's ChunkId in canonical
    /// ascending order. Supplied by the operation layer and carried
    /// opaquely by the storage layer; non-canonical and optional (a
    /// block need not have an entry). Lets a reader select or skip
    /// a block by causal frontier or stamp range without decoding
    /// its envelopes.
    pub operation_block_summaries:
        BTreeMap<ChunkId, OperationBlockSummary>,

    /// Optional operation index chunk: maps OperationId to
    /// (block, offset) for fast random access. If absent,
    /// implementations rebuild on demand by scanning blocks.
    pub operation_index_root: Option<ChunkRef>,

    /// The active canonical-base snapshot, if pruning has occurred.
    /// At most one canonical base is active at a time. When
    /// present, the canonical document is the base snapshot's
    /// materialized state plus the deterministic reduction of
    /// envelopes whose OperationIds are not covered by the base's
    /// causal frontier.
    pub canonical_base: Option<SnapshotRef>,
```

```
    /// Acceleration snapshots: caches that materialize state at
    /// various causal frontiers for fast incremental loading.
    /// These MAY be discarded freely without affecting canonical
    /// state.
    pub acceleration_snapshots: Vec<SnapshotRef>,

    /// Blob references for large opaque content (audio, images,
    /// custom fonts).
    pub blob_roots: Vec<BlobRef>,

    /// Profile declarations: which conformance profiles this
    /// manifest claims to satisfy.
    pub profile_declarations: Vec<ProfileDeclaration>,

    /// Extension declarations: which extensions contributed to
    /// this manifest, with their required/optional flag and
    /// edit barriers.
    pub extension_declarations: Vec<ExtensionDeclaration>,

    /// Text projection root chunk, if a text projection is
    /// maintained in this bundle. Non-canonical accelerator.
    pub text_projection_root: Option<ChunkRef>,

    /// Optional integrity index root chunk: cross-references chunk
    /// hashes for faster integrity verification of large bundles.
    /// Non-canonical accelerator; absence is permitted.
    pub integrity_root: Option<ChunkRef>,
}
```

8.4.1 Canonical and Non-Canonical Manifest Roots

The manifest's fields partition cleanly into two groups by their relation to canonical state.

Canonical roots (define the document):

- `operation_roots`: operation-envelope blocks containing the canonical operation set.
- `canonical_base`: the active canonical base snapshot, if pruning has occurred.
- `blob_roots`: blobs referenced by canonical operations or by canonical reduced state are themselves canonical; blobs referenced only by acceleration structures are non-canonical.
- `document_id`, `lineage_id`, `profile_declarations`, `extension_declarations`: structural metadata governing how the canonical document is interpreted.

Non-canonical reachable roots (accelerators only):

- `operation_index_root`: rebuildable from operation envelope blocks.
- `acceleration_snapshots`: caches at causal frontiers other than the canonical base.
- `text_projection_root`: derivable from canonical state.
- `integrity_root`: rebuildable from chunk hashes.

REQUIREMENT 8.5

The manifest **MUST** list every chunk that contributes to the current canonical state, transitively through its canonical roots (operation roots, canonical base, canonical blobs and extension chunks). It **MAY** additionally list non-canonical reachable roots (acceleration snapshots, operation index, text projection, integrity index, layout caches).
Chunks not reachable from any manifest root, canonical or non-canonical, are unreachable; readers **MUST NOT** rely on them and conservative garbage collection **MAY** reclaim them (Section 8.11).
Non-canonical reachable chunks **MAY** be rebuilt or replaced without altering canonical state. If a non-canonical chunk's hash fails verification, it **MUST** be discarded and (if needed) rebuilt; failed verification of a non-canonical chunk is *not* a corruption of the bundle.
Failed verification of a canonical chunk (operation envelope block, canonical base snapshot, canonical blob, manifest) *is* corruption and **MUST** be surfaced as a hard error.
User-facing metadata (title, composer, lyricist, copyright, and similar fields) is *not* part of the manifest. Such metadata lives in the operation set and the materialized state. The manifest carries only the structural roots and declarations that the format needs to locate and validate content.

8.4.2 File, Document, and Lineage Identity

The format distinguishes three identifiers, each serving a different identity question:

`file_uuid`. Physical bundle identity. Set at file creation; persists for the lifetime of the physical file. **Changes on Save As**: a copy created by Save As is a new physical bundle and **MUST** be assigned a fresh `file_uuid`. Used by tools that need to identify "this exact file on disk" (caches, backup software, file fingerprinting).

`document_id`. Logical work identity. Stable across Save As copies that are intended to remain the same work (the canonical case: "I'm saving my piece to a backup folder, and it's still the same piece"). **Persists across forks at the user's discretion**: a fork that represents a derivative work **SHOULD** have a new `document_id`, while a fork that is a working copy of the original **SHOULD** preserve it. The format does not enforce a policy; the editor surfaces the choice.

`lineage_id`. Optional shared ancestor identity. Records that two documents share a common ancestor for version-control or genealogy purposes. Multiple documents **MAY** share a `lineage_id`; the field is set when a fork explicitly preserves ancestry information.

REQUIREMENT 8.6

Implementations **MUST** assign a fresh `file_uuid` on any Save As operation. They **MUST NOT** carry the source file's UUID into the new file: doing so would conflate physical and logical identity and break tools that rely on `file_uuid` for content fingerprinting.
The choice of whether to preserve `document_id` across a Save As is exposed to the user: the editor **SHOULD** default to preserving `document_id` (the common case is "save a

copy of the same work to a new file") and **SHOULD** offer a clear "Save As Derivative Work" or equivalent action that mints a new `document_id`.
The format **MUST NOT** treat documents with the same `document_id` as automatically synchronizable: that decision belongs to the collaboration transport (Appendix A).

8.4.3 Manifest Encoding

The manifest chunk is special: it is the bootstrap entry from the superblock into the chunk graph. A cold reader needs to decode the manifest before it has any information from the chunk store.

REQUIREMENT 8.7

In this format version, the manifest chunk **MUST** be stored *uncompressed*. Its `CompressionAlgorithm` on disk is implicitly `None`; the superblock's `manifest_length` field is therefore both the on-disk length and the uncompressed payload length. Implementations **MUST** reject as malformed any bundle whose manifest payload is not directly parseable as a canonical manifest chunk's uncompressed bytes.
This rule eliminates a bootstrap chicken-and-egg problem: a reader can decode the manifest using only information present in the fixed header and the active superblock, without first consulting the manifest to learn how the manifest is compressed.
All other chunks (operation envelope blocks, snapshots, extension data, layout caches, blobs, integrity indexes, operation indexes, text projection chunks) **MAY** be compressed. Future format major versions **MAY** permit manifest compression, but doing so requires explicit superblock fields (`manifest_compression`, `manifest_uncompressed_length`) and is a non-backward-compatible change.

RATIONALE

The manifest is small (a few kilobytes at most for typical scores), so compression yields negligible space savings. Eliminating the bootstrap dependency is far more valuable than the few hundred bytes of compression headroom.

REQUIREMENT 8.8

The `manifest_id` field **MUST** be derived as

$$\texttt{ManifestId} = \text{trunc128}(\text{BLAKE3}(\texttt{"MUSCMNIF"} \| \texttt{document_id} \| \texttt{generation} \| \texttt{manifest_body}))\,,$$

where `document_id` is its 16-byte canonical form, `generation` is the 8-byte little-endian `u64`, and `manifest_body` is the canonical manifest encoding with the `manifest_id` field itself *excluded*. The exclusion is normative: a conforming writer **MUST** omit (equivalently, zero) the `manifest_id` field when computing the preimage, so that the identifier does not reference itself. trunc128 takes the leading 16 bytes of the 32-byte BLAKE3 output. Two conforming writers committing the same manifest body at the same generation of the same document therefore derive identical `ManifestId`s.

8.5 Content Hashing

Epiphany uses BLAKE3 as its single content-hashing algorithm.

REQUIREMENT 8.9

All content hashes in the bundle **MUST** be BLAKE3-256 outputs (32 bytes). Implementations **MUST NOT** use any other algorithm for content addressing; SHA-256, SHA-3, and other algorithms **MUST NOT** appear in this version of the format. Future major versions **MAY** introduce additional algorithms; such changes require a new format major version.

8.5.1 Domain-Separated Preimages

A naive content-addressing scheme that hashes only payload bytes is vulnerable to collisions across chunks of different semantic kinds. Epiphany uses domain-separated preimages.

```
/// Computes the canonical hash preimage for a chunk's content.
/// The chunk's compression algorithm is metadata on the ChunkRef,
/// not part of the content identity: identical uncompressed
/// payloads produce identical hashes regardless of how they are
/// physically compressed in storage. This preserves deduplication
/// across compression choices.
fn hash_preimage(
    domain_tag: &[u8; 8],
    chunk_kind: ChunkKind,
    schema_version: SchemaVersion,
    uncompressed_length: u64,
    uncompressed_payload: &[u8],
) -> Vec<u8> {
    let mut preimage = Vec::new();
    preimage.extend_from_slice(domain_tag);
    preimage.extend_from_slice(&chunk_kind.canonical_bytes());
    preimage.extend_from_slice(&schema_version.canonical_bytes());
    preimage.extend_from_slice(&uncompressed_length.to_le_bytes());
    preimage.extend_from_slice(uncompressed_payload);
    preimage
}
```

REQUIREMENT 8.10

The content hash of a chunk **MUST** be BLAKE3 of the canonical preimage above. The domain tag, chunk kind, schema version, and uncompressed length are part of the preimage; the compression algorithm is *not*. Two chunks with identical uncompressed payloads, identical kinds, and identical schema versions **MUST** produce identical hashes regardless of their compression choices.

RATIONALE

Including compression algorithm in the hash would cause identical content stored with different compression to have different addresses, defeating deduplication and forcing churn during recompression or repack. Including domain tag, kind, and schema version in the preimage closes a real collision: two semantically different chunks with identical raw bytes are no longer interchangeable.
The domain tag is a fixed 8-byte string identifying the bundle format and protecting against cross-format collisions if the same BLAKE3 hash were reused in unrelated contexts. The canonical domain tag for `.musc` chunks is `"MUSCCHNK"`; manifest chunks use `"MUSCMANI"`; blob payloads use `"MUSCBLOB"`.

8.6 Chunks

A chunk is an immutable, content-addressed unit of storage. Chunks are referenced from the manifest by `ChunkRef`.

```
pub struct ChunkRef {
    /// Content identifier: BLAKE3 of the canonical preimage.
    pub id: ChunkId,

    /// Kind of chunk. The reader uses this to dispatch parsing.
    pub kind: ChunkKind,

    /// Schema version targeted by this chunk's payload encoding.
    pub schema_version: SchemaVersion,

    /// Offset of the chunk's compressed payload in the bundle file.
    pub offset: u64,

    /// Length of the on-disk compressed payload.
    pub compressed_length: u64,

    /// Length of the uncompressed payload (used to size buffers
    /// before decompression).
    pub uncompressed_length: u64,

    /// Compression algorithm. Part of the ChunkRef, not part of
    /// the chunk's identity.
    pub compression: CompressionAlgorithm,

    /// Restated content hash, redundant with the id field but
    /// included in the ChunkRef for fast verification without
    /// fetching the chunk.
    pub hash: ContentHash,
}

pub enum ChunkKind {
    /// A block of operation envelopes.
    OperationEnvelopeBlock,
```

```
    /// An operation-id-to-block index accelerator.
    OperationIndex,

    /// A materialized snapshot of canonical state (full or
    /// partial). Canonical if referenced as the manifest's
    /// canonical_base; cache (acceleration) otherwise.
    Snapshot,

    /// A large opaque blob (audio, image, font, ML model).
    Blob,

    /// Extension-defined data, preserved opaquely by core readers.
    ExtensionData,

    /// The text projection's root document.
    TextProjection,

    /// Layout cache: derivative from canonical state, always
    /// discardable.
    LayoutCache,

    /// Integrity index: cross-references chunk hashes.
    IntegrityIndex,

    /// Manifest chunk. Special: referenced from the superblock,
    /// not from another chunk.
    Manifest,
}

pub enum CompressionAlgorithm {
    /// No compression. Stored payload bytes equal uncompressed.
    None,

    /// Zstandard compression. Implementations MUST support reading.
    Zstd { level: u8 },

    /// Other algorithms reserved for future versions.
    Reserved(u8),
}

pub struct ContentHash(pub [u8; 32]);

/// Chunk identifier. A newtype around ContentHash: the chunk's
/// identifier is identical to its content hash, but the type
/// distinction is preserved to make role visible at use sites
/// (a ChunkId is what you store in a ChunkRef and what you look
/// up; a ContentHash is what you compute by hashing). Both occupy
/// the same 32 bytes.
pub struct ChunkId(pub ContentHash);
```

REQUIREMENT 8.11

`ChunkKind::canonical_bytes()` is a single byte: the variant's declaration-order discriminant. Because that byte is part of the chunk hash preimage (Section 8.6, "Domain-Separated Preimages"), this assignment is normative and stable and **MUST NOT** be reordered:

Disc.	Variant	Disc.	Variant
0	`OperationEnvelopeBlock`	5	`TextProjection`
1	`OperationIndex`	6	`LayoutCache`
2	`Snapshot`	7	`IntegrityIndex`
3	`Blob`	8	`Manifest`
4	`ExtensionData`		

`CompressionAlgorithm` encodes as a fixed *two* bytes: a declaration-order discriminant byte followed by a single parameter byte that is always present. `None` = 0, and its parameter byte **MUST** be zero — a reader **MUST** reject a non-zero one rather than ignore it (Section 8.18, and the Binary Format companion's CHUNK REFERENCES). `Zstd{level}` = 1 with the `level` in the parameter byte; `Reserved(u8)` = 2 with the reserved value in the parameter byte. The parameter byte is written even for `None`, so the encoding is fixed-width and a reader always consumes two bytes. Compression is *not* part of a chunk's content identity (it is metadata on the `ChunkRef`), so these discriminants are stable but do not affect chunk hashes.

`ProfileId`'s discriminants are pinned in Section 8.15 (Requirement 8.31).

REQUIREMENT 8.12

Chunks **MUST** be immutable. Once a chunk is written and its bytes durably flushed, those bytes **MUST NOT** be modified. New state is encoded as new chunks with new identifiers; old chunks become unreachable garbage when their referencing manifests are no longer reachable.

Implementations **MUST** verify chunk hashes on first read of each chunk in a session: decompress the payload, compute the canonical preimage's BLAKE3, and confirm the result matches the declared `hash`. Subsequent reads **MAY** skip verification if the chunk has been validated within the current session. A chunk whose hash does not match **MUST** be treated as corrupt: if it is a cache chunk, discard and rebuild; if it is a canonical chunk (operation block, retained snapshot, or blob referenced from the manifest), surface as hard corruption.

Readers **MUST** verify that decompression of a chunk's payload produces exactly `uncompressed_length` bytes. Length mismatch is treated as corruption.

8.7 Operation Envelope Blocks

The canonical document is stored as operation-envelope blocks. Each block is a chunk of kind `OperationEnvelopeBlock` whose payload is a vector of envelopes. A block is identified by its `ChunkId`; because chunks are content-addressed, that identifier cannot appear inside the block payload itself. Summary metadata for a block travels in the *manifest* (`operation_block_summaries`),

keyed by the block's chunk id: the summary is computed by the operation layer from the block's envelopes, and the storage layer carries it as opaque bytes without interpreting it.

```rust
pub struct OperationEnvelopeBlock {
    /// Operation envelopes in this block. Order within a block is
    /// not normative for canonical reduction (envelopes are a set)
    /// but writers SHOULD preserve authoring order within a block
    /// for diagnostic purposes.
    pub envelopes: Vec<OperationEnvelope>,
}

/// Manifest-side summary of one block, keyed by the block's
/// ChunkId. Non-canonical and optional: a reader MUST NOT rely
/// on a summary being present, and canonical state never depends
/// on one.
pub struct OperationBlockSummary {
    /// DVV summary covering the envelopes in this block. Enables
    /// readers to determine causal coverage without parsing every
    /// envelope.
    pub dvv_summary: CausalContext,

    /// Earliest stamp in the block.
    pub min_stamp: OperationStamp,

    /// Latest stamp in the block.
    pub max_stamp: OperationStamp,
}
```

REQUIREMENT 8.13

Writers **SHOULD** begin a new operation-envelope block when adding another envelope would cause the uncompressed block payload to exceed 1 MiB, except when an individual envelope's encoded size exceeds 1 MiB, in which case the envelope occupies its own block.
Readers **MUST** accept valid blocks of any size up to a profile- declared maximum (default: 64 MiB uncompressed). Profile declarations **MAY** raise or lower this bound. Blocks exceeding the active profile's bound **MUST** be treated as malformed.
The `operation_roots` field of the manifest **MUST** list every operation-envelope block that contributes to the canonical document. The set of envelopes *is* the union of all envelopes across all referenced blocks; block boundaries are storage artifacts, not semantic structure.

8.7.1 The Operation Index

The optional `operation_index_root` chunk maps each `OperationId` to the `ChunkRef` of its enclosing block plus an offset within the block. It enables $O(\log n)$ lookup of an operation by id.

REQUIREMENT 8.14

The operation index is an acceleration structure, not canonical. If absent, readers rebuild it by scanning all blocks. If present but corrupt or stale, readers **MUST** reject the index and rebuild from blocks.
Writers **SHOULD** rebuild or incrementally update the operation index at commit time when the operation set has grown significantly since the last commit.

8.8 Snapshots and Canonical Bases

A snapshot is a materialized score state at a particular causal frontier. Snapshots have two distinct roles:

Acceleration snapshot. A cache of the materialized state at some point, used to skip reduction work. Listed in the manifest (or not), but *not* declared as a canonical base. Discardable.

Canonical base. A materialized state at a causal frontier that the manifest declares as the base after pruning. Together with the operation envelopes after the frontier, this constitutes the canonical document. Not discardable while it remains declared.

```rust
pub struct SnapshotRef {
    pub snapshot_id: SnapshotId,

    /// The causal frontier this snapshot materializes: the
    /// envelopes whose effects are baked into the snapshot.
    pub covers_causal_frontier: CausalContext,

    /// Reduction algorithm version under which the snapshot was
    /// produced. Snapshots produced under an earlier algorithm
    /// version cannot be used as canonical bases under a later
    /// algorithm without rebuilding.
    pub reduction_algorithm_version: ReductionAlgorithmVersion,

    /// Profile under which the snapshot was produced.
    pub profile_id: ProfileId,

    /// Root chunk of the snapshot's materialized state.
    pub root: ChunkRef,

    /// Hash of the snapshot's root chunk for fast verification.
    pub hash: ContentHash,
}
```

8.8.1 The Canonical Document Identity

REQUIREMENT 8.15

The canonical document is defined as follows:

- If `canonical_base` is `None`, the canonical document is the deterministic reduction (Section 6.3) of the union of envelopes in all `operation_roots` blocks.
- If `canonical_base` is `Some(base)`, the canonical document is the base snapshot's materialized state plus the deterministic reduction of all envelopes whose `OperationId` is *not covered* by the base's `covers_causal_frontier`. An `OperationId` is covered by a `CausalContext` if and only if it appears in the dotted version vector's per-replica contiguous range or among its dots (i.e., causal-precedes-or-equals under the DVV's induced partial order). HLC stamps **MUST NOT** be used to determine coverage; only the DVV frontier is canonical.
- A snapshot may serve as the canonical base only if its `reduction_algorithm_version` equals the active superblock's `reduction_algorithm_version` and its `profile_id` matches the manifest's profile declarations.

At most one `canonical_base` is active at any time. Pruning replaces the current canonical base with a new one whose frontier strictly dominates the previous frontier under the DVV partial order; the old base becomes unreachable and is subject to garbage collection.

Acceleration snapshots (`acceleration_snapshots`) **MUST** be ignored for the purposes of canonical state. They exist to accelerate cold open and incremental reduction; if their content disagrees with what the canonical reduction would produce, they **MUST** be rebuilt or discarded.

Caches, indexes, layout chunks, and other materialized artifacts in the bundle are acceleration structures only and **MUST** be ignored or rebuilt if they disagree with the canonical document.

REQUIREMENT 8.16

An implementation **MUST** name the reduction semantics it implements, and **MUST** validate a canonical base against that value rather than against a value the base itself supplied.

Comparing a base's `reduction_algorithm_version` only against the active superblock's, as Requirement 8.15 requires, is necessary but *not* sufficient: a writer that seeds the superblock from the base's own self-report makes that comparison an identity, so it holds for every conformingly written document and detects only tampering. The semantics the running implementation actually implements is a third value, and it **MUST** participate.

Accordingly:

- An implementation **MUST** expose the reduction-algorithm version it implements, and every reader and writer of a canonical base **MUST** state which semantics it implements rather than infer it from the document.

- A reader **MUST** refuse to open a document whose canonical base's `reduction_algorithm_version` differs from the semantics the reader implements. A document with no canonical base **MUST** open regardless, since it carries no reduced state that could be stale.
- A writer **MUST** refuse to emit or replace a canonical base whose `reduction_algorithm_version` differs from the semantics the writer implements. A commit that does not emit or replace the base **MUST NOT** be refused on this ground.
- The refusal **MUST NOT** degrade to a read-only open and **MUST NOT** be reported as an integrity anomaly. A stale base is not a restricted-but-correct view of the document; it is state computed under different rules, and serving it read-only would present incorrect canonical state as authoritative.
- A refusal on this ground **MUST** remain distinguishable from a malformed-document failure. A base whose version disagrees with its own superblock is corrupt, and corruption **MUST** be reported as such: the two conditions detect different faults, and collapsing them loses the distinction this requirement rests on.

Rebuilding is out of scope for opening. An implementation **MUST NOT** silently rebuild a stale base during open, because the envelopes required to rebuild may have been pruned (Requirement 8.17); a rebuild path is sound only where the full pre-base history is demonstrably present.

8.8.2 Pruning

REQUIREMENT 8.17

Pruning is the act of removing operation envelopes that are causally covered by a new canonical base snapshot. Pruning:

1. Materializes a snapshot covering the chosen causal frontier (a DVV).
2. Replaces the current `canonical_base` field with a `SnapshotRef` naming the new snapshot.
3. Removes operation-envelope blocks whose envelopes are *entirely* covered by the new frontier. Blocks with mixed coverage **MUST** be split or retained whole; uncovered envelopes **MUST** remain reachable.
4. Commits the new manifest via the atomic write protocol (Section 8.10).

Pruning **MUST NOT** alter the canonical document state: the reduction of (base snapshot + post-frontier envelopes) under the active algorithm **MUST** equal the reduction of the pre-pruning envelope set.
Implementations **MUST NOT** prune envelopes outside the explicit pruning protocol. Operation envelopes are not silently garbage- collected based on age or other heuristics.

8.9 Blobs

The blob store holds large opaque content: audio reference tracks, embedded images, custom fonts, ML model checkpoints.

REQUIREMENT 8.18

A `BlobId` **MUST** be BLAKE3 over the *bare* preimage `"MUSCBLOB" ‖ uncompressed_payload`: the 8-byte domain tag immediately followed by the uncompressed payload bytes, with *no* chunk-kind, schema-version, or length fields. This is deliberately *unlike* the structured chunk preimage of Section 8.6 (which commits to kind, schema version, and length): a blob is opaque payload with no schema, so its identity commits only to the domain tag and the bytes. As with chunks, the compression algorithm is metadata on the `BlobRef` and is *not* part of the blob's identity, so identical payloads stored under different compression share a `BlobId`.

```rust
pub struct BlobRef {
    pub blob_id: BlobId,
    pub media_type: String,  // RFC 6838 media type
    pub offset: u64,
    pub compressed_length: u64,
    pub uncompressed_length: u64,
    pub compression: CompressionAlgorithm,
    pub hash: ContentHash,

    /// Optional declared maximum size that readers MAY use to
    /// reject blobs that exceed reader resource limits.
    pub declared_max_uncompressed_length: Option<u64>,
}
```

REQUIREMENT 8.19

Readers **MUST** apply resource limits to blob loading: uncompressed length **MUST** be checked against the reader's policy before decompression begins; `media_type` **MUST** be validated against the reader's accepted list before any type-specific parsing. Implementations **MUST NOT** eagerly load all blobs at open; blobs **MUST** be streamed on demand. Operations requiring blob content **MUST** be designed for lazy access. Blob content is opaque to the core. The core stores, transmits, and integrity-checks blobs; it does not interpret their bytes.

8.10 The Atomic Write Protocol

A commit transforms the bundle from one canonical state to another. The protocol uses two superblock slots so that a crash at any point during commit leaves the bundle in either the pre-commit state (active superblock unchanged) or the post-commit state (active superblock now points at the new manifest); no intermediate state is visible.

REQUIREMENT 8.20

Commits **MUST** follow this sequence:

1. Write all new chunks (envelope blocks, possibly a new snapshot, possibly a new operation index, etc.) to file regions outside the prelude and outside currently-reachable chunks. Writers **MAY** append at end-of-file or fill known- unreachable regions, but **MUST NOT** overwrite any currently-reachable chunk.
2. Durably flush the file.
3. Compute the new manifest chunk's payload and write it.
4. Durably flush the file.
5. Compute the new `Superblock` with generation = active_generation + 1 and `manifest_offset` pointing at the new manifest chunk.
6. Write the new superblock to the currently-inactive superblock slot (a 256-byte write at a fixed offset).
7. Durably flush the file. *This is the commit point.* Before this flush returns successfully, the active superblock is the old one; after it returns successfully, the active superblock is the new one.

After step 7, the previously-active superblock slot becomes the inactive slot for the next commit. Garbage collection is a separate, optional, deferred operation (Section 8.11).

8.10.1 Crash Recovery

REQUIREMENT 8.21

On open after any crash, recovery follows the superblock selection rule (Section 8.3). The bundle is always in a recoverable state:

- Crash between steps 1 and 7: the active superblock is unchanged; the new chunks and the new manifest are unreachable garbage; the document opens at the pre-commit state. A later GC reclaims the orphaned bytes.
- Crash during step 7: the superblock slot write may be torn. The CRC check rejects torn writes. Recovery falls back to the other slot. The document opens at the pre-commit state. The partially-written slot will be overwritten by the next successful commit.
- Crash after step 7: the new superblock is active. The document opens at the post-commit state. No special recovery is needed.

Recovery **MUST NOT** synthesize or guess: if neither superblock validates, the file is corrupt and **MUST** be reported as such.

8.11 Garbage Collection and Retention

Garbage collection reclaims bytes occupied by chunks that are no longer reachable from any retained manifest. Retention policy governs which old manifests are preserved for rollback.

8.11.1 Retention Policy

```rust
pub struct RetentionPolicy {
    /// Maximum number of previous manifests to retain beyond the
    /// active one. 0 means "no rollback retention"; the active
    /// manifest is the only one preserved. The default declared by
    /// the Full profile is profile-defined.
    pub retain_previous_manifests: u32,

    /// Optional wall-clock duration beyond which old retained
    /// manifests may be reclaimed regardless of count. None means
    /// no time-based eviction.
    pub retain_duration: Option<WallClockDuration>,

    /// Whether to preserve manifests explicitly marked as named
    /// checkpoints (user-invoked "save a version" action) beyond
    /// the count and duration limits.
    pub retain_named_checkpoints: bool,
}
```

REQUIREMENT 8.22

The active conformance profile **MUST** declare a `RetentionPolicy`. Implementations **MUST** honor the declared policy: chunks reachable from any retained manifest **MUST NOT** be garbage-collected, and retention selection (which manifests are retained) **MUST** be deterministic given the manifest history and the policy.
Rollback is the act of writing a new superblock that points at a previously-active manifest still reachable in the chunk store. The fixed layout has exactly two superblock slots; rollback does not require additional on-disk superblocks. It requires only that the target manifest's reachable chunks remain in the bundle, which the retention policy guarantees.

8.11.2 Garbage Collection Rules

REQUIREMENT 8.23

Garbage collection is a conservative, optional, deferred operation. It **MUST** preserve every byte reachable transitively from:

1. The active superblock's manifest.
2. Every retained manifest selected by the active `RetentionPolicy`.
3. Any pending edit's working set (chunks written but not yet committed).

Garbage collection **MUST NOT** run as part of a commit's critical path. It **MAY** run as a background operation or as an explicit user-invoked compaction. Pre-GC and post-GC bundles **MUST** contain identical canonical state and identical retained-rollback reachability.

8.12 Forward Compatibility and Edit Barriers

The bundle must round-trip cleanly across implementations that may not all understand every extension referenced in it.

8.12.1 Extension Declarations

```rust
pub struct ExtensionDeclaration {
    pub extension_id: ExtensionId,
    pub version: SemVer,

    /// If true, an implementation that does not understand this
    /// extension MUST refuse to open the bundle for editing, but
    /// MAY open read-only.
    pub required: bool,

    /// Root chunks owned by this extension. Opaque to core
    /// readers; preserved across reads and writes.
    pub preserved_chunk_roots: Vec<ChunkRef>,

    /// Object kinds this extension contributes to or depends on,
    /// for edit-barrier evaluation.
    pub affected_object_kinds: Vec<ObjectKind>,

    /// Edit barriers declared by this extension.
    pub edit_barriers: Vec<EditBarrier>,
}

pub struct EditBarrier {
    /// Scope of the barrier: whole score, region, staff instance,
    /// analysis layer, object set, pitch space, tuning context, etc.
    pub scope: BarrierScope,

    /// Object kinds protected by this barrier.
    pub affected_object_kinds: Vec<ObjectKind>,

    /// Operation kind tags prohibited by this barrier. A barrier
    /// prohibits an operation class, not one exact payload, so it
    /// references OperationKindTag rather than OperationKind.
    pub prohibited_operation_kinds: Vec<OperationKindTag>,

    /// Additional condition narrowing the barrier (e.g., applies
    /// only when a specific attachment is present).
    pub condition: BarrierCondition,
}

/// A discriminator-only view of OperationKind (Chapter ch:semops),
/// without payload. Used by edit barriers, conformance reports,
/// and diagnostic surfaces where the operation's class matters
/// but its payload does not. There is a canonical mapping from
/// OperationKind to OperationKindTag.
pub enum OperationKindTag {
    InsertEvent,
```

```
    DeleteEvent,
    ModifyEvent,
    RespellPitch,
    Transpose,
    CreateCrossCutting,
    DeleteCrossCutting,
    ModifyCrossCutting,
    ChangeRegionTimeModel,
    InsertRegion,
    DeleteRegion,
    InsertStaffInstance,
    DeleteStaffInstance,
    SetUserSystemBreak,
    SetUserPageBreak,
    DeclareTransaction,
    Registered(OperationKindRegistryId),
    // Appended after Registered (append-only vocabulary; the
    // tag's wire discriminants never reorder). Phase-2/Phase-3
    // operation-catalog growth:
    InsertIdentifiedPitch,
    DeleteIdentifiedPitch,
    ModifyIdentifiedPitch,
    CreateVoice,
    DeleteVoice,
    SetMetadata,
    SetMetricGrid,
    InsertStaff,
    SetTimeSignature,
    SetTempoSegment,
    SetStaffLayout,
    // Schema-major-2 revision (repeat authoring). The pair keeps
    // its kind names verbatim, as the cross-cutting tags do.
    CreateRepeatStructure,
    DeleteRepeatStructure,
    // Push 4a: the faithful transpose. Transpose above is frozen.
    TransposeInterval,
    // Genesis tranche G1: name-verbatim, as the two most recent prior
    // additions are (the tag layer's older Create-to-Insert convention is
    // not followed here).
    CreateInstrument,
    // Genesis tranche G2a: the two major-0 settings setters.
    SetCanvasLayoutDefaults,
    SetSpellingPrecedence,
    // Genesis tranche G2b: name-verbatim, as the two prior additions are.
    SetTuningContext,
    // Genesis tranche G3a: the four remaining root-level entity mints,
    // name-verbatim, as the three most recent prior additions are.
    CreateStaffGroup,
    CreatePartDefinition,
    CreateAnalysisLayer,
    CreateView,
    // Genesis tranche G3b: the final rung, name-verbatim, as the four most
    // recent prior additions are. Closes the genesis ladder.
```

```
    CreateMeasure,
    // X3.1: name-verbatim, as the five most recent prior additions are.
    CreateTuplet,
    // X3.6: name-verbatim.
    SetClef,
    SetKeySignature,
}

pub enum BarrierScope {
    WholeScore,
    Region(RegionId),
    StaffInstance(StaffInstanceId),
    AnalysisLayer(AnalysisLayerId),
    ObjectSet(Vec<TypedObjectId>),
    PitchSpace(PitchSpaceId),
    TuningContext,
    /// Registered scope kind for grammar-specific scoping.
    Registered(BarrierScopeRegistryId),
}

/// Additional narrowing condition for an edit barrier. The barrier
/// applies only when its scope, affected object kinds, prohibited
/// operation kinds, and this condition all match the candidate edit.
pub enum BarrierCondition {
    /// The barrier applies unconditionally within its scope.
    Always,

    /// The barrier applies only while the named object exists
    /// (not tombstoned). Used when an extension's invariant
    /// depends on a specific object remaining live.
    ObjectExists(TypedObjectId),

    /// The barrier applies only when the named object carries
    /// data declared by the named extension. Used when the
    /// invariant protected by the barrier exists only for
    /// objects participating in the extension.
    ObjectHasExtensionData {
        object: TypedObjectId,
        extension: ExtensionId,
    },

    /// Conjunction: all conditions must match.
    All(Vec<BarrierCondition>),

    /// Disjunction: any condition matching is sufficient.
    Any(Vec<BarrierCondition>),

    /// Negation: the inner condition must not match.
    Not(Box<BarrierCondition>),

    /// Registered condition kind, evaluated by an extension-aware
    /// implementation. Core implementations evaluate Registered
    /// conditions as Always (conservative): they treat the barrier
```

```
    /// as active whenever an unknown extension's condition would
    /// otherwise narrow it.
    Registered(BarrierConditionRegistryId),
}
```

8.12.2 Behavior Under Unknown Extensions

REQUIREMENT 8.24

When an implementation opens a bundle declaring an extension it does not understand:

- If the extension is `required = true`, the implementation **MUST** either refuse to open the bundle or open it strictly read-only. It **MUST NOT** edit.
- If the extension is `required = false`, the implementation **MAY** open the bundle for editing, but **MUST** preserve the extension's `preserved_chunk_roots` across reads and writes. The chunks are treated as opaque bytes.
- Edits **MUST** be checked against every active edit barrier. An edit matching a barrier's scope, affected object kinds, and operation kinds is prohibited unless the user explicitly performs an *unsafe edit*.
- An unsafe edit is an explicit user action that acknowledges loss of extension data. The unsafe-edit operation **MUST** tombstone the relevant extension chunks (so they are no longer preserved) rather than silently breaking extension invariants.

The unsafe-edit mechanism prevents unknown extensions from rendering a document permanently unedittable while still preserving extension data by default.

REQUIREMENT 8.25

Barrier matching for target-free and opaque operations (ratified Pass 12). An operation with no graph target (`SetMetadata`, `DeclareTransaction`) is matched by *score-wide* barriers only (`BarrierScope::WholeScore`); a barrier with any narrower scope cannot match it, because there is no target to test the scope against. An opaque `Registered` operation is matched *fully conservatively*: since its targets and effects are unknown to a core implementation, it matches every active barrier whose remaining predicates do not exclude it.

REQUIREMENT 8.26

Unsafe-edit tombstone semantics (ratified Pass 12). Crossing a barrier by an unsafe edit immediately deactivates the owning extension's *remaining* barriers for the editing session — the extension's invariants are already forfeit, so its other barriers no longer protect anything. The crossing **MUST** be durably recorded at the next bundle commit: the pending tombstone set survives the session that performed the unsafe edit. A tombstoned extension that was declared `required = true` leaves the bundle openable read-only by implementations that depended on that extension (its invariants can no longer be trusted); implementations that never understood it proceed under the unknown-

extension rules above. The manifest-side *byte encoding* of the tombstone record is deferred to the Binary Format companion (the manifest schema is frozen at major 0, so the record must ride an append-safe channel); see the companion's open questions.

8.13 Text Projection

The format admits a deterministic projection to a canonical s-expression text form. The text projection is normative.

The form itself is supplied by the *Text Projection* companion (version 0.1.0), which this section delegates to: the syntax, the canonical layout, the parse rules, and the round-trip conformance requirement are its. This section states *what* is projected; the companion states *how*.

REQUIREMENT 8.27

The text projection **MUST** preserve, deterministically and bidirectionally with the binary form:

- All operation envelopes (with their identities, stamps, causal contexts, transaction groupings, and payloads).
- All profile declarations.
- All extension declarations *in full* — identity, semantic version, required flag, affected object kinds, edit barriers, and preserved chunk roots — with opaque payloads in a canonical text encoding.
- Every **canonical blob**: a blob referenced by a canonical operation or by canonical reduced state is a canonical root of this document (Section 8.4), so the projection carries its media type and its uncompressed payload. Blobs referenced only by acceleration structures are non-canonical and are not projected.
- The active canonical base snapshot's identity, frontier, and reduction-algorithm version, *with its payload inline*. The core specification once permitted the payload to be "referenced externally"; that is withdrawn, because a compacted document's base is derivable from nothing else and an external reference would make the projection lossy (Text Projection companion, `req:textproj:base-snapshot-inline`).
- All canonical reduced state — preserved by *determining* it, never by carrying a second literal copy. Reduced state is a deterministic function of the operation set and the canonical base, so a projection carrying both would hold two sources of truth for one fact. See the Text Projection companion, requirement `req:textproj:reduced-state-derived`.

The text projection **MUST NOT** be required to preserve:

- Chunk offsets within the file.
- Compression algorithm choices.
- Cache chunks (operation indexes, layout caches, integrity indexes).
- Garbage bytes from prior commits.

- Superblock generation numbers, slot assignments, or CRCs.

Two binary bundles whose canonical document semantics are identical **MUST** project to identical text. Two text projections that parse to identical canonical document semantics **MUST** re-serialize to binary bundles whose canonical document semantics are identical (though their physical layout, chunking, and compression **MAY** differ).

8.13.1 Use Cases

The text projection serves three primary use cases:

Version control. Text projections diff and merge in line-based tools. Merge conflicts surface at the operation- envelope level, which is the meaningful level for collaborative editing.

Format inspection and debugging. A canonical text form enables direct inspection of bundle contents without binary tooling.

Long-term archival. Text formats survive transitions between binary specifications. The text projection's structure is stable across binary format major versions where the semantic content is preserved.

For archival to hold, the text must be self-contained. Where a document has been compacted onto a canonical base and its prior operations pruned, that base is derivable from nothing else in the document, so the companion carries the snapshot payload *inline* rather than by reference (`req:textproj:base-snapshot-inline`). A reference-only projection of a compacted document would not determine the document it claims to project.

8.14 Schema Versioning

Chunk payloads are encoded against a schema declared by each chunk's `schema_version` field. Schemas evolve under a semantic versioning discipline.

```
pub struct SchemaVersion {
    pub major: u16,
    pub minor: u16,
}
```

REQUIREMENT 8.28

Schema versioning rules:

- These rules govern *schema* versions — the manifest's `manifest_schema_version` and each chunk's `schema_version`. The container's `format_major` is a separate axis, and a change to it now means more than a change of wire layout: it declares a new container *epoch*, with guarantees that a reader may rely on and that no later commit can confer on a container created before them (Section 8.3.1). A container format major boundary is therefore not satisfied by re-encoding bytes; a document produced under an older epoch acquires the new epoch's guarantees only by being repacked into a newly created container.

- Major version changes are non-backward-compatible. Readers that do not support the chunk's major schema version **MUST** handle it according to the chunk's role:
  - *Canonical chunks* (operation envelope blocks, the canonical base snapshot, the manifest, and blobs referenced by canonical operations or reduced state) **MUST** be either parseable or the bundle **MUST** be opened in read-only preservation mode (or refused). A reader that cannot parse a canonical chunk cannot know the canonical document.
  - *Non-canonical chunks* (acceleration snapshots, operation index, layout caches, text projection, integrity index, extension-defined chunks declared as preservable opaque content) **MAY** be treated as opaque preserved content; the reader does not need to understand them to open the bundle, and writes preserve them verbatim.
- Unknown canonical operation payloads (operation kinds not in the reader's understood catalog) require the relevant extension support or the reader **MUST** open the bundle read-only without materializing the unknown operations. Materializing an unknown operation **MUST NOT** proceed silently: doing so would fork canonical state against any reader that does understand the operation.
- Minor version changes are backward-compatible. A reader supporting major version N **MUST** accept any minor version $N.m$ for $m \leq$ the reader's supported minor. Under the positional, frozen-layout wire form, a minor change **MUST** only *append discriminants to the append-safe vocabularies the Binary Format companion defines* (its open `Registered` vocabularies, plus ratified appends to `OperationKind`, `OperationKindTag`, `OperationPayload`, and the closed value-layer unions), which leaves the bytes of every prior value untouched; it **MUST NOT** add or reorder a struct field — adding a field, even an `Option`, occupies a new positional slot and shifts subsequent bytes, so it is a *major* change (Binary Format companion, WHAT "ADDITIVE" MEANS HERE).
- The manifest's `manifest_schema_version` declares the schema of the manifest itself. Each chunk's `schema_version` field declares the schema of that chunk's payload.
- Schema evolution is governed by the Binary Format companion specification, which defines the wire encoding for each schema version.

Schema major 1 is the first data-model expansion major: it adds `Canvas.layout_defaults`, `Instrument.range`, and `Region.permits_spanning_slurs` to the graph, and unifies the non-canonical resolved-layout length-prefix width. Its wire form and the byte-for-byte migration from major 0 are defined in the Binary Format companion. The canonical-base `MaterializedState` embeds none of these values, so it is byte-identical across the bump and stays major 0. `Region.permits_spannin` also reaches the *canonical* operation layer, because `CreateRegion` embeds a full `Region`, so an operation-envelope block bearing a v1 `CreateRegion` is major 1. (As of the genesis operation tranche, `Canvas.layout_defaults` and `Instrument.range` do too — `SetCanvasLayoutDefaults` (G2a) and `CreateInstrument` (G1) embed them — but neither payload is major 1, and for *different* reasons. `SetCanvasLayoutDefaults` carries the leaf `CanvasLayoutDefaults`, which has exactly one layout: the versioning sits in the containing `Canvas` walk, never in the leaf, so the payload stamps major 0. `CreateInstrument` carries a whole `Instrument`, which does have

distinct major-0, major-1, and major-2 layouts — so it is not major 1 because it is *always major 2*: the schema-major-2 appends are mandatory rather than `Option`-hidden, so no lower-major layout for that payload exists at any value. See the Binary Format companion SCHEMA MAJOR 1 / SCHEMA MAJOR 2 for the operation-by-operation accounting.) Consequently a major-0-only reader opens a major-1 bundle *fully* only when the bundle carries no v1 `CreateRegion` operation; otherwise it reads the major-0 canonical base and manifest but opens read-only (it cannot replay the v1 canonical operations), per the canonical/non-canonical rules above. It always discards the higher-major non-canonical caches. A major-1 reader migrates a major-0 acceleration snapshot and a major-0 `CreateRegion` payload on read (default-filling the new fields).

Schema major 2 is the second data-model expansion major: it fills the truncated bodies of the cross-cutting structures (`Slur`, `Tie`, `Beam`, `Spanner`), `RepeatStructure` (kind and voltas), `Staff` and `StaffLineConfiguration`, `Instrument`, and `ScoreMetadata`, per this revision's Chapter 5 definitions. The same per-payload-type rules apply: the canonical-base `MaterializedState` embeds none of these values and stays major 0; the filled values reach the acceleration snapshot *and*, through the nine operation payloads that embed them (`CreateCrossCutting`, `ModifyCrossCutting`, `InsertStaff` and `InsertStaffInstance` (the Operation Catalog's `CreateStaff` / `CreateStaffInstance`), `SetStaffLayout`, `SetMetadata`, `CreateRepeatStructure` (the repeat-authoring revision — it embeds the filled `RepeatStructure` unconditionally, so it is *born at v2* and every block carrying one stamps major 2; its `DeleteRepeatStructure` sibling carries a bare identifier, a major-0 layout, and stamps major 0), `CreateInstrument` (genesis tranche G1 — it embeds the filled `Instrument` unconditionally, so it too is *born at v2*), and — transitively, because an embedded `Region` may carry a `StaffInstance` whose `staff_lines_override` embeds the filled `StaffLineConfiguration` — `InsertRegion` (`CreateRegion`)), the canonical operation layer. That is nine: `CreateCrossCutting`, `ModifyCrossCutting`, `InsertStaff`, `InsertStaffInstance`, `SetStaffLayout`, `SetMetadata`, `CreateRepeatStruct` `CreateInstrument`, and `InsertRegion`. A block is stamped with the *lowest* schema major whose layouts decode its bytes (the companion's minimal-stamping rule — deterministic, so identical content hashes identically): any v2-dependent bytes make it major 2, and a lower-major-only reader opens such a bundle read-only. The wire form and the total default-filling v1→v2 migration are defined in the Binary Format companion.

8.15 Format Profiles

A format profile defines a conformance subset that implementations **MAY** promise to support. Profiles enable lightweight readers to declare their capabilities precisely.

```rust
pub struct ProfileDeclaration {
    pub profile_id: ProfileId,
    pub version: SemVer,

    /// Constraints imposed by this profile (maximum block size,
    /// permitted compression algorithms, required extensions, etc.)
    pub constraints: ProfileConstraints,
}

pub enum ProfileId {
    /// Full profile: all features supported, no restrictions.
    Full,
```

```
    /// Read-only profile: bundles produced under this profile MAY
    /// be opened by readers that cannot edit. Editing implementations
    /// SHOULD upgrade the profile on first edit.
    ReadOnly,

    /// Lite profile: a reduced feature set for embedded or mobile
    /// readers. Restricts maximum block sizes, snapshot complexity,
    /// and permitted extensions.
    Lite,

    /// Custom profile defined by a registered profile declaration.
    Custom(ProfileRegistryId),
}
```

The constraints a profile imposes are carried in `ProfileConstraints`, which the GC and retention rules (Section 8.11) reference for the mandatory `RetentionPolicy`:

```
pub struct ProfileConstraints {
    /// Maximum uncompressed operation-envelope block size a reader
    /// must accept under this profile (default 64 MiB).
    pub max_uncompressed_block_size: u64,

    /// The RetentionPolicy this profile declares. Required: the GC
    /// and retention rules read it from the active profile.
    pub retention_policy: RetentionPolicy,
    // Additional constraint fields (permitted compression sets,
    // required-extension lists) are reserved for a later revision;
    // a reader MUST tolerate their future addition under the
    // minor-version compatibility rule.
}
```

REQUIREMENT 8.29

Every `ProfileDeclaration` **MUST** carry a `ProfileConstraints`, and every `ProfileConstraints` **MUST** include a `retention_policy` field. This satisfies the requirement (Section 8.11) that the active conformance profile declare a `RetentionPolicy`: the policy lives inside the active profile's constraints, not as a free-standing manifest field.
When a bundle declares more than one profile, the `RetentionPolicy` in force is the one declared by the *first* profile in the manifest's `profile_declarations` list (declaration order is canonical). A bundle that declares no profile constraints carrying a policy uses the Full profile's default retention policy.

REQUIREMENT 8.30

Every bundle **MUST** declare at least one profile in its manifest's `profile_declarations`. An implementation **MAY** open a bundle if it supports any of the declared profiles, sub-

ject to the required-extension rules (Section 8.12).

REQUIREMENT 8.31

`ProfileId` encodes as a fixed 20 bytes: a little-endian `u32` declaration-order discriminant (`Full` = 0, `ReadOnly` = 1, `Lite` = 2, `Custom` = 3) followed by a 16-byte `ProfileRegistryId`. The trailing 16 bytes carry the registry id for `Custom` and are zero for the other three variants; the field is always present, so the encoding is fixed-width (the superblock reserves a fixed slot for it). This assignment is normative and stable. `ProfileId` participates in superblock selection (Section 8.3, the load-bearing field set), so its discriminants are part of that comparison.

8.16 Compression

Each chunk **MAY** be stored compressed. Compression is per-chunk metadata, not part of content identity.

REQUIREMENT 8.32

Conforming implementations **MUST** support reading chunks compressed with Zstandard at any level zstd defines. Writers **MAY** choose to compress or not on a per-chunk basis; uncompressed chunks (`CompressionAlgorithm::None`) are always permitted.
Implementations **MUST NOT** require a specific compression algorithm for canonical conformance. Future major format versions **MAY** introduce additional algorithms via the `Reserved` variant; such changes require a new format major version.
Decompression **MUST** verify the output length against the declared `uncompressed_length` and reject chunks whose decompressed size disagrees.

8.17 Streaming Reads

Cold open of a large score **MUST** be fast: under one second to first interactive frame on a 100-page orchestral score, per the performance requirements (Chapter 10). The format's prelude and chunk-graph design support this.

8.17.1 Required Locatability

REQUIREMENT 8.33

The bundle **MUST** guarantee that:

- The fixed header is at offset zero of the file.
- Both superblock slots are at fixed offsets within the first 576 bytes.
- The active manifest is locatable from the active superblock without scanning.
- Operation-envelope blocks, the canonical base snapshot (if any), acceleration snap-

shots, blobs, and extension chunks are locatable from the manifest's roots without scanning.

- The operation index (if present) provides $O(\log n)$ operation-id lookup; if absent, scanning a small number of blocks suffices.

Locatable means: a reader can determine the offset and length of the chunk in the bundle using only previously read content.

8.17.2 Cold Open Procedure

The recommended cold open sequence:

1. Read the 64-byte fixed header. Verify magic and CRC.
2. Read both 256-byte superblock slots. Verify CRC on each. Select the active slot per the superblock selection rule (Section 8.3).
3. Read and verify the manifest chunk referenced by the active superblock. The manifest is stored uncompressed (Section 8.4.3), so this is a single read-and-parse.
4. If the manifest's `canonical_base` is `Some`, read and verify the canonical base snapshot. The snapshot provides the materialized state at its declared causal frontier.
5. Stream the operation-envelope blocks listed in `operation_roots`. Apply the canonical reduction (Chapter 6) over the envelopes whose `OperationId` is not covered by the canonical base's `covers_causal_frontier` (if a base is present), or over every envelope in the operation set (if no base is present). The result is the materialized score graph.
6. Invoke the layout pipeline (Chapter 7) over the materialized graph, producing the first system's layout. Layout caches (if reachable from the manifest as accelerators) **MAY** be consulted to skip work; cache misses fall back to fresh layout computation.
7. Load remaining regions, envelope blocks, snapshot ranges, and layout caches lazily as the user navigates.

For unsynchronized first-frame display, implementations **MAY** present the canonical base's materialized state immediately while the post-frontier reduction completes in the background; user edits **MUST** be deferred until reduction completes, since the visible state is provisional until then.

8.17.3 Memory Mapping

REQUIREMENT 8.34

Implementations **MAY** memory-map the bundle file for efficient random access to chunks and blobs. The format's content-addressing and chunk immutability guarantees make memory mapping safe: chunks cannot change underneath a reader holding a pointer into them. Superblock slots are 256-byte aligned and never overlap reachable chunk content, so writers updating an inactive superblock cannot corrupt readers accessing the active state.

8.18 Binary Format Companion

The byte-level encoding (varint conventions, exact field bit widths, record alignment, endianness, string encoding details, struct layouts) is delivered as a companion specification, the Binary Format document. The Binary Format document is normative; an implementation cannot conform to the file format specification without conforming to the Binary Format specification.

8.18.1 Ratified Convention Baseline

Although the full Binary Format companion is still to be written, the canonical encoding conventions shared by the core, operations, and bundle layers are ratified now, so those three layers do not drift apart before the companion formalizes them. The companion *inherits* this baseline rather than re-deriving it. Every ratified discriminant table, derivation preimage, and primitive encoding is consolidated in Appendix E (the Canonical Byte-Layout Reference); the companion imports that appendix as its starting point rather than recovering the layouts from the three crates.

REQUIREMENT 8.35

The canonical encoding of every composite structure (whole `Score`, operation envelopes, manifest) **MUST** follow these conventions:

- Integers are little-endian. A boolean is a single `0`/`1` byte.
- Counts and the length prefix of every variable-width *leaf* (an identifier, a `RationalTime`, a string) are `u32` little-endian. Length-prefixing every leaf makes the decoder width-agnostic: it never infers a boundary from context.
- A tagged union is encoded as a single discriminant byte followed by the variant payload.
- Free-text string fields are length-prefixed UTF-8 and are *not* NFC-folded by the codec: the in-memory value is preserved byte-exact, so `decode(encode(x)) == x`. Catalog identifiers are NFC-normalized at construction, not in the codec.

Certain fixed hashing preimages deliberately depart from the single-discriminant-byte convention where a wider or order-bearing tag is required — notably `TypedObjectId`'s 16-bit big-endian discriminant (Requirement 5.2). Such departures are pinned at their definitions and are not overridden by this baseline.

REQUIREMENT 8.36

The primitive scalar layouts referenced throughout the canonical encodings are ratified as follows:

- `RationalTime` encodes as a sign byte (`0` = zero, `1` = positive, `2` = negative), then the `u32`-little-endian-length-prefixed big-endian magnitude of the numerator, then the `u32`-little-endian-length-prefixed big-endian magnitude of the denominator. The value **MUST** always be stored reduced (lowest terms, positive denominator), so the encoding is canonical: equal rationals encode to equal bytes.
- Wall-clock integers (`WallClockTime`, `WallClockDuration`) are little-endian fixed-

> width integers, matching `QuantizedCoord`.

OPEN QUESTION

The remainder of the Binary Format companion — the full composite struct layouts, schema-version wire evolution, and varint details beyond the baseline above — is under development as a separate specification. Its development follows this chapter and inherits the ratified baseline (Sections 8.35 and 8.36); the remaining byte-level decisions depend on the structural decisions specified here.

8.19 Forward References

- The constraint-solver interface (Chapter 9) operates on the in-memory `ConstrainedLayoutIR`; layout caches stored in the bundle are derivative and not part of the canonical state.
- The full performance requirements that this format must satisfy are in Chapter 10.
- The Binary Format companion specifies byte-level encoding.
- The Text Projection companion specifies the canonical s-expression form.
- The Profile Conformance companion specifies the feature lists per profile.

Chapter 9

Constraint-Solver Interface

This chapter specifies the constraint-solver interface: the `ConstrainedLayoutIR` the solver consumes, the `ResolvedLayoutIR` it produces, the constraints it must respect, the determinism contract it must satisfy, the diagnostic information it must emit, and the conformance discipline by which implementations are judged.

The chapter deliberately specifies the *interface* and its contracts rather than a particular algorithm. Implementations are free to choose among approaches (Cassowary, linear programming relaxations, custom heuristics, differentiable layout, learned methods) and to evolve their choices over time. Conformance is established by hard-constraint validity, deterministic behavior, diagnostic completeness, and performance and quality on a reference suite of test scores.

9.1 Design Principles

The interface is normative; global optimality is not. The solver's input/output contract, its constraint families, its diagnostic surface, and the discipline by which it is evaluated are normative. Global aesthetic optimality (proving no better layout exists) is neither decidable nor practical to verify for arbitrary scores; it is therefore replaced by reference-suite thresholds.

Hard constraints are validity; quality metrics are evaluation axes. Hard constraints are the conditions under which a layout is valid at all. Quality metrics are the axes along which valid layouts are evaluated and compared. The solver **MUST NOT** trade off a hard-constraint violation for a better quality score: a layout that violates a hard constraint is not a layout, regardless of its quality vector.

Determinism within an implementation; equivalence across implementations. A given implementation at a fixed version **MUST** produce identical output for identical input. Different conforming implementations **MAY** produce different layouts; they conform by satisfying hard constraints, meeting diagnostic contracts, behaving deterministically internally, and passing reference-suite thresholds. Cross-implementation byte-identity is explicitly not required.

Pareto frontiers as design target, not conformance obligation. Solvers **SHOULD** seek layouts on or near the Pareto frontier of the normative quality metrics. They are *not* required to prove they have done so universally; conformance is reference-suite-based.

Deterministic budgets, not wall-clock budgets. Wall-clock time **MAY** stop interactive work, but the returned layout's canonical form depends on deterministic counters (iterations, constraint evaluations, search nodes), not on scheduler luck.

Incremental solving as observational equivalence. Incremental re-solving **MUST** be observationally equivalent to full solving of the declared invalidation scope under the same profile and budget. Implementations are free to widen scopes conservatively but **MAY**NOT return layouts that diverge from the canonical scoped result.

Structured diagnostics, never silent corruption. Solver failures are first-class outputs with diagnostic information sufficient for the editor to surface meaningful feedback. The score graph is never invalidated by a solver failure; a solver error means "we could not produce a layout," not "the score is broken."

9.2 The Solver Interface

```rust
pub trait ConstraintSolver: Send + Sync {
    /// The solver's identifying tier. Used by readers/editors to
    /// decide whether this solver is sufficient for the active
    /// score's declared solver tier requirement.
    fn tier(&self) -> SolverTier;

    /// The solver's implementation version. Identical input plus
    /// identical version produces identical output (within-
    /// implementation determinism).
    fn version(&self) -> SolverVersion;

    /// Solve from scratch.
    fn solve(
        &self,
        input: &ConstrainedLayoutIR,
        config: &SolverConfig,
    ) -> SolveReport;

    /// Solve incrementally over the declared invalidation scope.
    fn solve_incremental(
        &self,
        input: &ConstrainedLayoutIR,
        prior: &SolverState,
        invalidations: &InvalidationSet,
        config: &SolverConfig,
    ) -> SolveReport;
}

pub struct SolverConfig {
    /// Conformance profile under which to solve. Selects metric
    /// thresholds, profile-specific constraints, and the active
    /// extension set.
    pub profile: SolverProfile,

    /// Deterministic budget. Wall-clock time is advisory only.
    pub budget: SolverBudget,
```

```
    /// Tie-breaking weights for selecting among layouts of
    /// equivalent quality.
    pub tie_breaking: TieBreakingWeights,
}

pub struct SolverBudget {
    /// Maximum solver iterations (algorithm-defined unit; the
    /// reference algorithm counts Cassowary pivots).
    pub max_iterations: u64,

    /// Maximum search-tree nodes explored, for algorithms that
    /// perform discrete search.
    pub max_nodes: u64,

    /// Maximum constraint-evaluation count.
    pub max_constraint_evaluations: u64,

    /// Advisory wall-clock cap, in milliseconds. The solver MAY
    /// stop early when this is exceeded, but the layout it
    /// returns under stoppage MUST be the same layout that the
    /// deterministic budget would have produced at the
    /// corresponding deterministic counter state. Wall-clock time
    /// MUST NOT change the canonical layout.
    pub advisory_wall_time_ms: Option<u64>,
}

pub struct SolverBudgetUsed {
    pub iterations: u64,
    pub nodes: u64,
    pub constraint_evaluations: u64,
    pub wall_time_ms: u64,
}
```

REQUIREMENT 9.1

`solve` and `solve_incremental` **MUST** be pure functions of their inputs within the determinism contract (Section 9.7). They **MUST NOT** consult external state (system clock, environment, thread scheduling, random sources) in ways that influence the returned layout.
Implementations **MAY** use multi-threaded execution internally, but the result **MUST** be identical to a single-threaded execution under the same deterministic budget.

9.3 The Solver Report

Every solver invocation returns a `SolveReport`. There is no separate "error" return; failures and partial successes appear as report variants distinguished by `SolveStatus`.

```
pub struct SolveReport {
    pub status: SolveStatus,
```

```rust
    /// Whether every hard constraint is satisfied.
    pub satisfied_hard_constraints: bool,

    /// The layout produced. Always present; under failure
    /// statuses, the layout is diagnostic-only and MUST NOT be
    /// used as if it were valid.
    pub layout: ResolvedLayoutIR,

    /// Updated solver state for subsequent incremental calls.
    pub state: SolverState,

    /// Unsatisfied hard constraints, if any. Empty under
    /// SolveStatus::Solved.
    pub unsatisfied_constraints: Vec<ConstraintId>,

    /// Non-fatal warnings about the solution.
    pub warnings: Vec<SolverWarning>,

    /// Quality metric vector for the returned layout.
    pub metric_vector: QualityMetricVector,

    /// Budget consumed during this solve.
    pub budget_used: SolverBudgetUsed,
}

pub enum SolveStatus {
    /// All hard constraints satisfied, target quality reached.
    Solved,

    /// All hard constraints satisfied, but warnings were
    /// generated (e.g., a soft constraint had a large violation,
    /// or a layout decision was unusual and worth surfacing).
    SolvedWithWarnings,

    /// Deterministic budget exhausted before reaching target
    /// quality. The returned layout MUST still satisfy all hard
    /// constraints; it simply may not be optimal. If hard
    /// constraints could not be satisfied within budget, the
    /// status is Unsatisfiable, not PartialBudgetExhausted.
    PartialBudgetExhausted,

    /// Hard constraints cannot be simultaneously satisfied. The
    /// returned layout is diagnostic-only and MUST NOT be
    /// presented as a valid layout.
    Unsatisfiable,

    /// Solver bug or unexpected error. The returned layout is
    /// diagnostic-only. Editors SHOULD surface this as a defect
    /// to report.
    InternalError,
}
```

```rust
pub struct SolverWarning {
    pub kind: SolverWarningKind,
    pub affected_objects: Vec<TypedObjectId>,
    pub message: String,
}

pub enum SolverWarningKind {
    LargeSoftConstraintViolation { constraint: ConstraintId, magnitude: f64 },
    UnusualLayoutDecision(String),
    QualityFloorApproached { metric: QualityMetricKind },
    ExtensionWarning(ExtensionWarningId),
}
```

REQUIREMENT 9.2

The `SolveReport.layout` field is always populated, but its authority depends on `status`:

- `Solved` or `SolvedWithWarnings`: the layout is a valid layout that the editor may render.
- `PartialBudgetExhausted`: the layout is a valid layout (`satisfied_hard_constraints` is `true`) and the editor may render it, while noting that quality is below the profile's target.
- `Unsatisfiable` or `InternalError`: the layout is diagnostic-only. Editors **MAY** render it for the user to see what failed, but **MUST** clearly indicate the layout is not authoritative.

A solver **MUST NOT** return `PartialBudgetExhausted` with `satisfied_hard_constraints == false`. If hard constraints cannot be satisfied within the budget, the correct status is `Unsatisfiable`, with the partial layout marked as such.

REQUIREMENT 9.3

Sub-conformant report shape (ratified Pass 12). A below-conformance passthrough solver (`SolverTier::Stub`) presented with declared constraints it does not evaluate reports `SolvedWithWarnings` with `satisfied_hard_constraints == false` and a warning naming the unevaluated-constraint condition. The layout stays renderable; the report makes no conformance claim and satisfies no minimum-tier requirement. This is the one sanctioned case in which a renderable status carries `satisfied_hard_constraints == false`: the field reports *evaluated* satisfaction, and a non-evaluating solver has nothing to claim.

9.4 Constraint Families

The solver consumes constraints in normalized form. The constraint families enumerated here are the normative core; implementations **MAY** support additional families via `LayoutConstraint::Registered`

9.4.1 Strength Levels

```
pub enum ConstraintStrength {
    /// Hard constraint. The solver MUST satisfy this constraint,
    /// or return Unsatisfiable.
    Required,

    /// Soft constraint with an associated weight. The solver
    /// minimizes the weighted violation when optimizing.
    Preferred { weight: f64 },
}
```

REQUIREMENT 9.4

A solver **MUST NOT** treat a `Required` constraint as if it were `Preferred` for any reason, including for quality optimization. A layout that violates a `Required` constraint is not a valid layout, regardless of its quality vector.

REQUIREMENT 9.5

Strength is kind-determined (ratified Pass 12). A constraint instance carries no strength field; its strength is determined by its kind. Break constraints take their strength from `BreakKind`: `Hard` → `Required`, `Soft` → `Preferred` with weight 1.0. Every other core constraint family normalizes to `Required`. `Registered` constraint kinds normalize conservatively to `Required`. A future constraint family declares its strength (and weight, if `Preferred`) in its normative definition — the strength channel is the *kind*, not the instance.

9.4.2 Normative Constraint Families

The constraint catalog (spring, collision, alignment, containment, break, aesthetic, extension) is specified in Chapter 7. Implementations **MUST** support every family normatively required by the active solver tier (Section 9.6).

9.5 Quality Metrics

The solver's output is evaluated against a normative set of quality metrics. Each metric is normalized to the closed interval $[0.0, 1.0]$, with 0.0 representing the best achievable value and 1.0 representing the worst tolerable value before a profile threshold flags the metric as failing.

```
/// A quality metric value normalized to [0.0, 1.0].
/// Lower is always better.
pub struct NormalizedMetric(pub f64);

impl NormalizedMetric {
    /// Construct, panicking if the value is not finite or is out
    /// of range. Conforming implementations MUST construct only
    /// valid NormalizedMetric values.
```

```rust
    pub fn new(value: f64) -> Self {
        assert!(value.is_finite());
        assert!(0.0 <= value && value <= 1.0);
        NormalizedMetric(value)
    }
}

pub struct QualityMetricVector {
    pub collision_penalty: NormalizedMetric,
    pub spacing_distortion: NormalizedMetric,
    pub slur_shape_penalty: NormalizedMetric,
    pub beam_slope_penalty: NormalizedMetric,
    pub vertical_density_penalty: NormalizedMetric,
    pub system_break_penalty: NormalizedMetric,
    pub page_fill_efficiency: NormalizedMetric,
    pub casting_off_quality: NormalizedMetric,
    pub symbol_density_uniformity: NormalizedMetric,

    /// Additional metrics contributed by extensions. Each MUST be
    /// a valid NormalizedMetric (finite, in [0.0, 1.0], lower is
    /// better).
    pub extension_metrics: Vec<ExtensionMetric>,
}

pub struct ExtensionMetric {
    pub metric_id: ExtensionMetricId,
    pub value: NormalizedMetric,
}
```

REQUIREMENT 9.6

Every `NormalizedMetric` value **MUST** satisfy:

- It is finite (not NaN, not infinity).
- It lies in the closed interval $[0.0, 1.0]$.
- Lower values denote better layouts; higher values denote worse layouts.

Per-metric normalization functions (mapping raw measurements to $[0.0, 1.0]$) are specified in the Quality Metric Catalog companion document. Implementations **MUST** use the catalog's normalization; arbitrary normalization is non-conforming.
Extension metrics **MAY** define their own normalization function but **MUST NOT** change orientation or range: 0.0 remains best, 1.0 remains worst, and values must remain finite within range.

9.5.1 Pareto Frontiers as Design Target

Solvers should seek layouts on or near the Pareto frontier of the normative metrics. This is the design intent. It is not the conformance requirement.

REQUIREMENT 9.7

The normative metric set defines the axes along which layout quality is evaluated. Solvers **SHOULD** produce Pareto-efficient layouts with respect to these metrics: layouts for which no metric can be improved without worsening another. Because global Pareto optimality is not practically verifiable for arbitrary scores, conformance is determined by reference-suite thresholds (Section 9.9) rather than by proof of Pareto-optimality. When multiple Pareto-equivalent layouts exist, the configured `TieBreakingWeights` select among them. Tie-breaking is deterministic.

```
pub struct TieBreakingWeights {
    pub collision: f64,
    pub spacing: f64,
    pub slur_shape: f64,
    pub beam_slope: f64,
    pub vertical_density: f64,
    pub system_break: f64,
    pub page_fill: f64,
    pub casting_off: f64,
    pub symbol_density: f64,
}
```

REQUIREMENT 9.8

Tie-breaking weights **MUST** have normative defaults specified in the Quality Metric Catalog. Implementations **MAY** permit users to customize these weights; the defaults represent the reference engraving aesthetic and are the basis for reference-suite conformance.

9.6 Conformance Tiers

Solver conformance is graded into tiers, each declaring a subset of features the implementation supports. Tiers are *orthogonal* to file-format profiles (Chapter 8); a document may declare both an authoring file profile and a minimum solver tier, and the two axes are independent.

```
pub enum SolverTier {
    /// Not a conformance tier: an interface-only / passthrough
    /// solver that evaluates no constraints and computes no
    /// quality metrics. Ordered below every conformant tier.
    Stub,

    /// Minimal Layout Solver.
    Minimal,

    /// Standard Engraving Solver.
    Standard,

    /// Advanced / Extension-Aware Solver.
    Advanced,
```

```
}
```

The `Stub` variant exists so scaffolding, harnesses, and not-yet-conformant pipelines can label themselves honestly: it makes no conformance claim, and a document requiring any conformant tier is never satisfied by a solver declaring `Stub`.

9.6.1 Minimal Layout Solver

REQUIREMENT 9.9

A solver claiming the Minimal tier **MUST**:

- Satisfy every hard constraint or return `Unsatisfiable` (validity conformance).
- Be deterministic within its declared implementation version (Section 9.7).
- Produce well-formed `SolveReport` values with accurate `satisfied_hard_constraints`, `unsatisfied_constraints`, and metric vectors (diagnostic conformance).
- Support common-music-notation regions, metric time, and the standard constraint families.
- Pass the Minimal subset of the reference suite.

A Minimal solver **MAY** produce mediocre aesthetic quality; metric thresholds for the Minimal tier are relaxed relative to the Standard tier.

9.6.2 Standard Engraving Solver

REQUIREMENT 9.10

A solver claiming the Standard tier **MUST**:

- Satisfy every Minimal-tier obligation.
- Meet the Standard-tier metric thresholds on the reference suite for every quality metric.
- Support the incremental solving contract with at least Measure-local invalidation scope (Section 9.8).
- Support every Standard-tier constraint family declared in the Quality Metric Catalog.

The Standard tier corresponds to "professional engraving quality" for common-practice notation.

9.6.3 Advanced / Extension-Aware Solver

Requirement 9.11

A solver claiming the Advanced tier **MUST**:

- Satisfy every Standard-tier obligation.
- Support proportional and aleatoric region layout, not only metric regions.
- Support microtonal accidentals and their interactions with adjacent glyphs.
- Support extension-contributed constraint families and extension metric thresholds for each registered extension whose layout requirements are part of the Advanced reference suite.
- Support at least System-local invalidation scope under incremental solving.

9.6.4 Tier Declaration and Document Compatibility

Requirement 9.12

Documents **MAY** declare a minimum solver tier required for faithful engraving in their profile declarations (Chapter 8). A solver with tier lower than the document's declared minimum **MAY** open the document but **SHOULD** surface a clear warning that engraving quality may not match the document's intent.
Solver tier is independent of file-format profile. A document authored under the `Lite` file profile **MAY** declare a minimum solver tier of `Standard`; a document authored under the `Full` file profile **MAY** declare no minimum tier and be displayable by a `Minimal` solver.

9.7 Determinism

The solver's determinism contract has two parts. *Within- implementation determinism* is a strict byte-equality obligation; *cross-implementation conformance* is reference-suite-based.

9.7.1 Within-Implementation Determinism

Requirement 9.13

A solver implementation at a fixed version **MUST** produce byte-for-byte identical `SolveReport` output for identical inputs, where "identical inputs" means:

- Identical `ConstrainedLayoutIR`.
- Identical `SolverConfig` (profile, budget, tie-breaking weights).
- Identical font and glyph metrics (referenced by version and content hash).
- Identical reduction algorithm version (Chapter 6).
- For incremental calls: identical prior `SolverState` and identical invalidation set.
- Identical solver implementation version (i.e., the same binary).

Determinism **MUST** hold across runs on the same machine, across cold and warm starts, and across hardware threads. Implementations **MUST NOT** use fast-math flags, processor- dependent SIMD without canonical-output guarantees, or any compiler options that vary floating-point results across builds on the same platform.
Where parallelism is used internally, the solver **MUST** serialize the merge step so that thread scheduling does not affect output. Deterministic budgets **MUST** be enforced from shared counters that capture all parallel contributions.

9.7.2 Cross-Implementation Conformance

Requirement 9.14

Cross-implementation determinism (byte-equality of outputs across different conforming solvers) is *not* required. Different conforming solvers **MAY** produce different layouts for the same input. Conformance across implementations is established by:

1. All hard constraints satisfied on every reference suite score.
2. The implementation is internally deterministic per the within-implementation rule above.
3. `SolveReport` values are well-formed and diagnostically accurate.
4. For each reference-suite score, every quality metric is within the tier's declared threshold.

Rationale

Cross-implementation byte equality would freeze the ecosystem to whatever single algorithm first claimed conformance, and would preclude algorithmic innovation. Reference-suite thresholds give users a meaningful quality floor while permitting genuine implementation diversity. Within- implementation determinism remains essential for reproducible builds, regression testing, and predictable behavior under collaborative editing.

9.8 Incremental Solving

9.8.1 Invalidation Scopes

```
pub enum InvalidationScope {
    /// A single object's layout depends only on local context.
    ObjectLocal,
    /// Effects bounded within a single measure.
    MeasureLocal,
    /// Effects bounded within a single system.
    SystemLocal,
    /// Effects bounded within a single page.
    PageLocal,
    /// Effects bounded within a single region.
    RegionLocal,
```

```
    /// Effects may propagate across the entire score.
    WholeScore,
}

pub struct InvalidationSet {
    /// Declared scope. The solver MAY widen this scope
    /// conservatively; it MUST NOT narrow it.
    pub scope: InvalidationScope,

    /// Specific invalidated entries within the declared scope.
    pub slots: Vec<SpringSlotId>,
    pub bands: Vec<VerticalBandId>,
    pub constraints: Vec<ConstraintId>,
    pub glyphs: Vec<GlyphObjectId>,
}
```

9.8.2 Observational Equivalence

REQUIREMENT 9.15

For any invalidation set with declared scope S, `solve_incremental` **MUST** produce a layout observationally equivalent to `solve` called on the same input restricted to S, under the same profile, the same configuration, and the same deterministic budget. "Observational equivalence" means byte-identical `ResolvedLayoutIR` output for objects within S, and unchanged output for objects outside S.
The solver **MAY** widen the scope conservatively: an `ObjectLocal` invalidation may be solved as `MeasureLocal` if that simplifies the implementation, provided the widened result is also observationally equivalent to full solving at the widened scope.
The solver **MAY**NOT narrow the scope. If `SystemLocal` effects might propagate (e.g., an edit at the system's end shifts subsequent systems' contents), the invalidation **MUST** be declared at least `SystemLocal` or higher.

9.8.3 Propagation Documentation

REQUIREMENT 9.16

Implementations **MUST** document, per their published conformance contract, the maximum invalidation scope they can service for each constraint family. The documented bound is the implementation's contract for how far edit effects propagate before requiring whole-score re-solving.

9.9 Conformance: The Reference Suite

Solver conformance is established by performance on a normative reference suite of test scores. Each tier has its own subset of the suite and its own metric thresholds.

Requirement 9.17

The Reference Suite is delivered as a companion specification. Each suite entry consists of:

- A test score in canonical `.musc` form.
- Per-tier inclusion: whether the entry is required for Minimal, Standard, or Advanced conformance.
- Per-tier metric thresholds: the maximum permitted `NormalizedMetric` value for each quality metric.
- Optional fixed-expectation tests: specific layout properties (a particular bar's width, a specific system break location) that all conforming solvers **MUST** reproduce. Used sparingly, only where the suite intends to pin an unambiguous expectation.

A solver claiming a given tier **MUST** pass every entry required at that tier. Failure on any single entry is conformance failure at the claimed tier.
Suite versions are tied to specification versions; conforming implementations **MUST** declare which suite version they pass.

Rationale

The reference suite makes conformance checkable. The earlier universal-Pareto formulation quantified over all possible layouts of every possible score, which is undecidable. Reference-suite thresholds replace "no better layout exists" with "this layout is within tolerance on these specific scores," which is testable, falsifiable, and operationally useful. The cost is that conformance is only as good as the suite's coverage; that is acceptable, because suite coverage is visible, evolvable, and can be expanded as the spec matures.

9.10 Reference Algorithm

The reference algorithm is explanatory and diagnostic. Its purpose is to disambiguate constraint interactions where prose alone might leave a choice open, and to generate the expected outputs published with the reference suite. It is *not* the normative algorithm and *not* the aesthetic floor.

Requirement 9.18

Conforming implementations are *not* required to use the reference algorithm or to reproduce its layouts, except where a reference-suite entry explicitly fixes an expected behavior via a fixed-expectation test. Implementations **MAY** choose any algorithm satisfying the contract: hard constraints, determinism, diagnostics, reference-suite thresholds.

The reference algorithm is described in a non-normative companion document. A natural starting point uses the Cassowary linear- arithmetic constraint solver for the linear layer (positions, alignments, containment) together with a separate numerical optimization layer for aesthetic constraints (beam slopes, slur curvatures) that are nonlinear in nature. Hybrid ap-

proaches and alternative architectures (differentiable layout, constraint programming, learned methods) are all conforming choices.

9.11 Forward References

- End-to-end performance requirements, including frame budgets that the solver participates in, are in Chapter 10. Solver performance is measured in the context of the full layout pipeline.
- The Quality Metric Catalog companion document specifies the exact normalization functions for each metric, the default tie-breaking weights, and the per-tier thresholds.
- The Reference Suite companion document specifies the test scores, per-tier inclusion, and per-tier metric thresholds.
- The reference-algorithm companion document is non-normative and describes one starting-point implementation.

Chapter 10

Performance Requirements

This chapter specifies performance contracts for the *core layer*. Targets are normative; they bound what conforming implementations promise and what tests measure for conformance. The chapter restates targets introduced informally in earlier chapters and adds the measurement methodology.

10.1 Core/Product Boundary

Chapter 1 excluded the editor UI, the audio engine, the renderer beyond the layout IR, and platform integrations from the core's scope. Many performance properties of a working notation system depend on those out-of-scope components: first interactive frame after launch involves UI bootstrapping, edit response involves UI event handling, scrolling and selection involve a real renderer, audio acquisition involves a working audio engine. The core cannot promise end-to-end timings for behavior it does not own.

This chapter therefore distinguishes three obligation classes:

Core obligations. The core data structures, layout IR, solver, and file format **MUST** admit the stated bounds. The core's own work (reduction, layout, file writes, snapshot acquisition) **MUST** fit within the budgets allocated to it in the end-to-end frame budget. The core **MUST** expose APIs suitable for each downstream component to do its own work inside its remaining budget (e.g., a lock-free immutable snapshot API for audio engines, an incremental invalidation interface for renderers, a cancellable solve API for UIs).

Product obligations. End-to-end UI, audio, and rendering targets (first interactive frame after launch, end-to-end edit-to-pixel latency, end-to-end scroll smoothness, audio-thread snapshot acquisition latency under load) are the obligation of the product layers built atop the core. Those targets are specified in product-specific companion documents, not here.

Performance Reference Suite obligations. The Performance Reference Suite (Appendix A) measures core behavior only: layout time per system, file write/read time, reduction throughput, solver budget consumption, snapshot acquisition latency. End-to-end UI traces are out of scope for the core's reference suite.

Requirement 10.1

Where this chapter states a performance target, the target is a *core obligation* unless explicitly labeled product obligation. Core obligations **MUST** be satisfiable by a conforming core implementation independent of the product layers built atop it; product-layer code is permitted to consume budget on top of the core's portion, but the core's own portion **MUST** fit within the stated bound.

10.2 Design Principles

Targets are part of the contract. Core performance requirements are not aspirations. A conforming core implementation **MUST** meet the stated targets on the reference hardware profile under the stated measurement conditions.

Frame budgets dominate. User experience is ultimately governed by frame timing. The core's portion of each frame budget is stated explicitly; the product layer's portion is specified separately.

Reference hardware profile. All targets are stated against a specified hardware profile. Implementations **MAY** document additional profiles (lower-end mobile, higher-end workstation); the reference profile is the basis of conformance.

Measurement methodology is normative. Without specified measurement methodology, performance targets are unenforceable. This chapter specifies how each target is measured.

Tail latency matters. Average performance is insufficient; targets are stated as percentiles (typically p99 or worst-case) to reflect what users actually experience.

10.3 Reference Hardware Profile

The reference hardware profile defines the platform against which performance conformance is measured. Implementations **MAY** target additional profiles; this profile is the basis of conformance.

```
Reference Hardware Profile (2026 baseline):

  Desktop reference:
    - Apple silicon (M-series) or x86-64 equivalent
    - 8 performance cores (or equivalent throughput)
    - 16 GB RAM
    - NVMe SSD storage
    - Discrete or integrated GPU with Metal/Vulkan support
    - Display: 1440p at 60 Hz minimum

  Tablet reference:
    - iPad Pro M-series or equivalent
    - 8 GB RAM minimum
    - SSD storage
    - Display: 120 Hz Pro Motion or equivalent
```

Requirement 10.2

Conforming implementations **MUST** meet the targets in this chapter on the reference hardware profile. Implementations **MAY** miss targets on hardware below the reference profile (e.g., older devices, low-end embedded targets); such gaps **MUST** be documented in the implementation's published performance contract.
The reference profile is updated periodically as commodity hardware evolves. The version of the reference profile against which an implementation conforms **MUST** be declared.

10.4 Frame Budgets

10.4.1 Interactive Edit Latency

Requirement 10.3

The *core's portion* of a single-system edit (operation envelope construction, reduction, incremental layout through `ResolvedLayoutIR`) **MUST** complete within one frame (16.7 ms at 60 Hz) at the p99 percentile, measured on the reference hardware profile. This includes the case where the affected system is on a 100-page orchestral score: the core's incremental layout machinery (Chapter 7) and the constraint solver's incremental contract (Chapter 9) jointly satisfy this requirement.
On 120 Hz displays (tablet reference), the core's portion **SHOULD** complete within 8.3 ms at p99. This is not required for conformance but is the target for first-class tablet performance.
End-to-end edit-to-pixel latency (input handling, hit testing, render submission, display flip) is a product-layer obligation. The core's contribution is bounded by this target so that the product layer has the remaining frame budget to do its work.

10.4.2 Multi-System Edits

Requirement 10.4

Edits affecting up to ten systems (e.g., a transposition over a substantial passage) **MUST** complete within four frames (66.7 ms at 60 Hz) at p99. The four-frame budget gives the editor headroom to display a transient "thinking" indication without violating perception of responsiveness.

10.4.3 Full Re-Layout

Requirement 10.5

Full re-layout of a 100-page orchestral score (triggered, for example, by a tuning system change applied score-wide) **SHOULD** complete within two seconds at p99. This is a soft target; the hard target on full re-layout is that the operation be cancellable: a user beginning a different edit **MUST** cause the full re-layout to abort and the new edit to proceed.

10.5 Cold Open

Requirement 10.6

The *core's portion* of cold open of a 100-page orchestral score from a `.musc` bundle **MUST** reach *first-system-ready* state within one second on the reference hardware profile.
First-system-ready is defined as: the prelude has been read, the active superblock and manifest validated, the canonical base snapshot (if any) acquired, post-frontier envelopes reduced for at least the first system's coverage, and the first system's `ResolvedLayoutIR` produced. Rendering, hit-testing, and UI bootstrapping that build atop this state are product-layer obligations.
Background loading of remaining systems **MAY** continue after first-system-ready and **MUST NOT** block user interaction served from already-loaded state. not-yet-visible systems is permitted; full-score loading need not complete in one second.

10.5.1 Cold Open Measurement

Cold open is measured from process start (or, for warm processes opening a new score, from the open API invocation) to the first frame containing fully rendered content for the user's initial viewport.

10.5.2 Streaming Read Discipline

Requirement 10.7

The cold open target **MUST** be achieved through streaming reads per Chapter 8: the implementation **MUST NOT** load the full score into memory before displaying the first system. Implementations failing to meet the target by eager loading are non-conforming.

10.6 Memory Budgets

Requirement 10.8

Steady-state resident memory for a 100-page orchestral score **SHOULD** remain below 500 MB on the reference hardware profile, exclusive of evictable caches. Evictable caches **MAY** push total resident memory higher; they **MUST** be evicted under memory pressure without compromising correctness.

10.6.1 Memory Categories

For measurement, memory is partitioned into:

Core data. The score graph, attachments, indexes, and operation envelope set. Cannot be evicted without losing state.

Layout cache. The cached IR stages. Evictable; rebuilds on demand but at performance cost.

Glyph and font caches. Glyph render data and metric caches. Evictable; small-to-moderate size.

Audio buffers and reference tracks. Audio data loaded for playback or reference. Evictable.

Implementation overhead. Runtime, allocator metadata, OS-level mappings.

Requirement 10.9

Implementations **MUST** expose the core-data memory consumption separately from evictable caches. Reporting tools **MUST** be able to distinguish them. The 500 MB target applies to core data only.

10.6.2 Per-Object Memory Discipline

Requirement 10.10

Implementations **MUST NOT** embed glyph metrics in IR objects (Chapter 7). Implementations **MUST NOT** embed spelling-attachment payloads on pitch objects (Chapter 2); attachments are externally indexed. These discipline requirements bound per-object memory at known upper limits.

10.7 Core Data Structures for Audio Thread Use

The audio engine is outside the core specification. The core's obligation is to expose data structures that permit real-time audio safety; audio-engine performance is the obligation of the product audio layer (Section 10.1).

Requirement 10.11

The core's read-only data structures consumed by an audio thread (pitch resolution, tempo map lookup, event arena queries) **MUST** be designed for allocation-free, lock-free access. Specifically:

- Pitch frequency resolution given a tuning system **MUST** be performable without allocation.
- Tempo-map conversion (musical to wall-clock and vice versa) **MUST** be performable without allocation.
- Time-indexed event lookup for playback **MUST** be performable without allocation.

These are properties of the core's data layouts, not of any particular audio engine. A product-layer audio engine that consumes them is responsible for its own real-time discipline (no syscalls on the audio thread, no priority inversion, no unbounded loops); the core only promises that its data structures do not preclude that discipline.

10.7.1 Snapshot API

Requirement 10.12

The core **MUST** provide an immutable-snapshot API: the ability for any thread to acquire a read-only view of the score graph at a consistent point in time, suitable for use by an audio engine. Snapshot acquisition by the snapshot reader **MUST** be lock-free in the sense that it never blocks on edit-thread progress: an edit in flight **MUST NOT** prevent the snapshot reader from obtaining a valid (possibly older) snapshot.
Snapshot *release* **MAY** be deferred to a non-audio thread. The snapshot mechanism is implementation-defined (double-buffered immutable copies, persistent data structures, reference-counted immutable trees, hazard pointers); only the externally observable lock-free acquisition property is normative here.
End-to-end audio-thread acquisition latency under load is a product-layer obligation; the core's contribution is bounded by the lock-free guarantee.

10.8 Constraint-Solver Performance Targets

Requirement 10.13

The constraint solver (Chapter 9) **MUST** satisfy:

- Incremental solving of a single-system edit completes within 10 ms at p99 on the reference hardware profile. (This is a tighter target than the overall frame budget; the remaining 6.7 ms accommodates the rest of the pipeline.)
- Cold solving of a 100-page orchestral score completes within two seconds at p99.
- Incremental solving propagation is bounded as documented in the implementation's published contract.

10.9 File Format Performance

Requirement 10.14

File format operations **MUST** satisfy:

- Bundle write of a typical edit (one or more operation envelopes appended to the operation-envelope block stream, manifest rewrite, superblock flip) completes within 50 ms at p99 on the reference hardware profile.
- Bundle read of the manifest and bootstrap chunks (sufficient for first interactive frame) completes within 200 ms at p99 for a 100-page orchestral score.
- Operation-envelope reduction rate **MUST** exceed 10,000 envelopes per second on the reference hardware profile, measured during cold reduction from a fresh canonical base.

10.10 Measurement Methodology

Requirement 10.15

Performance conformance **MUST** be measured using:

- A reference test corpus including the conformance suite from Chapter 9, augmented with edit traces representative of typical authoring sessions.
- Measurements at the p99 percentile across at least 1000 iterations per scenario.
- Warm runs after a cold warm-up phase, unless the target is explicitly cold (cold open, cold load).
- Release builds with full optimization enabled, but without fast-math or precision-compromising flags.
- The reference hardware profile, with no other significant load on the system during measurement.

Implementations **MUST** publish their measurement results against this methodology for conformance claims.

10.11 Regression Testing

Requirement 10.16

Conforming implementations **SHOULD** maintain continuous performance regression testing against the reference corpus, with alerts for regressions exceeding 10% on any target. The regression suite **SHOULD** be a precondition for releases.

10.12 Forward References

- The full performance reference corpus is delivered as a companion specification, the Performance Reference Suite.
- Audio engine performance targets (latency, jitter, stream discipline) are specified in the audio engine specification, outside this document.
- Network and synchronization performance for the collaboration layer is specified in the collaboration specification, outside this document.

Chapter 11

Extension Points

This chapter consolidates the extension points specified across prior chapters into a single reference. Each extension point admits implementation-defined or plugin-contributed content that the core treats opaquely, subject to the contracts stated here.

The chapter is a navigation aid rather than a source of new requirements: every extension point referenced here is defined in its originating chapter, and the originating definition is authoritative. This chapter pulls the points together for plugin authors and future specification editors.

11.1 Design Principles

Extensibility without forking. Every extension point permits content to be added without modifying the core specification or its implementations. Plugins extend; they do not fork.

Identifiers, not strings. Extension points are identified by typed registry identifiers, not free-form strings. Registries are explicit, versioned, and discoverable.

Contracts, not contents. Extension points specify what an extension *must satisfy* (interfaces, invariants, reduction rules, performance bounds) rather than what an extension *must contain*.

Capability-bound at the runtime. Plugin runtime permissions (read score, write score, network, filesystem) are declared per extension and granted per user. This is enforced by the plugin runtime, which is outside this specification; the extension points are designed to admit such enforcement.

Forward compatibility. Extensions referenced in a stored score **MUST** be preserved across reads and writes even when the reading implementation does not understand them (per Chapter 8). Extensions thus survive distribution even where they cannot be exercised.

11.2 Notation Grammar Extensions

The notation grammar layer supports extension at several points:

`PositionStructure::Registered` (Chapter 4). Custom position structures for pitch spaces whose organization does not fit the chromatic, diatonic-over-chromatic, or JI lattice families. Used for maqam, gamelan, raga, and other non-Western grammars.

`IntervalAlgebra::Registered` (Chapter 4). Custom interval algebras for grammar-specific arithmetic.

`TranspositionBehavior::Registered` (Chapter 4). Custom transposition rules.

`SpellingAlgorithmId` (Chapter 2). Registered spelling-inference algorithms for the spelling pre-pass.

`NominalRegistry` (Chapter 4). Score-level or grammar-level extensions to the catalog of named position-letters.

`AccidentalRegistry` (Chapter 4). Score-level extensions to the accidental catalog, subject to the registry's `extensible` flag.

11.3 Tuning System Extensions

`TuningFunctionId` (Chapter 4). Registered procedural tuning functions (historical temperaments, custom meantone variants, generative tunings).

`AdaptiveTuningFunctionId` (Chapter 4). Registered adaptive tuning functions taking harmonic context as input.

`TuningFileFormat` (Chapter 4). Recognized external tuning file formats (Scala, MIDI Tuning Standard, additional formats via registry).

`CompatibilityMapping` (Chapter 4). Declared compatibility between pitch spaces and tuning systems whose underlying spaces differ.

11.4 Glyph and Font Extensions

`CustomGlyphId` (Chapter 4). Custom glyphs not in SMuFL, defined per score or per plugin.

`Composite` glyph references (Chapter 4). Composite glyphs constructed from multiple primary glyph references.

`ModificationRegistryId` (Chapter 4). Registered pitch-space modifications for grammar-defined accidentals.

11.5 Score Graph Extensions

`ExtensionOp` (Chapter 6). Implementation-defined operation kinds extending the core's semantic operations. Extension operations **MUST** satisfy the operation framework's requirements (preconditions, effects, inverse, reduction rule, re-anchoring).

`CustomMarkerId` (Chapter 5). Custom marker kinds beyond the core's standard catalog. Not yet defined: `MarkerKind` grows by appended variant, and schema major 5 defines none for a custom kind.

`CustomAnalyticalKindId` (Chapter 5). Custom analytical annotation kinds.

`Custom event variants` (Chapter 5). The event taxonomy is closed in the core, but the `Group` layout object and the `Registered` fields throughout permit composition of core types into extended structures.

11.6 Layout IR Extensions

`SynthesisRegistryId` (Chapter 7). Custom synthesis kinds for engraver-generated objects.

`ConstraintRegistryId` (Chapter 7). Registered layout constraint kinds extending the standard constraint families.

`MappingRegistryId` (Chapter 5). Custom parameter mappings for graphic-object time bindings.

`Custom view kinds` (Chapter 5). The `ViewKind::Custom` variant admits user-defined view kinds.

11.7 Constraint-Solver Extensions

`LayoutConstraint::Registered` (Chapter 7). Custom constraint families consumed by the solver. Extension constraints **MUST** declare their strength. When soft, extension constraints contribute to the quality vector through an extension metric subject to the `NormalizedMetric` discipline (Chapter 9) and to the tier-specific reference- suite thresholds.

11.8 File Format Extensions

`FormatProfile::Custom` (Chapter 8). Profile definitions beyond Full, ReadOnly, and Lite.

`Self-describing extension envelope` (Chapter 8). Plugin-contributed and forward-compatible content is carried in self-describing extension records that the core preserves opaquely.

11.9 Extension Registry Contract

Every extension registry shares a common structural contract.

```
pub trait ExtensionRegistry {
    type Id;
    type Definition;

    /// Resolve an identifier to its definition.
    fn resolve(&self, id: Self::Id) -> Option<&Self::Definition>;

    /// Enumerate all registered definitions.
    fn enumerate(&self) -> Vec<&Self::Definition>;

    /// Version of the registry. Score files declare which version
    /// their references target.
    fn version(&self) -> RegistryVersion;
}
```

Requirement 11.1

Every extension registry **MUST** be versioned. Stored references to registered definitions **MUST** include the registry version they target. Loading a score with references to registry versions newer than the reader understands **MUST** preserve those references per the forward-compatibility rules of Chapter 8.

11.9.1 Identifier Stability

Requirement 11.2

Registry identifiers **MUST** be stable across registry versions within a major version. Renumbering, reassignment, or repurposing of identifiers is forbidden within a major version. Removal of a definition **MUST** reserve its identifier; the identifier **MUST NOT** be reassigned to a different definition.

11.9.2 Discoverability

Registries **MUST** support enumeration: an implementation can list all known definitions and their identifiers. This enables tooling (plugin browsers, score migration assistants, conformance testers) to operate against unknown registries by inspection.

11.10 Plugin Runtime Considerations

The plugin runtime is outside this specification, but extension points are designed to admit a capability-bound runtime as described in the feature compendium:

- Extensions are isolatable: an extension's failure does not crash the host.
- Extensions are introspectable: their declared capabilities and inputs are inspectable before execution.
- Extensions are revocable: an extension can be disabled without invalidating the scores that reference it.

Requirement 11.3

Extension points **MUST** be designed such that disabling an extension does not invalidate stored content referencing it. The content remains in the score, opaque to the host, until the extension is re-enabled or the references are explicitly removed by the user.

11.11 Conformance and Extensions

Extensions do not affect core conformance directly, but they interact with it:

Requirement 11.4

A conforming implementation:

- **MUST** support every extension point as a structural slot (i.e., scores containing extension references are readable and writable).
- **MUST NOT** require any specific extension to be installed for core conformance.
- **MAY** require specific extensions for specific features (e.g., a particular plugin for a particular grammar).
- **MUST** clearly distinguish between core conformance and extension-dependent features in its documentation.

11.12 Forward References

- The plugin runtime (WebAssembly-based, capability-bound) is specified in the Plugin Runtime specification, outside this document.
- The complete extension registry catalog is delivered as a companion document.
- Implementation-defined extension contributions are governed by the implementation's published extension specification.

Chapter A

Intentionally Deferred Types and Specifications

This appendix catalogs types, concepts, and specifications that the core specification refers to but does not fully define. Each entry is *deliberately external*: it is the responsibility of a named companion specification or a future revision of this document. The purpose of this appendix is to distinguish deliberately external content from accidentally missing content, so that reviewers and implementers can quickly tell which unresolved references represent open work and which represent boundaries to other documents.

A.1 Companion Specifications

The following are companion specifications referenced throughout the core. None has been delivered as of this revision; each is named here with the chapter(s) that depend on it.

Binary Format companion. Byte-level encoding of all canonical chunks: varint conventions, exact field widths, record layouts, endianness rules, string encoding details. Referenced from Chapter 8. The on-disk representations specified in that chapter are structural; the Binary Format specifies how the bits are laid out.

Text Projection companion. The canonical s-expression text projection of the bundle: grammar, deterministic serialization rules, base64 encoding for opaque payloads, handling of extension data, round-trip semantics with binary form. Referenced from Chapter 8 Section 8.13.

Profile Conformance specification. The complete catalog of file-format profiles (Full, Read-Only, Lite, plus any normative custom profiles), with their constraints, permitted compression algorithms, maximum block sizes, required extensions, and conformance test discipline. Referenced from Chapter 8 Section 8.15.

Quality Metric Catalog. Per-metric normalization functions mapping raw measurements to `NormalizedMetric` values, default tie-breaking weights, per-tier metric thresholds, and the formal definition of each quality metric in the normative metric set. Referenced from Chapter 9.

Reference Suite. The collection of test scores against which solver conformance is established, with per-tier inclusion, per-tier metric thresholds, and any fixed-expectation tests. Versioned with this specification. Referenced from Chapter 9 Section 9.9.

Performance Reference Suite. The collection of edit traces and reference scores against which performance conformance is established. Referenced from Chapter 10.

Operation Catalog companion. The complete list of primitive operations and compound operations supported by the core, each with full preconditions, effects, inverse, and reduction rule. The representative operations in Chapter 6 illustrate the contract; the full catalog is delivered separately.

Reference Algorithm companion. A non-normative description of one starting-point implementation of the constraint solver. Explanatory and diagnostic only; conforming implementations are not required to use it. Referenced from Chapter 9 Section 9.10.

A.2 Externally Provided Components

The following are real components on which the core depends but which are owned by other specifications outside the Epiphany core scope.

SMuFL glyph catalog. The Standard Music Font Layout provides the normative glyph repertoire. Referenced from Chapter 4 and the layout chapters. SMuFL is maintained by the W3C Music Notation Community Group; the core spec declares its dependency on the SMuFL version targeted but does not redefine SMuFL.

Font metric provider. Implementations resolve glyph geometric data (bounding boxes, anchor points, advance widths) through a font metric provider. The provider's interface is part of the implementation contract; specific font metrics are external content.

Audio engine specification. Out of scope here per Chapter 1. The core exposes data structures designed for allocation-free, lock-free audio-thread access (Chapter 10); the audio engine's internals, latency obligations, and stream discipline are specified separately.

Plugin runtime specification. A capability-bound, sandboxed runtime hosts extension code. The runtime is out of scope here; the extension points (Chapter 11) are designed to admit such a runtime.

Collaboration transport. The wire protocol by which operation envelopes are exchanged between replicas is out of scope here. Chapter 6 specifies what must be exchanged and how it is reduced; the transport layer is separate.

User interface and editor. Editor commands, selection semantics, keyboard shortcuts, gesture grammar, and UI-level interaction patterns are out of scope. The core exposes a programmatic operation API; UI mapping is layered above.

A.3 Open Canonical Algorithms

The following algorithms affect canonical state but are not yet specified normatively, profile-declared, or marked non-canonical. Per Appendix D Section D.11, each **MUST** receive one of those dispositions before the document can claim full canonical score determinism across implementations.

Spelling pre-pass algorithm. The algorithm that computes default spellings from key signature, harmonic context, and notation grammar. Referenced from Chapter 2. Several established algorithms exist (Longuet-Higgins, Temperley pitch-spelling preference rules,

and others); the choice of normative algorithm or profile-declared algorithm set is open.

Notational decomposition algorithm. The algorithm that breaks a sounding duration into notehead values, augmentation dots, and ties under metric context. Referenced from Chapter 3. Conservatoire engraving manuals describe the conventions verbosely; the formalization is open.

Tempo curve integration. The numerical integration algorithm for converting musical position to wall-clock position under `TempoShape::Curve`. Referenced from Chapter 3. Possible candidates include Romberg integration and adaptive Gauss-Kronrod; the normative choice is open.

Wallclock-to-musical root-finding. The root-finding algorithm for the inverse of tempo curve integration. Referenced from Chapter 3. Possible candidates include bisection with monotonicity guarantees and Brent's method; the normative choice is open.

A.4 Extension Registry Catalogs

The extension registries enumerated in Chapter 11 (notation grammar extensions, tuning system extensions, glyph and font extensions, score graph extensions, layout IR extensions, solver extensions, file format extensions) are referenced by typed registry identifiers in the core. The concrete catalogs of registered extensions (the contents of each registry) are delivered as separate, versioned registry documents. Each registry document specifies its identifier numbering, the version under which each registered identifier was added, and the registered content's contract.

A.5 Support-Type Identity Semantics

Several identifier-like support types appear throughout the specification by name and shape, but their detailed encoding is delivered by companion specifications. Their semantic identity rules are normative here.

Type	Source	Semantic rule
`FileUuid`	128-bit random at bundle creation	Physical bundle identity. Fixed for the lifetime of the physical bundle; preserved by filesystem byte-copy; freshly minted on Save As, export-as-new-bundle, and derivative-work creation.
`DocumentId`	128-bit random at logical-work creation	Logical work identity. Stable across Save As copies intended to remain versions of the same work. May persist across forks at user discretion.
`LineageId`	128-bit random at fork-with-ancestry	Shared-ancestor identity. Optional. Multiple documents may share a `LineageId` declaring common ancestry for version-control or genealogy purposes.

Type	Source	Semantic rule
`ManifestId`	Content-derived from manifest payload	Unique per manifest version. Derived as `trunc128(BLAKE3("MUSCMNIF" || document_id || generation || manifest_body))` with the `manifest_id` field excluded from `manifest_body`; the derivation is ratified normatively in Section 8.8.
`SnapshotId`	Content-derived from snapshot payload and frontier	Stable for a snapshot's root and causal frontier. Two snapshots with identical materialized state at identical frontiers have identical ids. The Binary Format companion defines the exact derivation.
`BlobId`	BLAKE3 of blob content with domain tag `"MUSCBLOB"`	Content hash of the blob's uncompressed payload, identical to the blob's `ContentHash`.
`AuthorId`	Implementation-defined; persists across replica reseatings	Identifies the human author of operations, distinct from `ReplicaId` (which identifies the device or session). One author may operate multiple replicas; one replica may host one author at a time. The collaboration transport companion defines authentication.
`ReductionAlgorithmVersion`	Semantic version	Selects canonical reduction semantics. Two replicas with different `ReductionAlgorithmVersion` may produce different canonical states from the same operation set; the active superblock declares the version under which the bundle's canonical base was materialized (Section 8.3).
`SolverProfile`	Registered profile identifier	Selects the solver's hard-constraint set, normalized-metric thresholds, tie-breaking weights, and active extension catalog (Chapter 9). The Quality Metric Catalog companion defines the registered profile catalog.
`ProfileId` (file format)	Registered profile identifier	Selects the file-format conformance subset (Full, ReadOnly, Lite, or registered Custom). The Profile Conformance companion defines the registered profile catalog.
`ContentHash`	BLAKE3-256 output (32 bytes)	Cryptographic content hash. The canonical hash for all content-addressed objects in the bundle.

Type	Source	Semantic rule
`ChunkId`	Newtype around `ContentHash`	Same 32 bytes as the underlying `ContentHash`; the type distinction makes role visible at use sites.
`OperationId`	64-bit `ReplicaId` + 64-bit counter, both randomly initialized; counter monotonic	Stable identity of an operation, set by the authoring replica at the moment of authoring. Never reassigned.
`TransactionId`	128-bit, author-minted	Identifies a transaction declared by a `DeclareTransaction` envelope. Member envelopes reference this id in their envelope `transaction` field.
`ConflictId`	Content-derived via BLAKE3 truncation with domain tag `"MUSCCONF"`	Identifier of a conflict record. Derived from canonical kind, sorted causing operations, sorted affected objects. Section 6.5.3.
`Typed identifiers` (EventId, PitchId, VoiceId, StaffId, etc.)	64-bit `ReplicaId` + 64-bit counter; or `ReplicaId::SYSTEM_DERIVED` + BLAKE3-derived counter for system-derived identifiers	Stable identity within their typed namespace. Section 5.3.2.

REQUIREMENT A.1

Implementations **MUST** respect the semantic identity rules declared above. They **MAY** freely choose the bit-level encoding of each type (the Binary Format companion specifies the canonical wire encoding); they **MAY**NOT reinterpret the identity semantics. In particular, two distinct works **MUST NOT** share a `DocumentId`; two byte-distinct chunks **MUST NOT** share a `ContentHash`; two distinct operations **MUST NOT** share an `OperationId`.

A.6 Distinction from Open Questions

This appendix lists content that is *deliberately external* to the core specification: it is owned by a companion document, a future revision, or a third-party specification.

It does *not* list "open questions" in the core's own design space. Those are marked with the *Open Question* callout where they appear in the body of the text, and they represent unresolved decisions *within* the core specification's scope. The two categories overlap only at the four open canonical algorithms in Section A.3, which are unresolved in the core text *and* whose resolution will likely take the form of a companion specification or a profile declaration.

Chapter B

Glossary

This glossary defines key terms used throughout the specification. Definitions are drawn from their introducing chapters; the chapter reference points to the authoritative definition.

Acoustic pitch (Chapter 2). The frequency realization of a pitch under an active tuning system. One of the two intrinsic layers of a `Pitch`.

Adaptive tuning (Chapter 4). A tuning system whose frequency resolution depends on harmonic context, not on pitch-space position alone.

Aleatoric time (Chapter 3). A region time model in which events have partial or undefined ordering, expressed as a directed acyclic graph with optional interval bounds.

Anchor (Chapter 3). A reference to a point in time, expressed relative to an identified object (event, measure, region, wall-clock time) plus an offset. Anchors survive edits that do not delete their target.

Arena (event arena) (Chapter 5). The flat storage owned by a score holding all events of all kinds, with $O(1)$ lookup by `EventId`.

Bundle (Chapter 8). The on-disk container holding a score: header, manifest, chunk store, and blob store in a single `.musc` file.

Canonical reduction (Chapter 6). The deterministic procedure that materializes a score graph from the operation set: sort by canonical order, group transactions, apply each operation against working state, record effects.

Canvas (Chapter 5). The spatial root of a score: a 2D space partitioned into regions of staff-based, free-graphic, or hybrid content.

Causal predecessors (Chapter 6). The set of operations on which a given operation causally depends. Used by the CRDT layer to determine concurrency. Expressed in canonical form as a dotted version vector (`CausalContext`); see "Dotted version vector."

Chunk (Chapter 8). An immutable content-addressed unit of storage within a bundle. Identified by the cryptographic hash of its uncompressed payload, with a domain-separated preimage.

Compound operation (Chapter 6). A user-facing operation that expands into a sequence of primitive operations applied as a transaction.

Conflict record (Chapter 6). A first-class graph object recording an operation that could not apply cleanly under canonical reduction. Stable, addressable, user-visible, and resolv-

able by subsequent `ResolveConflict` operations.

CRDT (Chapters 6, 8). Conflict-free replicated data type. In Epiphany, the *operation set* is a true CRDT (a grow-only set of operation envelopes); the materialized score graph is *not* itself a CRDT but is the deterministic reduction of that set.

Cross-cutting structure (Chapter 5). A first-class object spanning multiple tree-nodes via references: slurs, ties, beams, spanners, markers, repeats, analytical annotations, comments, graphic gestures, lyrics, chord symbols.

Decomposition (Chapter 3). The notational breakdown of an event's sounding duration into notehead values, augmentation dots, and ties. Attached externally by the decomposition pre-pass.

Delta (Chapter 6). The structural record of an operation's effect, sufficient to reconstruct the pre-state of every object the operation modifies.

Document identifier (Chapter 8). The logical work identity (`DocumentId`), stable across Save As copies that are intended to remain versions of the same work. May persist across forks at the user's discretion. Distinct from `FileUuid` (physical bundle identity, changes on Save As) and `LineageId` (optional shared-ancestor identity).

Dotted version vector (Chapters 6, 8). The compact causal-context representation of operation history. Operational semantics (causal precedence, frontier coverage, monotonic growth) are specified in Chapter 6; the on-disk serialization is specified in Chapter 8.

Edit barrier (Chapter 8). A structured declaration by an extension that certain edits would invalidate the extension's data. Implementations not understanding the extension refuse matching edits unless the user performs an explicit unsafe edit.

Engraving decision (Chapter 7). A choice made by the engraver (stem direction, beam consolidation, etc.) recorded explicitly in the layout IR for inspection and override.

Engraving override (Chapter 7). A user assertion that the engraver's default decision should be replaced. Authoritative in the score graph; projected into the layout IR.

Event (Chapter 5). A rhythmic atom of the score: pitched, unpitched, rest, indeterminate, trajectory, graphic, or cue. Owned by exactly one voice.

Format profile (Chapter 8). A named subset of the file format that conforming implementations promise to support. Profiles enumerated: Full, ReadOnly, Lite, Custom.

Frame budget (Chapter 10). The time allowed for a layout-and-render operation to maintain 60 Hz (16.7 ms) or 120 Hz (8.3 ms) interactivity.

Glyph catalog identity (Chapter 7). The identifier (SMuFL version, font id, optional font version, metrics hash) under which layout output was produced. Required for any byte-equal-output layout conformance claim.

Graphic content (Chapter 5). Free-form vector graphics, drawn strokes, text, and images within free-graphic or hybrid regions.

Hard constraint (Chapter 9). A constraint that must be satisfied; violation produces a solver error.

Hybrid logical clock (Chapters 6, 8). A clock combining physical wall-clock time with a logical counter, used as a tie-breaker in canonical reduction order. Operational semantics

(per-replica monotonicity, role in canonical reduction ordering, malformed-envelope behavior) are specified in Chapter 6; the on-disk serialization is specified in Chapter 8.

Identifier (typed) (Chapter 5). A 128-bit stable identifier for a named object: `EventId`, `VoiceId`, `StaffId`, etc. Generated by replica-plus-counter scheme.

Incremental layout (Chapter 7). Re-layout of only the IR objects whose dependencies have changed, with cached values reused elsewhere.

Instrument (Chapter 5). An abstract definition of a sounding entity: name, transposition, range, default sound, default clef, default staff count.

Layout IR (Chapter 7). The pipeline of intermediate representations between score graph and renderer: `LogicalLayoutIR`, `ConstrainedLayoutIR`, `ResolvedLayoutIR`, `RenderIR`.

Manifest (Chapter 8). The mutable index of a bundle: a table of operation roots, optional canonical base snapshot, acceleration snapshots, blob roots, profile declarations, and extension declarations. Itself a content-addressed chunk; the active manifest is selected via the active superblock slot (Chapter 8).

Metric time (Chapter 3). A region time model with measures, beats, and exact rational positions. The dominant model for Western notated music.

Notation grammar (Chapter 4). The declarative definition of a notation system's pitch space, intervals, accidentals, and spelling rules. CMN is the default; others extend.

Operation envelope (Chapter 6). A transmitted, stored, and reducible record carrying an operation's payload together with its identity, ordering stamp, causal context, and optional transaction grouping. The unit of replication.

Operation kind (Chapter 6). The tagged enumeration of primitive operation variants: InsertEvent, DeleteEvent, RespellPitch, Transpose, ChangeRegionTimeModel, and so on, plus `Registered` for extension-defined primitives. The full catalog is delivered separately.

Operation log (Chapter 8). The physical storage of the canonical operation-envelope set on disk, as operation-envelope blocks. The canonical document is the deterministic reduction of this envelope set, or, after pruning, of the manifest's `canonical_base` snapshot plus envelopes whose `OperationId`s are not covered by the base's causal frontier.

Operation set (Chapter 6). The replicated grow-only CRDT of operation envelopes. The canonical document is its deterministic reduction.

Pareto frontier (design target) (Chapter 9). The set of layouts for which no quality metric can be improved without worsening another. Solvers **SHOULD** seek layouts near the frontier; conformance is reference-suite-based, not proof of Pareto optimality.

Part (Chapter 5). A per-instrument or per-section view onto the score for printed extraction. Parts are projections, not storage.

Pitch space (Chapter 4). The analytical universe of a pitch system: what positions exist, how they relate, what accidentals are valid. Distinct from tuning system.

Pitch spelling (Chapter 2). The notational appearance of a pitch: nominal, accidental, octave. Attached externally by the spelling pre-pass.

Primitive operation (Chapter 6). The atomic unit of editing. Primitive operations compose

into compound operations and transactions.

Provenance (Chapter 7). The record on each layout IR object tracing back to its score graph source, enabling selection, editing back-references, and incremental layout invalidation.

Proportional time (Chapter 3). A region time model in which horizontal position is wall-clock time, with no measures or beats.

Re-anchoring (Chapter 6). The normative response of a cross-cutting structure when its referent is deleted: cascade delete, truncate, re-anchor, mark orphaned, or refuse.

Reference pitch (Chapter 4). The anchor converting tuning system structure to absolute frequencies: a position and a frequency in Hertz. A property of the score, not the tuning system.

Region (Chapter 5). A spatial-temporal container in the canvas declaring a time model (metric, proportional, aleatoric) and a content model (staff-based, free-graphic, hybrid).

Repair record (Chapter 6). A deterministic compensating change made during reduction to preserve graph invariants (re-anchoring, spanner truncation, attachment tombstoning, voice promotion, tuplet compensation). Recorded as part of canonical state via `OperationEffect::AppliedWithRepair`.

Replica (Chapter 5, 8). An instance of a score on a device. Each replica has a unique identifier and authors operations with monotonic local sequence numbers. The replica identifier `ReplicaId::SYSTEM_DERIVED` is reserved for deterministically-derived system identifiers.

Retention policy (Chapter 8). The declared rule governing how many old manifests (and their reachable chunks) are preserved in the bundle for rollback purposes. Rollback is achieved through retained manifests in the chunk store, not by retaining multiple on-disk superblock generations.

Scale position (Chapter 2). The analytical identity of a pitch within a pitch space. One of the two intrinsic layers of a `Pitch`.

Score graph (Chapter 5). The in-memory representation of all musical content in a score: canonical truth from which layout, serialization, and editing operations derive.

Semantic operation (Chapter 6). A mutation of the score graph: insert event, delete measure, transpose, respell pitch, etc.

SMuFL (Chapter 4). Standard Music Font Layout. The normative source for glyph references in the core.

Snapshot (Chapter 8). A materialized score state. As an acceleration snapshot (listed in `acceleration_snapshots`), derivative and discardable. As a canonical-base snapshot (named in the manifest's `canonical_base` field with declared causal frontier and reduction algorithm version), the canonical base for pruned documents, in which case the canonical state is the base snapshot plus envelopes whose `OperationId`s are not covered by the base's causal frontier.

Soft constraint (Chapter 9). A constraint with a weight; violation is permitted but penalized in quality metrics.

Sounding duration (Chapter 3). The exact rational duration an event occupies in time. In-

trinsic; notational decomposition is attached separately.

Spring slot (Chapter 7). A time slot's horizontal spacing parameters (min, preferred, max widths; stretch, compress factors) consumed by the constraint solver.

Staff (Chapter 5). The global, abstract staff identity persisting across the entire score. A staff is declared once at the score level and referenced by staff instances in any region where it appears.

Staff instance (Chapter 5). The region-local manifestation of a staff: the voices, measures, clef changes, key changes, and metric grid that belong to that staff for one region. A single `Staff` may have multiple `StaffInstance`s across the score.

Tempo map (Chapter 3). The function mapping musical positions to wall-clock positions, piecewise over musical time with constant, linear, exponential, or curve segments.

Time anchor (Chapter 3). See "Anchor."

Time axis (Chapter 7). The polymorphic layout-IR component projecting time positions to horizontal positions. Three implementations: metric, proportional, aleatoric.

Transaction (Chapter 6). A replicated operation group that reduces atomically: either every member applies and the transaction reduces to `Applied`, or the entire transaction reduces to `Conflicted` with a recorded conflict.

Trajectory (Chapter 5). A continuous pitch-motion event: glissando, portamento, pitch bend. A sibling to pitched events under the event taxonomy.

Tuning system (Chapter 4). The mechanism mapping pitch-space positions to frequencies, given a reference pitch. Distinct from pitch space.

Tuplet (Chapter 3). A grouping object annotating irregular rhythmic groupings. Does not modify sounding durations; purely notational and analytical.

Typed object identifier (Chapter 5). A tagged union (`TypedObjectId`) over every identifier kind in the score graph. Used wherever an object is referenced generically (cross-cutting endpoints, conflict `affected_objects`, repair records, edit barriers).

Validation mode (Chapter 6). The strictness setting for operation precondition checking: strict authoring or lenient replay.

Vertical band (Chapter 7). A horizontal slice of the canvas with spring parameters for vertical spacing: staff bands, inter-staff gaps, inter-system gaps, margin bands.

View (Chapter 5). A recipe for displaying a configuration of the score: which layers are active, which parts are rendered, what overrides apply.

Voice (Chapter 5). A polyphonic line within a staff. Owns a sequence of events.

Wall-clock time (Chapter 3). Time measured by the playback engine or external sync source, in fixed-point nanoseconds. Distinct from musical time.

Chapter C

Bibliography and References

Engraving Treatises

Behind Bars. Elaine Gould. *Behind Bars: The Definitive Guide to Music Notation.* Faber Music, 2011. The principal modern reference for music engraving conventions.

Music Notation in the Twentieth Century. Kurt Stone. *Music Notation in the Twentieth Century: A Practical Guidebook.* W. W. Norton, 1980. Authoritative reference for contemporary notation.

Essentials of Music Notation. Tom Ross. *The Art of Music Engraving and Processing.* Hansen Books, 1970. Classical treatise on engraving conventions, still influential.

Music Notation. Gardner Read. *Music Notation: A Manual of Modern Practice.* Crescendo Publishers, 1969. Reference for engraving practices and history.

Music Theory and Analysis

The Cognition of Basic Musical Structures. David Temperley. MIT Press, 2001. Source of preference-rule approaches to spelling and meter analysis.

Pitch Spelling and the Line of Fifths. H. Christopher Longuet-Higgins. Foundational work on pitch-spelling algorithms.

Tonal Pitch Space. Fred Lerdahl. Oxford University Press, 2001. Pitch-space theory as analytical framework.

A Generative Theory of Tonal Music. Fred Lerdahl and Ray Jackendoff. MIT Press, 1983. Foundational work in formal music theory and the source of preference-rule methodology.

Tuning, Microtonality, and Just Intonation

Genesis of a Music. Harry Partch. University of Wisconsin Press, 2nd ed. 1974. Foundational text in extended just intonation and microtonal practice.

Tuning, Timbre, Spectrum, Scale. William Sethares. Springer, 2nd ed. 2004. Modern treatment of tuning systems and their psychoacoustic foundations.

Helmholtz-Ellis JI Pitch Notation. Marc Sabat and Wolfgang von Schweinitz. The HEJI accidental system referenced by the core's built-in JI pitch spaces.

Sagittal: A Microtonal Notation System. George D. Secor and David C. Keenan. The Sagittal microtonal accidental system.

Constraint-Based Layout

Cassowary Linear-Arithmetic Constraint Solver. Greg J. Badros, Alan Borning, and Peter J. Stuckey. *The Cassowary Linear Arithmetic Constraint Solving Algorithm.* ACM Transactions on Computer-Human Interaction, 8(4):267-306, 2001. Source of the constraint-solver model referenced in Chapter 9.

Music Engraving with Constraint Programming. Multiple authors. Survey of constraint-based approaches to music engraving.

The Gourlay Algorithm. J. S. Gourlay. *A Language for Music Printing.* Communications of the ACM, 1986. Source of the proportional spacing model referenced in Chapter 7.

CRDT and Distributed Systems

A Comprehensive Study of Convergent and Commutative Replicated Data Types. Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. INRIA Technical Report RR-7506, 2011. Foundational survey of CRDTs.

Dotted Version Vectors. Nuno Preguiça et al. Refinement of vector clocks used in the file format (Chapter 8).

Hybrid Logical Clocks. Sandeep S. Kulkarni et al. Clocks combining physical and logical time, used for total ordering of operation envelopes (Chapter 6).

Prior-Art Notation Formats

MusicXML. Recordare LLC, later W3C Music Notation Community Group. The dominant interchange format; surveyed for the interchange profile work referenced in Chapter 5.

MEI (Music Encoding Initiative). An XML-based encoding designed for scholarly editions and musicology. Influenced the spec's handling of editorial apparatus and analytical layers.

MNX. W3C Music Notation Community Group draft. The successor effort to MusicXML; informed several decisions on layered data models.

LilyPond. David Bauer, Han-Wen Nienhuys, Jan Nieuwenhuizen. Open-source music engraving software using a text-based input format and a sophisticated rule-based engraver. Significant influence on the spec's engraving philosophy.

SMuFL. W3C Music Notation Community Group. The Standard Music Font Layout, referenced as the normative glyph catalog in Chapter 4.

Additional References

The Anatomy of a Note. Multiple papers on the data representation of musical notation in computer systems.

Notation in Computer Music. Various proceedings of the International Computer Music Conference (ICMC) and the New Interfaces for Musical Expression (NIME) workshops.

Chapter D

Determinism Contract

This appendix consolidates the reproducibility obligations that appear throughout the specification. It defines exactly which values are admissible in canonical state, how they are serialized, how comparisons work, and where implementation freedom begins.

The appendix is the document's legal code for reproducibility: where the body of the specification says "deterministic," this appendix says what that means.

D.1 Thesis

Requirement D.1

Canonical document state **MUST** be independent of platform, CPU, locale, thread scheduling, hash-map iteration order, floating-point environment, compression settings, and wall-clock timing.
Where exact equality is required, this specification defines canonical representations. Where numerical approximation is unavoidable, this specification defines deterministic tolerances, quantization, and reporting rules.
Implementations **MAY** freely choose internal data structures, algorithms, and parallelism strategies. The canonical externally observable result **MUST NOT** depend on those choices.

D.2 Layers of Determinism

The specification distinguishes five layers. Each layer has its own determinism contract; conflating them is the most common source of impossible requirements.

Canonical score determinism (Chapter 6). The reduction of an operation-envelope set to a materialized score graph is fully deterministic *when every algorithm contributing to that reduction has received one of the dispositions in Section D.11* (normatively specified, profile-declared by versioned identifier, or explicitly non-canonical). Under that condition, identical operation sets produce identical materialized score states across all conforming implementations, with identical object identifiers, tombstones, conflicts, and effects. For algorithms still in open-question state (Section D.11), this byte-equal cross- implemen-

tation guarantee **MUST NOT** be claimed; the output of such algorithms is either non-canonical or governed by the profile's declared algorithm version.

Canonical serialization determinism (Chapter 8). The bundle format's chunk hashes, manifest encoding, text projection, and the ordering of canonical maps and sets are deterministic. Two implementations producing the same canonical state **MUST** produce identical chunk content hashes and identical text-projection bytes.

Layout determinism (Chapter 9). Within a single solver implementation at a fixed version, with fixed profile, fixed budget, and fixed font/glyph metrics, layout output is byte-identical across runs. Across different solver implementations, layout output **MAY** differ aesthetically within reference-suite thresholds.

Conformance determinism . Reference-suite-based: conforming implementations need not produce byte-identical output to each other; they need to fall within declared metric thresholds on every reference-suite entry. See Chapter 9 Section 9.9.

Non-canonical cache determinism . Implementations maintain caches (operation index, layout cache, glyph metric cache, render artifact cache). These **MAY** vary across runs, platforms, and implementations. They **MUST NOT** affect canonical state and **MUST** be discardable (Chapter 6 Section 6.3.5).

REQUIREMENT D.2

Byte-equality obligations **MUST** be applied only to the layer for which they are stated. Implementations **MUST NOT** interpret "deterministic" more strongly than its layer specifies, and **MUST NOT** interpret it more weakly either.

RATIONALE

The conditional in canonical score determinism is essential. Several algorithms affecting canonical state remain open questions in this draft: the spelling pre-pass (Chapter 2), the notational decomposition algorithm (Chapter 3), and the `wallclock_to_musical` root-finding algorithm (Chapter 3). Until those algorithms receive Section D.11 dispositions, cross-implementation byte equality cannot be promised for outputs derived from them. This is not a weakening of the determinism contract; it is a precise statement of what the contract currently covers. Once the open algorithms are specified or profile-declared, the conditional resolves and full cross-implementation byte equality applies to all of canonical state.

D.3 Floating-Point Values in Canonical State

D.3.1 Permitted Forms

REQUIREMENT D.3

Canonical stored floating-point values **MUST** be finite IEEE 754 binary64 values. NaN and infinity **MUST NOT** appear in canonical chunks: implementations **MUST** reject them at serialization time and **MUST** treat their presence on read as data corruption.
The value `-0.0` (negative zero) **MUST** be canonicalized to `+0.0` before storage. Two floating-point values that differ only in zero sign are equal under canonical comparison.
Floating-point values appear in advisory, acoustic, tuning, tempo, layout, and quality-metric contexts. Where exact identity is needed (object identifiers, operation ordering, graph membership, hash identity), floating point **MUST NOT** be used.

D.3.2 Serialization

REQUIREMENT D.4

Canonical `f64` values are serialized as little-endian IEEE 754 binary64 octets after applying the `-0.0` $\rightarrow$ `+0.0` canonicalization. The eight serialized bytes are the canonical representation; two values whose canonical bytes are equal are canonically equal.
Hash preimages incorporating floating-point values (Section 8.5) **MUST** use the canonical serialized bytes, not platform-native byte orders or representations.

D.3.3 Equality

REQUIREMENT D.5

Canonical equality of two floating-point values is byte equality of their canonical serialized representations. The IEEE 754 notion that `NaN` $\neq$ `NaN` is irrelevant in canonical state, because canonical state cannot contain NaN.
For `NormalizedMetric` values (Chapter 9), additional invariants apply: the value is finite, in $[0.0, 1.0]$, and lower is better. Comparisons of `NormalizedMetric` use IEEE 754 ordered comparison; the `-0.0` $\rightarrow$ `+0.0` canonicalization applies before comparison.

D.4 Rounding and CPU Behavior

REQUIREMENT D.6

Numerical operations whose results enter canonical state **MUST** use IEEE 754 round-to-nearest, ties-to-even. The ambient floating-point rounding mode (which on most platforms defaults to round-to-nearest-ties-to-even but can be changed) **MUST NOT** affect canonical results: implementations **MUST** either save/restore the rounding mode or use operations insensitive to it.
Implementations **MUST NOT** compile canonical numerical algorithms with

`-ffast-math`, `-funsafe-math- optimizations`, or equivalent flags that permit non-IEEE-conformant transformations. Implementations **MUST NOT** reorder, reassociate, or distribute floating-point expressions in ways that can change the canonical result. Implementations **MUST NOT** silently fuse multiplications and additions into fused-multiply-add operations unless the algorithm explicitly requests FMA. Implementations **MUST NOT** depend on, or be sensitive to, ambient floating-point exception flags or host rounding mode.
Where implementations use SIMD, GPU acceleration, or platform math libraries (`libm`, vendor math libraries) in canonical algorithms, they **MUST** produce results identical to a strict-IEEE single-threaded baseline. Differences in transcendental function implementations (`sin`, `cos`, `exp`, `log`) across libm versions are a real source of nondeterminism; implementations using transcendentals in canonical algorithms **MUST** use a documented portable implementation (e.g., a vendored math library or a software fallback path).

D.5 Exact and Quantized Representations

The specification prefers exact representations over floating-point wherever possible. The following are exact by design:

Time and duration. `MusicalPosition` and `MusicalDuration` are rational (Chapter 3). Inline- or-promoted representation; arithmetic is exact.

Operation ordering. Causal order from DVV; total order by HLC and identifier (Chapter 6). All integer- valued.

Identifiers. All graph identifiers are 128-bit values composed of replica plus counter (Chapter 5).

Hashes. All canonical hashes are BLAKE3-256 (Chapter 8).

D.5.1 Quantized Layout Coordinates

Spatial layout coordinates require numeric computation, but canonical output must be stable. The specification quantizes canonical coordinates to a fixed grid.

```rust
/// A canonical spatial coordinate, quantized to 1/1024 of a
/// staff space. Used for canonical serialized output of
/// ResolvedLayoutIR positions. Internal solvers MAY use floating
/// point during computation; canonical serialization rounds to
/// QuantizedCoord.
pub struct QuantizedCoord {
    /// Coordinate value in units of 1/1024 staff space.
    pub units: i64,
}
```

REQUIREMENT D.7

The canonical spatial coordinate grid is $1/1024$ staff space per unit. Layout coordinates emitted into canonical `ResolvedLayoutIR` **MUST** be quantized to this grid before serialization. The quantization rule is round-to-nearest, ties-to-even on the integer unit value. Implementations **MAY** use floating-point coordinates internally during layout computation. The act of quantization at serialization time absorbs all floating-point variation from the canonical output. Two implementations whose internal computations agree to better than $1/2048$ staff space at every coordinate produce identical canonical output after quantization.

The grid spacing is fixed at $1/1024$ staff space for this format version. Future major versions **MAY** adjust the spacing; doing so is a non-backward-compatible change.

RATIONALE

$1/1024$ staff space is finer than visual resolution at any practical printing density (a staff space is typically 1.5–2 mm, so $1/1024$ is roughly 1.5–2 μm, well below print resolution). The grid is fine enough that quantization is inaudible musically and invisible visually, but coarse enough that floating-point noise from different solver implementations routinely rounds away.

D.6 Tolerance Classes

The specification refers to numerical tolerances in several places (acoustic comparison, layout quality thresholds, tempo integration, solver residual). All such tolerances belong to one of a small set of named classes, each with declared semantics.

```
pub enum ToleranceClass {
    /// Acoustic pitch comparison, in cents.
    AcousticCents,

    /// Layout coordinate comparison, in staff spaces.
    LayoutCoordinate,

    /// Quality metric comparison; absolute on the [0.0, 1.0]
    /// NormalizedMetric scale.
    QualityMetric,

    /// Tempo integration residual: maximum permitted error in
    /// musical_to_wallclock or wallclock_to_musical conversion.
    TempoIntegration,

    /// Solver residual: maximum permitted constraint violation
    /// for a soft constraint to be considered satisfied.
    SolverResidual,
}

pub struct Tolerance {
    pub class: ToleranceClass,
```

```
    /// Absolute tolerance. The unit is implied by the class.
    pub absolute: f64,

    /// Optional relative tolerance, applied to nonzero values.
    pub relative: Option<f64>,

    /// What the tolerance governs.
    pub governance: ToleranceGovernance,
}

pub enum ToleranceGovernance {
    /// Tolerance affects canonical equality (rare; usually only
    /// for diagnostic comparison).
    Equality,

    /// Tolerance is a validation threshold (constraint considered
    /// satisfied if violation is below tolerance).
    Validation,

    /// Tolerance affects only diagnostic output; canonical state
    /// is unaffected.
    Diagnostic,
}
```

REQUIREMENT D.8

Specifications and conformance documents **MUST NOT** introduce ad-hoc epsilon constants. Every numerical tolerance that affects normative behavior **MUST** be declared as a named `Tolerance` value belonging to a defined `ToleranceClass`, with an explicit unit, absolute and optional relative bounds, and governance category.
Tolerances **MUST NOT** apply to identity: `PitchId`, `EventId`, `OperationId`, graph membership, hash identity, and operation ordering are exact, never within tolerance.
The Quality Metric Catalog, the Reference Suite, and the Performance Reference Suite enumerate the specific tolerance values for each profile and tier. Their values are normative per the companion specifications.

D.7 Ordered Iteration over Sets and Maps

Canonical output frequently depends on iterating a collection. Hash-map and B-tree implementation order leaks platform-specific behavior into canonical output if not controlled.

REQUIREMENT D.9

Whenever canonical output, canonical serialization, or canonical hashing depends on iterating a set, map, or other collection, iteration **MUST** occur in a specified total order. The following total orders are normative:

- Typed identifiers (`EventId`, `PitchId`, `VoiceId`, etc.): lexicographic ascending on

the identifier's canonical byte form (16 bytes: 8-byte replica, 8-byte counter, big-endian).

- Operation envelopes: causal order first (from DVV closure), then HLC physical time, then HLC logical counter, then `OperationId` as final tie-break.
- Chunk references: ascending by `ChunkKind` discriminant, then by content hash lexicographic, then by file offset.
- Conflict records: ascending by `ConflictId`.
- Extension declarations: ascending by `ExtensionId`, then by semantic version lexicographic.
- Strings (text fields): byte-lexicographic on the UTF-8 NFC representation (Section D.8).
- Rationals (`RationalTime` sequences): by numeric value ascending.
- Composite keys: lexicographic on the canonical representation of the key tuple's component fields, in declaration order.

Hash-map iteration order, B-tree implementation differences, and similar implementation artifacts **MUST NOT** leak into canonical output.

D.8 Text and Unicode

Locale-dependent text behavior is a routine source of canonical divergence. The specification fixes this at the encoding layer.

REQUIREMENT D.10

Canonical text fields **MUST** be encoded as UTF-8 with Unicode NFC (Normalization Form Canonical Composition) applied. Two texts whose NFC byte representations are equal are canonically equal regardless of decomposition state.
Comparisons of canonical text fields for identity **MUST** be byte comparisons of NFC-encoded UTF-8. User-facing display collation (sorting by locale rules, case-insensitive matching, diacritic folding) is non-canonical and **MUST NOT** affect canonical state.
Locale settings **MUST NOT** affect:

- Parsing of canonical text projections.
- Serialization of canonical text fields.
- Sorting of canonical map keys.
- Decimal formatting in canonical text projections (the text projection uses a fixed decimal-point convention; locale comma-vs-period preferences are irrelevant).
- Date and time formatting in canonical data (canonical timestamps use a fixed format; locale formats are for UI only).

Two implementations operating under different locale settings on the same canonical document **MUST** produce identical canonical output, including identical chunk hashes and identical text projections.

D.9 Randomness and Parallelism

D.9.1 Randomness

REQUIREMENT D.11

Randomness **MUST NOT** affect canonical output unless the random seed is an explicit canonical input.
Algorithms whose results enter canonical state and that use pseudorandom generation (e.g., probabilistic constraint solvers, randomized rounding) **MUST** consume their seed as a declared algorithm parameter that becomes part of the algorithm's conformance configuration. Identical seed plus identical input yields identical output.
Identifier minting (`ReplicaId` initialization) is the only conforming use of platform-level randomness in this specification, and even there the random output enters canonical state only via the resulting identifiers.

D.9.2 Parallelism

REQUIREMENT D.12

Parallel implementations of canonical algorithms **MUST** produce results identical to a strict single-threaded baseline. Race timing, work-stealing order, and thread count **MUST NOT** affect canonical state.
Where parallel reductions are used (summation over many contributions, parallel constraint evaluation), the merge step **MUST** be deterministic: ordered reduction over a canonical ordering of contributions, not arbitrary tree-merge order.
Implementations **MAY** use parallel execution as a performance optimization. They **MAY**NOT use it as a license for nondeterminism.

D.10 Compression and File Bytes

REQUIREMENT D.13

Canonical content identity is the uncompressed payload bytes with the domain-separated preimage (Chapter 8). Compression algorithm choice and compression level **MUST NOT** affect canonical identity. Identical uncompressed content has identical `ContentHash` regardless of how it was physically compressed.
Canonical chunk payload encoding **MUST** be deterministic: the same in-memory canonical structure **MUST** produce identical uncompressed bytes across runs and across implementations, within the same schema version. The Binary Format companion specification defines the canonical encoding.
Compression bytes (the physical bytes stored on disk after zstd or another algorithm is applied) are *not* canonical unless a profile explicitly requires bit-identical bundle reproduction. Implementations **MAY** recompress chunks (e.g., during compaction or repack) without altering canonical state.

Readers **MUST** verify decompressed payload against the declared content hash before treating the payload as authoritative.

D.11 Open Algorithm Hooks

Two algorithms remain open questions in the body of the specification: tempo integration for arbitrary curves and root-finding for `wallclock_to_musical` (Chapter 3). These algorithms affect canonical state. The spelling pre-pass (Chapter 2) and the notational decomposition algorithm (Chapter 3) were resolved in Pass 12 to the profile-declared disposition below, each with a ratified default identifier (Requirements 2.10 and 3.16).

REQUIREMENT D.14

Any algorithm that affects canonical state **MUST** satisfy one of:

- It is specified normatively in this document or a named companion specification. Conforming implementations **MUST** implement the specified algorithm.
- It is profile-declared: the active conformance profile names a specific algorithm by versioned identifier. Two conforming implementations on the same profile use the same algorithm. Profiles may differ in their algorithm choices.
- It is explicitly marked non-canonical or advisory: its output appears in the score only in non-canonical layers (caches, diagnostics, user-facing rendering hints) and **MUST NOT** appear in canonical chunks.

Implementations **MUST NOT** introduce algorithms affecting canonical state outside of these three categories. "Vendor's spelling preference" or "proprietary decomposition heuristic" **MUST NOT** silently determine canonical content.

D.12 Conformance Statement

REQUIREMENT D.15

Conforming implementations **MUST** declare, as part of their published conformance statement:

- Their platform and floating-point library, with the libm version or vendored math library reference.
- Their parallel execution strategy and the demonstrated equivalence to single-threaded baseline.
- Their handling of the `-0.0` canonicalization and NaN/infinity rejection.
- Their adherence to round-to-nearest-ties-to-even.
- Their compliance with the canonical iteration orders in Section D.7.
- Their NFC normalization implementation reference.
- Their declared algorithms for any open-question area (spelling, decomposition,

> tempo curves), with versioned identifiers.

Chapter E

Canonical Byte-Layout Reference

E.1 Status of this reference

This appendix consolidates, in one place, every byte layout pinned by the Pass 11 ratification: the discriminant tables, derivation preimages, and primitive encodings that the core, operations, and bundle layers already implement and lock with golden-bytes tests. Its purpose is to give the Binary Format companion (Section 8.18) a single point to import, rather than re-deriving these layouts from three separate crates.

This reference is a *consolidation, not a second source of truth.* Every entry cites the requirement that defines it normatively; where this appendix and a cited requirement appear to disagree, the cited requirement governs. The appendix introduces no obligation beyond what its citations already impose. The layouts that Pass 11 deliberately left to the interchange companions are listed in Section E.7 so the boundary is explicit.

E.2 Shared encoding conventions

All composite canonical encodings follow the convention baseline that the Binary Format companion inherits (Requirement 8.35): little-endian integers; a boolean as a single `0/1` byte; `u32` little-endian counts and a length prefix on every variable-width leaf; a tagged union as a single discriminant byte followed by its variant payload; and free-text strings as length-prefixed UTF-8 that the codec does *not* NFC-fold (catalog identifiers are NFC-normalized at construction, not in the codec). Certain hashing preimages deliberately depart from the single-discriminant-byte convention where a wider or order-bearing tag is required — notably the `TypedObjectId` 16-bit big-endian discriminant; such departures are pinned at their definitions and noted in the tables below.

E.3 System-derived identifier function

Several identifiers below are content-addressed rather than counter-allocated. They share one function (Section 5.3.2): the 64-bit counter of a `ReplicaId::SYSTEM_DERIVED` (`0xffff_ffff_ffff_ffff`) identifier is BE_u64(BLAKE3(`domain_tag`‖`canonical_inputs`)[0..8]). The domain tag is exactly eight bytes. The reserved built-in tags *for this derivation* are the closed set {`MUSCSVCE`,

MUSCSPCH, MUSCSANM} — the three system-derived-identifier tags only; the other reserved domain tags catalogued in Section E.5 feed plain hashing preimages, not this counter. Tags introduced by registered extensions **MUST** begin with MUSCS, be exactly eight bytes, and not collide with the reserved three.

E.4 Discriminant tables, derivations, and encodings

E.4.1 Object-graph identity (canonical document state)

Layout	Canonical bytes / preimage	Requirement
TypedObjectId tag	16-bit big-endian discriminant (0–27) ‖ variant payload. Registered = 27 encodes reg (16 BE) ‖ raw (16 BE), 34 bytes total. ObjectKindRegistryId is 128-bit	5.2
Promoted VoiceId	System-derived under MUSCSVCE over a 64-byte preimage: staff_instance ‖ original_voice ‖ winning_op ‖ losing_op, each 16-byte big-endian	§5.6.3
System-derived PitchId	System-derived under MUSCSPCH over the intrinsic identity (scale position, acoustic realization); variable-width strings length-prefixed and NFC-normalized at the boundary; the tuning reference is always present (Inherit as a distinct presence marker)	5.8
IntegrityAnomalyId	System-derived under MUSCSANM over IntegrityAnomalyKind::to_canonical_bytes()	5.5
ObjectKind discriminant	Single byte: Voice = 0, Pitch = 1, Registered = 2. Enters the IntegrityAnomalyId preimage	5.6

E.4.2 Bundle layer (chunk store, manifest, blobs)

Layout	Canonical bytes / preimage	Requirement
Chunk content hash	BLAKE3 of domain_tag ‖ ChunkKind byte ‖ SchemaVersion (4) ‖ uncompressed_length (u64 LE) ‖ payload; tag MUSCCHNK for chunks, MUSCMANI for the manifest. Compression is *not* in the preimage	§8.5
ChunkKind	Single declaration-order byte, 0–8 (in the chunk hash preimage, so stable)	8.11

Layout	Canonical bytes / preimage	Requirement
`Compression Algorithm`	Fixed two bytes: discriminant ‖ parameter. `None` = `[0, 0]`, `Zstd{level}` = `[1, level]`, `Reserved(v)` = `[2, v]`. Not part of chunk identity	8.11
`SchemaVersion`	`major` (u16 LE) ‖ `minor` (u16 LE), four bytes	§8.5
`BlobId`	Bare BLAKE3 of `MUSCBLOB` ‖ `uncompressed_payload`: the domain tag immediately followed by payload, with no kind, schema, or length fields	8.18
`ManifestId`	`trunc128` (leading 16 bytes) of BLAKE3 of `MUSCMNIF` ‖ `document_id` (16) ‖ `generation` (u64 LE) ‖ `manifest_body`; the `manifest_id` field is excluded from the body	8.8
`ProfileId`	Fixed 20 bytes: discriminant (u32 LE; `Full` = 0, `ReadOnly` = 1, `Lite` = 2, `Custom` = 3) ‖ `ProfileRegistryId` (16, zero unless `Custom`). Load-bearing in superblock selection	8.31

E.4.3 Operation layer (transaction metadata)

Layout	Canonical bytes	Requirement
`Transaction Category`	Single byte: `NoteEntry` = 0, `Structural` = 1, `Layout` = 2, `Import` = 3, `Registered` = 4	6.19
`ResolutionAction`	Single byte: `AcceptLoser` = 0, `KeepWinner` = 1, `Override` = 2, `Reanchor` = 3, `Dismiss` = 4, `Registered` = 5. Encoded into the operation content hash	6.14

E.4.4 Primitive scalar encodings

Layout	Canonical bytes	Requirement
`RationalTime`	Sign byte (`0` zero, `1` positive, `2` negative) ‖ numerator magnitude (u32-LE length-prefixed, big-endian) ‖ denominator magnitude (u32-LE length-prefixed, big-endian). Always stored reduced (lowest terms, positive denominator), so equal rationals encode to equal bytes	8.36
Wall-clock integers	Fixed-width little-endian (matching `QuantizedCoord`)	8.36

E.5 Domain-tag registry

Every reserved built-in eight-byte domain tag, in one place. The nine *canonical* tags — whose preimages produce identifiers and content hashes that are part of the interoperable, durably persisted form every conforming implementation must reproduce bit-identically — are listed first; `MUSCFNTM` and `MUSCLOID` are *non-canonical* (layout-only) and listed last, for completeness (see Section E.7). Registered extensions may additionally mint *system-derived* tags, which **MUST** begin with `MUSCS`, be exactly eight bytes, and not collide with a reserved tag (Section E.3).

Tag	Use
`MUSCCHNK`	Chunk content-hash preimage (§8.5)
`MUSCMANI`	Manifest chunk content-hash preimage (§8.5)
`MUSCBLOB`	Blob identity, bare preimage (Requirement 8.18)
`MUSCMNIF`	Manifest-id derivation (Requirement 8.8)
`MUSCCONF`	`ConflictId` derivation (§6.5.3)
`MUSCENVH`	Operation-envelope hash (`EnvelopeHash`) (§6.2.5)
`MUSCSVCE`	System-promoted voice id (§5.6.3)
`MUSCSPCH`	System-derived pitch id (Requirement 5.8)
`MUSCSANM`	Integrity-anomaly id (Requirement 5.5)
`MUSCFNTM`	Font-metrics hash for layout conformance — *non-canonical* (§7.12.1)
`MUSCLOID`	Layout-object id — *non-canonical*, wired by the reference implementation as of P12-I2 (Requirement 7.4)

E.6 Reference-implementation locks

Each ratified layout above is anchored by a test in the reference implementation that fails deliberately if the layout drifts. Most are *golden-bytes* tests pinning the literal encoding; the two noted exceptions (`BlobId` and `RationalTime`) are anchored by round-trip and canonicalization tests, which catch a drift in decode or normalized form but not every literal-byte change. These anchors make the spec-to-code correspondence checkable.

Layout	Anchoring test
`TypedObjectId`	`epiphany-core/src/ids.rs`: `typed_object_id_byte_form_is_locked`
Promoted `VoiceId`	`epiphany-core/src/graph.rs`: `promoted_voice_id_byte_form_is_locked`
System `PitchId`	`epiphany-core/src/pitch.rs`: `system_pitch_id_byte_form_is_locked`
`IntegrityAnomalyId`	`epiphany-ops/src/anomaly.rs`: `integrity_anomaly_id_byte_form_is_locked`
`ObjectKind`	`epiphany-ops/src/support.rs`: `object_kind_discriminants_are_golden`
`ChunkKind`	`epiphany-bundle/src/chunk.rs`: `chunk_kind_discriminants_are_golden`
`Compression Algorithm`	`epiphany-bundle/src/chunk.rs`: `compression_algorithm_encoding_is_golden`
`ProfileId`	`epiphany-bundle/src/superblock.rs`: `profile_id_discriminants_are_golden`
`ManifestId`	`epiphany-bundle/src/ids.rs`: `manifest_id_is_content_derived_and_deterministic`
`Transaction Category`	`epiphany-ops/src/payload.rs`: `transaction_category_discriminants_are_golden`
`ResolutionAction`	`epiphany-ops/src/conflict.rs`: `resolution_action_discriminants_are_golden`
`BlobId`	`epiphany-determinism/src/hash.rs`: `ContentHash::of_blob` and its round-trip locks (*round-trip, not a literal-byte golden*)
`RationalTime`	`epiphany-core/src/time.rs`: `equal_rationals_encode_identically` (*canonical-equality lock, not a literal-byte golden*)

The `MUSCLOID` layout-object-id derivation (Requirement 7.4), wired in P12-I2, is anchored differently because it is *non-canonical*: its ids never enter document state or a content hash, so there is no durable byte layout to golden-lock. It is instead pinned by a derivation reference-lock test (`epiphany-layout-ir/src/provenance.rs`: `stable_id_uses_the_ratified_muscloid_derivation`), which fixes the tagged preimage; by the tag-spelling lock (`epiphany-determinism/src/domain.r` `exact_tag_spellings_match_spec`); and by the renderer's provenance goldens, whose `data-prov` hex changes if the derivation drifts.

E.7 Layouts deferred to the companions

The following are intentionally *not* ratified here; they remain provisional pending the Binary Format companion and the Operation Catalog, and are recorded so the companion authors know precisely which layouts they own versus inherit:

- `OperationKindTag` / `OperationKind` discriminants and the full per-operation payload encodings. The reference implementation assigns provisional declaration-order discriminants (`0`–`16`) locked only for mutual distinctness, not to literal values; pinning the literal

wire form is the Operation Catalog's and Binary Format companion's job.

- The full composite struct layouts — whole-`Score`, operation envelopes, the manifest body's field order — together with schema-version wire evolution and varint details beyond the convention baseline (Requirement 8.35). The companion inherits the baseline and formalizes these.

Chapter F

Revision History

Date	Section	Change
October 11, 2026	All	Initial scaffold; chapter structure and front matter established.
October 11, 2026	Ch. 2	Pitch chapter written: layered pitch type, scale position, acoustic realization, spelling subsystem with attachments, precedence, pre-pass, and analytical layers. Chapter structure expanded: pitch separated from time/tuning; semantic operations chapter added.
October 11, 2026	Ch. 3	Time and duration chapter written: exact rational time with inline-or-promoted representation, distinct position and duration newtypes, wall-clock time as fixed-point nanoseconds, time anchors, time signatures with explicit beat groups including irrational meters, tempo maps with deterministic conversion, notational decomposition pre-pass, tuplets as grouping objects, and three region time models (metric, proportional, aleatoric).
October 11, 2026	Ch. 4	Tuning systems and pitch spaces chapter written: pitch spaces as analytical universes distinct from tuning systems, position structures (chromatic, diatonic-over-chromatic, JI lattice, registered), interval algebras, nominal and accidental registries, SMuFL glyph references with versioning, tuning resolution variants including adaptive tuning, score-level reference pitch, hierarchical resolution rules, and the normative built-in catalog of pitch spaces and tuning systems.

Date	Section	Change
October 11, 2026	Ch. 5	Score graph chapter written: hybrid tree-plus-cross-cutting topology, canvas-region-staff-voice-event hierarchy with regions carrying time and content models, typed 128-bit identifiers with replica-plus-counter generation, flat event arena, complete event taxonomy (pitched, unpitched, rest, indeterminate, trajectory, graphic, cue), cross-cutting registry partitioned by type, graphic content with region-local coordinates and time bindings, parts as projections separate from the canvas, analysis layers and views, graph invariants, and required indexes.
October 11, 2026	Ch. 6	Semantic operations chapter written: operation framework with dual command/delta layering, primitive and compound operation distinction, transactions as atomic user units, precondition classification (invariant vs. advisory) with strict authoring and lenient replay modes, the normative re-anchoring rule table, inverse-based undo with single-replica scope, semantic merge semantics with five merge families, and representative operations illustrating each major category. The full operation catalog is deferred to a separate conformance specification.
October 11, 2026	Ch. 7	Layout IR chapter written: pipeline of four stages (LogicalLayoutIR, ConstrainedLayoutIR, ResolvedLayoutIR, RenderIR-at-interface), shared provenance discipline, spatial primitives in staff spaces with f32 precision, polymorphic time axis trait with metric/proportional/aleatoric implementations, composite logical objects flattened to glyph-level constrained objects, spring-based horizontal and vertical layout, explicit engraving decision recording, hard and soft override mechanism with preservation rules, fine-grained dependency tracking with invalidation rules, glyph catalog interface, and uniform region containers across kinds.

Date	Section	Change
October 11, 2026	Ch. 8	File format chapter written: bundle container structure with header, manifest, chunk store, and blob store; custom schema-defined binary format with self-describing extension envelope; content-addressed immutable chunks with manifest as the only mutable index; multi-level chunking (score-level, staff-region, locality-grouped cross-cutting, attachment chunks); operation log as canonical source of truth with dotted version vectors and hybrid logical clocks; snapshots as derivative caches; schema versioning with lazy migration and forward compatibility; format profiles (Full, ReadOnly, Lite); per-chunk zstd compression; atomic write protocol; deterministic s-expression text projection; streaming read guarantees. DRM and encryption explicitly excluded.
October 11, 2026	Ch. 9	Constraint-solver interface chapter written: interface-and-contract specification rather than algorithm; the solver trait with from-scratch and incremental modes; Cassowary- style Required and Preferred constraint strengths; the constraint family catalog (spring, collision, alignment, containment, break, aesthetic, extension); the normative quality metric set (optical spacing, collisions, page fill, casting-off, beam angle, slur curvature, vertical balance, symbol density uniformity); Pareto- bounded quality target with configurable slack and tie-breaking weights for ambiguous cases; within-implementation byte-determinism with cross-implementation Pareto equivalence; incremental solving contract with propagation bounds; structured error reporting with partial solutions on budget exhaustion; reference suite for conformance; non-normative recommendation of Cassowary as a starting algorithm.

Date	Section	Change
October 11, 2026	Chs. 10, 11	Performance Requirements chapter expanded from skeleton: reference hardware profile (2026 desktop and tablet baselines), frame budgets for interactive edits at p99, cold-open targets, memory budgets separating core data from evictable caches, audio thread discipline requirements crossing the core/audio boundary, constraint-solver performance targets, file format performance targets, measurement methodology, regression testing recommendations. Extension Points chapter written from skeleton: consolidated catalog of every extension point in the spec organized by chapter of origin (notation grammar, tuning system, glyph, score graph, layout IR, constraint solver, file format extensions), the extension registry contract requiring versioning and identifier stability, plugin runtime considerations, conformance interactions.
October 11, 2026	Appendices	Glossary expanded with 50+ defined terms cross-referenced to their originating chapters. Bibliography populated with engraving treatises, music theory references, tuning and microtonality sources, constraint-based layout literature, CRDT references, and prior-art notation format citations.
October 11, 2026	All	Platform named *Epiphany*. Typography refreshed to TeX Gyre Pagella (body), Heros (sans), and Cursor (mono). Color palette refined to deep teal, antique gold, warm parchment neutrals, and deep crimson accents. Title page redesigned with ornamental rules and an oldstyle classical feel. Status section, introduction, and PDF metadata updated to reflect the platform's name.

Date	Section	Change
October 11, 2026	Pass 1 revision (Chs. 2, 3, 4, 5, 6)	Type-seam corrections in response to external review. Introduced `IdentifiedPitch` wrapping every embedded pitch with a stable `PitchId`; updated `PitchedEvent`, `TrajectoryEndpoint`, and `TrajectoryShape::Stepwise` accordingly. Introduced `EventDuration` union covering musical, wall-clock, and indeterminate cases, with concrete-only `DurationBounds` avoiding recursive indeterminacy. Aleatoric regions now declare an explicit `AleatoricAnchoringDiscipline`. `PitchSpelling` carries a stack of accidentals (`Vec<AccidentalId>`) with normative additive semantics and duplicate-detection invariants, replacing the prior `Option<AccidentalId>`. `PitchSpacePosition` gains a `JiVector` variant for first-class just-intonation lattice positions. The default CMN pitch space is now correctly described as `cmn-12` (diatonic-over-chromatic), not pure chromatic. Chapter 2's reference-pitch text rewritten to defer to Chapter 4, resolving the prior ownership contradiction. `MusicalPosition` and `MusicalDuration` now derive `Clone`, not `Copy`, with a permitted inline-or-pointer-tagged implementation noted as conforming. `Tie::pitch_pairing` now uses `Vec<(PitchId, PitchId)>` instead of positional indices. The duplicate `AnalysisLayer` definition in Chapter 2 was removed in favor of the authoritative definition in Chapter 5. Graph invariants expanded to cover identified pitches, tombstoning, time-model agreement, and `PitchId`-pairing tie targets. Representative operations (`InsertEvent`, `DeleteEvent`, `RespellPitch`, `Transpose`) updated for atomic ID minting, pitch tombstoning, voice-promotion on concurrent insertion, and `PitchId` preservation through transposition. Status downgraded to "structural working draft."

Date	Section	Change
October 11, 2026	Pass 2 revision (Chs. 3, 5, 6)	Graph and time invariants strengthened in response to external review. `TimeAnchor` offsets generalized via the `AnchorOffset` union (Musical, WallClock, Zero), with normative agreement between offset variant and target region's time model. `TempoSegment` restructured to carry explicit `end: Option<TimeAnchor>`, `start_tempo`, `end_tempo: Option<Tempo>`, and `shape`; tempo segments are required to be monotonic in musical time so that wall-clock-to-musical conversion is single-valued. Polymeter admitted natively: `StaffBasedContent` now carries `Vec<StaffInstance>` (region-local), plus `default_metric_grid: Option<MetricGrid>` and `barline_alignment_groups: Vec<BarlineAlignmentGroup>`. Measures and metric grids are per-staff-instance, not per-region. Staff identity split into the global `Staff` (declared once at score level) and the region-local `StaffInstance`; `Score` top-level structure expanded with `staves: Vec<Staff>` and `staff_groups: Vec<StaffGroup>`. `DeleteEvent` preconditions strengthened: target events belonging to tuplets require explicit `TupletCompensation` (`NotInTuplet`, `ReplaceWithRest`, `RewriteTuplets`, `CascadeDeleteTuplets`). Tie validation reframed from universal immediate-adjacency to class-specific profiles: `TieClass` (Standard, Editorial, CrossVoice, LaissezVibrer, Registered) admits editorial, cross-voice, and laissez-vibrer ties without relaxing standard-tie invariants. System-promoted voices formalized: `VoiceOrigin` enum (UserDeclared, Imported, SystemPromoted); promoted voices have deterministically-derived `VoiceId`s, are first-class graph objects, are visible to users, and are normalized only by explicit user operation. Graph invariants expanded from 14 to 18 normative invariants reflecting staff-instance ontology, per-staff measures, anchor-offset agreement, system-promoted voice identity, and barline-alignment-group containment. New identifier types: `StaffInstanceId`, `StaffGroupId`, `BarlineAlignmentGroupId`, `TimeSignatureId`.

Date	Section	Change
October 11, 2026	Pass 3 revision (Ch. 6, 8)	Concurrent semantics rewritten from pairwise operation merge to operation-set deterministic reduction. The replicated operation set is now framed as a true CRDT (a grow-only set of envelopes); the materialized score graph is its deterministic reduction, not itself a CRDT. `OperationId` (replica + counter) separated from `OperationStamp` (HLC + id); canonical reduction order is causal-then-HLC-then-lexicographic. Causal contexts expressed as dotted version vectors (`CausalContext`), not exhaustive predecessor lists; exhaustive lists are explicitly forbidden as canonical representation. `OperationEnvelope` introduced with id, author, stamp, causal context, optional transaction, payload. Four-phase operation lifecycle specified: Prepare, Commit, Reduce, Report. Operation effects formalized: `Applied`, `AppliedWithRepair`, `Conflicted`, `TombstonedTarget`, `NoOp{reason}`; every replica reducing the same set records the same effects with the same reasons. Conflict records made first-class graph objects in a score-level `ConflictRegistry`: stable, addressable, user-visible, resolvable by subsequent `ResolveConflict` operations. Tombstones formalized with explicit retention rules (pending replication, undo, conflict resolution, attachment resolution). Re-anchoring rewritten as a total deterministic function: "nearest" is now a strict lexicographic minimum over (containment proximity, time distance, direction preference, object id), with no implementation discretion. Re-anchoring rule table updated for the new staff-instance ontology and the deterministic "nearest" function. Transactions formalized as atomic replicated groups: primitive members **MUST NOT** materialize as independently visible intermediate states; failures conflict the whole transaction. Undo redefined as a forward compensating operation (`UndoTransaction`) with three policies (`StrictInverse`, `BestEffort`, `Cascade`); bit-identical undo demoted to content equivalence. LWW reduction restricted to explicitly-marked `LwwAdvisory` scalar fields (metadata, view preferences, layout hints, system/page break preferences, optional cross-cutting display hints); structural musical fields **MUST NOT** default to LWW. Cache invalidation rule: caches are acceleration only, never canonical; invalidated on operation-set, causal-order, reduction-algorithm, or profile change. Per-operation merge families renamed to reduction rules and rewritten in the new model; `ChangeRegionTimeModel` now correctly conflicts rather than silently coercing incompatible contained events; `SetUserSystemBreak` explicitly marked LwwAdvisory; `RespellPitch` concurrent col-

Date	Section	Change
October 11, 2026	Pass 4 revision (Ch. 8)	File-format restructured around Pass 3's operation-set canonical-document model. The earlier "single mutable manifest" story replaced with a fixed 64-byte header plus two fixed-size 256-byte superblock slots; the superblocks are the only mutable on-disk objects, and atomic commit is achieved by writing the inactive superblock slot with generation +1. Active superblock selected by highest valid generation. Generation gap greater than one detected as integrity anomaly with read-only recovery mode. Manifest reframed as a table of roots and declarations (operation_roots, retained_snapshots, blob_roots, profile_declarations, extension_declarations, text_projection root, integrity_root); user-facing metadata explicitly removed from the manifest in favor of operation envelopes and reduced state. BLAKE3 fixed as the single content-hashing algorithm, replacing the prior "BLAKE3 or SHA-256" uncertainty. Hash preimage domain-separated and explicitly excludes `compression_algorithm` to preserve deduplication across compression choices. Operation-envelope blocks specified as the canonical storage unit with a soft 1 MiB uncompressed target size and a default 64 MiB upper bound per profile. Snapshots separated into acceleration snapshots (caches, discardable) and canonical-base snapshots (declared in `retained_snapshots`, required for pruning). Pruning protocol specified: canonical state preserved as (base snapshot + post-frontier envelopes). Atomic write protocol specified step-by-step with explicit durable-flush points; the durable flush is platform-abstracted to cover `fsync`, `FlushFileBuffers`, and equivalents. Crash recovery rules enumerated by failure point. Garbage collection specified as conservative, optional, deferred; never on the commit critical path. Edit barriers formalized as a structured first-class concept (`EditBarrier` with scope, affected object kinds, prohibited operation kinds, and condition), with an unsafe-edit escape hatch that tombstones extension data rather than silently corrupting it. Text projection reframed: semantic round-trip required (envelopes, declarations, reduced state); physical-layout fidelity (chunk offsets, compression choices, cache chunks, garbage regions, superblock generation) explicitly not required. Schema versioning, format profiles (Full, ReadOnly, Lite, Custom), and compression sections updated for the new architecture. Streaming-read cold-open procedure rewritten to use snapshot + envelope-stream model.

Date	Section	Change
October 11, 2026	Pass 5 revision (Ch. 9)	Solver conformance reframed from universal Pareto optimality to reference-suite-based conformance with profile-specific thresholds. The earlier "no better Pareto layout exists" obligation, which quantified over an undecidable space of possible layouts, has been replaced with checkable, falsifiable thresholds on a reference suite. Solver obligations split into four classes: validity conformance (hard constraints, provenance, unit system, invariants), determinism conformance, diagnostic conformance (`SolveStatus`, `SolveReport`, warnings, unsatisfied-constraint reporting, metric vectors), and quality conformance (reference-suite thresholds). Three solver tiers introduced (`Minimal`, `Standard`, `Advanced`) with declared obligations per tier; tiers are orthogonal to file-format profiles. Deterministic budgets formalized (`max_iterations`, `max_nodes`, `max_constraint_evaluations`); wall-clock time demoted to advisory only. Wall-clock **MUST NOT** change the canonical layout. Quality metrics normalized: `NormalizedMetric` is a finite `f64` in $[0.0, 1.0]$ with 0.0 best and 1.0 worst; orientation and range fixed across all metrics including extension metrics. Hard constraints explicitly separated from quality metrics: a solver **MUST NOT** trade off a hard-constraint violation for a better quality score. Within-implementation determinism (byte-equal output for identical input + version) separated from cross-implementation conformance (reference-suite thresholds, not byte equality). `SolveReport` unified the prior `Result<SolveOutput, SolverError>` return: every invocation yields a report with a `SolveStatus` (`Solved`, `SolvedWithWarnings`, `PartialBudgetExhausted`, `Unsatisfiable`, `InternalError`). `PartialBudgetExhausted` preserves hard-constraint validity; budget exhaustion that cannot satisfy hard constraints returns `Unsatisfiable`. Six invalidation scopes specified (`ObjectLocal`, `MeasureLocal`, `SystemLocal`, `PageLocal`, `RegionLocal`, `WholeScore`); incremental solving contract reframed as observational equivalence to scoped full solving (solvers **MAY** widen scopes conservatively but **MAY**NOT narrow them). Reference algorithm demoted from "recommended starting point" to explanatory and diagnostic only; conforming implementations **MAY** choose any algorithm. Pass 4 rollback-language corrected: rollback is achieved by retained manifests in the chunk store, not by retaining multiple on-disk superblock generations (the fixed layout has only two slots).

Date	Section	Change
October 11, 2026	Pass 6 revision (App. D)	New Determinism Contract appendix consolidates reproducibility obligations scattered across Chapters 2–9 into a single normative reference. Five layers of determinism distinguished: canonical score determinism (byte-equal cross-implementation), canonical serialization determinism (byte-equal chunk hashes and text projection), layout determinism (byte-equal within implementation version), conformance determinism (reference-suite-based, not byte-equal cross-implementation), and non-canonical cache determinism (caches may vary, MUST NOT affect canonical state). Canonical floating-point values restricted to finite IEEE 754 binary64; NaN and infinity prohibited from canonical chunks; `-0.0` canonicalized to `+0.0` before storage and hashing. Little-endian IEEE 754 binary64 serialization fixed. IEEE 754 round-to-nearest, ties-to-even mandated for canonical numerical algorithms; `-ffast-math` and equivalents prohibited; reassociation, distribution, and implicit fused-multiply-add forbidden where they could change canonical output. Quantized layout coordinates introduced (`QuantizedCoord` at 1/1024 staff space) so canonical `ResolvedLayoutIR` positions are stable across implementations whose internal computations agree to within half-quantum. Tolerance classes formalized (`AcousticCents`, `LayoutCoordinate`, `QualityMetric`, `TempoIntegration`, `SolverResidual`) with governance category (`Equality`, `Validation`, `Diagnostic`); ad-hoc epsilons prohibited; tolerances MUST NOT apply to identity. Canonical iteration order specified for every enumerable collection (identifiers lexicographic on canonical byte form; operations causal-then-HLC-then-id; chunk references by kind, hash, offset; conflicts by id; extensions by id then version; strings byte-lexicographic on NFC UTF-8). Text normalization fixed at UTF-8 NFC; locale prohibited from affecting canonical parsing, serialization, sorting, decimal formatting, or projection. Randomness MUST NOT enter canonical state unless the seed is an explicit canonical input. Parallelism permitted but MUST produce results identical to a strict single-threaded baseline through deterministic ordered reductions. Compression confirmed as non-canonical (identical uncompressed content has identical hash regardless of compression choice); recompression permitted without altering canonical state. Open algorithm hooks (spelling pre-pass, decomposition, tempo integration, root-finding) required to be either normatively specified, profile-declared by versioned

Date	Section	Change
October 11, 2026	Pass 7 revision (editorial, all chapters)	Editorial consistency pass: no architectural changes, only cleanup. Resolved the contradiction between Appendix D's "byte-equal canonical state cross-implementation" and the existence of open canonical algorithms by adding an explicit conditional on the disposition of Section D.11. Stale section-number cross- references replaced with symbolic labels (Section 2.4 → `sec:pitch:spelling`; Section 2.4.4 → `sec:pitch:precedence`; Section 5.9.3 → `sec:graph:timebinding`; Section 6.6 → `sec:semops:conflicts`; Section 6.8 → `sec:semops:undo`). Terminology normalized: "merge family" → "reduction rule" where the operation framework was meant; "Pareto-bounded quality model" → reference-suite/normalized-metric language in extension points; glossary entries for Operation log, Operation set, Operation envelope, Canonical reduction, Pareto frontier (design target), Staff, Staff instance, Conflict record, CRDT, Snapshot, and Transaction rewritten or added. New Appendix A (Intentionally Deferred Types and Specifications) catalogs companion specifications (Binary Format, Text Projection, Profile Conformance, Quality Metric Catalog, Reference Suite, Performance Reference Suite, Operation Catalog, Reference Algorithm), externally provided components (SMuFL, font metric provider, audio engine, plugin runtime, collaboration transport, user interface), open canonical algorithms, and extension registry catalogs; distinguishes deliberately external content from accidentally missing content. Status section rewritten to reflect structural stabilization: substantive review passes complete, architecture not under active redesign, remaining blockers are companion specifications and open canonical algorithms; the section enumerates what conformance can be established now and what awaits companion delivery. Revision history entries for Passes 1–7 read as the document's architectural-stabilization log, consolidating the design decisions taken in response to external review.

Date	Section	Change
October 11, 2026	Pass 8 revision (post-review correctness pass)	Determinism correctness pass addressing review of the Pass 7 output. `ConflictId` reframed as content-derived via BLAKE3 truncation with domain tag `"MUSCCONF"`, fixing a determinism leak where each replica would have minted its own conflict identifiers from local replica-counter sequences; conflict records produced during reduction now have identifiers that agree across replicas by construction. `RepairRecord` defined (was referenced by `OperationEffect::AppliedWithRepair` without definition), with a `RepairKind` enumeration covering `Reanchored`, `SpannerTruncated`, `Orphaned`, `CascadeDeleted`, `AttachmentTombstoned`, `VoicePromoted`, `TupletCompensated`, and `Registered`. `TypedObjectId` defined as a tagged union over every identifier kind in the score graph, with canonical-bytes serialization for hashing/ordering/equality. `OperationKind` defined as a tagged enumeration of primitive operation variants, plus a `Registered` variant for extension-defined primitives. System-derived identifier namespace formalized: `ReplicaId::SYSTEM_DERIVED` reserved at `0xffff_ffff_ffff_ffff`, with the 64-bit counter derived deterministically via BLAKE3 truncation; domain tags for system identifiers (`"MUSCSVCE"` for system-promoted voices, `"MUSCSPCH"` for system-derived pitches) enumerated; user-authored replicas **MUST NOT** use the reserved value. Envelope-acceptance rules added: a well-formed envelope requires non-system replica id, finite physical time, well-formed DVV, deserializable payload; per-replica monotonicity enforced; clock-skew warnings allowed but reordering for plausibility prohibited; authentication deferred to collaboration transport. Canonical-base snapshot semantics repaired: manifest split into single `canonical_base: Option<SnapshotRef>` plus `acceleration_snapshots: Vec<SnapshotRef>`; coverage test specified as DVV-frontier membership, explicitly forbidding HLC-stamp comparison for canonicality determination; pruning replaces the canonical base atomically. Header CRC moved to the last 4 bytes of the 64-byte fixed header, resolving a circular-definition bug where the CRC field appeared inside its own coverage range. Manifest chunks **MUST** be stored uncompressed in this format version, resolving the bootstrap chicken-and-egg problem (a reader needs to know how to decode the manifest before it has any manifest information). `ChunkId` defined as a newtype around `ContentHash`, clarifying the prior comment that the hash and id were redundant. Three-layer identity

Date	Section	Change
October 11, 2026	Pass 9 revision (correctness and consistency pass)	Operation-envelope duplicate handling formalized into three outcomes: `IdempotentDuplicate` (same id, byte-identical envelope: silently dropped), `FirstAcceptance` (new id: accepted after well-formedness check), and `EnvelopeEquivocation` (same id, different canonical bytes: divergent envelope rejected, integrity diagnostic surfaced, bundle non-conforming if equivocation appears on-disk). Well-formedness invariant `envelope.stamp.id == envelope.id` made normative. HLC monotonicity reframed as a property of the set of accepted envelopes, not arrival order: out-of-order delivery is fine; on detection of inconsistency, both envelopes are retained but the offending replica's stream from the violating counter onward is quarantined and the document is flagged with a replica-equivocation anomaly. The free-form `NoOpReason::PreconditionFailedUnderReduction reason: String` replaced with a typed `PreconditionFailureReason` enum, closing a determinism leak (free-form strings in canonical state could otherwise differ across implementations). Transaction descriptor ordering integrated into canonical reduction via causal dependency: every primitive member of a transaction **MUST** causally depend on its `DeclareTransaction` envelope; members lacking such a dependency reduce to `NoOp{TransactionConflict}` with a synthesized conflict record. The fixed-header `file_uuid` requirement reworded to match the Pass 8 identity model: fixed for the lifetime of a physical bundle, preserved by filesystem byte-copy, freshly minted on Save As. Manifest `integrity_root` made `Option<ChunkRef>` (it is an accelerator); new "Canonical and Non-Canonical Manifest Roots" subsection partitions the manifest's fields by their role in canonical state, with the rule that failed verification of non-canonical chunks is rebuildable rather than corruption. Cold-open procedure rewritten to reflect the canonical-base / operation-envelope-block model (the prior text retained pre-Pass-3 region-chunks language). Streaming- read locatability list updated. System-derived counter collisions: mandatory collision check during reduction; collision becomes hard corruption via new `ConflictKind::SystemIdentifierCollision` record naming both colliding canonical-input sets. `GlyphCatalogIdentity metrics_hash` scope broadened: hash covers glyphs referenced by the `ConstrainedLayoutIR` (solver inputs), not just glyphs in the final `ResolvedLayoutIR`. `CommitState` validation added to superblock selection: non-

Date	Section	Change
October 11, 2026	Pass 10 revision (order-independence and integrity-anomaly cleanup)	`OperationId` equivocation reworked from a first-arrival rule (which made canonical state arrival-order-dependent across replicas observing the same envelopes in different orders) to an order-independent `OperationSlot` model. The slot is either `Single(OperationEnvelope)` or `Equivocated { operation_id, candidates: BTreeSet<EnvelopeHash> }`; observing a second distinct canonical envelope under an `OperationId` transitions the slot to `Equivocated` regardless of arrival order. Equivocated slots produce no canonical reduction effect; both candidates are retained for diagnostic recovery. Operations causally depending on an equivocated id are held pending resolution. Resolution is by explicit policy: transport-level credential revocation, local user reconciliation, or a profile-declared deterministic selection function. No arrival-order default applies. HLC monotonicity-anomaly handling sharpened: a new `AnomalousReplicaSegment` type captures the offending replica, the first violating counter, the reason, and the excluded envelope ids. Operations from the violating counter onward are retained but *excluded from canonical reduction*; canonical state is derived only from envelopes that satisfy the monotonicity invariant. The earlier "quarantined" wording is now backed by an explicit structural exclusion rule. System-derived ID collisions moved out of `ConflictKind` and into a new `IntegrityAnomaly` type with kinds `SystemIdentifierCollision`, `OperationSlotEquivocated`, and `ReplicaStreamQuarantined`. The distinction is that conflict records are ordinary canonical-state facts (a representable musical conflict between two valid edits) while integrity anomalies indicate the structural assumptions underlying canonical state have failed and require external recovery; an integrity anomaly takes the document out of ordinary canonical operation. Stale "refined in a later revision" parenthetical removed from the `DeleteEvent` reduction rule; the re-anchoring total order is part of the current architecture and needs no further refinement. Typo fixed in the superblock-selection requirement (`MAY-be-valid` replaced with the prose "invalid for ordinary selection but may be inspected in diagnostic recovery mode"). Glossary chapter references for "Dotted version vector" and "Hybrid logical clock" updated ~~"Chapters 6, 8"~~ with explicit "defined operationally in Chapter 6; serialized in Chapter 8" wording, reflecting that the Pass 3 architectural shift made causal context part of operation semantics

Date	Section	Change
October 11, 2026	Pass 11 revision (spec ratification)	Converted the v0 implementation's provisional, golden-locked byte choices into ratified normative spec text; the architecture is unchanged. *Adopt-and-pin (bytes):* pinned the `TypedObjectId` 16-bit big-endian discriminant table (`0–27`, `Registered` = 27; added the `Tuplet`/`RepeatStructure`/`LyricLine`/`ChordSymbol`/`View` variants the implementation carried); the promoted-voice (`MUSCSVCE`), system-pitch (`MUSCSPCH`, tuning always part of identity), and integrity-anomaly (`MUSCSANM`, promoted to a reserved built-in tag) derivations; the `ChunkKind` / `ProfileId` / `CompressionAlgorithm` discriminants; the `ManifestId` preimage (with `manifest_id` excluded); and the `RationalTime`/scalar primitive layouts plus the shared codec convention baseline the Binary Format companion inherits. *Decide-and-pin:* `TempoShape::Linear` interpolates *speed* (whole notes/second), not bpm; a `StructuralFieldCollision` tags the *winner* `Conflicted`; lifted the >2-way voice-promotion generalization (partial-overlap, lowest-id-survivor) to normative; pinned the `TransactionCategory` and `ObjectKind` core vocabularies; added `ResolutionAction::Dismiss` so the `Dismissed` resolution state is reachable by an authored operation; and specified the (non-canonical) `LayoutObjectId` derivation with a `MUSCLOID` tag as the Track A target (then deferred to Track A; subsequently wired by the reference implementation in P12-I2). *Fixes:* blob hashing is the bare `"MUSCBLOB" || payload` (deleted the contradictory "identically to chunks" phrasing); added the equal-generation superblock rule (`DivergentSameGeneration` on load-bearing-field divergence); defined `ProfileConstraints` and placed the required `RetentionPolicy` in it with first-declared multi-profile precedence; made the DVV zero-based counter floor normative; reconciled the graph-invariant count to 19 (was 18 in QUICKSTART) and named the three construction-time MUSTs (time-signature beat-group sum, ordering-DAG acyclicity, and non-degenerate `TupletRatio`, now enforced at construction). *Consolidation:* added the Canonical Byte-Layout Reference (Appendix E) gathering every ratified discriminant, derivation preimage, and primitive encoding into one place for the Binary Format companion to import, with a domain-tag registry and golden-lock anchors; and closed two follow-through gaps where the discriminant *bytes* of `ObjectKind` (`Voice` = 0, `Pitch` = 1, `Registered` = 2; feeds the `IntegrityAnomalyId`

Date	Section	Change
October 11, 2026	Pass 12 tranche 1 (audit spec-alignment)	Aligned spec text with dispositions the project had already made and the reference implementation exercises; no byte-layout or architecture change. The re-anchoring rule table's Slur and Spanner rows now state the Operation Catalog's surviving-endpoint semantics (re-anchor while $\geq$ 1 endpoint survives, cascade-delete only when none does; proximity-aware re-targeting deferred). `TupletCompensation::RewriteTuplets` adopts the catalog's id-only v1 payload (`TupletRewrite` removed): graph-aware reduction refuses the variant rather than fabricate rewritten values, and a value-carrying payload revision reopens it. Operation-envelope blocks are pure envelope vectors (a content-addressed chunk cannot embed its own id); the per-block summary metadata (`dvv_summary/min_stamp/max_stamp`) relocated to the manifest's `operation_block_summaries` map (opaque ops-supplied bytes, non-canonical, ChunkId-keyed), and the order-blocks-by-`min_stamp` **SHOULD** was dropped because the canonical manifest encoding sorts chunk references by encoded form. The spelling and decomposition pre-passes are now specified as producing *canonical derived annotations* — deterministic functions of (materialized graph, profile, versioned algorithm id), recomputed on materialization and never stored — replacing the stored-`Inferred`-attachment model; the incremental re-run requirement is demoted from **MUST** to **MAY** with caching and incrementality required to be unobservable. Spelling-precedence ties now break by canonical attachment order (attachments carry no creation timestamp). The `ConflictRegistry` moved off the `Score` root into canonical materialized state, matching Chapter 6. `SolverTier` gains the non-conformance `Stub` variant. Companion: Operation Catalog 0.2.0 $\rightarrow$ 0.3.0 (ModifyEvent materializes metric placement changes behind a voice-occupancy placement precondition). Dispositions logged in `spec/PASS12_RATIFICATION_LOG.md`; the Pass 12 batch itself remains open.

Date	Section	Change
October 11, 2026	Pass 12 tranche 1 addendum (Push-3 enablers)	Two enabler ratifications for the machinery-wiring work. `ResolveEquivocation`, previously a dangling prose reference in the equivocation section, is now a defined meta-operation: `OperationPayload` gains the variant here, and the Operation Catalog (0.3.0 → 0.4.0) pins its payload schema (`{ target: OperationId, chosen: EnvelopeHash }`), the order-independent earliest-resolve-governs promotion, idempotence, differing-resolve meta-conflict, and precondition no-ops; the profile-declared selection-policy path remains open (P12-K5). Engraving break overrides became position-addressable: `OverrideKind::SystemBreak/PageBreak` carry the break's `TimeAnchor` (the `ScoreGraph` target names the owning region; projected break overrides carry `Internal` origin pending P11-C8 authorship surfacing), closing the representational gap that blocked the logical-stage override projection §Engraving Overrides requires.
October 11, 2026	Phase-3 first tranche (casting-off + catalog)	Companion movements, no core-spec byte or architecture change. The Operation Catalog (0.4.0 → 0.5.0) gains the Phase-3 schema-fill tranche — `CreateStaff` (set-union global-staff mint, plus a staff-liveness precondition on `CreateStaffInstance`), `SetTimeSignature` and `SetTempoSegment` (LWW structural overwrites beneath the whole-grid `SetMetricGrid`), `SetStaffLayout` (LWW advisory) — and a rewritten UNDOTRANSACTION: `StrictInverse/BestEffort` now perform *value restoration* for overwrite primitives via canonical-order write chains (delete resurrection, Transpose inversion, and Cascade closure stay deferred, P11-C8 narrowed). The Binary Format companion (0.1.0 → 0.2.0) appends the corresponding wire discriminants. This document's `OperationKind` listing gains the four kinds. The create score/canvas slots remain deliberately unavailable pending an addressable root model (P12-K8). (Reversed in part by `spec/RULING_GENESIS_PERSISTENCE.md`, ratified with the genesis operation tranche: the root/canvas *objects* stay unavailable as ratified here, but the score's *contents* are no longer outside the operation set — see THE CANVAS, §5.5.)

Date	Section	Change
October 11, 2026	Schema major 1 (data-model expansion)	Defines the two referenced-but-undefined types `CanvasLayoutDefaults` (with `CanvasSize` / `CanvasMargins`, staff-space page geometry, A4/8 mm default; closes P12-I7) and `PitchRange` (advisory pitch compass, used by `Instrument.range` and `IndeterminacyHints`); adds `Region.permits_spanning_slurs` (default false, P12-K7). Records that schema major 1 is the first data-model expansion major: the canonical-base `MaterializedState` embeds none of these values and stays major 0, byte-identical, but `Region.permits_spanning_slurs` reaches the *canonical* `CreateRegion` operation payload, so a major-0 reader opens a bundle carrying v1 `CreateRegion` ops read-only. The wire form, the resolved-layout length-prefix unification (the manifest-embedded barrier blobs stay `u64`), and the byte-for-byte v0→v1 migration are ratified in the Binary Format companion (0.2.0 → 0.3.0).
October 11, 2026	Schema major 1 Phase F (engrave ratify-only)	Ratifies three engrave/layout-ir dispositions the implementation already made (Pass 12 batch I8/I9/I10, now struck), with no byte-layout or architecture change: the *break-constraint satisfaction* predicate (Requirement 7.12 — a `SystemBreakAt` / `PageBreakAt` is satisfied iff the final `ResolvedLayoutIR` starts a system/page at that slot, a region-first slot trivially); *break-override attribution* threaded through a `ConstrainedLayoutIR.break_origins` sidecar, declining to widen the normalized constraint record (Requirement 7.8); and the *system-continuation* synthesis kind `Registered(SYSTEM_CONTINUATION_SYNTHESIS)` with an `(original, ordinal)` instance key (Requirement 7.3). Flags one open item: which region governs a cross-region slur's spanning permission (implemented as conservative AND, P12-K12).

Date	Section	Change
October 11, 2026	Pass 12 Ratification (the batch pass)	Retires the open Pass-12 batch (28 rows; dispositions in `PASS12_RATIFICATION_LOG.md`). Algorithm dispositions: the spelling pre-pass and notational decomposition move to profile-declared-by-versioned-identifier with ratified v1 defaults (Requirements 2.10 and 3.16; closes P12-H1/H3/H4/H5/C5); decomposition precedence pinned fixed (P12-H6); authored annotations for inference-ineligible targets surface (Requirement 2.11, P12-H7). Identity: system-derived intrinsic content immutable under reduction (P12-K3). Genesis ratified outside the operation set (P12-K8 — reversed in part by `spec/RULING_GENESIS_PERSISTENCE.md`, as annotated above: the root/canvas objects stay unavailable, the score's contents no longer do). Cross-region slur permission = AND (P12-K12). Re-anchoring: cue cascade-on-any-source, Range truncate defined, annotation orphaning sanctioned, `ReanchorReason::SameCanvasNearer` appended (P12-C1/C2/C3/C4). Barriers: target-free and opaque-operation matching (Requirement 8.25, P12-E4); unsafe-edit tombstone semantics (Requirement 8.26, P12-E5; encoding deferred to the Binary Format companion). Solver: kind-determined strength (Requirement 9.5, P12-I4), sub-conformant report shape (Requirement 9.3, P12-I5), the Minimal-tier constraint floor (Requirement 7.9, P12-I6). The stale `OperationKindTag` listing gains the eleven appended tags. Companion movements: Operation Catalog 0.5.0 → 0.6.0, Binary Format 0.3.0 → 0.4.0.

Date	Section	Change
October 11, 2026	Schema major 2 (data-model expansion, Phase A)	Defines the leaf types Chapter 5's ratified shapes referenced but never pinned: `SlurKind`, `CurveDirection`, `CurvatureOverride`, the shared `SpanStyle` record (consolidating the formerly-sketched identical `SlurStyle`/`TieStyle`/`SpannerStyle` triplet), `SubBeam`, `BeamGeometryOverride`, the `SpannerKind` payload types (`HairpinDirection`, `OctaveOffset`, `PedalKind`, `TextLineDefinition`, `BracketKind`) — `SpannerKind` gains a leading `Generic` variant as the honest migration default — `Volta`, `MetadataValue`, `Timestamp`, `SoundConfiguration`, `TranspositionInterval` (structural, advisory until the Chapter 4 tuning catalog per the P12-K2 discipline), `UnpitchedMember`, `SpaceUnit`, `LineStyle`, and `StaffBracketKind`. Ratifies the clef/key content model into Chapter 5 (`Clef`, `ClefShape`, `KeySignature`, `ClefChange`, `KeySignatureChange` — the visible-slice value types) and retires the `ClefId` identifier sketch: `Staff.default_clef` and `Instrument.default_clef` embed `Clef` values, and `Instrument.default_staff_lines` is reconciled from `u8` to `StaffLineConfiguration`. Metadata timestamps are pinned strictly authored (Requirement 5.1: nothing writes them implicitly). Chapter 8 gains the schema-major-2 paragraph (canonical base stays major 0; the filled values reach the snapshot and the seven embedding operation payloads, `InsertRegion` transitively). Wire forms and the total default-filling migration: Binary Format companion 0.4.0 → 0.5.0.
October 11, 2026	Schema major 2 (repeat authoring, Phase D)	The dedicated `CreateRepeatStructure`/`DeleteRepeatStructure` pair enters the operation set: Chapter 6's `OperationKind` enum and Chapter 8's `OperationKindTag` listing gain the two variants (repeats are not `CrossCuttingValue` kinds on the wire — the reconciliation note follows the `CrossCuttingStructure` listing), and the re-anchoring rule table gains the Repeat-structure / Anchor row (re-anchor to the nearest surviving anchor across *every* event-referencing anchor site — `start`/`end`, jump targets, volta spans — cascade-delete only when none survives). Six-part schemas: Operation Catalog Repeat Structures, 0.6.0 → 0.7.0. Wire discriminants 28/29 and the per-payload stamping (create born at v2; delete a minor kind append at major 0): Binary Format companion 0.5.0 → 0.6.0.

Date	Section	Change
October 11, 2026	Schema major 2 (rendering, Phase F)	The rendering consumers of the schema-major-2 data model are ratified into Chapter 7. `ResolvedLayoutIR` gains the non-glyph primitive vocabulary it has always carried in implementation — line `Stroke`s (staff lines, stems, barlines, volta brackets) and cubic-Bézier `Curve`s (slurs) — as 7.13 (provenance-traced, non-canonical). Two Minimal-tier rendering floors join the constraint floor: 7.15 (E1: repeat barlines at resolved boundaries, volta brackets with ending numbers, honest non-drawing of unresolvable boundaries, jump-kind marks and cross-region repeats deferred) and 7.16 (E2: a slur renders as a cubic Bézier honoring `CurvatureOverride`, honest non-drawing of unresolvable or cross-staff spans, a non-`Solid` line style surfaced not silently rendered solid). The curvature-computing algorithm and the `RenderIR` encoding stay forward-referenced out (7.13, 7.9). Layout geometry is non-canonical, so no wire form and no companion version moves — an engrave-side ratify-as-implemented, mirroring the schema-major-1 Phase F precedent.
October 11, 2026	Layout IR (slur rendering completion, Push 3)	The three slur refinements 7.16 deferred are now landed and its requirement extended: an authored non-`Solid` `SpanStyle` line renders faithfully (dashed / dotted — the `LineStyle` rides the `Curve`, whose listing gains the field), a slur spanning a system break splits into per-system sub-curves (de Casteljau; first segment keeps the slur's provenance, the rest synthesized continuations), and `slur_shape_penalty` moves off its pinned 0.0 to a real measurement (Quality Metric Catalog SLUR_SHAPE_PENALTY rationale refreshed; formula unchanged, no version move). Layout geometry stays non-canonical — no wire form, no companion version moves.

Date	Section	Change
October 11, 2026	Layout IR (primitive band ownership, Push 3)	Chapter 7 gains 7.11: every primitive the projection presents to the solver — `GlyphObject`, `Stroke`, `Curve` — declares the `VerticalBandId` that owns it, naming a band that exists, and a vertical solver takes ownership from that declaration rather than inferring it from geometry. Ratifies-as-implemented the contract a vertical solver needs and an inference cannot supply: a slur's endpoints are lifted clear of its own staff, so its owner is not recoverable from where it is drawn. The `ConstrainedLayoutIR` listing gains `strokes` / `curves` (present since staff lines, never listed) and the `Stroke` / `Curve` listings gain `vertical_band`; a band now exists for every staff of a region and for its margin, members or not. 7.14 carries the declaration through the resolved stage on strokes and curves (a `ResolvedGlyph` keeps none — its ownership is already baked into its position). Band ownership is non-canonical attribution metadata: it enters no content hash and no canonical encoding, so no wire form and no companion version moves.

Date	Section	Change
October 11, 2026	The Fixed Header (container epoch)	Chapter 8 gains The Container Epoch and 8.3 (`spec/CONTRACT_FORMAT_EPOCH_MAJOR1.md`). `format_major` becomes 1 and `format_minor` restarts at 0. A reader classifies the major three ways — legacy 0, current 1, unsupported otherwise — and carries the classification as a value rather than re-deriving it by comparison, so a legacy container is decoded deliberately instead of refused. The epoch exists because a canonical base is only meaningful when it has been validated against a reduction authority, and nothing in a document could previously attest that: the reduction-algorithm version is a bare integer that a writer copies from the base's own self-report, so the one check that would detect a stale base necessarily passes for a base that was propagated conformingly. The header can attest it because the header never changes after creation — which makes the epoch a property of the file's creation that no later commit can confer. A legacy container may therefore open while it carries no base, **MUST NOT** acquire one, and is refused outright if it already has one; a repack into a freshly created major-1 container is the only path forward, and read-only degradation is explicitly not offered, since a restricted view of unverifiable canonical state is still unverifiable canonical state. Corruption keeps precedence over every epoch rule in both epochs, so tampering is never reported as staleness. Schema Versioning gains the corresponding distinction between the schema axis and the container axis.

Date	Section	Change
October 11, 2026	Ch. 8	The Canonical Document Identity gains Requirement 8.16: an implementation **MUST** name the reduction semantics it implements and validate a canonical base against *that*, not against a value the base itself supplied. Comparing the base's `reduction_algorithm_version` only against the active superblock's is necessary but not sufficient, because a writer that seeds the superblock from the base's own self-report makes the comparison an identity — so it holds for every conformingly written document and detects only tampering. The requirement therefore introduces the implementation's own semantics as a third participant, required at both boundaries: a reader refuses a base that disagrees with the semantics it implements, and a writer refuses to emit or replace one. A document with no canonical base opens regardless, carrying no reduced state that could be stale, and a commit that does not touch the base is not refused on this ground. The refusal may not degrade to a read-only open nor be reported as an integrity anomaly — a stale base is not a restricted-but-correct view but state computed under different rules — and it stays distinguishable from the malformed-document failure a base disagreeing with its own superblock produces, since the two detect different faults. Rebuilding during open is prohibited outright, because the envelopes a rebuild would need may have been pruned.

Date	Section	Change
October 11, 2026	TOP-LEVEL SCORE STRUCTURE, GRAPH INVARIANTS	**P13-S16: `StaffGroup.members` becomes a maintained projection (disposition A), superseding genesis tranche G3a's disposition B, and graph invariant 21 is appended.** `Staff.group` was already the sole authority for group membership, but `members` was stored exactly as carried and neither maintained nor trusted, so *both* disagreeing states were permitted outcomes rather than defects — a sound ruling while the tranche minted only, since no authoring order could repair a disagreement once made. This revision makes `members` derived: `CreateStaffGroup` refuses a non-empty carried `members` (`ContainerNotEmpty`, an *empty-container* precondition on the carried value and therefore enforced base-free, unlike the sibling mints' graph-aware liveness checks), `CreateStaff` carrying a group appends the staff to that group's `members`, and undo of a staff strips it back out. The two **MUST** now agree in both directions, which new graph invariant 21 (`StaffGroupMembershipAgreement`) enforces between live objects while abstaining on dangling membership — an undeclared member is invariant 10's reference-resolution concern, not a disagreement. The enumeration count moves from 20 to 21 accordingly.

Date	Section	Change
October 11, 2026	Graph Invariants, Aleatoric Time	**P13-S26: graph invariant 10 names its complete reference surface, and aleatoric reference locality becomes a labelled requirement.** Invariant 10 previously read only that "every cross-cutting structure's references resolve to extant objects", naming no individual class — while the checker enforces a surface spanning cross-cutting structures, structural top-level references, meter and time-signature references at every level a `MeterChange` appears, an attachment-internal reference, event-internal references and the tempo map's segment anchors. That sentence is also scoped to cross-cutting structures alone, a defined term narrower than the surface beneath it, so the item now opens with a universal subordinated to the re-anchoring rules that explicitly permit transient dangling, and retains the original sentence unchanged beneath it. The surface is enumerated, derived from the checker rather than copied from either prior summary, since the normative enumeration and the implementation's own doc comment were incomplete in different places and neither could be repaired from the other. Tempo-map ordering and non-overlap are named as *not* this invariant's, being governed by Requirement 3.11. Separately, Aleatoric Time gains Requirement 3.23: an aleatoric region's ordering and bounds references **MUST** name events of that same region — a locality rule the checker has always enforced and no requirement stated.
October 11, 2026	Graph Invariants, Time and Duration	**P13-S29: the violation tag stops multiplexing.** Graph invariant 10 reported Chapter 3 and Chapter 4 failures under its own number, through a public API. The violation type becomes `WellFormednessViolation` carrying a two-armed `ViolationKind`: an invariant arm and a requirement arm naming a `req:` label. Invariant 10 keeps only reference resolution; tempo segment shape, tempo segment order, aleatoric reference locality and accidental modification expressibility now report under their own requirements. Time and Duration gains Requirement 3.12, stating the enforced shape/`end_tempo` compatibility without determining whether a constant segment records an `end_tempo` at all. The enumeration is unchanged.

Date	Section	Change
October 11, 2026	Sounding Duration and Notational Decomposition	**X3.3 and X3.4: the default decomposition algorithm reaches version 2.** Requirement 3.16's scope bounds widen: barlines from each staff instance's measures, so every governing meter is honoured; a pickup keeps its place in its bar; a span one value expresses within a bar is that value, dotted or double-dotted. Derived output is invalidated, not migrated.
October 11, 2026	Chapter 6 `OperationKind`, Chapter 8 `OperationKindTag`, Re-Anchoring	**X3.1: tuplets enter the operation set.** `OperationKind` and `OperationKindTag` gain `CreateTuplet`, which mints a `Tuplet` (Tuplets as Grouping Objects) into the cross-cutting registry: semantics in the Operation Catalog CreateTuplet, wire discriminant 40 and epoch 13 in the Binary Format companion, the `create-tuplet` production in the Text Projection companion. The re-anchoring table's "Tuplet / Member event" row gains its uncompensated case: a member tombstoned with no compensation to declare cascade-deletes the tuplet. The reduction algorithm version moves to `2` (Operation Catalog, version 0.16.0 note).
October 11, 2026	Graph Invariants	**X3.5: a pickup's successor is consistent.** Graph invariant 20's boundary clause measured the second measure's distance from a first measure against a full bar, so every pickup's successor was flagged (P13-S19). A first measure may now be shorter than its bar; every later distance is still a full bar. `CreateMeasure` admits the same successor (Operation Catalog, version 0.16.0 note).
October 11, 2026	Chapter 6 `OperationKind`, Chapter 8 `OperationKindTag`	**X3.6: clef and key changes enter the operation set.** `OperationKind` and `OperationKindTag` gain `SetClef` and `SetKeySignature`, which set, replace or remove the change at a musical offset in a live staff instance's clef or key sequence: semantics in the Operation Catalog SetClef and SetKeySignature, wire discriminants 41 and 42 and epoch 14 in the Binary Format companion, the `set-clef` and `set-key-signature` productions in the Text Projection companion. The reduction algorithm version does not move: no operation reducible before changes its verdict or the state it produces.

Date	Section	Change
October 11, 2026	Tuplets as Grouping Objects	**X3c: a tuplet's display.** `Tuplet.display`, listed here since the first revision but never defined, nor carried, gains its type: `TupletDisplay`, a `TupletNumber` (`Actual`, `None`) and a `TupletBracket` (`Auto`, `Hidden`), each growing by appended variant. It is canonical state, on the wire from schema major 4 (Binary Format companion Schema Major 4), and Requirement 3.18 binds an engraver to it. The reduction algorithm version does not move: the display enters no verdict and no canonical reduced state.
October 11, 2026	Tuplets as Grouping Objects	**X4a.0: the `Tuplet` listing in wire order.** The listing gains `required_total`, carried since the first revision but never listed, and gives the fields in the order the binary layout writes them, `display` last; it had put `display` before `parent`. The Text Projection companion writes a struct's fields in the order its listing here declares (`req:textproj:value-projection`), so the listing now agrees with the text form, which always followed the binary layout. No byte and no requirement changes.
October 11, 2026	Pitched Events, Markers, Lyrics, Graph Invariants	**X4b: expression and text** (roadmap D58). An event's `articulations`, `ornaments` and `grace`, listed since the first revision over placeholder types, take their types: `marks` (`EventMark`), `Ornament` and `Grace`, marks and ornaments canonical sets (Requirement 5.15); `GraceKind` keeps its first two variants. A grace note has zero duration (Requirement 5.14), which invariant 3 now states. The `Marker` listing, which named kinds no writer produced and a display the engraver owns, becomes the point marks the ruling defines; `LyricLine` becomes `Lyric`, one syllable an event and verse (Requirement 5.27, invariant 10, and a cascade-delete row in the rule table); `Text` is held to NFC (Requirement 5.28); `LineStyle` gains `Wavy`, `SpannerKind` `PedalBracket`, and `Score` `voice_homes`, written by `SetVoiceHome`. On the wire from schema major 5 (Binary Format companion Schema Major 5); the reduction algorithm version is `4`, which X4b opened.